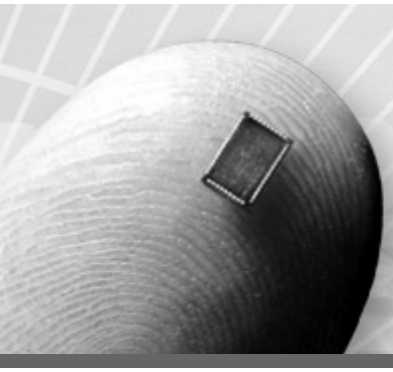
ON Semiconductor®



CFX DSP Architecture Manual

M-20470-010

ON Semiconductor

September 2011

This document may not be redistributed or transmitted in any form.

Copyright © 2011 Semiconductor Components Industries, LLC d/b/a ON Semiconductor.

ON Semiconductor and the ON logo are registered trademarks of Semiconductor Components Industries, LLC (ON Semiconductor). All other brands, product names and company names mentioned herein may be trademarks or registered trademarks of their respective holders.

ON Semiconductor reserves the right to make changes without further notice to any products herein. ON Semiconductor makes no warranty, representation or guarantee regarding its products including suitability for any particular purpose, nor does ON Semiconductor assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation special, consequential or incidental damages. "Typical" parameters which may be provided in ON Semiconductor data sheets and/or specifications can and do vary in different applications and actual performance may vary over time. All operating parameters, including "Typicals" must be validated for each customer application by customer's technical experts. ON Semiconductor does not convey any license under its patent rights nor the rights of others. ON Semiconductor products are not designed, intended, or authorized for use as components in systems intended for surgical implant into the body, or other applications intended to support or sustain life, or for any other application in which the failure of the ON Semiconductor product could create a situation where personal injury or death may occur, unless otherwise expressly agreed to in writing by ON Semiconductor and BUYER. Should Buyer purchase or use ON Semiconductor products for any such unintended or unauthorized application, Buyer shall indemnify and hold ON Semiconductor and its officers, employees, subsidiaries, affiliates, and distributors harmless against all claims, costs, damages, and expenses, and reasonable attorney fees arising out of, directly or indirectly, any claim of personal injury or death associated with such unintended or unauthorized use, even if such claim alleges that ON Semiconductor was negligent regarding the design or manufacture of the part. ON Semiconductor is an Equal Opportunity/Affirmative Action Employer. Literature is subject to all applicable copyright laws and is not for resale in any manner.

To maintain the quality of our publications, we request your comments on the accuracy, clarity, organization, and value of this document.

Address correspondence to:

Americas and Asia
ON Semiconductor
200-611 Kumpf Drive
Waterloo, Ontario, Canada N2V 1K8
Tel: +1 519 884 9696

Europe, Middle East and Africa
ON Semiconductor
Champs-Montants 12a
2074 Marin, Switzerland
Tel: +41 32 755 74 30

Website
www.onsemi.com

E-mail
dsp.support@onsemi.com

TABLE OF CONTENTS

1. Introduction 1

1.1 Purpose . . . 1
1.2 Intended Audience . . . 1
1.3 Conventions . . . 1
1.4 Manual Organization . . . 1
1.5 Further Reading . . . 2

2. CFX Architecture Overview 3

2.1 Types of Operations and Parallel Execution . . . 5
2.2 Execution Units . . . 5
2.3 Registers . . . 6
2.4 Memory Spaces . . . 6
2.5 Buses . . . 7

3. Instruction Types 9

3.1 Introduction . . . 9
3.2 Long Instructions . . . 9
3.3 Arithmetic-Move Instructions . . . 10
3.3.1 Arithmetic Instructions . . . 10
3.3.2 Move Instructions . . . 12
3.4 Move-Move Instructions . . . 12

4. Data Computation Unit 15

4.1 Introduction . . . 15
4.2 DCU Architecture . . . 15
4.2.1 Data Registers . . . 15
4.2.1.1 Accumulators . . . 16
4.2.1.2 General-Purpose XY Data Registers . . . 16
4.2.2 Arithmetic Logic Units . . . 16
4.2.3 Multipliers . . . 17

4.3 Data Formats and Representation 18
4.4 Operand Conversions for Arithmetic-Class Instructions . . 19
4.5 Conditional Arithmetic Instructions 21
4.6 Additive Arithmetic and Logical Operations 21
4.6.1 Additive Arithmetic Operations 22
4.6.2 Logical Operations 23
4.7 Shifter Operations 25
4.7.1 Shifts 25
4.7.2 Leading Bit Detection 26
4.7.3 Normalization 27
4.8 Multiplication 27
4.8.1 Arithmetic of Multiplication 28
4.8.2 Multiplication Instructions 28
4.8.3 Timing, Concurrency, and Restrictions 30
4.8.3.1 Multiplication Guidelines 31
4.8.3.2 Details of Multiplication Restrictions 32
4.9 Miscellaneous Operations 34

5. Data Movement Unit *37*

5.1 Introduction 37
5.2 DMU Architecture 37
5.3 Data Sizes 38
5.4 Data Size Conversions 40
5.4.1 Single-Width Data Transfers 40
5.4.2 Extended Single-Width Data Transfers 41
5.4.3 Double-Width Data Transfers 41
5.5 Rounding and Saturation 42
5.5.1 Rounding 42
5.5.2 Saturation 43
5.6 Data Movement Operations 43

6. Address Generation Units *45*

6.1 Introduction 45

6.2 AGU Architecture 45
6.3 Stack Addressing 46
6.3.1 Allocating and Releasing Stack Space 46
6.3.2 Accessing Data on the Stack 46
6.4 Indirect Addressing 47
6.4.1 Circular Updates 48
6.4.1.1 Circular Buffers (Bit 14=1). 49
6.4.1.2 Bit-Reversed Access (Bit 14=0) 49
6.5 Address Arithmetic Operations 50

7. Program Control Unit 51

7.1 Introduction 51
7.2 PCU Architecture 51
7.3 Execution Pipeline 52
7.4 Status Register 53
7.5 Controlling Program Flow 55
7.5.1 Delay Slots 56
7.5.2 Branching 56
7.5.2.1 Conditional Branches 58
7.5.3 Calling Subroutines 60
7.5.4 Sleep Instructions 61
7.5.5 Program Flow Delay Slot Restrictions. 62
7.6 Hardware Loops 63
7.6.1 Hardware Looping Instructions 63
7.6.2 Hardware Loop Modification Restrictions. 64
7.6.3 Instruction Restrictions in Hardware Loops 64
7.6.4 Hardware Loop Stack Registers 66
7.6.4.1 Using the Loop Registers 69
7.7 Handling Interrupts 71
7.7.1 Interrupt Configuration 71
7.7.2 Invoking Interrupt Handlers 72
7.7.3 RETURNI.2D Delay Slot Restrictions 73
7.8 Restrictions on Concurrent Register Accesses 74
7.8.1 Timing of Concurrent Register Reads and Writes 74

7.8.2 Concurrent Updates to the Status Register 75
7.8.3 Concurrent Updates to Registers Other than SR 76

8. Memory Spaces *79*

8.1 Introduction 79
8.2 Memory Architecture 79
8.3 X Data Memory 79
8.4 Y Data Memory 80
8.5 XY Double Data Memory 80
8.6 P Memory 80

9. Processing Modes *81*

9.1 Introduction 81
9.2 Run Mode 81
9.3 Sleep Mode 81
9.4 Debug Mode 81

10. CFX Instruction Set *83*

10.1 Introduction 83
10.2 Instruction Function Groups 83
10.2.1 Data Movement Instructions 83
10.2.1.1 XY Data Movement Instructions 85
10.2.1.2 P Data Movement Instructions 85
10.2.2 Stack Manipulation Instructions 86
10.2.3 Control Flow and Looping Instructions 87
10.2.4 Pointer Arithmetic Instructions 88
10.2.5 Status Register Manipulation Instructions 89
10.2.6 Logic Instructions 89
10.2.7 Arithmetic Instructions 90
10.2.8 Multiplication Instructions 94
10.2.9 Miscellaneous Instructions 96
10.3 Instruction Execution Groups and Parallelism 96

10.4 Instruction Details 107
10.4.1 Instruction Types 107
10.4.2 Operation Descriptions 108
10.4.3 Instruction Encodings 108
10.4.4 Operand Classes 109
ABS d-reg, d-reg 119
ABS a-reg, ab-reg 120
ADD SP, imm16 121
ADD SP, imm7s 122
ADD xp-reg, SP, imm16 123
ADD xp-reg, SP, imm6n 124
ADD xp-reg, xp-reg, xstep-reg 125
ADD xp-reg, xp-reg, imm16 126
ADD xp-reg, xp-reg, imm3s 127
ADD yp-reg, yp-reg, ystep-reg 128
ADD yp-reg, yp-reg, imm3s 129
ADD d-reg, d-reg, d-reg 130
ADD a-reg, a-reg, ab-reg 131
ADD b-reg, b-reg, ab-reg 133
ADD d-reg, d-reg, imm3s 135
ADD d-reg, d-reg, cond3 136
ADD d-reg, imm3s, cond3 137
ADD.DW d-reg, d-reg, imm3s 138
ADD.DW d-reg, imm3s, cond3 139
ADD.FR d-reg, d-reg, imm3s 140
ADD.NX d-reg, d-reg, d-reg 141
ADD.NX a-reg, a-reg, ab-reg 142
ADD.NX d-reg, d-reg, imm3s 143
ADD.NX.DW d-reg, d-reg, imm3s 144
ADDMUL ab-reg, xy-reg, xy-reg, xy-reg 145
ADDMULADD ab-reg, xy-reg, xy-reg, xy-reg 147
ADDMULNEG ab-reg, xy-reg, xy-reg, xy-reg 149
ADDMULSUB ab-reg, xy-reg, xy-reg, xy-reg 151

ADDSH d-reg, d-reg, d-reg 153
ADDSH a-reg, a-reg, ab-reg 154
ADDSH b-reg, b-reg, ab-reg 156
AND d-reg, d-reg, d-reg 158
AND a-reg, a-reg, ab-reg 159
AND d-reg, d-reg, imm3s 160
AND.DW d-reg, d-reg, imm3s 161
BITCLR SR, imm4u 162
BITCLR d-reg, bit-index 163
BITSET SR, imm4u 164
BITSET d-reg, bit-index 165
BITTGL d-reg, bit-index 166
BITTST d-reg, bit-index 167
BREAKPOINT 168
CALL imm16u 169
CALL xp-reg 170
CALL.1D imm16u 171
CALL.2D xp-reg 172
CLR X[single-mode(X)] 173
CMP d-reg, d-reg 174
CMP a-reg, ab-reg 175
CMP d-reg, imm3s 176
CMP.DW d-reg, imm3s 177
CMPU.NX d-reg, d-reg 178
CMPU.NX a-reg, ab-reg 179
CMPU.NX d-reg, imm3s 180
CMPU.NX.DW d-reg, imm3s 181
DIVST d-reg, d-reg, d-reg 182
ENDLOOP 183
GOTO imm16u 184
GOTO imm16u, cond5 185
GOTO xp-reg 186
GOTO xp-reg, cond5 187

GOTO.0D imm16u 188
GOTO.0D imm16u, cond5 189
GOTO.1D imm16u 190
GOTO.2D imm16u, cond5 191
GOTO.2D xp-reg 192
GOTO.2D xp-reg, cond5 193
INTERRUPT imm5u 194
LOAD reg, X[stack-mode16] 195
LOAD stack-reg6, X[stack-mode6] 196
LOAD stack-reg4, X[stack-mode4] 197
LOAD reg, X[imm16u] 198
LOAD xmem-reg, X[single-mode(X)] 199
LOAD ax-reg, X[dual-mode(X)] 200
LOAD reg, Y[imm16u] 201
LOAD by-reg, Y[single-mode(Y)] 202
LOAD by-reg, Y[dual-mode(Y)] 203
LOAD reg, P[imm16u] 204
LOAD reg, P[single-mode(X)] 205
LOAD reg, P[single-mode(Y)] 206
LOAD reg, imm20s 207
LOAD d-reg, imm24s 208
LOAD d-reg, imm5s 209
LOAD d-reg, imm6s 210
LOAD l-reg, imm24s 211
LOAD xstep-reg, imm5s 212
LOAD ystep-reg, imm5s 213
LOAD ax-reg, imm2s 214
LOAD by-reg, imm2s 215
LOAD.DW ab-reg, XY[imm16u] 216
LOAD.DW ab-reg, XY[single-mode(X)] 217
LOAD.DW ab-reg, imm24s 218
LOAD.DW ab-reg, imm5s 219
LOAD.DW d-reg, imm6s 220

LOAD.EX ab-reg, P[imm16u] 221
LOAD.EX ab-reg, P[single-mode(X)] 222
LOAD.EX ab-reg, P[single-mode(Y)] 223
LOG2ABS d-reg, d-reg 224
LOOP imm12u, imm16u 225
LOOP d-reg, imm16u 226
LOOP.2D imm12u, imm16u 227
LOOP.2D d-reg, imm16u 228
MAX d-reg, d-reg, d-reg 229
MAX a-reg, a-reg, ab-reg 230
MIN d-reg, d-reg, d-reg 231
MIN a-reg, a-reg, ab-reg 232
MOVE reg, reg 233
MOVE ab-reg, x-reg 234
MOVE xy-reg, a-reg 235
MOVE ab-reg, y-reg 236
MOVE xy-reg, b-reg 237
MOVE d-reg, d-reg 238
MOVE d-reg, d-reg, cond3 239
MOVE.DW ab-reg, ab-reg 240
MOVE.EX ab-reg, ab-reg 241
MOVE.NX d-reg, d-reg 242
MOVE.NX a-reg, ab-reg 243
MUL ab-reg, xy-reg, xy-reg 244
MUL a-reg, x-reg, xy-reg 245
MUL b-reg, y-reg, xy-reg 247
MUL.SU ab-reg, xy-reg, xy-reg 249
MUL.UU ab-reg, xy-reg, xy-reg 250
MULADD ab-reg, xy-reg, xy-reg 251
MULADD a-reg, x-reg, xy-reg 252
MULADD b-reg, y-reg, xy-reg 254
MULADD.SU ab-reg, xy-reg, xy-reg 256
MULADD.UU ab-reg, xy-reg, xy-reg 257

MULNEG ab-reg, xy-reg, xy-reg 258
MULNEG a-reg, x-reg, xy-reg 259
MULNEG b-reg, y-reg, xy-reg 261
MULNEG.SU ab-reg, xy-reg, xy-reg 263
MULNEG.UU ab-reg, xy-reg, xy-reg 264
MULSUB ab-reg, xy-reg, xy-reg 265
MULSUB a-reg, x-reg, xy-reg 266
MULSUB b-reg, y-reg, xy-reg 268
MULSUB.SU ab-reg, xy-reg, xy-reg 270
MULSUB.UU ab-reg, xy-reg, xy-reg 271
NEG d-reg, d-reg 272
NEG a-reg, ab-reg 273
NEG d-reg, d-reg, cond3 274
NLOG2ABS d-reg, d-reg 275
NLOG2ABS a-reg, ab-reg 276
NOP 277
NOP 278
NOP 279
NOP 280
NOT d-reg, d-reg 281
OR d-reg, d-reg, d-reg 282
OR a-reg, a-reg, ab-reg 283
OR d-reg, d-reg, imm3s 284
OR.DW d-reg, d-reg, imm3s 285
RETURN 286
RETURN.2D 287
RETURNI 288
RETURNI.0D 289
RETURNI.2D 290
SHLL d-reg, d-reg, d-reg 291
SHLL d-reg, d-reg, imm3u 292
SHLL d-reg, imm6u 293
SHLL d-reg, d-reg, cond3 294

SHLL d-reg, imm3u, cond3 295
SHLL.NX d-reg, d-reg, d-reg 296
SHLL.NX d-reg, d-reg, imm3u 297
SHLL.NX d-reg, imm6u 298
SHRA d-reg, d-reg, d-reg 299
SHRA d-reg, d-reg, imm3u 300
SHRA d-reg, imm6u 301
SHRA d-reg, d-reg, cond3 302
SHRA d-reg, imm3u, cond3 303
SHRL.NX d-reg, d-reg, d-reg 304
SHRL.NX d-reg, d-reg, imm3u 305
SHRL.NX d-reg, imm6u 306
SLEEP 307
SLEEP imm16u 308
STORE X[stack-mode16], reg 309
STORE X[stack-mode6], stack-reg6 310
STORE X[stack-mode4], stack-reg4 311
STORE X[imm16u], reg 312
STORE X[single-mode(X)], xmem-reg 313
STORE X[dual-mode(X)], a-reg 314
STORE Y[imm16u], reg 315
STORE Y[single-mode(Y)], by-reg 316
STORE Y[dual-mode(Y)], b-reg 317
STORE P[imm16u], reg 318
STORE P[single-mode(X)], reg 319
STORE P[single-mode(Y)], reg 320
STORE X[imm16u], imm8s 321
STORE Y[imm16u], imm8s 322
STORE P[imm16u], imm8s 323
STORE.DW XY[imm16u], ab-reg 324
STORE.DW XY[single-mode(X)], ab-reg 325
STORE.DW XY[imm16u], imm8s 326
STORE.EX P[imm16u], ab-reg 327

STORE.EX P[single-mode(X)], ab-reg 328
STORE.EX P[single-mode(Y)], ab-reg 329
SUB SP, imm16 330
SUB SP, imm7sn 331
SUB xp-reg, SP, imm16 332
SUB xp-reg, SP, imm6p 333
SUB xp-reg, xp-reg, imm16 334
SUB xp-reg, xp-reg, imm3sn 335
SUB yp-reg, yp-reg, imm3sn 336
SUB d-reg, d-reg, d-reg 337
SUB a-reg, a-reg, ab-reg 338
SUB b-reg, b-reg, ab-reg 340
SUB d-reg, d-reg, imm3sn 342
SUB d-reg, d-reg, cond3 343
SUB d-reg, imm3sn, cond3 344
SUB.DW d-reg, d-reg, imm3sn 345
SUB.DW d-reg, imm3sn, cond3 346
SUB.FR d-reg, d-reg, imm3s 347
SUB.NX d-reg, d-reg, d-reg 349
SUB.NX a-reg, a-reg, ab-reg 350
SUB.NX d-reg, d-reg, imm3sn 351
SUB.NX.DW d-reg, d-reg, imm3sn 352
SUBMUL ab-reg, xy-reg, xy-reg, xy-reg 353
SUBMULADD ab-reg, xy-reg, xy-reg, xy-reg 355
SUBMULNEG ab-reg, xy-reg, xy-reg, xy-reg 357
SUBMULSUB ab-reg, xy-reg, xy-reg, xy-reg 359
SUBSH d-reg, d-reg, d-reg 361
SUBSH a-reg, a-reg, ab-reg 362
SUBSH b-reg, b-reg, ab-reg 364
SUBSTEP xp-reg 366
SUBSTEP yp-reg 367
SWAP d-reg, d-reg, cond3 368
XOR d-reg, d-reg, d-reg 369

XOR d-reg, d-reg, imm3s 370
XOR.DW d-reg, d-reg, imm3s 371

A. Instruction Set Summary *373*

A.1 Introduction 373
A.2 Explanation of Columns in the Summary Table 373
A.3 Summary of Instruction Types 375
A.4 Alphabetical Instruction Summary Table 377

B. Opcode Map *395*

B.1 Long Instructions 396
B.2 Move Instructions 398
B.3 Arithmetic Instructions 400

C. Glossary *407*

1 Introduction

1.1 Purpose

This manual provides an architecture description and instruction set reference for engineers using the CFX Digital Signal Processor (DSP). The CFX DSP a processing core that is designed to work with the HEAR Configurable Accelerator. The CFX DSP is used to configure the system and coordinate the flow of signal data progressing through the system. This DSP can also be programmed with unique signal processing applications, while the other core, the HEAR Configurable Accelerator, is designed primarily for performing routine signal processing algorithm structures. The HEAR is a microcode configurable signal processing engine that reduce the CFX DSP load. For more information about the HEAR, see the *HEAR Configurable Accelerator Reference Manual*.

You can use the information in this manual to develop complex algorithms to process, filter and enhance digital signals in the most efficient and compact way. If the CFX DSP is new to you, you can learn about the architecture in Chapters 2 to 9. If you are familiar with the CFX DSP, you can go directly to the instruction-set reference in Chapter 10, "CFX Instruction Set" on page 83.

1.2 Intended Audience

This manual is for software development professionals or signal processing engineers with experience developing applications in assembly language. We assume you are somewhat familiar with digital signal processing, embedded software development, or both.

1.3 Conventions

The following conventions are used in this manual:

- Sample code and instructions that are embedded in text appear in a `monospace font`.
- Instruction types appear in an *italic typeface*.
- Instruction extensions (if they are not part of an example instruction) appear in a **`bold monospace font`**.
- imm*N*s represents a signed immediate operand where *N* is the size in bits.
- imm*N*u represents an unsigned immediate operand where *N* is the size in bits.
- imm*N*n represents a negative immediate operand where *N* is the size in bits.
- imm*N*p represents a positive immediate operand where *N* is the size in bits.
- imm*N*sn represents a negated signed immediate operand where *N* is the size in bits.
- imm*N* represents a value where either a signed or unsigned value is acceptable and *N* is the size in bits.
- In general, numbers are presented in decimal notation. In cases where hexadecimal notation is more convenient, these numbers are identified by the prefix "0x". For example, the decimal number 123456 can also be represented as 0x1E240.

1.4 Manual Organization

The *CFX DSP Architecture Manual* contains the following chapters and appendices:

- **Chapter 1: Introduction**, describes the purpose of the manual, the intended audience, outlines how the book is organized, and lists related documents.
- **Chapter 2: CFX Architecture Overview**, identifies the main components of the CFX DSP and briefly describes each one.
- **Chapter 3: Instruction Types**, explains the different groups of instructions in the CFX assembly language and describes which operations comprise instructions in each group.
- **Chapter 4: Data Computation Unit**, describes the architecture of the DCU, which performs arithmetic and logical operations, shifting, and multiplication.
- **Chapter 5: Data Movement Unit**, presents the architecture of the DMU, including data size conversions, rounding and saturation, and the routing of data both within the CFX and to memory.
- **Chapter 6: Address Generation Units**, explains how the AGUs work, and describes the addressing registers and data addressing.
- **Chapter 7: Program Control Unit**, explains how the PCU manages the execution pipeline and program control flow, including branching, subroutine calls, interrupts, and hardware-assisted looping. The status register, instruction timing, and restrictions on instruction usage are also detailed.
- **Chapter 8: Memory Spaces**, describes the different areas of memory that can be accessed by the CFX DSP.
- **Chapter 9: Processing Modes**, describes the states that are possible for processing CFX instructions.
- **Chapter 10: CFX Instruction Set**, describes the syntax, use and effect of each instruction and operation.
- **Appendix A: Instruction Set Summary**, provides all the CFX instructions and operations in a table for quick lookup.
- **Appendix B: Opcode Map**, shows opcodes for all the CFX instructions and operations.
- **Appendix C: Glossary**, defines the specialized abbreviations and terms used in this manual.

1.5 FURTHER READING

For more information, refer to the following documents:

- *Integrated Development Environment User's Guide*
- *CFX Toolchain Reference*
- *HEAR Configurable Accelerator Reference Manual*
- *Firmware Reference Manual*
- *Hardware Reference Manual*
- *Introduction to Programming*
- Hennessy, John L., and Patterson, David A. *Computer Architecture: A Quantitative Approach*. 3rd ed. (Morgan Kaufmann Publishers)

2 CFX Architecture Overview

The CFX is a highly flexible DSP core combining a dual-Harvard, dual-MAC DSP architecture with a parallel instruction set for simultaneous control of multiple computation units. The essential components of the CFX are shown in Figure 1 and described below.

The CTRL arrows from the PCU in the figure represent control signals from the PCU to all other computational units in the CFX.

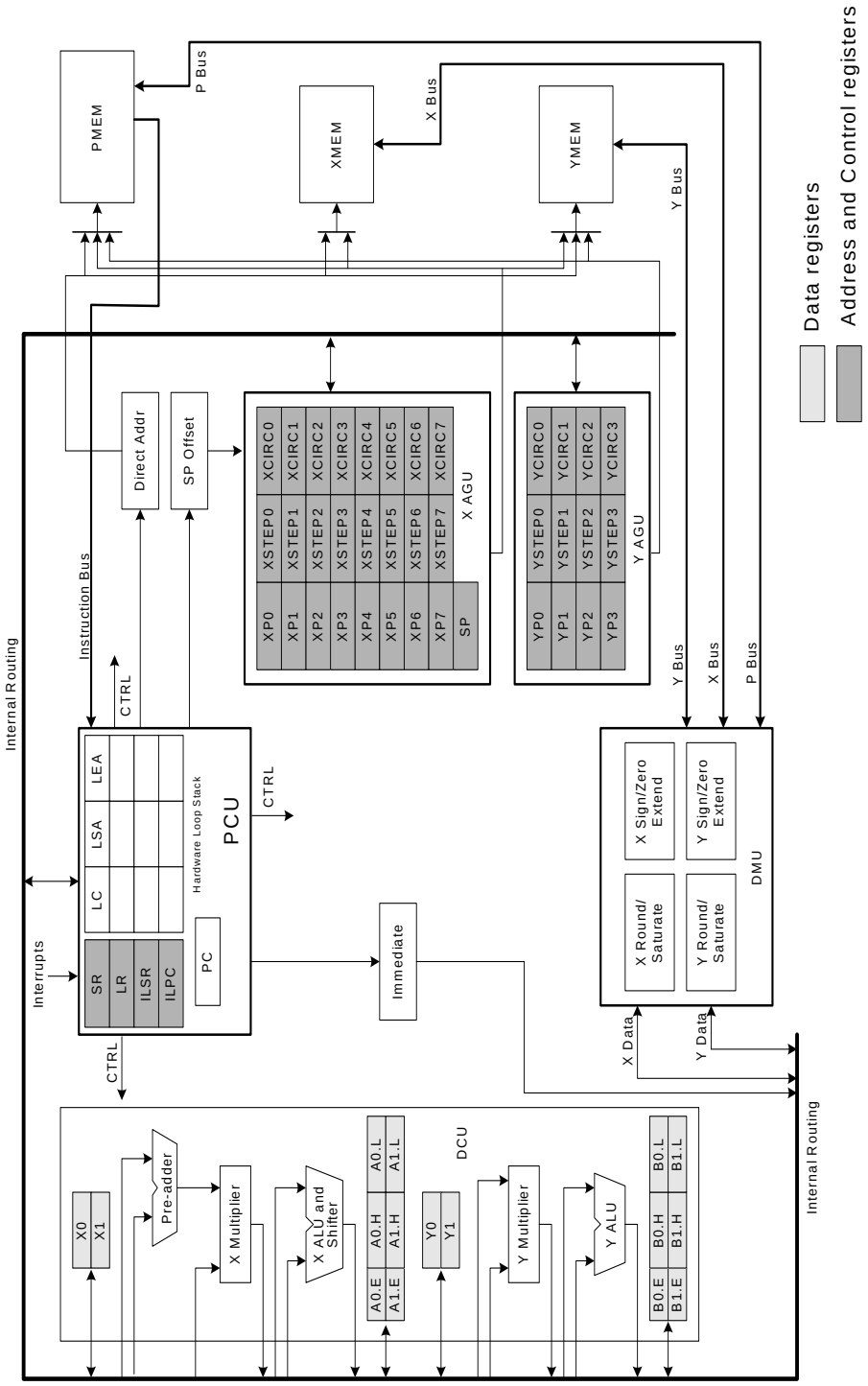


Figure 1: CFX Component Overview

2.1 TYPES OF OPERATIONS AND PARALLEL EXECUTION

Three kinds of operations are executed by the CFX:

- Computations on data
- Addressing and movement of data
- Changing the flow of program control

The DSP contains execution units dedicated to each kind of operation, and multiple operations can be given in a single instruction. Extensive instruction-level parallelism is thereby available, in a very compact format. The DSP can execute up to four computation operations in parallel with two data transfers (including rounding and/or saturation as well as complex address updates), while simultaneously changing control flow. For more information, see Chapter 3, "Instruction Types" on page 9.

2.2 EXECUTION UNITS

Five execution units operate in parallel to process CFX instructions: the Data Computation Unit (DCU), the Data Movement Unit (DMU), two Address Generation Units (AGUs), and the Program Control Unit (PCU). The DSP's computational power is further increased by parallelism within these blocks.

DCU The DCU contains two 56-bit Arithmetic Logic Units (ALUs) and two 24-bit x 24-bit multipliers. The ALUs and multipliers are paired to support parallel data flow: the X multiplier result can be used by the X ALU, and the Y multiplier result can be used by the Y ALU. The X multiplier has a pre-add/subtract unit and supports combinations of signed and unsigned data types. The X ALU has a 56-bit barrel shifter for shifting and normalizing. Combined, these perform arithmetic, logical, multiplication and multiply-accumulate (MAC) operations. The ALUs support multiple data types and have some "high-level" instructions like division-step, add/subtract with shift, and maximum/minimum. For more information, see Chapter 4, "Data Computation Unit" on page 15.

DMU The DMU handles loading data from memory, storing data to memory, and moving data between data registers. Two round/saturate units and two sign/zero-extend units support parallel data type conversion. Separate X, Y and P memory buses support parallel data movement operations. For more information, see Chapter 5, "Data Movement Unit" on page 37.

X AGU and Y AGU The AGUs provide addresses for indirect memory accesses, generating up to two memory addresses and updating up to two address registers simultaneously. The X AGU generates addresses for accessing P, X and Y memory. The Y AGU generates addresses for accessing P and Y memory. Both AGUs support:

- Post-increment by 1, 2, or step
- Post-decrement by 1, 2, or step
- Circular addressing
- Bit-reversed address updates

The X AGU also manages a stack in X memory. For more information about the AGUs, see Chapter 6, “Address Generation Units” on page 45.

PCU

The PCU fetches, decodes, and executes instructions in a 3-stage pipeline. Parallel instructions are executed in a single cycle, and operations changing the flow of program control are supported with delay slots to reduce or eliminate pipeline stalls. The PCU supports four levels of nesting for hardware-assisted, zero-overhead loops. For more information, see Chapter 7, “Program Control Unit” on page 51.

2.3 REGISTERS

Four types of registers store different kinds of information.

Data Registers

Four 56-bit accumulators (A0, A1, B0 and B1) and four 24-bit data registers (X0, X1, Y0 and Y1) store inputs and outputs for arithmetic, logic and multiplication operations. These registers are separated into two A and two B accumulators and two X and two Y data registers to allow the X or Y multipliers and ALUs easier access to them. The accumulators are also accessible as two 24-bit data registers and one 8-bit data register. For more information, see Section 4.2.1, “Data Registers” on page 15.

Address Registers

The X and Y AGUs contains eight and four pointer registers, respectively (XP*, YP*). There is one stack pointer (SP), used to access a stack in X memory. For more information, see Section 6.2, “AGU Architecture” on page 45 and Section 6.3, “Stack Addressing” on page 46.

Addressing Configuration Registers

Each pointer register has associated step (XSTEP* and YSTEP*) and circular configuration (XCIRC* and YCIRC*) registers that control address updates in the AGUs. For more information, see Section 6.4, “Indirect Addressing” on page 47.

Program Control Registers

The PCU state includes a program counter (PC) and a status register (SR). Subroutine calls and returns are managed through a link register (LR). The interrupt-link program counter (ILPC) and interrupt-link status register (ILSR) store the state during interrupts. The hardware loop state is stored in an internal loop stack, with the top entry visible through the loop count (LC), loop start address (LSA), and loop end address (LEA) registers. The loop stack can be unloaded and reloaded in software using these registers. For more information on the program control registers, see Section 7.2, “PCU Architecture” on page 51.

2.4 MEMORY SPACES

The CFX memory is split into X memory (XMEM) and Y memory (YMEM) for data, and program memory (PMEM) for program instructions as well as data. All memory spaces use 16-bit addresses, and support a 64 Kword addressing range.

XMEM and YMEM are 24 bits wide. Some instructions support accessing both memories as a logical 48-bit memory called XY memory (XYMEM).

PMEM is 32 bits wide, but can be accessed as 24-bit or 32-bit data.

For more information about memory spaces, see Chapter 8, “Memory Spaces” on page 79.

2.5 BUSES

The CFX has independent data and address buses to provide simultaneous access to all memory spaces for both instructions and data. This supports data-intensive signal processing operations such as multiply-accumulate and filtering.

The X Bus, Y Bus and Instruction Bus can be used simultaneously. The Instruction Bus is only used for program memory access by the PCU. The X and Y Buses can be used simultaneously in two ways: via an XY data access to load a 48-bit double-width value, or using a dual data movement instruction to perform separate X and Y memory accesses.

The P Bus is used to access the program memory space as data. The P and Instruction Buses can be used simultaneously unless they are both being used to access the same bank of CFX program memory. The P Bus cannot be used simultaneously with the X Bus or Y Bus.

This page left blank for printing purposes.

3 Instruction Types

3.1 INTRODUCTION

The CFX program memory is composed of 32-bit instruction words. Each instruction word is self-contained: the PCU fetches and decodes one instruction word at a time, and there are no two-word instructions. However, instruction words can contain multiple independent sub-instruction fields that are dedicated to the various execution units, allowing programmers to specify parallel operations compactly.

Instructions are therefore categorized into several types, as determined by their encoding format in an instruction word and the execution unit to which the instruction is dedicated. This chapter describes the available instruction types, the composition of instructions into program memory words, and the general rules about instruction timing. More details about instructions, organized according to the execution unit that handles them, are available in Chapter 4, "Data Computation Unit" on page 15, Chapter 5, "Data Movement Unit" on page 37, and Chapter 7, "Program Control Unit" on page 51.

The three main instruction categories are those that occupy a full instruction word in memory. They are named according to the kinds of instructions they comprise, and are described in the following sections.

3.2 LONG INSTRUCTIONS

Long instructions have the simplest structure: the whole instruction word contains just a single instruction. These full-word instructions generally have large immediate fields or are used infrequently. *Long* instructions perform the following types of operations:

Control Flow

All instructions which affect the flow of program control are *Long* instructions.

Many of these require the whole 32-bit instruction word because they contain a 16-bit target address in program memory, as well as other fields. The following instructions contain a target address: `LOOP`, direct `GOTO` and `CALL`, and `SLEEP` with wakeup address.

Control flow instructions that do not have a target address as an immediate field include indirect `GOTO` and `CALL`, `RETURN` (from subroutine), `RETURNI` (from interrupt), `ENDLOOP`, and `INTERRUPT` (software interrupt).

Memory Load and Store with Full-Sized Address/Offset

All load and store instructions that contain the direct memory address as a 16-bit immediate field are *Long* instructions. Similarly, load and store operations to/from the stack with a full-sized (16-bit) stack pointer offset must be *Long* to allow space for the stack offset field.

Load and Store Indirect to P Memory

Using P memory as a data space is a rare operation, so most forms of indirect load from and store to P memory—with non-extended (24-bit) registers or via X pointers—are *Long* instructions.

Load Full-Sized Immediate

More compact instructions exist for working with common, short immediate values, but when a register (which is typically 16 bits for a pointer or 24 bits for a data register) must be loaded with a larger value, the *Long* instructions for loading immediates can be used to load all of the bits of any register.

Pointer Arithmetic with Full-Sized Immediate

The *Long* class contains three forms of `ADD` that allow a full-sized 16-bit immediate value to be added to an X pointer or the stack pointer `SP`, and stored back into an X pointer or `SP`.

System Control

Basic system control instructions are in the *Long* class because they are executed infrequently or they affect the PCU/pipeline state. This includes the `BITSET` and `BITCLR` instructions that are used to control system mode bits in the status register `SR` (commonly used to globally enable and disable system interrupts), as well as the `SLEEP` and `BREAKPOINT` instructions that affect the system's processing mode (see Chapter 9, "Processing Modes" on page 81) and `NOP`.

Since *Long* instruction words contain just a single instruction, these instructions have no special syntax; the single instruction appears on its own, as the whole instruction line. The following are examples of *Long* instructions containing each of the six categories of operation described above:

```
goto process_it, not_zero
load x0, x[filter_coeff]     // load filter coefficient from XMEM
store.ex p[xp0++], a0        // store a0.e|a0.h to table in PMEM
load b1.l, 0x123456
add xp5, sp, -500            // point xp5 far back in stack
bitset sr, SR_IE_POS         // enable interrupts
```

3.3 ARITHMETIC-MOVE INSTRUCTIONS

As the name implies, instructions in the *Arithmetic-Move* category contain two parts: an arithmetic part that is executed by the DCU and a data movement part that is executed by the DMU and AGUs. Syntactically, the two parts are separated by "|||"—indicating that they are executed in parallel—so an abstract *Arithmetic-Move* instruction is written like this:

```
<arithmetic part>  |||  <move part>
```

The following sections examine first the arithmetic part, and then the data movement part of the *Arithmetic-Move* instructions.

3.3.1 Arithmetic Instructions

Arithmetic instructions are executed in the DCU, performing arithmetic and logical computations with the DCU's data registers and immediate values. Arithmetic instructions are classified as either *Single* or *Dual*: *Single Arithmetic* instructions carry out a single operation using the X side of

the DCU; *Dual Arithmetic* instructions carry out two operations using both the X and Y sides of the DCU.

Single Arithmetic instructions can make full use of the X multiplier and X ALU functionality:

- The full range of arithmetic, logical, and shift operations are available.
- Operands are very flexible: destinations can be chosen from any of the eight data registers, and source operands can typically be any data register or one of several short-immediate forms.
- Several data types are supported: there are instruction forms for double-width, no-extension, and signed/unsigned operations.
- Conditional instructions are available to perform operations on data registers if a specified condition holds, without the overhead of control-flow changes.
- The pre-addition or pre-subtraction functionality of the X multiplier is available.
- A *Single Arithmetic* `NOP` instruction is available so that an *Arithmetic-Move* instruction can contain only a *Move* operation.

The following shows some examples of *Single Arithmetic* instructions used in *Arithmetic-Move* instructions, with the data movement part of the instruction left abstract for the moment.

```
add a0, b0, x0             |||  <move part>
shll a1, 32                |||  <move part>
max a0, b0, b1             |||  <move part>
add.dw b1, -1, not_zero    |||  <move part>
bittst x1, 11              |||  <move part>
swap a0, b0, carry         |||  <move part>
muladd.uu b0, x1, y1       |||  <move part>
submulsub b0, x0, y0, y1   |||  <move part>
```

The arithmetic part of an *Arithmetic-Move* instruction can also contain two independent instructions for the X and Y sides of the DCU. The two instructions form a *Dual Arithmetic* instruction. There are fewer instructions available—only the most common instruction forms—because there are fewer bits for encoding operands and because the Y multiplier and ALU have more limited functionality.

The X part of the *Dual Arithmetic* instruction is referred to as the *X Arithmetic* instruction. It uses the X multiplier and ALU, while the Y part—the *Y Arithmetic* instruction—uses the Y multiplier and ALU. In the assembler syntax, the two parts are separated by "`!!`"—indicating parallel execution of the two instructions—so an abstract *Arithmetic-Move* instruction containing *Dual Arithmetic* instructions looks like this:

```
<X arith op> !! <Y arith op>  |||  <move part>
```

Some examples are as follows. Note that the *X Arithmetic* instructions have an `A` accumulator for their destination, while *Y Arithmetic* instructions' destinations are `B` accumulators. The choices for source operands are limited, as are the operations that can be used. Multiplications in *Dual Arithmetic* instructions are always signed/signed, with no pre-addition or pre-subtraction. There is also an *X Arithmetic* `NOP` instruction that allows you to execute a Y multiplication without an X side operation.

```
and a0, a0, b0    !! sub b1, b1, a1     |||  <move part>
sub a1, a0, a1    !! muladd b0, y0, y1  |||  <move part>
mulsub a0, x1, y1 !! add b0, b0, a0     |||  <move part>
muladd a1, x0, x1 !! mul b1, y0, y1     |||  <move part>
```

3.3.2 Move Instructions

Move instructions use the DMU, the AGUs, and the memory buses. They move data between the registers and memory banks, and perform arithmetic on addressing registers.

If the instruction moves just one datum or performs addressing register arithmetic, it is called a *Single Move* instruction. This instruction type includes memory accesses to load and store register values indirectly (via an address stored in a pointer register) to all of the memory spaces, as well as register-to-register transfers, short forms for loading registers with small immediate values, and arithmetic operations between pointer registers, step registers, and short immediate values.

The load and store instructions include transfers between the accumulators and the double-width XY memory space where the least significant 24 bits are in X memory and the most significant 24 bits are in Y memory. Both X and Y memories are accessed, but there is just one double-width datum (there is one accumulator, and one memory address for both memories), so XY memory accesses are *Single Move* instructions.

A *Single Move* `NOP` instruction is also available so that an *Arithmetic-Move* instruction can contain only an *Arithmetic* operation. You can also use the *Single Move* `NOP` to skip the first, second, or both cycles of a *Move-Move* instruction to consume delay slots or allow multiplication operations to complete.

Here are some examples of *Single Move* instructions used in *Arithmetic-Move* instructions, with the arithmetic part of the instruction left abstract:

```
<arith part>  |||  load  y0, x[xp1++]          // From XMEM
<arith part>  |||  store y[yp2 -= ystep2], b1  // To YMEM
<arith part>  |||  load  yp1, x[sp - 6]        // From stack frame
<arith part>  |||  store.ex p[yp1 += 2], a0    // High 32 bits to PMEM
<arith part>  |||  load.dw  a0, xy[xp0]        // Double-width from XYMEM
<arith part>  |||  move xp6, b0.h
<arith part>  |||  load xstep5, -10
<arith part>  |||  add  sp, 20                 // Allocate stack frame
<arith part>  |||  add  xp7, sp, -12           // Point xp7 into stack frame
<arith part>  |||  add  yp3, yp3, 2            // Increment yp3 by 2
```

There are also *Dual Move* instructions which contain two independent data movement instructions. As with the *Dual Arithmetic* instructions, only the most common functions are available: memory transfers with common addressing modes and some pointer/register limitations, register transfers, and immediate loads. There is no *Dual Move* `NOP` instruction; each *Dual Move* instruction has a corresponding *Single Move* instruction.

The two parts of the *Dual Move* instruction are called *X Move* and *Y Move* instructions, and as with *Dual Arithmetic* instructions, they are executed in parallel as indicated by the "`!!`" separator in the assembly language syntax. Here are some examples:

```
<arith part>  |||  load x0, x[xp1++] !! load y0, y[yp2++]
<arith part>  |||  load x1, x[xp2]   !! store y[yp0++], b1
<arith part>  |||  store x[xp3], a0  !! move x1, b0
<arith part>  |||  load a0, -1       !! load y0, y[yp3]
```

3.4 MOVE-MOVE INSTRUCTIONS

Instruction words in the *Move-Move* category contain two *Move* instructions with the full flexibility described in Section 3.3.2, "Move Instructions" on page 12. They are effective for writing compact code for operations that require extensive data movement, such as setting up the

parameters for a function call. The two *Move* instructions (each of which can be *Single* or *Dual*) are executed serially, as indicated by their assembly-language separator "`>>>`", so an abstract *Move-Move* instruction is written like this:

```
<first move part>  >>>  <second move part>
```

Move-Move instructions provide only a space optimization. The above abstract *Move-Move* instruction is equivalent to these two *Arithmetic-Move* instructions:

```
nop  |||  <first move part>
nop  |||  <second move part>
```

These two instructions perform the same function as the *Move-Move* instruction, in the same amount of time: two processor cycles. However, the *Move-Move* instruction occupies only a single instruction word in program memory.

Examples of *Move-Move* instructions can be created by combining any pair of *Single Move* or *Dual Move* instructions:

```
store.ex p[yp1 += 2], a0                 >>>  load b1, y[yp2 -= ystep2]
load x0, x[xp1++] !! load y0, y[yp2++]   >>>  store.dw xy[xp0], a0
load x1, x[xp2]   !! store y[yp0++], b1  >>>  store x[xp3], a0 !! move x1, b0
```

This page left blank for printing purposes.

4 Data Computation Unit

4.1 INTRODUCTION

This chapter describes the Data Computation Unit (DCU) which executes the *Arithmetic* part of *Arithmetic-Move* instructions. These instructions include additive operations, comparisons, multiplication/accumulation, logical operations, single-bit operations, shifting, normalization, and some "high level" instructions like division-step, add/subtract-with-shift, and maximum/minimum. The data registers and formats are also described.

4.2 DCU ARCHITECTURE

The main computational blocks in the DCU are:

- The X multiplier, X MUL, which has a pre-adder on one of its inputs
- The X arithmetic logic unit (ALU) and barrel shifter
- The Y multiplier, Y MUL
- The Y ALU

Additionally, the DCU contains eight *data registers*, which are divided into four pairs: two pairs of general-purpose data registers (`X0`, `X1`, `Y0` and `Y1`), which also act as inputs for the multipliers, and two pairs of accumulators (`A0`, `A1`, `B0` and `B1`), each of which comprises extension, high, and low subregisters.

Figure 1 on page 4 shows the inputs and outputs of the DCU's execution units connected to general internal routing. However, the actual connections supported are restricted in two ways:

1. The inputs and outputs of all DCU execution units are restricted to the data registers, immediate values, or the inputs/output of other DCU execution units.
2. The DCU interconnection architecture and instruction set further restrict options for each execution unit. Specific restrictions for each DCU execution unit are given in the following sections, and are detailed in Section 10.4, "Instruction Details" on page 107.

4.2.1 Data Registers

The DCU contains eight registers which can be used as source and destination registers for *Arithmetic* instructions computed by the DCU's execution units. In fact, these registers provide *all* destinations and sources other than immediate values; instructions executed by the DCU cannot directly use values stored in memory or in other CFX registers. Such operands must be moved into the DCU registers if *Arithmetic* instructions are to use them.

Note: There are *Long* and *Move* instructions for performing address arithmetic. The AGUs handle these instructions, not the DCU.

The DCU registers are arranged in pairs as *register files*. Within each pair, the two registers are equivalent; the instruction set is specified in terms of register files, so wherever one of the

registers in a pair can be used, the other is also valid. For example, if X0 is valid, X1 is equally valid.

There are two pairs of accumulators and two pairs of general-purpose data registers, as described in the following sections.

4.2.1.1 Accumulators

The A accumulator register file contains accumulators A0 and A1, while the B accumulator register file contains B0 and B1. Each of these four accumulators is 56 bits in width.

The accumulators can be used as source and destination registers for all Arithmetic-class instructions with the single exception that they cannot be used as source operands to multiply and multiply-accumulate (MAC) instructions. In some cases, an operand *must* be an accumulator:

- As the destination operand in multiply and MAC instructions
- All operands for Dual Arithmetic instructions other than multiply and MAC instructions

In *Arithmetic* instructions in the DCU, the use of an accumulator named as above generally means that all 56 bits are used or stored by the instruction. However, parts of the accumulators can be specified for certain operations, as follows:

- Each accumulator has three subregisters, shown in Table 1.

TABLE 1: ACCUMULATOR SUBREGISTERS

Accumulator Subregister	Notation (e.g. for A0)	Bits
extension	A0.E	48 to 55
high	A0.H	24 to 47
low	A0.L	0 to 23

- Arithmetic-class instructions with the **.NX** (no extension) suffix ignore the extensions of accumulator operands. These instructions use the accumulators as 48-bit registers containing 24-bit high and low halves.
- While Arithmetic-class instructions naturally use 56 bits of precision, data-movement instructions in the *Long* and *Move* classes transfer either 24, 32 or 48 bits. The interpretation of an accumulator as an operand for these instructions is therefore different from the above rules, and is described in Section 5.4, “Data Size Conversions” on page 40.

4.2.1.2 General-Purpose XY Data Registers

The X general-purpose data register file contains the X0 and X1 registers and the Y register file contains Y0 and Y1. Each of these four registers is 24 bits in width.

The XY registers can be used as source and destination registers for most Arithmetic-class instructions. They *must* be used as the source operands to multiply and MAC instructions.

4.2.2 Arithmetic Logic Units

The DCU has two arithmetic logic units (ALUs), termed the *X ALU* and *Y ALU*. Each accepts two 56-bit inputs and produces a 56-bit result. The X ALU is used for *Single Arithmetic* instructions and the *X Arithmetic* half of *Dual Arithmetic* instructions, while the Y ALU is used only for the *Y*

Arithmetic half of *Dual Arithmetic* instructions. All ALU instructions execute in a single clock cycle.

The X ALU is used for most data computations. In addition to typical ALU functionality it contains a 56-bit barrel shifter and a leading-bit detector, and it supports several data formats. It is used to perform the following operations:

Additive arithmetic
Addition and subtraction with optional division by two, comparison, accumulation of products, negation

Logical operations
AND, OR, Exclusive OR

Shifting
Arithmetic and logical shifts with immediate or signed register shift counts, detection of leading bit for exponent computation

Single-bit operations
Bit test, bit set, bit clear, bit toggle

Conditional operations
Conditional add, subtract, negate, shift, move, swap

Other operations
Absolute value, maximum, minimum, division step

Most of the X ALU's binary operations support a general three-operand form with both sources and the destination chosen from any of the data registers. A small immediate value can be used instead of the second source register. Operand flexibility is reduced for the following general instruction forms to allow the instructions to be encoded in the available opcode space:

- Conditional instructions
- Larger immediate values, especially for shift counts
- *Dual Arithmetic* instructions

The functionality of the Y ALU is much more limited, as it is designed primarily to be used for MAC instructions and is only used in the *Y Arithmetic* half of *Dual Arithmetic* instructions. It supports addition and subtraction with optional division by two, and multiply and MAC instructions use it for negation and accumulation. Its destination and one source operand must be B accumulators, while the other source operand is an A or B accumulator, or the output from the Y multiplier.

4.2.3 Multipliers

The DCU has two multipliers called the *X multiplier* (X MUL) and *Y multiplier* (Y MUL). Each accepts two 24-bit fractional inputs and produces a full 48-bit fractional product. X MUL is used for *Single Arithmetic* instructions and the *X Arithmetic* half of *Dual Arithmetic* instructions, while Y MUL is used only for the *Y Arithmetic* half of *Dual Arithmetic* instructions.

Both multipliers produce their product in a single clock cycle. However, multiplication is performed as the first cycle of a two-cycle multiply or MAC instruction; the second cycle performs an ALU operation between the product and an accumulator, in the corresponding ALU. During this

second cycle (using the ALU), another multiply or MAC instruction can be initiated, using the multiplier in a pipelined fashion. Thus each multiplier can produce a product in every cycle.

X MUL is very flexible when used in a *Single Arithmetic* instruction. The source registers are any of the XY data registers, and the destination is any AB accumulator. One or both operands can be treated as unsigned fractional numbers instead of the default signed/signed multiplication. X MUL's pre-adder can be used in a three-input form where two inputs are added or subtracted prior to multiplication.

When used in *Dual Arithmetic* instructions, both the X and Y multipliers have similar, more limited functionality:

- Operand register flexibility is reduced: one source must be from the same side as the multiplier (so an X data register for X MUL or a Y register for Y MUL), and the destination must be on the same side (i.e., A accumulators for X MUL or B accumulators for Y MUL).
- X MUL's pre-adder is not available.
- All source operands are treated as signed fractional data.

Multiplier products are typically used in multiply-accumulate (MAC) operations: the product is either added to or subtracted from an accumulator before being stored back into the accumulator. It is also possible to use zero for the ALU input instead of the source accumulator, which allows a multiply-only operation to store the product or negated product in the destination accumulator.

4.3 DATA FORMATS AND REPRESENTATION

The CFX supports the following data types:

Signed Fractional

In the "usual" format: two's complement with the high-order bit as the sign bit, and a fixed-point binary point after the sign bit. This data type has a range of -1.0 to just less than +1.0.

Unsigned Fractional

With no sign, the high-order bit becomes the most significant fraction bit. The **`.NX`** (no extension) instructions support the use of accumulators for storing unsigned fractional numbers because they ensure that the extension part of the destination accumulator is the same as the high bit of the 48-bit unsigned fractional result, so saturation does not change the value. The `CMPU.NX` instructions provide unsigned comparisons.

Extended Signed Fixed-Point

In the accumulators, the extension part allows the data range to extend beyond the fractional domain, giving a range of -256.0 to just less than +256.0.

Signed Integer

Stored in two's complement format. The **`.NX`** (no extension) instructions support the use of accumulators for storing integers. The CFX does not directly support integer multiplication; you must shift products from the multipliers to compensate for their fractional output format.

Unsigned Integer

The **.NX** (no extension) instructions support the use of accumulators for storing unsigned integers because they ensure that the extension part of the accumulator is the same as the high bit of the unsigned integer, so saturation will not change the value. The CMPU.NX instructions provide unsigned comparisons. The CFX does not directly support integer multiplication; you must shift products from the multipliers to compensate for their fractional output format.

All of the above data types are supported in both single-width and double-width representations. Single-width (24-bit) fractional and integer numbers can be stored in any of the data registers; the high part of the accumulators is used for single-width data. Double-width (48-bit) fractional and integer numbers can be stored in the accumulators. Extended signed fixed-point numbers use the accumulators' extensions, using 32 bits and 56 bits for single-width and double-width extended values, respectively.

In most of the Arithmetic-class instructions that have a short immediate source operand, the immediate operand is sized either double-width (in the **.DW** instructions) or single-width (otherwise). This is necessary because the short immediate is an integer value; it is aligned with the low-order bits of an integer according to this size, and (if it is a signed immediate) it is sign extended upwards. The **.DW** instructions, then, align their immediate operand with the low-order bits of an accumulator's low part, while the immediate value in single-width instructions is aligned with the low-order bits of the XY registers or the high part of the accumulators. The single exception to these rules is the ADD.FR instruction, where the immediate operand is fractional, so it is aligned with the *high-order* bits of the other operands. The high-order bits of single-width and double-width fractional data are the same—the high-order bits of XY registers and the accumulators' high parts—so this instruction does not have single-width and double-width immediate forms.

4.4 OPERAND CONVERSIONS FOR ARITHMETIC-CLASS INSTRUCTIONS

All Arithmetic-class instructions use an ALU; even those instructions that only move data (the MOVE, SWAP, and LOAD immediate instructions) transfer the data through the X ALU. While both the X ALU and the Y ALU are 56 bits in width, the data registers, immediate values, and multiplier results that are used for ALU inputs and outputs have a variety of widths, requiring conversions. Since these conversions are similar for all Arithmetic-class instructions, and they apply equally to instructions in the X and Y ALUs, this section describes these conversions prior to the sections that describe the specific instruction groups.

IMPORTANT: The conversions done by DCU arithmetic instructions are quite different from those performed by the DMU for data-movement instructions in the *Long* and *Move* classes. Section 5.4, "Data Size Conversions" on page 40 describes conversions in data-movement instructions.

Source Operand Conversions

The X and Y ALUs perform all instructions using two 56-bit input operands, with a single exception: the *count* operand to shift instructions, which is described below. The following points describe how the various source operands are converted to 56 bits prior to their use in an ALU instruction:

- **Accumulator registers** are used unmodified.

- **XY data registers** are aligned with the high part (bits 24 to 47) of the ALU input, and the value is sign extended upward through to bit 55. The low 24 bits of the ALU input are set to zero.
- **Multiplier products** are converted to 56-bit fractional values at the outputs of the multipliers (as described in Section 4.8, "Multiplication" on page 27), so these are used unmodified.
- **Double-width immediate values** (in the **.DW** instructions) are aligned with the low part (bit 0 and up) of the ALU input, and sign extended upward through to bit 55.
- **Single-width immediate values** are aligned with the high part (bit 24 and up) of the ALU input, and sign extended (if signed) or zero extended (if unsigned) upward through to bit 55. The low 24 bits of the ALU input are set to zero.
- **Fractional immediate values** define specific bit patterns for the common fractions, which are placed in bits 45 to 55 of the ALU input (see "ADD.FR d-reg, d-reg, imm3s" on page 140 for details). The lower bits (0 to 44) are set to zero.

After source operands have been converted to 56 bits, if the instruction is a **.NX** (no extension) instruction, the extension bits (bits 48 to 55) of the ALU inputs are set to zero. The ALU inputs are now converted and ready for the actual ALU operation.

The *count* operand in shift instructions is the only exception to the preceding source operand conversion procedure. A shift count in a register is a 7-bit, signed two's complement integer which is inherently single-width, and is therefore stored in bits 0 to 6 of an XY register or bits 24 to 30 (in the *high* part) of an accumulator. The remaining bits of the register are ignored in shift operations.

Destination Conversions

All X and Y ALU instructions produce a 56-bit output, again with a single exception: the results of LOG2ABS and its negative form, which are described below. This section describes how the 56-bit output is converted for storage in the destination register.

If the instruction is a **.NX** (no extension) instruction, the extension bits (bits 48 to 55) of the ALU output are overwritten by sign extending the output value up from bit 47.

Most instructions that update the Zero, Negative, and Extension In Use flags in the status register SR perform that update based on the ALU output at this point. The SR flags that are updated correspond to the ALU used to perform the instruction, and which flags are updated depends on the instruction and the Flag Updates mode bit in SR. There are some exceptions to this general rule, and updates to Overflow and Carry flags are more specific to the particular instruction. See Section 7.4, "Status Register" on page 53 for more information about SR flags and modes, and Chapter 10, "CFX Instruction Set" on page 83 for specific flag updates.

If the destination register is an accumulator, all 56 bits of the ALU output are copied to that accumulator. This occurs regardless of whether one or more of the source operands were XY data registers, or single-width immediate values. If the destination register is an XY data register, the high part (bits 24 to 47) of the ALU output is copied to the XY register *with no rounding or saturation*. Again, this operation is independent of the types of the source operands, as well as the rounding and saturation mode settings in the SR. If there is no destination register (for example, in a comparison instruction), the ALU output is discarded.

IMPORTANT: If the destination register is an XY data register, the high part (bits 24 to 47) of the ALU output is copied to the XY register *with no rounding or saturation***.**

The result of the LOG2ABS and NLOG2ABS instructions is the only exception to the preceding ALU output conversion procedure. This result value is like a shift count (and is often used as such): it is a signed two's complement integer which is inherently single-width. When this ALU result is stored in an XY destination register, the value is sign extended upward through to bit 23. When stored in an accumulator, the result is stored into the *high* part of the accumulator and sign extended upward through to bit 55; the low part of the accumulator is set to zero.

4.5 CONDITIONAL ARITHMETIC INSTRUCTIONS

Some arithmetic instructions support conditional execution. Conditional binary operations (add, subtract, and shift) must use the same register for their first source operand and their destination. Conditional arithmetic instructions do not update the status flags.

The arithmetic instructions that support conditional execution are: MOVE, SWAP, ADD, ADD.DW, SUB, SUB.DW[1], NEG, SHLL, and SHRA.

The eight possible conditions are described in Table 2.

TABLE 2: ARITHMETIC INSTRUCTION CONDITIONS

Condition	Meaning	Logic Expression
CARRY or X_CARRY	Carry	X Carry
NO_CARRY or X_NO_CARRY	No carry	NOT (X Carry)
EQ, X_EQ, ZERO or X_ZERO	Equal, or zero	X Zero
NE, X_NE, NOT_ZERO or X_NOT_ZERO	Not equal, or not zero	NOT (X Zero)
LT, X_LT, LTZ[1] or X_LTZ[1]	Less than	X Negative XOR X Overflow
LE, X_LE, LEZ[1] or X_LEZ[1]	Less than or equal	X Zero OR (X Negative XOR X Overflow)
GE, X_GE, GEZ[1] or X_GEZ[1]	Greater than or equal	NOT (X Negative XOR X Overflow)
GT, X_GT, GTZ[1] or X_GTZ[1]	Greater than	NOT (X Zero OR (X Negative XOR X Overflow))

1. This condition requires version 3 or greater of the CFX assembler.

4.6 ADDITIVE ARITHMETIC AND LOGICAL OPERATIONS

The X ALU, Y ALU, and X MUL's pre-adder unit perform additive arithmetic and logical operations; operand options are introduced in Section 4.2.2, "Arithmetic Logic Units" on page 16 and general operand conversions are detailed in Section 4.4, "Operand Conversions for Arithmetic-Class Instructions" on page 19. This section describes the arithmetic and logical properties of these operations, including general information about how they affect the status register flags.

Note: The DCU performs arithmetic on data values stored in data registers. These instructions can be used for address arithmetic and arithmetic on values in

1. This instruction requires version 3 or greater of the CFX assembler.

other CFX registers, only by copying those registers' values to data registers. Instructions for performing address arithmetic, in the *Long* and *Move* classes, are executed by the AGUs, and are described in Section 6.5, "Address Arithmetic Operations" on page 50.

4.6.1 Additive Arithmetic Operations

In addition to the standard addition, subtraction, comparison and negation operators, the DCU provides some higher-level additive instructions: add and shift right, subtract and shift right, absolute value, maximum, and minimum. Addition, subtraction and negation—with both register and immediate source operands—can be performed conditionally.

Since all ALU operations are performed with 56 bits of precision, support for single-width and double-width arithmetic operations is not provided with separate instructions but rather through:

- The use of 24-bit XY registers or 56-bit accumulators for source and/or destination operands
- High-aligned single-width immediate values or low-aligned double-width immediates
- The data moved into and out of the accumulators; single-width arithmetic can be performed simply by loading accumulators with 24-bit or 32-bit data

Arithmetic performed with accumulator operands is naturally signed, and can equivalently be considered signed integer or signed fractional since the additive arithmetic operations are identical for these two types when two's complement is used for negative values. The 8-bit extension part of the accumulators allows data growth, so that single-width computations can grow up to 32 bits, while double-width computations can use the full 56 bits.

Non-extended integer or fractional operations are supported partially by storing data in the XY data registers, which have no extension. However, extensive additional support for both single-width and double-width non-extended data—including unsigned integer or fractional types—is provided by the **.NX** (no extension) instructions. Since the X ALU clears the extension bits of the source operands, the instruction has unsigned semantics (for comparisons and right shifts, for example). The sign extension performed on the result prevents saturation (even when it is enabled) when the value is stored. Applications can thereby intermix control code and signal processing operations without changing the saturation mode.

X MUL's pre-adder unit performs 24-bit two's complement addition or subtraction between two XY data registers. The result is used (without conversion) as an input to X MUL.

The status register SR contains a set of status flags for each ALU. An additive arithmetic operation (which includes the ALU operation performed in the second cycle of a multiply or MAC instruction) updates the corresponding set of SR flags based on the 56-bit, two's complement ALU operation. The flags are set to the conventional values for additive ALU operations, but note the following general rules that describe variations for some groups of CFX additive arithmetic operations:

- When it is updated, the Carry flag is always set to the carry out of bit 55 of the ALU. Some architectures complement the carry from the ALU when updating the Carry flag after a subtraction or comparison, so that the Carry flag is 1 if a borrow occurred. The DCU does *not* complement the ALU's carry out on subtraction or comparison, so the Carry flag is 0 if a borrow occurred.
- Conditional instructions do not update the status flags.
- X MUL's pre-adder does not modify the flags.
- Absolute value, negation, **.NX** instructions (except for comparisons), and the add-zero/negate performed in the second cycle of a multiply instruction do not update the Carry flag.

- The **.NX** instructions do not modify the Extension In Use flag and they unconditionally set the Overflow flag to 0.
- The add/subtract and shift instructions (ADDSH and SUBSH) update the Zero, Negative, and Extension In Use flags according to the final result, while the Carry and Overflow flags are updated from the addition or subtraction operation only.
- The maximum and minimum instructions update the flags as if a comparison were performed with the two source operands.

Status flag updates also depend on the Flag Updates mode. When Flag Updates mode is disabled, the comparison instructions are the only additive arithmetic instructions that update the status flags. Section 7.4, "Status Register" on page 53 has more information about the flags, and precise flag settings are detailed for each instruction in Chapter 10, "CFX Instruction Set" on page 83.

Table 3 provides a quick guide to the DCU operations that perform additive arithmetic on data. For detailed information about each instruction, see Chapter 10, "CFX Instruction Set" on page 83.

TABLE 3: ADDITIVE ARITHMETIC OPERATIONS

Operation	ALU	Mnemonics
Addition	X ALU, Y ALU, X MUL pre-adder	ADD, ADD.DW, ADD.FR, ADD.NX, ADD.NX.DW, *MULADD*, ADDMUL*
Subtraction	X ALU, Y ALU, X MUL pre-adder	SUB, SUB.DW[1], SUB.FR[1], SUB.NX, SUB.NX.DW[1], *MULSUB*, SUBMUL*
Comparison	X ALU	CMP, CMP.DW, CMPU.NX, CMPU.NX.DW
Negation	X ALU, Y ALU	NEG, *MULNEG*
Add & shift right	X ALU, Y ALU	ADDSH
Subtract & shift right	X ALU, Y ALU	SUBSH
Absolute value	X ALU	ABS
Maximum	X ALU	MAX
Minimum	X ALU	MIN

1. This instruction requires version 3 or greater of the CFX assembler.

4.6.2 Logical Operations

The DCU provides the usual binary logical operations AND, OR, and XOR (Exclusive OR), as well as instructions for testing and modifying a single bit in a data register. Only the X ALU is capable of logical operations, so these instructions are all in the *Single Arithmetic* or *X Arithmetic* classes.

As with the additive arithmetic operations, the ALU's logical operations are performed with 56 bits of precision, so single-width and double-width logical operations are supported through:

- The use of 24-bit XY registers or 56-bit accumulators for source and/or destination operands
- High-aligned single-width immediate values or low-aligned double-width immediates

- The data moved into and out of the accumulators; single-width logical functions can be performed simply by loading accumulators with 24-bit or 32-bit data

Non-extended single-width logical operations can be performed by using an XY data register for the destination. For non-extended double-width (48-bit) logical operations, be aware that transferring a non-extended accumulator[2] triggers saturation if the high bit (bit 47) is set and the saturation mode is enabled. To avoid this without changing the saturation mode, sign extend the accumulator value using the `MOVE.NX` instruction prior to the transfer.

Single-bit operations can test a bit's value, set a bit to 1, clear a bit to 0, or toggle (complement) a bit, in any of the data registers. These are "in-place" operations; just one register operand is both the source and destination.

Logical operations update some of the X ALU status flags, as follows:

- AND, OR, and XOR instructions update the Zero, Negative, and Extension In Use flags according to the result from the ALU; the Negative flag is set from bit 55. The Carry flag is not modified and the Overflow flag is unconditionally set to 0.
- Single-bit instructions update the Carry flag to the value of the indexed bit *prior* to the instruction. No other flags are modified.

Status flag updates also depend on the Flag Updates mode. When Flag Updates mode is disabled, the `BITTST` instruction is the only logical instruction that updates the status flags. Section 7.4, "Status Register" on page 53 has more information about the flags, and precise flag settings are detailed for each instruction in Chapter 10, "CFX Instruction Set" on page 83.

Table 4 provides a quick guide to the X ALU operations that perform logical operations. For detailed information about each instruction, see Chapter 10, "CFX Instruction Set" on page 83.

TABLE 4: LOGICAL OPERATIONS

Operation	Mnemonics
AND	`AND, AND.DW`
OR	`OR, OR.DW`
XOR (Exclusive OR)	`XOR, XOR.DW`
NOT (Complement)	`XOR` with -1, `XOR.DW` with -1, `NOT`[1]
Test bit	`BITTST`
Set bit to 1	`BITSET`
Clear bit to 0	`BITCLR`
Toggle bit	`BITTGL`

1. This instruction requires version 3 or greater of the CFX assembler.

2. Transferring non-extended accumulators includes register copy or store instructions in the *Long* and *Move* classes, but excluding the **`.EX`** (extended) instructions.

4.7 SHIFTER OPERATIONS

The X ALU provides a variety of shifting instructions as well as leading-bit-detection instructions for numeric normalization. The ALU contains a 56-bit barrel shifter, so shift and leading-bit detection instructions execute in a single cycle.

4.7.1 Shifts

Shift operations are performed by the ALU on a 56-bit value. Section 4.4, “Operand Conversions for Arithmetic-Class Instructions” on page 19 explains how the first source operand (the value to shift) is converted to 56 bits prior to the operation, and how the single-width shift count is obtained from either an immediate value, an XY data register, or the *high* part of an accumulator. Shifts on single-width data can be performed by using the XY data registers as source and destination operands, or simply by loading an accumulator with single-width data prior to shifting it.

There are four different shift functions available. You can choose either left or right, and either extended or non-extended; all four allow an immediate or a register-based shift count. Left shifts are logical: zeros are shifted into bit 0. Extended right shifts are arithmetic—bit 55 is duplicated—while **`.NX`** (no extension) right shifts are logical since the extension bits of the input are zeros.

When a register operand provides a shift count, the count is a 7-bit signed two’s complement number. The shift count must be between -56 and 56. A negative value reverses the shift direction indicated by the instruction’s mnemonic; the behavior of the shift instruction becomes that of its opposite shift function according to the equivalents shown in Table 5. For detailed information about each instruction, see Chapter 10, “CFX Instruction Set” on page 83.

TABLE 5: EQUIVALENT SHIFTS FOR NEGATIVE COUNT

Shift with negative count register	Equivalent shift
`SHLL -N`	`SHRA N`
`SHRA -N`	`SHLL N`
`SHLL.NX -N`	`SHRL.NX N`
`SHRL.NX -N`	`SHLL.NX N`

The shift instructions have several source operand combinations:

- In three-operand form, you can choose the value to shift, the shift count, and the destination register independently. The shift count is either the sign-extended value of the low 7 bits of a data register (in the range -56 to 56), or an unsigned 3-bit immediate value (in the range 0 to 7).
- In two-operand full-immediate form, the value to shift and the destination are the same data register. The shift count is an unsigned 6-bit immediate value in the range 0 to 56.
- In two-operand conditional form, the shift only occurs if the condition you specify is true. The value to shift and the destination are the same data register. The shift count is either the sign-extended value of the low 7 bits of a data register (in the range -56 to 56), or an unsigned 3-bit immediate value (in the range 0 to 7). The conditional forms are only available for the “with-extension” shift instructions `SHLL` and `SHRA`.

Shift instructions update the X ALU status flags only when the Flag Updates mode is enabled. The conditional shift instructions never update status flags. Flag updates for the various shift instructions, with the equivalents for negative shift counts assumed (for example, the description below for SHRA also applies to a SHLL instruction with a negative count), are as follows:

- **No-extension** shifts—SHLL.NX and SHRL.NX—update the Zero and Negative flags from the ALU's result, and unconditionally set the Overflow flag to 0.
- **SHRA** shifts update the Zero, Negative, and Extension In Use flags from the ALU's result, update the Carry flag to the value of the last bit shifted out of bit 0 of the ALU, and unconditionally set the Overflow flag to 0.
- **SHLL** shifts update the Zero, Negative, and Extension In Use flags from the ALU's result, update the Carry flag to the value of the last bit shifted out of bit 55 of the ALU, and update the Overflow flag to indicate whether bit 55 of the ALU changed value at any time during the shift operation.

Section 7.4, "Status Register" on page 53 has more information about the flags, and precise flag settings are detailed for each instruction in Chapter 10, "CFX Instruction Set" on page 83.

4.7.2 Leading Bit Detection

The ALU's LOG2ABS instruction determines the base-2 logarithm of the absolute value of its 56-bit input. This is sometimes referred to as an *exponent* operation because the result is used as the input number's exponent when the number is converted to a floating-point representation. The result can also be used to *normalize* a number (see Section 4.7.3, "Normalization" on page 27), or simply as an approximation of the number's scale.

LOG2ABS scans its 56-bit input, starting with the most significant bit, for a bit which is different from bit 55. The position of the bit found, as an offset from bit 46 (the most significant magnitude bit for numbers in the fractional domain) is LOG2ABS's output. More precisely, LOG2ABS performs the following operation:

Output = 9 - *number of identical leading bits*

Thus the result is positive when the extension is in use (this can only happen with an accumulator source operand whose value exceeds the fractional range), and negative when the magnitude of the source operand is less than 0.5. The result is treated as a single-width value and is stored in the destination register as described in Section 4.4, "Operand Conversions for Arithmetic-Class Instructions" on page 19, so that it is ready to be used as a register-based shift count.

LOG2ABS has one special case: when the input is 0, the result is 0.

The NLOG2ABS instruction performs the same function as LOG2ABS, but the result is negated. In combination with shift operations taking a register-based count, these instructions provide the scaling facilities required for many mathematical function approximations as well as conversions to and from floating-point representations. More information about such conversions is provided in the following section.

The leading bit detection instructions update the X ALU status flags only when the Flag Updates mode is enabled. The Zero and Negative flags are set from the result value (*after* negation for NLOG2ABS), and the Overflow flag is unconditionally set to 0. The Carry and Extension In Use flags are not modified. Section 7.4, "Status Register" on page 53 has more information about the status flags, and the precise settings are detailed in Chapter 10, "CFX Instruction Set" on page 83.

4.7.3 Normalization

A number is *normalized* in an accumulator when it is shifted to the greatest magnitude that does not exceed the fractional range. A nonzero normalized number contains sign bits in the extension register and bit 23 of the high register, and the most significant magnitude bit (bit 22 of the high register) has the opposite value of the sign bits. The value 0 is a special case, and is considered normalized.

The LOG2ABS instruction determines the shift count (as a signed value) required to normalize a value in an accumulator. To normalize the value in an accumulator, use the following two instructions:

```
log2abs d-reg, ab-reg
shra ab-reg, ab-reg, d-reg
```

After these operations, the destination accumulator and selected data register contain a floating-point representation of the fractional number that was originally in the accumulator. The destination accumulator contains the normalized mantissa and the data register contains the signed exponent.

There are two special cases to consider:

- If the input is 0, LOG2ABS produces a result of 0, and the value is normalized correctly: mantissa = 0, exponent = 0.
- If the input is all 1 (the negative fractional number nearest to zero), LOG2ABS produces a result of -47, and the value is again normalized correctly: mantissa = 0xFF800000000000, exponent = -47.

No special handling is required for either of these cases.

You can also use the shifter to denormalize a number, returning it to the fractional domain. Load the sign-extended normalized value (mantissa) in an accumulator and the exponent in a data register, then execute the following instruction:

```
shll ab-reg, ab-reg, d-reg
```

After this instruction, the destination accumulator contains the fractional representation of the floating point number.

4.8 MULTIPLICATION

The DCU contains two multipliers that each multiply 24-bit inputs to produce a 48-bit fractional-format product in a single cycle. However, multiplication operations are pipelined so that a multiplier computes the product in the first cycle and an ALU accumulates the result into an accumulator in the second cycle. A new multiplication can be initiated during the ALU operation of the previous multiplication, so each multiplier can compute a product per cycle.

Multiplication instructions are divided into two categories by the ALU operation that occurs in their second cycle: *multiply-accumulate* (MAC) instructions either add the product to an accumulator or subtract the product from an accumulator, while *multiply* instructions (sometimes referred to as *multiply-only* for clarity) use zero for the first input to the ALU operation, and thus store the product or its negation into the destination accumulator. The mnemonics for these

instructions as well as their operands, variations such as the signs of the inputs, and status flag settings are described in Section 4.8.2, “Multiplication Instructions” on page 28.

Since multiplication operations are pipelined, their results are not available in the cycle after they are initiated. As they are still using resources (their ALU and the destination accumulator) during the second cycle, there are restrictions on the instructions that can be initiated. These aspects of multiplications are examined in Section 4.8.3, “Timing, Concurrency, and Restrictions” on page 30.

First, however, the arithmetic properties of the multipliers are described, including the product output data format, in Section 4.8.1, “Arithmetic of Multiplication”.

4.8.1 Arithmetic of Multiplication

Both the X and Y multipliers take two 24-bit inputs from XY data registers and produce a 48-bit output which is converted to 56 bits for use in the corresponding ALU. The second input to X MUL comes from a pre-adder that performs standard two’s complement addition or subtraction with no overflow handling. This section discusses the multiplication that is performed with the multipliers’ two inputs, one of which is the *output* of the pre-adder, in the case of X MUL.

X MUL can treat its inputs as signed or unsigned numbers, for some forms of the multiplication instructions. Therefore, this section considers multiplication between two inputs of 24 bits, each of which has a signed/unsigned attribute. For Y MUL and some instruction forms for X MUL, the signedness attribute for both inputs is always **signed**.

To generate the product of the inputs, the multipliers first perform an integer multiplication, with signed or unsigned operands according to the attributes of the inputs. This produces a full 48-bit (integer) product that is correct for all input values; there are no special cases such as the square of 0x800000[3], due to the use of integer arithmetic.

The integer product is then converted to a 56-bit fractional form for use in the following ALU operation. For unsigned/unsigned multiplications, the 48-bit product is zero extended upward above bit 47, while all other products are sign extended upward from their high bit (bit 47) through to bit 55. The 56-bit product is then shifted left by one bit (inserting zero in the least-significant bit and discarding the value shifted out of the most-significant bit) to remove the duplicate sign bit that was generated in the product for signed/signed inputs, and thereby convert products of signed/signed inputs into the usual fractional output format.

Note: These steps to convert the 48-bit integer product into the multiplier’s 56-bit output—sign/zero extension followed by left shift—occur *regardless* of the signedness attributes of the multiplier’s inputs. That is, they occur for signed/unsigned and unsigned/unsigned products, as well as for signed/signed products.

4.8.2 Multiplication Instructions

The DCU executes multiplication instructions in the *Single Arithmetic* and *Dual Arithmetic* classes. *Single Arithmetic* instructions use X MUL and the X ALU. In the *Dual Arithmetic* class, multiplication instructions can be executed in parallel on both sides, using the X MUL/X ALU and

3. 0x800000 is the fractional value -1.0; its square +1.0 cannot be represented in two’s complement fractional form unless extension bits are added to indicate that the value is positive.

Y MUL/Y ALU respectively, or one side can perform an ALU operation while the other initiates a multiplication.

Single Arithmetic multiplications using X MUL and the X ALU have considerably more flexibility in their source and destination operands than *Dual Arithmetic* instructions. For two-input instructions, any pair of XY data registers can be used for the source operands, which can be treated as signed/signed, signed/unsigned, or unsigned/unsigned inputs, and the destination can be any AB accumulator. Alternatively, the three-input form uses three XY register source operands (only certain combinations are valid); the second and third are added or subtracted in X MUL's pre-adder to form the second input to X MUL. In this case, both inputs to the multiplier are considered signed.

Note: Regardless of operands (for example, even when a B accumulator is the destination), *Single Arithmetic* multiplications *always* execute in X MUL and the X ALU.

X Arithmetic and *Y Arithmetic* multiplications require their first source operand and their destination accumulator to be from the "same side"—X and A registers for the X instruction, Y and B registers for the Y instruction—as the MUL/ALU units performing the instruction. In addition, the second source operand on each side must be opposite (i.e., one side must be an X register, and the other side must be a Y register). Both inputs to the multiplier are considered signed.

All multiplication instructions perform an ALU operation in the second cycle of their execution, after the product is generated in the first cycle. There are four options for the ALU operation, as shown in Table 6.

TABLE 6: ALU OPERATIONS FOR MULTIPLIES

Description	Operation
Multiply-Add	`accum = accum + product`
Multiply-Subtract	`accum = accum - product`
Multiply	`accum = 0 + product`
Multiply (negated product)	`accum = 0 - product`

When the Flag Updates mode is enabled, multiplication instructions update the flags in the status register `SR` from the ALU operation in the second cycle of execution. The flag settings are the same as for the corresponding additive arithmetic operations except that the multiply-only instructions do not update the Carry flag. See Section 4.6.1, "Additive Arithmetic Operations" on page 22, and note that the flags updated are those corresponding to the instruction's ALU. Section 7.4, "Status Register" on page 53 has more information about the flags, and precise flag settings are detailed for each instruction in Chapter 10, "CFX Instruction Set" on page 83.

Table 7 provides a quick guide to the DCU operations that perform multiplications. Where operand signedness is not indicated, it is signed/signed. For detailed information about each instruction, see Chapter 10, "CFX Instruction Set" on page 83.

TABLE 7: MULTIPLICATION OPERATIONS

Operation	DCU Side	Mnemonics
Multiply	X MUL/ALU, Y MUL/ALU	MUL
Multiply (negated product)	X MUL/ALU, Y MUL/ALU	MULNEG
Multiply-Add	X MUL/ALU, Y MUL/ALU	MULADD
Multiply-Subtract	X MUL/ALU, Y MUL/ALU	MULSUB
Multiply signed/unsigned	X MUL/ALU	MUL.SU, MULNEG.SU
Multiply-Accumulate signed/unsigned	X MUL/ALU	MULADD.SU, MULSUB.SU
Multiply unsigned/unsigned	X MUL/ALU	MUL.UU, MULNEG.UU
Multiply-Accumulate unsigned/unsigned	X MUL/ALU	MULADD.UU, MULSUB.UU
Multiply with pre-add	X MUL/ALU	ADDMUL, ADDMULNEG
Multiply-Accumulate with pre-add	X MUL/ALU	ADDMULADD, ADDMULSUB
Multiply with pre-subtract	X MUL/ALU	SUBMUL, SUBMULNEG
Multiply-Accumulate with pre-subtract	X MUL/ALU	SUBMULADD, SUBMULSUB

4.8.3 Timing, Concurrency, and Restrictions

Multiplication instructions take two cycles to execute, occupying a multiplier in the first cycle to compute the product, and the corresponding ALU in the second cycle. All multiplication instructions use either X MUL and the X ALU, or Y MUL and the Y ALU; for example, the Y ALU is never used for the second cycle of an *X Arithmetic* multiplication.

These instructions are pipelined, so a multiplication instruction can be initiated in every cycle in each MUL/ALU pair. After the first such parallel multiply-accumulate instructions are initiated, parallel MAC instructions initiated in subsequent cycles will be executing four operations in parallel: while the multipliers are computing the products for the current MAC instructions, the ALUs are accumulating the products from the previous MACs. In this way, each MUL/ALU pair can compute the sum of *N* products in *N*+1 cycles.

Other combinations of parallel arithmetic are possible, such as multiplications in one MUL/ALU pair in parallel with ALU instructions (arithmetic, logic, shifts, etc.) in the opposite ALU. Any of these parallel arithmetic options can be combined with one or two data-movement instructions (in an *Arithmetic-Move* instruction; see Section 3.3, “Arithmetic-Move Instructions” on page 10) which *also* execute concurrently with the Arithmetic-class instructions. Finally, since the ALU part of a multiplication instruction is executed a cycle after the instruction is initiated, in principle *any* instruction type—including control-flow instructions in the *Long* class—can be executed during the ALU operation part of the multiply or MAC instruction.

Not surprisingly, the CFX has some restrictions on what operations can execute concurrently, with well-defined behavior. The following sections describe these restrictions, primarily as they relate to multiplication instructions.

4.8.3.1 Multiplication Guidelines

The following is a summary of the code constructions to avoid during the ALU part of a pipelined multiplication instruction; that is, in the next instruction.

Using the same ALU

The ALU is already in use in the cycle following the multiplication, so it cannot be used for another operation. Another multiplication *can* be scheduled because it uses the multiplier in its first cycle.

```
mul a0, x0, y0  |||  nop
add b0, b1, -1  |||  nop   // X ALU already in use!
```

Using the same accumulator pair in arithmetic

The data registers are paired in *register files*, with interconnections working on the register file level within the DCU. Therefore, an *Arithmetic* instruction following a multiplication cannot write to the destination accumulator *or its pair*. Some forms of read accesses are also not possible, so it is best to avoid using the accumulator pair entirely, in an *Arithmetic* instruction following a multiplication.

```
sub a1, a0, a1 !! mul b0, y0, y1  |||  nop
add b1, a0, -1  |||  nop
    // Cannot write B accumulators - already in use!

sub a1, a0, a1 !! muladd b0, y0, y1  |||  nop
add a0, b1, -1  |||  nop
    // Cannot read B accumulators - already in use!
```

Accessing outputs

After a multiplication, there must be at least one other instruction before any code that:

- Accesses the result in the accumulator
- Accesses the status flags that are updated if Flag Updates mode is enabled

For example:

```
// Flag Updates mode is enabled here
mul a0, x0, y0  |||  nop
goto neg_product, lt   // Flags are not ready yet!
```

Conversely, however, it is generally not a good idea to *depend* on the delayed results, by either reading the destination accumulator with the assumption that the old value will be read, or (when Flag Updates mode is enabled) by performing a conditional branch with the assumption that the flags will have the values from code prior to the multiplication. These assumptions fail if an interrupt occurs after the multiplication is initiated; in this case the multiplication will fully complete before the next instruction is executed (after returning from the interrupt handler), so the destination accumulator and flags will have new values from the multiplication.

```
mul a0, x0, y0  |||  nop
store x[some_var], a0         // Old a0 might have been overwritten!
```

While the “same accumulator pair” guideline does not allow reads from the accumulator or its pair in an *Arithmetic* instruction, reading the destination accumulator in a data-movement instruction is also dangerous.

Two writes to the destination register

The behavior is not defined when the same register is written by two concurrently-executing instructions. While the “same accumulator pair” guideline does not allow writes to the accumulator or its pair in an *Arithmetic* instruction, writing the destination accumulator in a data-movement instruction is also undefined.

```
mul a0, x0, y0  |||  nop
nop  |||  move a0, b0    // Undefined result in a0!
```

4.8.3.2 Details of Multiplication Restrictions

The guidelines in Section 4.8.3.1, “Multiplication Guidelines” on page 31 are sufficient for avoiding problems related to the pipelined multiplication instructions. Some of the guidelines are more restrictive than necessary, however, for simplicity.

This section elaborates on some sections of the guidelines to provide comprehensive information about multiplication instruction combinations that are well-defined and those that are not. The limitations of the hardware are detailed, with more examples of specific legal and illegal instruction combinations. This allows you to write programs that use the multipliers and ALUs to their full extent.

The main elaboration on the guidelines relates to the use of the same accumulator pair in both the multiplication instruction and the following *Arithmetic* instruction. While the guidelines treat read and write operations as equivalent, the restrictions actually differ for these operations, as detailed in the next two sections.

Writing the same accumulator pair in arithmetic

As the multiplication guidelines show, if a multiplication instruction is followed by an *Arithmetic-Move* instruction, the *Arithmetic* part cannot write to *either* accumulator in the pair containing the multiplication instruction’s destination.

This restriction is due to the number of write ports on the DCU’s register files, so it does *not* affect *Move* or *Long* instructions, although they can only write to the other accumulator of the pair without violating the “two writes to destination” restriction. The following modification to the example from the guidelines is well-defined:

```
sub a1, a0, a1 !! mul b0, y0, y1  |||  nop
nop  |||  move b1, a0
    // Legal write to b1 in Move-class instruction
```

Note: The `SWAP` instruction is the only *Arithmetic* instruction with *two* destination registers. The two register operands to `SWAP` are both sources and destinations.

Reading the same accumulator pair in arithmetic

There is also a limit to the number of read ports on the register files. However, the design of the DCU’s interconnections allows many combinations of multiplication instructions with *Arithmetic* instructions that read the same accumulator pair. Only certain combinations are illegal.

The *multiply-only* instructions—those multiplication instructions that do not add or subtract the multiplier product with the accumulator—do *not* read their accumulator destination. Thus there is no restriction on reads of the accumulator pair for *Arithmetic* instructions that follow MUL, MUL.SU, MUL.UU, ADDMUL, SUBMUL, or the MULNEG variants of these instructions.

The following variant on the example from the guidelines is well-defined, because it uses MUL instead of MULADD:

```
sub a1, a0, a1 !! mul b0, y0, y1  |||  nop
add a0, b1, -1  |||  nop
    // Legal read of b1, after mul or mulneg
```

The MAC instructions—all variants of MULADD and MULSUB—do read their accumulator destination. When you use these multiplication instruction forms, a following *Arithmetic* instruction cannot use *either* accumulator in the same register pair for its *first* source operand. There is no restriction for the second source operand, so the following variation on the example from the guidelines is also legal:

```
sub a1, a0, a1 !! muladd b0, y0, y1  |||  nop
add a0, a1, b1  |||  nop
    // Legal read of b1 as 2nd source, after muladd or mulsub
```

The rules for identifying the *first* source operand for the various forms of *Arithmetic* instructions are as follows:

- For three-operand instructions that implement binary operators with two source operands and a destination, the first source operand is always the instruction's second operand, following the destination.
- For the comparisons and BITTST there is no destination, so the instruction's first source is its first operand.
- For conditional instructions or those with 6-bit immediate operands, the first source and the destination are the same register: the instruction's first operand.
- Instructions that implement unary operators (ABS, LOG2ABS, MOVE, MOVE.NX, NEG, and NLOG2ABS) do not have a first source operand. Their source operand is routed in the DCU as a second source operand, so there are never any restrictions on an accumulator used as the source operand.

As with writing to accumulators, this read-port restriction does not affect *Move* or *Long* instructions, which can read either accumulator in the pair. For example:

```
sub a1, a0, a1 !! muladd b0, y0, y1  |||  nop
nop  |||  move a0, b1
    // Legal read of b1 in Move-class instruction
```

The following sections provide additional information about two of the other potential pitfalls discussed in the guidelines.

Using the same ALU

This resource conflict applies to both the X ALU and the Y ALU. However, all of the instruction combinations that violate this restriction with the Y ALU also violate the "writing the same accumulator pair" restriction. For example:

```
mul a0, x0, y0 !! mul b0, y1, x1  |||  nop
muladd a0, x0, x1 !! add b1, b1, a1  |||  nop
    // Y ALU already in use!
    // Cannot write B accumulators - already in use!
```

Two writes to the status register

As stated in the multiplication guidelines, a destination register cannot be written twice in the same cycle. It is legal and well-defined, however, for two instructions to write the SR simultaneously.

This can occur during the second cycle of a multiplication instruction, when the flags in the SR are updated (if Flag Updates mode is enabled) by the ALU operation. Simultaneous updates to SR are discussed in Section 7.8, “Restrictions on Concurrent Register Accesses” on page 74.

4.9 MISCELLANEOUS OPERATIONS

The X ALU supports some instructions for data movement, and a division-step instruction that implements one iteration of a non-restoring division algorithm. Operand options are introduced in Section 4.2.2, “Arithmetic Logic Units” on page 16 and general operand conversions are detailed in Section 4.4, “Operand Conversions for Arithmetic-Class Instructions” on page 19. While the data-movement instructions might appear to be different from other Arithmetic-class instructions since they are not computational, they do execute in the X ALU and follow all other conventions for ALU operations. For example, they never round or saturate data (as would data-movement instructions in the *Long* or *Move* classes) because DCU operand conversions never round or saturate. This section describes these miscellaneous operations, including status register flag updates.

The division step instruction DIVST implements one iteration of a standard non-restoring division algorithm.[4] It has two data register source operands and a data register destination, and implements the following operations:

```
input1 = source1; // Usual source operand conversions
input2 = source2 & 0xFFFFFFFFFFFF; // 48-bit divisor; clear extension

if (X ALU Negative flag is set)
    output = input1 * 2 + input2;  // And update X ALU Negative flag
else
    output = input1 * 2 - input2;  // And update X ALU Negative flag

if (X ALU Negative flag is not set)
    output |= 1;                   // Do NOT update X ALU Negative flag

destination = output;              // Usual destination conversions
```

As shown, DIVST updates the Negative flag. This is the only flag that it modifies, and the update is performed regardless of whether the Flag Updates mode is enabled or disabled.

The X ALU provides a small set of useful data-movement operations, which are summarized in Table 8. For detailed information about each instruction, see Chapter 10, “CFX Instruction Set” on page 83.

4. See, for example, *Computer Architecture: A Quantitative Approach* by Hennessy and Patterson, Appendix A.

TABLE 8: DATA-MOVEMENT OPERATIONS

Operation	Mnemonics
Load immediate	LOAD, LOAD.DW
Copy register	MOVE[1]
Copy register conditionally	MOVE
Copy register and sign extend	MOVE.NX
Swap register contents conditionally	SWAP

1. This instruction requires version 3 or greater of the CFX assembler.

The load immediate instructions allow a 6-bit signed immediate value, aligned as a single-width value or a double-width value, to be loaded into any of the data registers.

The two conditional instructions are very useful for implementing data-conditional logic without branch instructions. The MOVE instruction is destructive to the destination, while SWAP is not. The operands to SWAP must be in different register files; for example, it is possible to swap A0 with B0 or X0 with Y1, but not A0 with A1 or X0 with X1.

Note: DCU *Arithmetic* instructions provide the sole method for copying all 56 bits of one accumulator register to another accumulator with a single instruction. The MOVE* instructions in the *Move* class always transfer 24, 32, or 48 bits. While the MOVE and SWAP instructions in this section are conditional, it is possible to copy a full 56-bit accumulator unconditionally using an additive arithmetic, logical, or shift instruction with an immediate operand such that the destination receives the unmodified value of the source operand. Some examples are:

```
add  a0, b0, 0   |||  nop
and  a0, b0, -1  |||  nop
xor  a0, b0, 0   |||  nop
shll a0, b0, 0   |||  nop
min  a0, b0, b0  |||  nop
```

Version 3 or greater of the CFX assembler supports "`move d-reg, d-reg`" as a synonym for "`xor.dw d-reg, d-reg, 0`".

Beware of status flag updates when using these computational instructions to move data. The actual data-movement instructions shown in Table 8 do not update any status register flags.

This page left blank for printing purposes.

5 Data Movement Unit

5.1 INTRODUCTION

This chapter describes the Data Movement Unit (DMU) which executes data-movement instructions in the *Long* and *Move* classes. This includes:

- All instructions which load data from memory into CFX registers or store data from registers into memory
- All instructions which load immediate values into CFX registers or store immediate values into memory
- Register copy instructions, except those in the *Arithmetic* class that are executed by the DCU

Most importantly, this chapter extensively describes the conversions that are performed during these data transfers.

5.2 DMU ARCHITECTURE

The DMU manages *all* transfers between CFX registers and memory, and register-to-register transfers other than those executed by the DCU. It is therefore connected to all three system data buses—the X Bus, Y Bus, and P Bus—and has X and Y data connections to internal subsystems such as the register files. These connections can be seen in Figure 1 on page 4.

While the DMU manages all transfers of data to and from memory, the *addresses* accessed in memory are managed by the PCU for direct addresses from the instruction stream, and by the AGUs for indirect addresses via pointer registers. The DMU is not involved in address generation and so is not connected to any of the system's address buses.

The DMU contains the following logic blocks for performing X and Y data transfers in both directions:

- When an accumulator is the data source for a transfer (when storing to memory or as the source operand in a register-to-register copy), the accumulator's 56-bit value passes through a Round/Saturate unit (see Section 5.5, "Rounding and Saturation" on page 42) to convert it to the destination's size.
- When an accumulator is the destination for a transfer (when loading from memory or as the destination operand in a register-to-register copy), the source value passes through a Sign/Zero Extend unit to sign extend it up and/or zero extend it down to produce the 56-bit value for the accumulator.

Conversions for other CFX registers are less complex and therefore do not have dedicated logic blocks. All data conversions are described in Section 5.4, "Data Size Conversions" on page 40.

Data can be transferred concurrently on the X and Y data buses; this is supported in the instruction set with:

- *Dual Move* instructions which access the X and Y memories independently

- Accesses to the logical XY memory space (composed of both X memory and Y memory) where a value is accessed in each memory at the same address

The DMU contains separate conversion blocks (Round/Saturate and Sign/Zero Extend) for X and Y memory transfers to support concurrent access to these memories. The X Round/Saturate unit and Sign/Zero Extend unit are also used for P memory and XY memory transfers, so these units have functionality to support extended and double-width data conversions. There are restrictions on concurrent register access as described in Section 7.8, “Restrictions on Concurrent Register Accesses” on page 74.

5.3 DATA SIZES

The CFX has registers of several sizes, and can access memory with memory buses of three different widths. This section describes and names these entities, introducing notation for use in the following sections.

Table 9 summarizes the groups of similar registers in the CFX. Registers can be source or destination operands in data-movement instructions; in fact, the same register (or parts of the same accumulator) could be both the source and destination in a register-to-register move instruction.

TABLE 9: CFX REGISTER GROUPS

Group Name	Contains	Width	Signedness
Whole accumulators	Accumulators `A0`, `A1`, `B0`, and `B1`	56 bits	signed
Single-width registers	General-purpose XY data registers `X0`, `X1`, `Y0`, and `Y1` Accumulator high parts `A0.H`, `A1.H`, `B0.H`, and `B1.H` Accumulator low parts `A0.L`, `A1.L`, `B0.L`, and `B1.L` Loop end address `LEA`[1]	24 bits	signed
Pointer and control registers	X pointers `XP0` to `XP7` Stack pointer `SP` Y pointers `YP0` to `YP3` X circular configuration registers `XCIRC0` to `XCIRC7`[2] Y circular configuration registers `YCIRC0` to `YCIRC3` Status register `SR` Linking registers `LR`, `ILPC`, and `ILSR` Loop start address `LSA` Loop count `LC`[3]	16 bits	unsigned

TABLE 9: CFX REGISTER GROUPS (CONTINUED)

Group Name	Contains	Width	Signedness
Step registers	X step registers XSTEP0 to XSTEP7 Y step registers YSTEP0 to YSTEP3	16 bits	signed
Accumulator extension registers	Accumulator extension parts A0.E, A1.E, B0.E, and B1.E	8 bits	signed

1. Although loop start and end addresses are 16 bits, the LEA register logically contains 24 bits, as explained in Section 7.6.4, "Hardware Loop Stack Registers" on page 66.

2. The circular configuration registers only store 15 bits, but as they are unsigned they are treated as 16 bits in width with a zero bit in position 15.

3. The loop count register only stores 12 bits, but as it is unsigned it is treated as 16 bits in width with four zero bits in positions 12 to 15.

The DMU manages three different types of data transfers, as follows:

Single-Width

Single-width data transfers are 24-bit operations. All LOAD, STORE, and MOVE instructions (in the *Long* and *Move* classes) that do *not* have a suffix (**.EX** or **.DW**) are single-width. Any register can be a source or destination for a single-width data transfer, as can the three single-width memory areas: X memory, Y memory, and P memory. LOAD, STORE, and CLR (where the immediate value zero is implicit) instructions also use immediate values as source operands.

Extended Single-Width

Extended single-width (or simply *extended*) data transfers are 32-bit operations. The instruction mnemonics use the **.EX** suffix: LOAD.EX, STORE.EX, and MOVE.EX. Only whole accumulators and P memory can be source and destination operands for extended data transfers. Note that both single-width and extended single-width instructions can access P memory.

Double-Width

Double-width data transfers are 48-bit operations. The instruction mnemonics use the **.DW** suffix: LOAD.DW, STORE.DW, and MOVE.DW. Whole accumulators and XY memory can be source and destination operands for extended data transfers. LOAD.DW instructions also use immediate values as source operands.

5.4 DATA SIZE CONVERSIONS

This section describes the actions taken by the DMU as it executes all data-movement instructions. The register groups and data transfer types introduced in Section 5.3, “Data Sizes” on page 38 are used extensively in these descriptions.

IMPORTANT: The conversions done by DMU data-movement instructions are quite different from those performed by the DCU for *Arithmetic* instructions. Section 4.4, “Operand Conversions for Arithmetic-Class Instructions” on page 19 describes conversions in *Arithmetic* instructions.

Data transfers of all types use the same general scheme for performing conversions:

1. If the source is memory, transfer the memory value.

   Otherwise, convert the source operand to the transfer width (single, extended single, or double), if necessary.

2. If the destination is memory, transfer and store the resulting value.

   Otherwise, convert the resulting value to the destination register’s width, if necessary, and store it.

The following sections detail the actions required to implement the two conversion steps in this procedure, for each of the three data transfer types.

5.4.1 Single-Width Data Transfers

Single-width transfers are 24-bit operations involving all of the register groups described in Section 5.3, “Data Sizes” on page 38. An immediate value or register source operand is converted to 24 bits as follows:

- A 24-bit immediate value or single-width register does not require conversion.
- A smaller immediate value is converted to 24 bits by sign extending or zero extending, if the signedness of the immediate field is signed or unsigned, respectively.
- A smaller register’s value is converted to 24 bits by sign extending or zero extending, if the register is signed or unsigned, respectively.
- A whole accumulator’s value is passed through the Round/Saturate unit. The value is rounded and the low part is discarded, and then it is saturated and the extension is discarded, leaving the rounded and saturated 24-bit high part. The rounding and saturating operations function according to the currently-selected modes in the status register `SR`, as described in Section 5.5, “Rounding and Saturation” on page 42.

If the destination is a register, the transferred or converted source value is converted to the register’s width as follows:

- No conversion is required for a single-width destination register.
- For a smaller register, the value is converted by simply ignoring the high bits. The low-order bits corresponding to the width of the destination register are stored in the register.
- For a whole accumulator, the 24-bit value is stored in the high part of the accumulator, and the Sign/Zero Extend unit sign extends the value upward from bit 47 through to bit 55, while filling the low part (bits 0 to 23) with zeros.

Single-width transfers to and from P memory are a special case. When reading a single-width value from P memory, the upper 8 bits are discarded and saturation is not performed. When writing a single-width value to P memory, the 24-bit value is sign extended to 32 bits.

Since whole accumulator conversions are the most complex, it is instructive to examine a single-width register-to-register copy with two accumulator operands:

```
nop ||| move a0, b0     // copy b0 to a0, single-width
```

The following conversions are performed during the data transfer:

1. The value from B0 is converted to 24 bits by rounding and saturating it.
2. The resulting 24-bit value is written to the high part of A0.
3. The value is sign extended and zero padded, overwriting the whole 56-bit value of the accumulator.

5.4.2 Extended Single-Width Data Transfers

Extended transfers are 32-bit operations involving only whole accumulators and P memory.

An accumulator source operand's value is converted to 32 bits by passing it through the Round/Saturate unit. Only rounding is performed, however, and the low part is discarded, leaving the high and extension parts (32 bits) of the rounded value. The rounding operation functions according to the current rounding mode selected in SR, as described in Section 5.5, "Rounding and Saturation" on page 42.

If the destination is an accumulator, the 32-bit transferred or converted source value is stored in the extension and high parts of the accumulator, and the Sign/Zero Extend unit fills the low part (bits 0 to 23) with zeros.

An extended register-to-register copy uses both the source and destination conversions:

```
nop ||| move.ex a0, b0     // copy b0 to a0, extended
```

The following conversions are performed during the data transfer:

1. The value from B0 is converted to 32 bits by rounding it.
2. The resulting 32-bit value is written to the extension and high parts of A0.
3. The value is zero padded, overwriting the whole 56-bit value of the accumulator.

5.4.3 Double-Width Data Transfers

Double-width transfers are 48-bit operations involving only whole accumulators, immediate values, and XY memory.

An immediate source operand is converted to 48 bits by sign extension. All LOAD.DW instructions that load an immediate value have a signed immediate operand.

An accumulator source operand's value is converted to 48 bits by passing it through the Round/Saturate unit. Only saturation is performed, however, and the extension part is discarded, leaving the high and low parts (48 bits) of the saturated value. The saturation operation functions according to the current saturation mode selected in SR, as described in Section 5.5, "Rounding and Saturation" on page 42.

If the destination is an accumulator, the 48-bit transferred or converted source value is stored in the high and low parts of the accumulator, and the Sign/Zero Extend unit sign extends the value upward from bit 47 through to bit 55.

A double-width register-to-register copy uses both the source and destination conversions:

```
nop ||| move.dw a0, b0     // copy b0 to a0, double-width
```

The following conversions are performed during the data transfer:

1. The value from B0 is converted to 48 bits by saturating it.
2. The resulting 48-bit value is written to the high and low parts of A0.
3. The value is sign extended, overwriting the whole 56-bit value of the accumulator.

5.5 ROUNDING AND SATURATION

The Round/Saturate unit is used when a full 56-bit accumulator is the source of a transfer. There are two Round/Saturate units to allow parallel transfers. The Rounding, Bias, and Saturation bits in SR (See Section 7.4, “Status Register” on page 53) control the behavior of both Round/Saturate units.

5.5.1 Rounding

Rounding is applied when a full 56-bit accumulator is the source of a single-width (24-bit) or extended (32-bit) transfer. For single-width transfers, rounding is applied before saturation.

Rounding is performed to eliminate the low part of the value from the accumulator. The Rounding and Bias bits in SR control the behavior of the rounding operation.

Rounding = 0, Bias = 0

Simple truncation. Round toward $-\infty$ to the nearest whole value of the high part and extension (i.e. both positive and negative values are rounded down).

- Discard the low part.

Rounding = 0, Bias = 1

Sign-magnitude truncation. Round toward 0 to the nearest whole value of the high part and extension (i.e. positive values are rounded down, and negative values are rounded up).

1. If the accumulator is negative and the low part is non-zero, add 1 to the value from the high part and extension.
2. Discard the low part.

Rounding = 1, Bias = 0

Simple biased rounding. Round toward the nearest whole value of the high part and extension, with a bias toward $+\infty$ (i.e. values at the mid-point between two whole values are rounded up).

1. If the low part is greater than or equal to 0x800000, add 1 to the value from the high part and extension unless this would cause them to overflow. If this would cause overflow, do not add 1.

2. Discard the low part.

Rounding = 1, Bias = 1

Convergent rounding. Round toward the nearest whole value of the high part and extension, with a bias toward even values (i.e. values at the mid-point between two whole values are rounded up if the high part is odd, and down if the high part is even).

1. If the low part is greater than 0x800000, add 1 to the value from the high part and extension unless this would cause them to overflow. If this would cause overflow, do not add 1.
2. If the low part is equal to 0x800000, and the high part is odd (bit 24 is 1), add 1 to the value from the high part and extension unless this would cause them to overflow. If this would cause overflow, do not add 1.
3. Discard the low part.

5.5.2 Saturation

Saturation is applied when a full 56-bit accumulator is the source of a single-width (24-bit) or double-width (48-bit) transfer. For single-width transfers, rounding is applied before saturation.

Saturation is performed to eliminate the extension part of the value from the accumulator. The Saturation bit in `SR` controls the behavior of the saturation operation.

Saturation = 0

Saturation is disabled. The extension part of the value from the accumulator is simply discarded, and the Saturated bit (Sat) in `SR` is not affected.

Saturation = 1

Saturation is enabled. If any bit of the extension part of the accumulator is not equal to bit 47 of the accumulator, then the Saturated bit (Sat) is set in `SR`, and the result is saturated to the maximum-magnitude fractional domain value with the same sign as the input, as shown in Table 10.

TABLE 10: SATURATED RESULT VALUES

Input Sign	Output Width	
	24 bits	48 bits
Bit 55 = 0	0x7FFFFF	0x7FFFFFFFFFFF
Bit 55 = 1	0x800000	0x800000000000

5.6 DATA MOVEMENT OPERATIONS

Table 11 provides a quick guide to the DMU data-movement operations. For detailed information about each instruction, see Chapter 10, “CFX Instruction Set” on page 83.

TABLE 11: MOVE OPERATIONS

Operation	Transfer Type	Instruction Types	Mnemonics
Load immediate	Single-width	Long, Single Move, X Move, Y Move	LOAD
Load immediate	Double-width	Long, Single Move	LOAD.DW
Register copy	Single-width	Single Move, X Move, Y Move	MOVE
Register copy	Extended	Single Move	MOVE.EX
Register copy	Double-width	Single Move	MOVE.DW
Clear X memory indirect	Single-width	Single Move	CLR
Store immediate to X direct	Single-width	Long	STORE
Store immediate to Y direct	Single-width	Long	STORE
Store immediate to P direct	Single-width	Long	STORE
Store immediate to XY direct	Double-width	Long	STORE.DW
Load/store to/from stack	Single-width	Long, Single Move	LOAD, STORE
Load/store to/from X direct	Single-width	Long	LOAD, STORE
Load/store to/from Y direct	Single-width	Long	LOAD, STORE
Load/store to/from P direct	Single-width	Long	LOAD, STORE
Load/store to/from P direct	Extended	Long	LOAD.EX, STORE.EX
Load/store to/from XY direct	Double-width	Long	LOAD.DW, STORE.DW
Load/store to/from X indirect	Single-width	Single Move, X Move	LOAD, STORE
Load/store to/from Y indirect	Single-width	Single Move, Y Move	LOAD, STORE
Load/store to/from P indirect	Single-width	Long	LOAD, STORE
Load/store to/from P indirect	Extended	Long, Single Move	LOAD.EX, STORE.EX
Load/store to/from XY indirect	Double-width	Single Move	LOAD.DW, STORE.DW

6 Address Generation Units

6.1 INTRODUCTION

This chapter describes the Address Generation Units (AGUs), including address registers (pointers) and addressing configuration registers which are used to configure the updates that are performed in some indirect addressing modes. Stack addressing and the arithmetic operations executed by the AGUs are also described.

6.2 AGU ARCHITECTURE

Two address generators are available to simultaneously generate and update two addresses in a single cycle. The X AGU generates addresses for X memory, XY memory, or P memory, while the Y AGU generates addresses for Y memory or P memory.

The X AGU contains the stack pointer register `SP` and 24 indirect addressing registers arranged in triples as shown in Table 12.

TABLE 12: X ALU INDIRECT ADDRESSING REGISTERS

Pointer	Step Register	Circular Configuration
XP0	XSTEP0	XCIRC0
XP1	XSTEP1	XCIRC1
XP2	XSTEP2	XCIRC2
XP3	XSTEP3	XCIRC3
XP4	XSTEP4	XCIRC4
XP5	XSTEP5	XCIRC5
XP6	XSTEP6	XCIRC6
XP7	XSTEP7	XCIRC7

Pointer registers are 16 bits wide. Step registers contain 16-bit signed two's complement values. The circular configuration registers are 15 bits wide. Step register `XSTEP`*N* and `XCIRC`*N* control the behavior of updates to `XP`*N*.

The X AGU manages the stack pointer `SP` since the stack is in X memory.

The Y AGU contains 12 indirect addressing registers arranged in triples as shown in Table 13.

TABLE 13: Y ALU INDIRECT ADDRESSING REGISTERS

Pointer	Step Register	Circular Configuration
YP0	YSTEP0	YCIRC0
YP1	YSTEP1	YCIRC1
YP2	YSTEP2	YCIRC2
YP3	YSTEP3	YCIRC3

Pointer registers are 16 bits wide. Step registers contain 16-bit signed two's complement values. The circular configuration registers are 15 bits wide. Step register YSTEP*N* and YCIRC*N* control the behavior of updates to YP*N*.

6.3 STACK ADDRESSING

The CFX architecture and instruction set provide hardware support for a single stack in X memory. A stack-frame model is supported with indirect-plus-offset addressing via the stack pointer. Negative short-immediate offsets in some addressing modes support stack growth towards higher memory addresses.

A CFX program should initialize the stack pointer SP to the low address of a RAM segment in XMEM that is reserved for the stack, at program start-up.

Note: Although CFX subroutine calls and interrupt vectoring do not *require* the use of the stack (see Section 7.5.3, "Calling Subroutines" on page 60 and Section 7.7, "Handling Interrupts" on page 71 for details), most subroutines and interrupt handlers *will* use the stack for saving the register state and allowing nested calls. Therefore, we recommend initializing SP very soon after your program starts.

6.3.1 Allocating and Releasing Stack Space

In typical usage, the stack pointer points to the next unused word at the top of the stack; that is, to the address one greater than the address of the item at the top of the stack. You can allocate *N* words of space on the stack by adding *N* to the stack pointer. Subtract *N* from SP to release the top *N* words from the stack. Memory accesses via the stack pointer do not have auto-increment or auto-decrement update modes, as is typically the case on machines that allocate stack space using PUSH and POP instructions.

The *Long* and *Single Move* instruction classes have instructions for adding a signed immediate value to the stack pointer to allocate and release a stack frame. Stack frames that are not large can be allocated and released with the 7-bit signed immediate value of the *Move* instruction, while the *Long* form has a 16-bit immediate value so that stack frame size is not limited.

6.3.2 Accessing Data on the Stack

You can access data on the stack, with an address relative to the top of the stack, using a LOAD or STORE instruction with an XMEM address of the form SP + offset or SP - offset. The *offset* from the stack pointer is usually negative since the stack grows upwards with SP pointing just above the top of the stack.

The SP-relative LOAD and STORE instructions have three forms:

- The *Long* instructions can load or store any register, using a 16-bit SP offset.
- *Single Move* load and store instructions can access accumulators and their sub-registers, XY data registers, X pointer registers, and the registers LR, SR, ILPC, and ILSR using a 6-bit negative offset (with range -64..-1) from SP.
- *Single Move* load and store instructions can also access Y pointer registers, the loop-stack registers LC, LSA, and LEA, and the stack pointer SP itself. This instruction form has only a 4-bit negative offset (range -16..-1) from SP, however.

Stack data can also be accessed with the other XMEM addressing forms:

- If the datum is at a known constant address in the stack—for instance, in the first stack frame at the bottom of the stack—you can use the direct addressing forms of the *Long* instructions, with their 16-bit immediate address specifying a location relative to the bottom of the stack.
- The XMEM indirect addressing forms described in Section 6.4, “Indirect Addressing” can address stack data. To load an X pointer with an address in the stack, provide it as the destination register in an ADD instruction with source operands SP and an immediate value. The *Long* form of this instruction allows a full 16-bit immediate value, while the *Single Move* instruction has a 6-bit negative immediate field (range -64..-1).

6.4 INDIRECT ADDRESSING

Indirect addressing is used to load values from X, Y, XY or P memory at an address found in an X or Y pointer. Indirect addressing modes allow the pointer to be automatically updated after the value is loaded from memory. Using indirect addressing, you can use X pointers to access values in X, XY or P memory, and Y pointers to access values in Y or P memory.

The following indirect addressing modes are supported by the X and Y AGUs for *Long* and *Single Move* instructions. The XP0 pointer is used for illustration, but you can use any X or Y pointer.

`xp0`	No update.
`xp0++`	Post-increment.
`xp0--`	Post-decrement.
`xp0 += 2`	Post-increment by 2.
`xp0 -= 2`	Post-decrement by 2.
`xp0 += xstep0`	Post-increment by the signed 16-bit value in the step register. The pointer and step register must match (XSTEP*N* for XP*N*, and YSTEP*N* for YP*N*).
`xp0 -= xstep0`	Post-decrement by the signed 16-bit value in the step register. The pointer and step register must match (XSTEP*N* for XP*N*, and YSTEP*N* for YP*N*).
`xp0 +%= xstep0`	Post-increment by the 16-bit value in the step register, and perform circular wrapping modulo the value in the XCIRC0 register. Depending on the value in XCIRC0, this mode will also perform bit-reversed addressing (Section 6.4.1, “Circular Updates” on page 48). The pointer and step register must match (XSTEP*N* for XP*N*, and YSTEP*N* for YP*N*). The circular

configuration register is selected implicitly (XCIRC*N* for XP*N*, and YCIRC*N* for YP*N*).

Version 3 or greater of the CFX assembler supports the following synonyms for addressing modes:

`xp0+` **and** `xp0 += 1` synonyms for `xp0++`

`xp0-` **and** `xp0 -= 1` synonyms for `xp0--`

`xp0+!` synonym for `xp0 += xstep0`

`xp0-!` synonym for `xp0 -= xstep0`

`xp0+%!` synonym for `xp0 +%= xstep0`

Dual Move instructions with indirect addressing have the following restrictions:

- In *X Move* instructions, only the first four X pointers (XP0 to XP3) can be used.
- Only four indirect addressing modes can be used:

```
xp0
xp0++
xp0 += xstep0
xp0 +%= xstep0
```

The updated pointer value is calculated in parallel with the instruction, and the pointer is modified at the end of the processor cycle. In *Move-Move* instructions, pointer modifications in the first *Move* instruction are committed before executing the second *Move* instruction in the next processor cycle. As a result, you can use indirect addressing to load two values from different memory locations using the same pointer register. For example, the following instruction loads two values from consecutive addresses in X memory.

```
load x0, x[xp0++] >>> load x1, x[xp0++]
```

Do not load a value into a pointer and update the same pointer during the same processor cycle. For example, the value in XP0 is undefined after executing the following instruction.

```
nop ||| load xp0, x[xp0++]      // xp0 value is undefined
```

You can safely store a value from a pointer and update the same pointer during the same processor cycle. The value stored is the value before the pointer modification.

```
load xp0, 0
nop ||| store x[xp0++], xp0    // store 0 at x[0], xp0 now 1
```

6.4.1 Circular Updates

The circular indirect addressing mode (`xp0 +%= xstep0`) supports accessing circular buffers, and bit-reversed memory buffer access. All other addressing modes ignore the circular configuration registers, so there is no need to clear these registers to perform regular pointer updates.

The circular buffer mode is determined by the high bit (bit 14) of the circular configuration register.

6.4.1.1 Circular Buffers (Bit 14=1)

Set the high bit of the circular configuration register to 1 to access circular buffers.

The lower 14 bits of the circular configuration register are the unsigned size of the buffer through which the pointer values cycle. The value must be at least 1, and at least as large as the magnitude of the step register's value.

The base of the circular buffer is determined by the pointer register and buffer size. If *N* is the smallest integer such that $2^N \geq size$, the base of the circular buffer is the current pointer value with the bottom *N* bits set to 0. The last address in the circular buffer is *base* + *size* - 1.

For example, for sizes between 5 and 8 inclusive, *N* is 3. If the pointer is 0x3A, then the base of the circular buffer is 0x3A AND 0xFFF8 = 0x38.

When the pointer is updated, the step is added to the current pointer value. If the step is non-negative and the new address is greater than the last address in the circular buffer, the buffer size is subtracted from result. If the step is negative and the new address is less than the base of the circular buffer, the buffer size is added to the result. The pointer is then updated with the new address.

6.4.1.2 Bit-Reversed Access (Bit 14=0)

Set the high bit of the circular configuration register to 0 to enable bit-reversed addressing. Bit-reversed addressing is typically used with a step that is a power of 2 to implement butterflies for algorithms like the FFT and Viterbi decoding.

The low 14 bits of the circular configuration register are ignored for bit-reversed addressing.

When the pointer is updated, these steps are followed to calculate the next address:

1. The pointer register is reversed (LSB becomes MSB and vice-versa).
2. The step register is reversed.
3. The two reversed numbers are added.
4. The result is reversed.

Consider the following example, using 4-bit values for simplicity. The start pointer value is `1000` and the step register value is `0100`. Repeated updates to the pointer will cycle through the following values:

```
1000 +%= 0100 = 1100
1100 +%= 0100 = 1010
1010 +%= 0100 = 1110
1110 +%= 0100 = 1001
1001 +%= 0100 = 1101
1101 +%= 0100 = 1011
1011 +%= 0100 = 1111
1111 +%= 0100 = 1000
```

6.5 ADDRESS ARITHMETIC OPERATIONS

Table 14 provides a quick guide to the address arithmetic operations which are executed by the AGUs. Other AGU operations execute in parallel with and provide addresses for DMU load and store operations. For detailed information about each instruction, see Chapter 10, “CFX Instruction Set” on page 83.

TABLE 14: ADDRESS ARITHMETIC OPERATIONS

Operation	Instruction Types	Mnemonics
Add immediate to stack pointer, result to X pointer	Long, Move	ADD
Subtract immediate from stack pointer, result to X pointer	Long, Move	SUB[1]
Add pointer and immediate	Long, Move	ADD
Subtract immediate from pointer	Long, Move	SUB[1]
Add pointer and step register	Move	ADD
Subtract step register from pointer	Move	SUBSTEP

1. These instructions require version 3 or greater of the CFX assembler.

7 Program Control Unit

7.1 INTRODUCTION

This chapter describes the Program Control Unit (PCU), which is the master controller in the CFX DSP core. To execute a program, the PCU reads and analyzes the instructions from P memory, generating control signals for all of the DSP's functional units.

7.2 PCU ARCHITECTURE

In its role as the program controller, the PCU's main logic blocks relate to analysis and control of program flow:

- The program decoder reads and decodes the program and the pipeline manager pipelines its execution
- The hardware loop controller supports looping with no control-flow overhead
- The interrupt/subroutine controller deals with subroutine call/return and interrupt handling

These blocks use several sets of registers. Some are visible directly or indirectly to other parts of the system and to the programmer, while others are internal to the PCU.

Program Counter

The `PC` contains the address of the instruction that is currently executing. The pipeline manager maintains three copies of the `PC` that track the addresses of the instructions in the stages of the pipeline. These are updated accordingly for changes in program flow due to branches, subroutines, loops, and interrupts. The `PC` is not accessible directly; it is modified using control-flow instructions.

When used in this chapter and the instruction descriptions in Chapter 10, "CFX Instruction Set" on page 83 and Appendix A, "Instruction Set Summary" on page 373, `PC` refers to the address, in program memory, of the currently executing instruction.

Status Register and Interrupt-Link Status Register

The `SR` is a 16-bit register containing flag bits indicating the status of *Arithmetic* instructions and data-movement conversions, and mode bits to control the behavior of some functional units. A copy of the `SR` is saved in the `ILSR` as part of the interrupt-handling process; `SR` is restored from `ILSR` when returning from an interrupt. The program can directly access both registers.

Link Register and Interrupt-Link Program Counter

When a subroutine is called, the PCU saves the return address into `LR`. The PCU saves the address to which control will return (after handling an interrupt) into a separate register, `ILPC`, so that interrupts can occur while a subroutine is executing. The program can directly access both registers.

Loop Stack Registers

The hardware loop controller maintains a stack of register triplets that describe the currently-executing loops. The stack is four triplets deep, so hardware loops can be nested to four levels. The controller maintains a loop stack pointer that is not visible directly to the programmer, and the loop stack register triplets are not accessible either. However, the most-current loop's parameters *are* visible through the `LC`, `LSA`, and `LEA` register names, which are aliases for the loop register triplet at the top of the stack.

The main input to the PCU is the program instruction stream from P memory, which the PCU reads using the dedicated Instruction Bus. When accessing the P memory data space, programs use the P memory bus (see Section 8.2, "Memory Architecture" on page 79) to avoid contention with reading the instruction stream.

External interrupt signalling provides another input to the PCU; specifically to the interrupt controller. The PCU also requires access to the registers managed by other functional units, to perform indirect branches and subroutine calls, for example, and to use data registers for loop counts.

The PCU has many control outputs, but also generates some values from the instruction stream that are used by other blocks in the core:

- **Memory addresses** from direct `LOAD` and `STORE` instructions are routed to the memory address buses.
- **Stack offsets** from accesses to the stack in X memory via the stack pointer are routed to the X AGU to generate the address for the stack data access.
- **Immediate values** are routed to the appropriate execution unit (depending on the instruction type) for use as data operands.

7.3 EXECUTION PIPELINE

The execution pipeline has three stages: *fetch*, *decode*, and *execute*. Straight-line execution of single-cycle (*Long* and *Arithmetic-Move*) instructions is shown in Figure 2. Each column indicates the address of the instruction being fetched, decoded or executed at each clock cycle.

Clock Cycle	Fetch	Decode	Execute
n	**i**	i-1	i-2
n+1	i+1	**i**	i-1
n+2	i+2	i+1	**i**
n+3	i+3	i+2	i+1

Figure 2: Execution of Single-Cycle Instructions

Move-Move instructions take two cycles to execute, as shown in Figure 3.

Clock Cycle	Fetch	Decode	Execute
n	**i**	i-1	i-2
n+1	i+1	**i**	i-1
n+2	i+2	i+1	**i (First Move)**
n+3	i+2	i+1	**i (Second Move)**
n+4	i+3	i+2	i+1

Figure 3: Execution of a Move-Move Instruction

The ALU part of a multiplication instruction (both the *multiply* and *multiply-accumulate* forms) executes in parallel with the execution of the next instruction. These instructions do not interrupt the pipeline flow, as shown in Figure 4.

Clock Cycle	Fetch	Decode	Execute
n	**i**	i-1	i-2
n+1	i+1	**i**	i-1
n+2	i+2	i+1	**i (Multiply)**
n+3	i+3	i+2	**i (Mult-ALU)**, i+1

Figure 4: Execution of Multiplication Instructions

In all cases, instructions enter the execution pipeline at least two clock cycles before they are executed. Instructions that control program flow (`GOTO`, `CALL`, `RETURN` and `RETURNI`) include delay slots to avoid stalling the pipeline. The arrangement of instructions in the delay slots, and the flow of instructions through the pipeline are described in Section 7.5, “Controlling Program Flow” on page 55.

7.4 STATUS REGISTER

The status register `SR` contains status flags which indicate the results of *Arithmetic* instructions and conversions done in data-movement operations, as well as mode bits which are used to control or configure some aspects of the core’s behavior.

Due to concurrent execution of multiple arithmetic and data-movement instructions, it is possible that part or all of the `SR` is updated by multiple instructions, simultaneously. Such updates are well-defined. The behavior and several scenarios in which such updates can occur are detailed in Section 7.8, “Restrictions on Concurrent Register Accesses” on page 74.

The flags and mode bits are named and used as follows:

Rounding Mode (bit 0)

Enables rounding when 1. When it is 0, 56-bit accumulator values are converted to 24 bits or 32 bits by truncating the low part. See the Bias mode bit below, and Section 5.5, “Rounding and Saturation” on page 42.

Bias Mode (bit 1)

Controls bias when rounding (if Rounding is 1) or when truncating (if Rounding is 0). This bit is *inverted*; when it is 0, rounding or truncation is biased. Setting the Bias bit to 1 removes bias by using convergent rounding (if Rounding is 1) or symmetric sign-magnitude truncation (if Rounding is 0). See Section 5.5, “Rounding and Saturation” on page 42.

IE: Interrupt Enable (bit 2)

Global interrupt enable (when 1). This bit is cleared to 0 after the `SR` is saved to `ILSR` during the interrupt vectoring operation.

X Carry Flag (bit 3)

The carry out from the X ALU for the last *Arithmetic* or *X Arithmetic* instruction that updated this status flag.

Note: This flag is *not* complemented by comparison or subtraction operations; for these operations, a borrow occurred if Carry = 0.

X Negative Flag (bit 4)

Bit 55 of the X ALU result for the last *Arithmetic* or *X Arithmetic* instruction that updated this status flag.

X Zero Flag (bit 5)

True if all 56 bits were 0 in the X ALU result of the last *Arithmetic* or *X Arithmetic* instruction that updated this status flag.

X Overflow Flag (bit 6)

True if a signed overflow occurred in the X ALU result of the last *Arithmetic* or *X Arithmetic* instruction that updated this status flag.

Note: This flag is *not* sticky. 0 indicates that no signed overflow occurred.

Y Carry Flag (bit 7)

The carry out from the Y ALU for the last *Y Arithmetic* instruction that updated this status flag.

Note: This flag is *not* complemented by comparison or subtraction operations; for these operations, a borrow occurred if Carry = 0.

Y Negative Flag (bit 8)

Bit 55 of the Y ALU result for the last *Y Arithmetic* instruction that updated this status flag.

Y Zero Flag (bit 9)

True if all 56 bits were 0 in the Y ALU result of the last *Y Arithmetic* instruction that updated this status flag.

Y Overflow Flag (bit 10)

True if a signed overflow occurred in the Y ALU result of the last *Y Arithmetic* instruction that updated this status flag.

Note: This flag is *not* sticky. 0 indicates that no signed overflow occurred.

Saturation Mode (bit 11)

Enables saturation in both round/saturate units when set to 1. This affects data-movement instructions in the *Long* and *Move* classes.

X Extension In Use Flag (bit 12)

True if one or more of the extension bits (the high 8 bits) were different from the fractional sign bit (bit 47) in the X ALU result of the last *Arithmetic* or *X Arithmetic* instruction that updated this status flag.

Y Extension In Use Flag (bit 13)

True if one or more of the extension bits (the high 8 bits) were different from the fractional sign bit (bit 47) in the Y ALU result of the last *Y Arithmetic* instruction that updated this status flag.

Saturated Flag (bit 14)

True if saturation occurred when moving a whole accumulator to a 24-bit or 48-bit destination during a *Long* or *Move* instruction. This flag is *sticky*; it is not cleared when saturation does not occur.

FUP: Flag Updates Mode (bit 15)

When the Flag Updates mode is enabled (bit 15 is 1), most unconditional Arithmetic-class instructions update the status flags for the ALU on which the instruction executed.

When Flag Updates mode is disabled, only the `CMP*`, `BITTST`, and `DIVST` instructions update the status flags.

The Saturated flag is not affected by the Flag Updates mode.

7.5 CONTROLLING PROGRAM FLOW

Control-flow instructions, including branching and subroutine instructions, interrupt the execution pipeline by changing the program counter, causing instructions to be fetched from a new address:

- These instructions have *delay slots* which are described in Section 7.5.1, “Delay Slots” on page 56.
- Section 7.5.2, “Branching” on page 56 describes the execution of branching instructions and the behavior of the execution pipeline with and without delay slots.
- Section 7.5.3, “Calling Subroutines” on page 60 describes the execution of subroutine instructions and the behavior of the execution pipeline with and without delay slots.
- Section 7.5.5, “Program Flow Delay Slot Restrictions” on page 62 describes restrictions placed on the instructions that can be executed in delay slots.
- The `SLEEP` instruction also affects the execution pipeline, and is described in Section 7.5.4, “Sleep Instructions” on page 61.
- Hardware loops and interrupts also affect the program counter and execution pipeline. They are described separately in Section 7.6, “Hardware Loops” on page 63 and Section 7.7, “Handling Interrupts” on page 71.

All control-flow instructions are *Long* instructions.

7.5.1 Delay Slots

When a control-flow instruction executes, instructions after the control-flow instruction are already in the fetch and decode stages of the execution pipeline. The pipeline controller can flush these instructions, causing the pipeline to stall until new instructions can be fetched and decoded. Alternatively, these instructions can be executed while the new instructions are fetched and decoded. This is called executing *delay slots*.

The number of delay slots available following a control-flow instruction is counted in clock cycles. If an instruction has one delay slot, you must fill this with a *Long* or *Arithmetic-Move* instruction. If an instruction has two delay slots, you can fill these with two *Long* or *Arithmetic-Move* instructions, or one *Move-Move* instruction. Further restrictions are placed on the instructions that can be used to fill delay slots. These restrictions are described in Section 7.5.5, "Program Flow Delay Slot Restrictions" on page 62.

An instruction with delay slots and the instructions in the delay slots are executed atomically. If an interrupt is received while the branch instruction or instructions in the delay slots are executing, the interrupt vectoring operation does not occur until the instructions in the delay slots finish. Similarly, the debugger steps over a branch instruction and all instructions in its delay slots with a single step, and you cannot set a breakpoint on an instruction in a delay slot.

7.5.2 Branching

Branching instructions affect the control flow by changing the program counter to a new address and subsequently changing the location from which instructions are fetched, decoded, and executed. Branching instructions come in six forms, listed in Table 15. For detailed information about each instruction, see Chapter 10, "CFX Instruction Set" on page 83.

TABLE 15: BRANCHING INSTRUCTIONS

Operation	Instructions
Unconditional branch to immediate address	GOTO.0D imm16u GOTO.1D imm16u GOTO imm16u
Conditional branch to immediate address	GOTO.0D imm16u, cond5 GOTO.2D imm16u, cond5 GOTO imm16u, cond5
Unconditional branch to indirect address (X Pointer)	GOTO.2D xp-reg GOTO xp-reg
Conditional branch to indirect address (X Pointer)	GOTO.2D xp-reg, cond5 GOTO xp-reg, cond5

First consider the forms with two delay slots (those ending with **`.2D`**). These instructions take 3 clock cycles to execute, but two clock cycles are delay slots used to execute one or two subsequent instructions. If the next two instructions are both *Long* or *Arithmetic-Move* instructions, then two instructions are executed in the delay slots as shown in Figure 5 (*k* is the target address of the branch).

Clock Cycle	Fetch	Decode	Execute
n	**i (Branch)**	i-1	i-2
n+1	i+1	**i (Branch)**	i-1
n+2	i+2	i+1	**i (Branch)**
n+3	k	i+2	i+1 **(Delay Slot)**
n+4	k+1	k	i+2 **(Delay Slot)**
n+5	k+2	k+1	k

Figure 5: Branch with Two Delay Slots Filled with Long or Arithmetic-Move Instructions

If the next instruction is a *Move-Move* instruction, then this instruction requires both delay slots to execute, as shown in Figure 6. The second instruction after the branch instruction is fetched but not executed.

Clock Cycle	Fetch	Decode	Execute
n	**i (Branch)**	i-1	i-2
n+1	i+1	**i (Branch)**	i-1
n+2	i+2	i+1	**i (Branch)**
n+3	k	i+2	i+1 (First Move) **(Delay Slot)**
n+4	k+1	k	i+1 (Second Move) **(Delay Slot)**
n+5	k+2	k+1	k

Figure 6: Branch with Two Delay Slots Filled with One Move-Move Instruction

The CFX performs pre-decoding of unconditional, immediate branch instructions during the fetch stage. Because the branch is always taken and the target address is available in the instruction word, the pipeline can begin fetching instructions from the target address one cycle early, before the branch instruction reaches the execute stage. As a result, unconditional, immediate branch instructions can have one delay slot that you must fill with a *Long* or *Arithmetic-Move* instruction, as shown in Figure 7.

Clock Cycle	Fetch	Decode	Execute
n	**i (Branch)**	i-1	i-2
n+1	i+1	**i (Branch)**	i-1
n+2	k	i+1	**i (Branch)**
n+3	k+1	k	i+1 **(Delay Slot)**
n+4	k+2	k+1	k

Figure 7: Branch with One Delay Slot

The immediate branch instructions have a second form with no delay slots (`GOTO.0D`). These instructions do not execute instructions after the branch is taken; instead, they stall the pipeline

for 1 or 2 clock cycles to fetch and decode instructions from the target address, as shown in Figure 8 for unconditional branches, and Figure 9 for conditional branches.

Clock Cycle	Fetch	Decode	Execute
n	**i (Branch)**	i-1	i-2
n+1	i+1	**i (Branch)**	i-1
n+2	k	i+1	**i (Branch)**
n+3	k+1	k	— **(Delay Slot)**
n+4	k+2	k+1	k

Figure 8: No-Delay Unconditional Branch

Clock Cycle	Fetch	Decode	Execute
n	**i (Branch)**	i-1	i-2
n+1	i+1	**i (Branch)**	i-1
n+2	i+2	i+1	**i (Branch)**
n+3	k	—	— **(Delay Slot)**
n+4	k+1	k	— **(Delay Slot)**
n+5	k+2	k+1	k

Figure 9: No-Delay Conditional Branch Taken

A conditional no-delay branch not taken does not cause a pipeline stall. The pipeline continues to execute the instructions that were already fetched and decoded before the branch instruction, as shown in Figure 10.

Clock Cycle	Fetch	Decode	Execute
n	**i (Branch)**	i-1	i-2
n+1	i+1	**i (Branch)**	i-1
n+2	i+2	i+1	**i (Branch)**
n+3	i+3	i+2	i+1
n+4	i+4	i+3	i+2

Figure 10: No-Delay Conditional Branch Not Taken

Use the no-delay forms of the branch instructions when you cannot find useful instructions to insert into the delay slots. Using these instructions is similar to filling the delay slots with `NOP` instructions, except that you save program memory words.

7.5.2.1 Conditional Branches

Conditional branch instructions cause a branch if a condition is true. If the condition is false, then execution continues with instructions after the branch instruction in program memory.

When executing a conditional branch instruction with delay slots (GOTO.2D), the instructions in the delay slots are always executed atomically with the branch instruction, even if the branch is not taken (the condition is not true).

The 32 possible condition codes for conditional branches are described in Table 16.

TABLE 16: BRANCH INSTRUCTION CONDITIONS

Condition	Meaning	Logic Expression
CARRY or X_CARRY	Carry	X Carry
NO_CARRY or X_NO_CARRY	No carry	NOT (X Carry)
EQ, X_EQ, ZERO or X_ZERO	Equal, or zero	X Zero
NE, X_NE, NOT_ZERO or X_NOT_ZERO	Not equal, or not zero	NOT (X Zero)
LT, X_LT, LTZ[1] or X_LTZ[1]	Less than	X Negative XOR X Overflow
LE, X_LE, LEZ[1] or X_LEZ[1]	Less than or equal	X Zero OR (X Negative XOR X Overflow)
GE, X_GE, GEZ[1] or X_GEZ[1]	Greater than or equal	NOT (X Negative XOR X Overflow)
GT, X_GT, GTZ[1] or X_GTZ[1]	Greater than	NOT (X Zero OR (X Negative XOR X Overflow))
EXTEN or X_EXTEN	Extension in Use	X Extension In Use
NO_EXTEN or X_NO_EXTEN	Extension not in use	NOT (X Extension In Use)
OFLOW or X_OFLOW	Overflow	X Overflow
NO_OFLOW or X_NO_OFLOW	No overflow	NOT (X Overflow)
FULL	Loop stack is full	Loop stack depth = 4
NOT_FULL	Loop stack is not full	Loop stack depth < 4
EMPTY	Loop stack is empty	Loop stack depth = 0
NOT_EMPTY	Loop stack is not empty	Loop stack depth > 0
Y_CARRY	Carry	Y Carry
Y_NO_CARRY	No carry	NOT (Y Carry)
Y_EQ or Y_ZERO	Equal, or zero	Y Zero
Y_NE or Y_NOT_ZERO	Not equal, or not zero	NOT (Y Zero)
Y_LT or Y_LTZ[1]	Less than	Y Negative XOR Y Overflow
Y_LE or Y_LEZ[1]	Less than or equal	Y Zero OR (Y Negative XOR Y Overflow)
Y_GE or Y_GEZ[1]	Greater than or equal	NOT (Y Negative XOR Y Overflow)
Y_GT or Y_GTZ[1]	Greater than	NOT (Y Zero OR (Y Negative XOR Y Overflow))
Y_EXTEN	Extension in Use	Y Extension In Use
Y_NO_EXTEN	Extension not in use	NOT (Y Extension In Use)
Y_OFLOW	Overflow	Y Overflow
Y_NO_OFLOW	No overflow	NOT (Y Overflow)
SAT	Saturation has occurred	Saturated
NO_SAT	Saturation has not occurred	NOT (Saturated)

TABLE 16: BRANCH INSTRUCTION CONDITIONS (CONTINUED)

Condition	Meaning	Logic Expression
FALSE or 0[1]	False	False (Branch is never taken)
TRUE or 1[1]	True	True (Branch is always taken)

1. This condition requires version 3 or greater of the CFX assembler.

7.5.3 Calling Subroutines

Subroutine call and return instructions modify the PC and affect the execution pipeline similar to branching instructions, but also interact with the Link Register LR to support calling and returning from subroutines.

Invoke a subroutine using one of the subroutine call instructions listed in Table 17. For detailed information about each instruction, see Chapter 10, "CFX Instruction Set" on page 83.

TABLE 17: CALL INSTRUCTIONS

Operation	Instruction
Call subroutine at immediate address	CALL.1D imm16u CALL imm16u
Call subroutine at indirect address (X Pointer)	CALL.2D xp-reg CALL xp-reg

Call instructions store the return address in LR. The return address is calculated relative to PC when the call instruction is executed, and depends on the number of delay slots and how they are filled. This is described in Table 18.

Note: To support nested subroutines (one subroutine calling another), your subroutine must save the contents of LR, typically on the stack.

TABLE 18: RETURN ADDRESS CALCULATION FOR CALL INSTRUCTIONS

Call Instruction	Instructions in Delay Slot(s)	Return Address
CALL.1D imm16u	One *Long* or *Arithmetic-Move* instruction	PC + 2
CALL.2D xp-reg	Two *Long* or *Arithmetic-Move* instructions	PC + 3
CALL.2D xp-reg	One *Move-Move* instruction	PC + 2

Call instructions have one delay slot (with an immediate target address) or two delay slots (if the target address is taken from an X pointer). The execution pipeline behavior is the same as the unconditional branch instructions, shown in Figure 5, Figure 6 and Figure 7. As with unconditional, immediate branches, the CFX performs pre-decoding of immediate call instructions during the fetch stage, so the pipeline can begin fetching instructions from the target address one cycle early, before the call instruction reaches the execute stage.

Use the RETURN.2D instruction to return from a subroutine. This instruction restores the return address from LR to PC. If you have stored the contents of LR to call another subroutine, you must restore it before executing the return instruction. The execution pipeline behavior is the same as the unconditional, indirect branch instruction, with the branch target taken from LR (see Figure 5 on page 57 and Figure 6 on page 57). The RETURN.2D instruction has two delay slots.

Table 19 provides a quick guide to the return instruction. For detailed information about this instruction, see Chapter 10, "CFX Instruction Set" on page 83.

TABLE 19: RETURN INSTRUCTION

Operation	Instruction
Return from subroutine	RETURN.2D RETURN

7.5.4 Sleep Instructions

The SLEEP instruction puts the CFX into a low-power mode with most of the gated clocks turned off. The CFX stops executing instructions until an enabled interrupt is received. Sleep mode is described in Section 9.3, "Sleep Mode" on page 81.

The two forms of the SLEEP instruction are listed in Table 20. The difference is the address where execution resumes after the CFX leaves Sleep mode. This can be the address following the SLEEP instruction, or an address specified in the SLEEP instruction. For detailed information about each instruction, see Chapter 10, "CFX Instruction Set" on page 83.

TABLE 20: SLEEP INSTRUCTIONS

Operation	Instruction
Sleep. Resume at next address.	SLEEP
Sleep. Resume at specified address.	SLEEP imm16u

When the CFX exits Sleep mode, an enabled interrupt must be pending, so the CFX immediately performs the interrupt vectoring operation described in Section 7.7, "Handling Interrupts" on page 71. The address stored in the interrupt-link program counter (ILPC) is determined by the form of the SLEEP instruction. If the SLEEP instruction specifies a restart address, this address is stored in ILPC. Otherwise, the address of the instruction after the SLEEP instruction is stored in ILPC.

The SLEEP instruction executes in 3 clock cycles, and has no delay slots, as shown in Figure 11. When the CFX exits sleep mode, the first two cycles are spent fetching and decoding the instructions at the interrupt vector address, *k* in the figure.

Clock Cycle	Fetch	Decode	Execute
n	**i (Sleep)**	i-1	i-2
n+1	i+1	**i (Sleep)**	i-1
n+2	i+2	i+1	**i (Sleep)**
n+3	i+3	—	**i (Sleep)**
n+4	i+4	—	**i (Sleep)**
…	Sleep Mode		
m	k	—	—
m+1	k+1	k	—
m+2	k+2	k+1	k

Figure 11: Sleep Pipeline

7.5.5 Program Flow Delay Slot Restrictions

Not all instructions are allowed in delay slots. The instructions in delay slots must execute completely before the end of the delay slots. Multiplication instructions are an exception to this restriction *if* the instruction that will execute in the following cycle satisfies the multiplication restrictions described in Section 4.8.3, “Timing, Concurrency, and Restrictions” on page 30. For a multiplication in the last delay slot of a conditional `GOTO.2D`, the multiplication restrictions must be satisfied for both the instruction at the `GOTO.2D` target address and the instruction following the delay slots.

If an instruction has one delay slot, it can be filled with:

- Any non-control-flow single-cycle *Long* instruction
- Any *Arithmetic-Move* instruction; if the *Arithmetic* part involves a multiplication, the instruction at the control-flow target address must satisfy the multiplication restrictions

If an instruction has two delay slots, they can be filled with:

- Any two *Arithmetic-Move* or non-control-flow single-cycle *Long* instructions; if the second instruction is an *Arithmetic-Move* instruction involving a multiplication, the instruction that will execute in the following cycle must satisfy the multiplication restrictions
- A *Move-Move* instruction (all *Move-Move* instructions use two cycles)

In particular, the following instructions *cannot* be used to fill any delay slot:

- Any multi-cycle *Long* instruction
- Any control-flow instruction

In the last delay slot, you must be aware that:

- Two-cycle *Move-Move* instructions cannot be used
- For any *Arithmetic-Move* instruction involving one or two multiplication instructions, the multiplication restrictions must be satisfied for the instruction that will execute in the following cycle

7.6 HARDWARE LOOPS

The CFX hardware loops provide efficient zero-overhead looping, nested up to four levels deep.

7.6.1 Hardware Looping Instructions

The PCU maintains a stack of up to four nested hardware loops. Start a hardware loop with a `LOOP.2D` instruction, which pushes the *loop count*, *loop start address*, and *loop end address* for a new loop on top of the hardware loop stack.

Loop count The loop count is the number of times that the body of the loop will be executed.

The loop count is a 12-bit integer, and can be an immediate value or a value from a data register. When the loop count is taken from an XY data register, only the low 12 bits are used. When the loop count is taken from an accumulator, the low 12 bits of the high part of the accumulator are used.

If the loop count is between 1 and 4095, inclusive, it indicates the number of times that the body of the loop will be executed. If the loop count is 0, the body of the loop is executed 4096 times.

Loop start address The loop start address is the address of the first instruction in the body of the loop.

Loop instructions have two delay slots. The rules for filling these delay slots are the same as for other control flow instructions, and are summarized in Section 7.5.5, “Program Flow Delay Slot Restrictions” on page 62. The loop start address is the first address *after* the instructions in the delay slots. The instructions in the delay slots are only executed the first time through the loop.

Loop end address The loop end address is the address of the last instruction in the body of the loop.

The loop end address is specified in the instruction word.

IMPORTANT: For nested hardware loops, no two hardware loops can have the same loop end address.

When an instruction is fetched, the CFX compares the address of the newly fetched instruction to the loop end address of the top entry on the loop stack. If an instruction is fetched from the loop end address, the CFX performs the following actions:

1. Decrement the loop count for the top entry on the loop stack.
2. If the new loop count is not zero, store the new loop count and change the address to fetch the next instruction to the loop start address for the top entry on the loop stack; the instruction at the loop start address is the next instruction fetched.
3. If the new loop count is zero, pop the top entry from the loop stack, and continue fetching and executing instructions normally.

Use the ENDLOOP instruction to pop the top entry from the loop stack. This instruction can be used to implement loop "breaks" by following the ENDLOOP instruction with a control-flow instruction whose target is outside the loop.

Interrupt handlers with hardware loops share the loop stack with non-interrupt code. You must take this into account when designing code structures with nested hardware loops to avoid overflowing the loop stack. The depth of the loop stack is not directly accessible, but you can use the conditions FULL, NOT_FULL, EMPTY and NOT_EMPTY to perform conditional branches based on the depth of the loop stack. When the hardware loop stack is full, the LOOP.2D instruction is ignored. Similarly, when the loop stack is empty, the ENDLOOP instruction is ignored.

Table 21 summarizes the hardware loop instructions. For detailed information about each instruction, see Chapter 10, "CFX Instruction Set" on page 83.

TABLE 21: LOOP INSTRUCTIONS

Operation	Instruction
Start a hardware loop with an immediate loop count.	LOOP.2D imm12u, imm16u LOOP imm12u, imm16u
Start a hardware loop with a count from a data register.	LOOP.2D d-reg, imm16u LOOP d-reg, imm16u
Terminate a hardware loop before its loop count reaches zero.	ENDLOOP

7.6.2 Hardware Loop Modification Restrictions

The PCU pipelines the decrement, test, and instruction prefetch steps that occur at the end of a hardware loop. These operations occur in parallel with the last instructions of the loop body so the next iteration of the loop can begin immediately after the last instruction of the current iteration.

During these pipelined looping operations, CFX instructions must not modify the hardware loop stack. This restriction applies to the last two instruction words of a hardware loop, and the third-last instruction word is also restricted unless one or both of the following are true:

- The second-last instruction word is a *Move-Move* instruction.
- The third-last instruction word is a *Move-Move* instruction and the loop-stack modification occurs in the first *Move* instruction.

The CFX instructions that modify the hardware loop stack (and thus are affected by these restrictions) are described in the following sections with a reference to this section.

7.6.3 Instruction Restrictions in Hardware Loops

The delay slots of LOOP.2D have the same restrictions as other control-flow instructions, as described in Section 7.5.5, "Program Flow Delay Slot Restrictions" on page 62. However, since the program flow does not immediately change (control flows straight through the delay slots into the first loop iteration), it is straightforward to ensure that the multiplication restrictions are satisfied if the last delay slot involves a multiplication because the cycle following the

multiplication is simply the next instruction of the program: the first instruction of the hardware loop.

Further restrictions on the instructions that you can use in a hardware loop are as follows:

- The last **two** instruction words of a hardware loop cannot be any of these *Long* instructions:

```
CALL imm16u
CALL.1D imm16u
GOTO imm16u
GOTO.0D imm16u
GOTO.1D imm16u
INTERRUPT imm5u
```

  CALL is synthesized as CALL.1D followed by NOP so it is two words.

- The last **three** instruction words of a hardware loop cannot be any of these *Long* instructions:

```
CALL xp-reg
CALL.2D xp-reg
GOTO imm16u, cond5
GOTO.0D imm16u, cond5
GOTO.2D imm16u, cond5
GOTO xp-reg
GOTO.2D xp-reg
GOTO xp-reg, cond5
GOTO.2D xp-reg, cond5
RETURN
RETURN.2D
RETURNI
RETURNI.0D
RETURNI.2D
```

  CALL, GOTO indirect and RETURN are synthesized as the corresponding **.2D** form followed by *Move-Move* NOPs, so they are two words.

- The last **two or three** instruction words of a hardware loop cannot be any of the hardware looping instructions. Section 7.6.2, “Hardware Loop Modification Restrictions” on page 64 specifies whether the last two words or the last three words are affected. The restricted instructions are:

```
ENDLOOP
LOOP imm12u, imm16u
LOOP.2D imm12u, imm16u
LOOP d-reg, imm16u
LOOP.2D d-reg, imm16u
```

  LOOP is synthesized as the corresponding LOOP.2D form followed by *Move-Move* NOPs, so it is two words.

- If the last instruction word in the loop is an *Arithmetic-Move* instruction involving one or two multiplication instructions, the multiplication restrictions described in Section 4.8.3, “Timing, Concurrency, and Restrictions” on page 30 must be satisfied for both the first instruction in the hardware loop and the instruction after the loop.

7.6.4 Hardware Loop Stack Registers

You can access the loop count, loop start address and loop end address of the top loop on the loop stack (the most-nested, currently-executing loop) using the LC, LSA and LEA registers, respectively. Table 22 summarizes the basic characteristics of these registers.

TABLE 22: LOOP STACK REGISTERS

Register	Width	Signedness
LC	12 bits	unsigned
LSA	16 bits	unsigned
LEA	24 bits[1]	signed

1. Although LEA is an address register, it has more than 16 bits and is signed for reasons that are explained in "Reading LEA" and "Writing LEA" below.

The following paragraphs describe the behavior of accesses to the loop stack registers.

Reading LC

The value of the LC register is the number of iterations remaining, including the current iteration. During the first iteration, the value of LC is the count given in the LOOP.2D instruction that initiated the loop, so it is zero if the count was zero, although 4096 iterations will be performed.

If LC is read in the last two instruction words of the loop, the effects of the pipelined implementation of the hardware loop controller are visible. The controller "executes ahead" to prefetch code from the appropriate addresses in parallel with the loop body's execution; thus LC is already modified in the last two instruction words of the loop. If the second-last instruction word in the loop is a *Move-Move* instruction, only the *Move* instruction in the second cycle will read the modified value for LC.

The loop controller's modification of LC depends on the current loop context. The effect on the value of LC in the last two instruction words of the loop is as follows:

- In most loop iterations—other than the special cases described by the following points—LC's value is decremented.
- In the last iteration of a nested loop, the loop controller pops the current loop from the loop stack, so LC's value is the count from the loop in which the current loop is nested.
- In the last iteration of a loop which is *not* nested, the loop controller pops the loop from the loop stack as usual. The loop stack is then empty, so the LC register continues to refer to the now-unused loop-stack entry for the soon-to-be-finished current loop. Thus LC's value is 1 since the loop count is not written back to the loop stack when it is decremented from 1 to 0.
- If the loop body contains only one instruction word, that instruction is both the last and the second-last instruction in the loop, due to the loop controller's pipelining. Thus if the loop-body instruction is a *Long* or *Arithmetic-Move* instruction, LC's value in that instruction has been

modified twice by the loop controller: once for the current loop iteration and once for the following iteration. If the loop-body instruction is a *Move-Move* instruction, LC's value in the first *Move* instruction is the once-modified value, and LC's value in the second *Move* instruction is twice modified.

Writing LC

You can change the number of iterations remaining in the current loop by writing a new value into the LC register. Writing the value 1 means that this iteration is the last; the loop will exit at the end of this iteration. Writing the value 0 has the same effect as an initial loop count of 0: the DSP executes 4096 more iterations (including the current iteration) of the loop.

The new count is not used by the loop controller in this loop iteration if it is written by an instruction that is restricted for loop-stack modifications, as described in Section 7.6.2, "Hardware Loop Modification Restrictions" on page 64.

Changing the number of iterations remaining in a loop during the loop's execution is an unusual operation; more typically, a new value is written into LC as part of a system-level operation to execute a new loop, as described in Section 7.6.4.1, "Using the Loop Registers" on page 69.

Reading LSA

The loop start address does not change in each loop iteration, so the LSA register contains the same address in each loop iteration.

As when reading the LC register, the value of LSA in the last two instruction words of the loop is the "next" value; in the second-last instruction word, only the second *Move* of a *Move-Move* instruction reads the "next" value. The special cases described for reading LC—in the last iteration of a loop and if the loop body is only one instruction—apply to reading LSA.

Writing LSA

You can change the start address of the loop—that is, the address to which control flows if the loop count is not decremented to zero at the end of the loop—by writing the new loop start address into the LSA register. The new start address is not used by the loop controller in this loop iteration when it is written by an instruction that is restricted for loop-stack modifications, as described in Section 7.6.2, "Hardware Loop Modification Restrictions" on page 64.

As with writes to LC, writes to LSA are usually part of a system-level operation to execute a new loop, as described in Section 7.6.4.1, "Using the Loop Registers" on page 69.

Reading LEA

You can read the loop end address of the current loop using the LEA register. As when reading the LC register, the value of LEA in the last two instruction words of the loop is the "next" value; in the second-last instruction word, only the second *Move* of a *Move-Move* instruction reads the "next" value. The special cases described for reading LC—in the last iteration of a loop and if the loop body is only one instruction—apply to reading LEA.

In addition to the basic functionality of accessing the end address of the loop at the top of the loop stack, read accesses to LEA can be used for loop stack management, as follows:

- Although reading LC or LSA when the loop stack is empty simply produces the value from the last loop that occupied the bottom loop-stack entry, reading LEA when the loop stack is empty results in the special 24-bit value 0x800000, which is distinct from any 16-bit loop end address.
- If the loop stack is *not* empty, reading LEA has a side effect: after the end address of the loop at the top of the stack is retrieved, the loop at the top of the loop stack is popped off of the loop stack. If the popped loop was nested, the enclosing loop becomes the new current, top-of-stack loop and subsequent accesses to LC, LSA and LEA access that loop's parameters; otherwise, the loop stack is now empty.

Reading LEA within a loop causes the loop controller to use a new address when it is checking whether program control has reached the end of the current loop. If the loop stack is empty after reading LEA, the loop controller is not checking for a loop end address at all because there is no current loop. However, the new loop end address (or lack thereof) is not used by the loop controller if LEA is read by an instruction that is restricted for loop-stack modifications, as follows:

- The loop controller still detects the old loop end address—the end address of the loop that was popped by reading LEA—in this iteration of the old most-nested loop, if LEA is read in the part of that loop that is restricted for loop-stack modifications. The affected instructions are described in Section 7.6.2, "Hardware Loop Modification Restrictions" on page 64.
- The loop controller does not detect the new loop end address in this iteration of the new most-nested loop if LEA is read in the part of that loop that is restricted for loop-stack modifications. The affected instructions are described in Section 7.6.2, "Hardware Loop Modification Restrictions" on page 64.

Due to their loop-stack manipulating behaviors, accesses to LEA are usually part of a system-level operation to execute a new loop, as described in Section 7.6.4.1, "Using the Loop Registers" on page 69.

Writing LEA

Writing a value to the LEA register is similar (but opposite) to reading LEA: a loop end address is updated *and* the loop stack is manipulated. Writes to LEA are usually part of a system-level operation to execute a new loop, as described in Section 7.6.4.1, "Using the Loop Registers" on page 69. The details of the operation are as follows:

- Writing to LEA has no effect when the high bit (bit 23; LEA has 24 bits) of the value written is 1. No modifications are made to the loop stack or the parameters of any hardware loop.
- The loop stack must not be full (there can be at most three nested loops in progress) when you write to LEA, *if* the high bit of the value written is 0. Writing such a value to LEA when the loop stack is full results in **undefined** behavior.

- When the high bit of the value to be written to LEA is 0, a new entry is created on the loop stack *prior* to writing the new value to the end address for the current top-of-stack loop. This new entry on the loop stack becomes the current loop, so the loop controller begins watching for an instruction fetch from the new loop end address that was written. The loop count and loop start address for this new entry on the loop stack *are not initialized*; they retain their values (if any) from the previous loop that was executed at this level of nesting.

Writing LEA within a loop causes the loop controller to use a new address when it is checking whether program control has reached the end of the current loop. However, the new loop end address is not used by the loop controller if LEA is written by an instruction that is restricted for loop-stack modifications, as follows:

- The loop controller still detects the old loop end address—that is, the end address of the old most-nested loop in which the new current loop is now nested—in this iteration of the old most-nested loop, if LEA is written in the part of the old most-nested loop that is restricted for loop-stack modifications. The affected instructions are described in Section 7.6.2, “Hardware Loop Modification Restrictions” on page 64.
- The loop controller does not detect the new loop end address in this iteration of the new most-nested loop if LEA is written in the part of that loop that is restricted for loop-stack modifications. The affected instructions are described in Section 7.6.2, “Hardware Loop Modification Restrictions” on page 64.

Note: If LEA is both read and written in the same clock cycle, the behavior is **undefined**. Therefore, the *Single-Move* instruction “MOVE LEA, LEA” is illegal.

7.6.4.1 Using the Loop Registers

The loop stack registers provide access to the loop stack at a lower level of abstraction than the looping instructions LOOP.2D and ENDLOOP. You have more flexibility when initiating loops using the loop stack registers, but this flexibility must be used with care.

With the LOOP.2D instruction, the start and end addresses of the loop are static; they are determined at assembly time or linking time. In a situation where you would like to determine either or both of these addresses based on runtime state or logic, you can use the loop stack registers. Suppose that x0 contains a loop count, xp0 contains the start address of the loop and xp1 contains its end address. The following code executes the loop:

```
// Assumes that there is room on the loop stack
nop ||| move lea, xp1  // Create new loop on stack; set end address
nop ||| move lsa, xp0  // Set new loop's start address
nop ||| move lc, x0    // Set new loop's count
goto.2d xp0            // Jump to start of loop
nop >>> nop            // [delay slots 1 and 2]
```

Setting up the loop on the loop stack does not transfer control to the loop; you must do that explicitly, as shown in the example. When the loop exits, the loop is automatically popped off of the loop stack, but you must ensure that the program correctly continues its execution. The example above further assumes that the loop end address is determined independently from the

start address; if only the start address is dynamic, it is probably easier to use static loops (using LOOP.2D), with an indirect jump or subroutine call to the appropriate loop.

Most systems will achieve their best performance by statically partitioning the loop stack—with three loop levels reserved for "mainline" code, for example, leaving one hardware loop for interrupt handlers—which has the benefit that testing for the availability of a hardware loop on the loop stack is unnecessary. This simple scheme does not work for all systems, however, and the loop stack registers allow you to manage the loop stack dynamically and even allocate parts of it in RAM. There are many possible scenarios; approaches for handling some of them are presented below. The examples assume that space on the stack has already been allocated.

- You can test whether the loop stack is full using conditional GOTO with the FULL or NOT_FULL condition. If the loop stack is full, the following code saves the top (most-nested) loop to the system stack in XMEM and pops it from the loop stack in the process. It then executes its own loop, and finally restores the saved loop:

```
    load x0, 0                    // Loop stack not full
    goto.0d done_save, not_full   // Only save if full
    load x0, 1                    // Loop stack full
    nop ||| store x[sp-3], lc     // Save LC
    nop ||| store x[sp-2], lsa    // Save LSA
    nop ||| store x[sp-1], lea    // Save LEA; pop/free loop
done_save:

    loop.2d 256, loop_end
    ...                           // Delay slots and loop
loop_end:
    ...                           // Last loop instruction

    cmp x0, 0 ||| nop             // Was loop stack full?
    goto.0d done_restore, eq      // Restore if yes
    nop ||| load lea, x[sp-1]     // Push loop; restore LEA
    nop ||| load lsa, x[sp-2]     // Restore LSA
    nop ||| load lc, x[sp-3]      // Restore LC
done_restore:
```

- To save the entire contents of the loop stack, save loop stack entries repeatedly using code like the above, until the EMPTY condition holds (or equivalently, while NOT_EMPTY holds). The saving code can count the levels as they are saved, so that the restoring code knows how many loops to restore.
- The number of clock cycles used by the above approaches varies depending on how full the loop stack is. For systems where fixed execution times are preferred, note that the save and restore sequences in the above code can be executed unconditionally; the code behaves correctly regardless of whether the loop stack is empty, full, or partly filled, due to the special value returned for LEA when the loop stack is empty and to the special behavior when that value is written back to LEA. To unconditionally free up a loop stack entry and then restore it (if any loop was running initially), the conditional aspects of the example above can simply be removed, as follows:

```
    nop ||| store x[sp-3], lc     // Save LC
    nop ||| store x[sp-2], lsa    // Save LSA
    nop ||| store x[sp-1], lea    // If empty save 0x800000
                                  // Else save LEA & pop loop
```

```
    loop.2d 256, loop_end
    ...                             // Delay slots and loop
loop_end:
    ...                             // Last loop instruction

    nop ||| load lea, x[sp-1]       // If was empty do nothing
                                    // Else push loop with LEA
    nop ||| load lsa, x[sp-2]       // Restore LSA
    nop ||| load lc, x[sp-3]        // Restore LC
```

- To save the entire contents of the loop stack with a fixed execution time, save the loop stack entries unconditionally as above, repeating four times (and using 12 words on the stack). You need not count the number of loops that were in use; you can restore the whole loop stack by repeating the unconditional restore operations four times.

7.7 HANDLING INTERRUPTS

When enabled, interrupts allow the CFX to respond to external events from interfaces, peripherals or software by automatically diverting control to an interrupt handler. Interrupts are also the only mechanism for getting the CFX out of Sleep mode.

7.7.1 Interrupt Configuration

The CFX supports 32 interrupts. Interrupt 0 is the reset interrupt. The interrupt sources for the other interrupts depend on the system in which the CFX DSP is contained. Refer to the *Hardware Reference Manual* for more information about this and all other system-dependent details described below. All 32 interrupts can be triggered by either hardware or software.

The interrupt vector table contains code to handle each interrupt. The reset interrupt vector (hardware interrupt 0) is always located at address 0 in program memory. The remainder of the interrupt vector table is located at a system-dependent location in program memory. Software interrupts 0-31 and hardware interrupts 1-31 are handled by the indexed entry in the interrupt vector table. Before enabling an interrupt, the corresponding 32-bit entry in the interrupt vector table must contain one of the following instructions:

- `GOTO.0D` (unconditional form) to branch to the interrupt handler
- `RETURNI.0D` to return immediately; this can only be used for software interrupts

An external interrupt controller is responsible for enabling and disabling individual interrupts, acknowledging and auto-acknowledging interrupts, and maintaining interrupt priorities. These details are system-dependent.

Setting or clearing the interrupt enable (`IE`) bit in the status register (`SR`) globally enables or disables external (hardware) interrupts. You cannot disable software interrupts, neither individually nor globally.

The `BITSET SR, imm4u` instruction has special behavior when the index is 2 (to set the `IE` bit). If interrupts are pending, they are handled immediately after this instruction is executed. To support this behavior, the CFX flushes the execution pipeline. If no interrupts are pending, two additional clock cycles are required to fetch and decode the next instruction. Effectively, `BITSET SR, 2` takes 3 clock cycles with no delay slots.

We recommend globally disabling interrupts (by clearing the IE bit in SR) before configuring interrupts.

7.7.2 Invoking Interrupt Handlers

Two types of events cause interrupt handlers to be invoked:

Software interrupts The CFX executes an INTERRUPT instruction. You cannot disable software interrupts, neither individually nor globally.

Hardware interrupts An external interface or peripheral triggers an interrupt, that interrupt is individually enabled, and interrupts are globally enabled (IE = 1). The hardware interrupt can be triggered when the interrupt is disabled or interrupts are globally disabled, but it is only handled when both are enabled.

In either case, the CFX takes the following actions.

1. Finish executing the current atomic instruction group.
   - If executing a *Move-Move* instruction, finish both *Move* instructions.
   - If executing an instruction with delay slots or an instruction in a delay slot, finish executing the instruction and all instructions in delay slots.
   - Otherwise, finish executing all cycles of the current instruction, including both cycles of pipelined multiplication instructions.
2. Flush the pipeline.
3. Set the interrupt-link program counter (ILPC) to the address of the instruction that would have been executed if the interrupt were not handled.
4. Set the interrupt-link status register (ILSR) to the current value of the status register (SR).
5. Clear the IE bit in SR.
6. Branch to the entry in the interrupt vector table corresponding to the highest-priority, enabled, pending interrupt.
7. Deliver an acknowledgement of the interrupt that was handled to the interrupt controller. The response to this acknowledgement is system-dependent, but might result in the interrupt being cleared automatically.

Nested interrupts are not directly supported in hardware. You can support nested interrupts in software by saving the ILPC and ILSR registers to the stack, and re-enabling interrupts (globally) after enabling or disabling individual interrupts if necessary.

Use the RETURNI.0D or RETURNI.2D instructions to return from an interrupt handler. This restores SR from ILSR, and resumes execution at the address in ILPC. Because the status register is restored to its value before the interrupt was handled, you do not need to enable interrupts (by setting IE) before returning from an interrupt handler.

Table 23 summarizes the interrupt-related instructions. For detailed information about each instruction, see Chapter 10, “CFX Instruction Set” on page 83.

TABLE 23: INTERRUPT INSTRUCTIONS

Operation	Instruction
Invoke a software interrupt.	INTERRUPT imm5u
Return from an interrupt.	RETURNI.0D RETURNI.2D RETURNI
Globally enable interrupts. Handle any enabled, pending interrupts immediately.	BITSET SR, 2

7.7.3 RETURNI.2D Delay Slot Restrictions

The instructions that execute in the delay slots of the `RETURNI.2D` instruction have further restrictions (in addition to those described in Section 7.5.5, "Program Flow Delay Slot Restrictions" on page 62) because `RETURNI.2D` overwrites the status register `SR` with the value from `ILSR` during its delay slots.

Two general groups of instructions must be avoided, as follows:

1. Instructions that modify the `SR` cannot be used; this prevents the interrupt handler context from modifying the flag state of the interrupted context.

   Most *Arithmetic* instructions can be used in these delay slots even when the Flag Updates mode is enabled, because the PCU internally disables the Flag Updates mode during the delay slots of `RETURNI.2D`, regardless of the setting of the mode bit in the `SR`.

   Note: The PCU disables Flag Updates **only** *during the two cycles of* `RETURNI.2D`*'s delay slots*. If a multiplication instruction were initiated in the last delay slot, Flag Updates would not necessarily be disabled in the second cycle of the multiplication. This is a further reason—in addition to the unknown instruction at the return address—why multiplications must be avoided in the last delay slot of `RETURNI.2D`.

2. Instructions whose behavior depends on the values of bits in the `SR` should not be used because the `SR`'s value is changing.

Thus you must avoid the following instructions in the delay slots of `RETURNI.2D` in addition to the usual restrictions for the delay slots of control-flow instructions:

- `BITTST`, `DIVST` and all of the comparison instructions (`CMP`, `CMP.DW`, `CMPU.NX`, `CMPU.NX.DW`) because they update flags in the `SR` even when Flag Updates is disabled. Furthermore, `DIVST`'s behavior depends on the X Negative flag.
- Conditional *Arithmetic* instructions because they depend on the flags in the `SR`. (The conditional `GOTO` instructions must be avoided as well; they are excluded by the usual delay slot restrictions.) The instructions are:

```
ADD     d-reg,  d-reg,  cond3
ADD     d-reg,  imm3s,  cond3
ADD.DW  d-reg,  imm3s,  cond3
MOVE    d-reg,  d-reg,  cond3
NEG     d-reg,  d-reg,  cond3
```

```
SHLL    d-reg,  d-reg,  cond3
SHLL    d-reg,  imm3u,  cond3
SHRA    d-reg,  d-reg,  cond3
SHRA    d-reg,  imm3u,  cond3
SUB     d-reg,  d-reg,  cond3
SUB     d-reg,  imm3s,  cond3
SUB.DW  d-reg,  imm3s,  cond3
SWAP    d-reg,  d-reg,  cond3
```

- `BITCLR SR` and `BITSET SR` which explicitly modify bits in the `SR`.
- `MOVE SR` and `LOAD SR` (both loading from memory and loading an immediate value) which modify the whole of the `SR`.
- Instructions that copy the `SR` (and so depend on its value). The instruction forms are:

```
MOVE   <register>,  SR
STORE  <memory>,  SR
```

- *Long* and *Move* instructions that move a whole accumulator's value; they depend on the Saturation, Rounding and Bias mode bits in the `SR`. Those that can saturate the value from the accumulator might also update the Saturated flag. (*Arithmetic* instructions never round or saturate an accumulator's value.) The instructions forms are:

```
MOVE      <register>,  <accumulator>
MOVE.DW   <register>,  <accumulator>
MOVE.EX   <register>,  <accumulator>
STORE     <memory>,  <accumulator>
STORE.DW  <memory>,  <accumulator>
STORE.EX  <memory>,  <accumulator>
```

7.8 RESTRICTIONS ON CONCURRENT REGISTER ACCESSES

Conflicts can arise when several execution units operate concurrently. Such situations are discussed, with rules for avoiding problems, in Section 4.8.3, "Timing, Concurrency, and Restrictions" on page 30, Section 6.4, "Indirect Addressing" on page 47, and Section 7.5.5, "Program Flow Delay Slot Restrictions" on page 62.

Those sections describe conflicts and related restrictions for certain types of instructions. This section discusses a kind of conflict—concurrent updates to CFX registers—that can occur for several types or combinations of instructions. Some situations where this kind of conflict arises have been discussed in the aforementioned sections; this section describes *all* of the situations that can occur, and what the core's behavior is for each.

In summary, multiple concurrent reads of any register are well-defined as are concurrent updates to the `SR`, while concurrent updates to any other register are ill-defined. The following sections describe the behavior and timing of register accesses, and analyze the specific cases for updates to the `SR` and to other core registers.

7.8.1 Timing of Concurrent Register Reads and Writes

During each cycle of program execution, a given register can be read multiple times and written once; the `SR` can be written multiple times as described in Section 7.8.2, "Concurrent Updates to the Status Register".

All register reads occur at the start of the cycle, before any registers are written. Any register can be read multiple times; each instruction or sub-instruction reads the value of the register prior to any register updates that occur in the cycle.

For example, reading a multiplication's destination accumulator (with a *Move* or *Long* instruction) during the multiplication's second cycle is restricted (see Section 4.8.3, "Timing, Concurrency, and Restrictions" on page 30) only because the behavior changes if an interrupt occurs prior to the read. There is nothing ill-defined about the accumulator read occurring concurrently with the multiplication instruction's write (and read, if it is a MAC) of the same accumulator; if this occurs in a code section that cannot be interrupted, it will reliably read the value of the accumulator prior to it being updated by the second cycle (the ALU cycle) of the multiplication instruction.

7.8.2 Concurrent Updates to the Status Register

The bits of the SR are treated independently, and the register as a whole has priority logic for arbitrating some cases. The following sections describe the scenarios where the SR is updated by multiple instructions in the same cycle:

Concurrent X ALU and Y ALU operations

When Flag Updates mode is enabled, instruction combinations executing concurrently in the X and Y ALUs are simultaneously updating the corresponding ALU's arithmetic status flags. These flag updates work as expected because the SR bits are independent and each instruction only modifies the flag bits for its ALU.

The most obvious instruction combination where this scenario occurs is in *Dual Arithmetic* instructions. When both instructions in the *Dual Arithmetic* instruction are multiplication instructions, the simultaneous updates occur in the following cycle, and no conflict can occur with *Arithmetic* instructions executing in that following cycle because such instructions cannot use the ALUs.

Similarly, when just one multiplication instruction is initiated, the following cycle can only execute instructions that use the *other* ALU, so no simultaneous updates can occur to the *same* flag bits in the SR.

The situations with multiplication instructions are described in detail in Section 4.8.3.1, "Multiplication Guidelines" on page 31.

Concurrent updates to arithmetic flags and the Saturated flag

As with concurrent instructions that independently update the X ALU and Y ALU arithmetic status flags, the Saturated flag can also be updated concurrently by a *Long* or *Move* instruction which stores or moves an accumulator when the Saturation mode is enabled. The Saturated flag is updated independently from the arithmetic status flags, so such concurrent updates work as expected.

Dual Move **instructions that both saturate**

When the Saturation mode is enabled in the SR and a *Dual Move* instruction contains MOVE and STORE instructions that have accumulators for their sources, it is possible that both saturate. The conflict with both instructions updating the Saturated flag in the SR is resolved in the obvious

way: the Saturated flag is updated to 1 if *either or both* of the instructions saturates its value before transferring it.

Arithmetic-Move **instructions that move to** SR

When the *Move* part of an *Arithmetic-Move* instruction is a LOAD or MOVE whose destination is the SR, any arithmetic status flag updates from the *Arithmetic* part of the instruction *are entirely overwritten* by the new value moved into the SR. The conflicting updates are resolved by updating the whole SR to the new value.

Multiplication instruction followed by move to SR

When a multiplication instruction is immediately followed by a *Long* or *Move* instruction with SR as the destination of a LOAD or MOVE, any arithmetic status flag updates from the multiplication instruction *are entirely overwritten* by the new value moved into the SR.

Multiplication followed by BITSET SR **or** BITCLR SR

When a multiplication instruction is immediately followed by a *Long* BITSET or BITCLR instruction, the bit updated by BITSET or BITCLR takes precedence. If that bit is one of the arithmetic status flags updated by the multiplication instruction, the update from the multiplication is overwritten by 1 or 0 for BITSET or BITCLR, respectively, and the other arithmetic status flag updates from the multiplication occur as expected. If the bit updated by BITSET or BITCLR is not one of those updated by the multiplication instruction, all updates take place independently, as expected.

Multiplication followed by return from interrupt handler

When a multiplication instruction is immediately followed by a RETURNI.0D or RETURNI.2D instruction, any arithmetic status flag updates from the multiplication are overwritten because the return-from-interrupt instruction loads the SR with the value from the ILSR register.

7.8.3 Concurrent Updates to Registers Other than SR

Multiple concurrent updates to any CFX register other than SR are illegal and will not produce useful results. When an accumulator is a destination, all 56 bits are updated, so a concurrent update to any of that accumulator's subregisters (see Section 4.2.1, "Data Registers" on page 15) is illegal. The following instruction combinations should be avoided:

Arithmetic-Move **instruction with identical destinations**

You cannot use the same destination register in the *Arithmetic* and *Move* parts of an *Arithmetic-Move* instruction, because the two parts execute in parallel.

Note that the SWAP instruction is the only *Arithmetic* instruction with *two* destination registers. The two register operands to SWAP are both sources and destinations.

If the *Arithmetic* or *Move* part (or both) are *Dual*, you must ensure that you avoid using the same destination register(s) in both parts of the *Dual* instructions. *X Arithmetic* instructions can only modify A accumulators and

Y Arithmetic instructions can only modify B accumulators, but note that *X Move* instructions can modify any of the eight data registers, as can *Y Move* instructions. You must avoid conflicting destination registers in all combinations.

Multiplication instruction followed by update of accumulator

Multiplication instructions update their accumulator destination in their second (pipelined) execution cycle. *Arithmetic* instructions that update the multiplication's destination accumulator in this second cycle violate several DCU-specific restrictions as described in Section 4.8.3.1, "Multiplication Guidelines" on page 31. *Long* or *Move* instructions that update the multiplication instruction's accumulator destination in this second cycle are also illegal.

`LOAD` **of pointer register also used for indirect address with update**

Long or *Move* instructions, which `LOAD` a pointer register via a post-updating indirect addressing mode with the same pointer, are illegal because the pointer is updated twice. See Section 6.4, "Indirect Addressing" on page 47 for an example.

Dual Move **instruction with identical destinations**

The destination operand of an *X Move* instruction must not be the same register as the destination operand of the paired *Y Move* instruction.

This page left blank for printing purposes.

8 Memory Spaces

8.1 INTRODUCTION

This chapter describes the memory spaces that the CFX can access.

The CFX can address three memory spaces: X memory (XMEM), Y memory (YMEM), and program memory (PMEM). The CFX can also address X and Y memory together as a double-width XY memory (XYMEM).

Various kinds of memory areas can be mapped into the three address spaces, depending on the system implementation:

- RAM is commonly used for data and instructions.
- ROM areas are used to store boot code and data tables.
- The interrupt vector table resides in PMEM.
- Peripheral control registers are typically memory-mapped in PMEM.
- Storage shared with other system processors can be memory-mapped.

Refer to the *Hardware Reference Manual* for details on the layout of memory areas, and the usage of peripherals, shared memory, and interrupts. Refer to Section 7.7, “Handling Interrupts” on page 71 for information on the interrupt vector table.

8.2 MEMORY ARCHITECTURE

The CFX has independent data and address buses for each memory space, allowing simultaneous access to XMEM, YMEM and PMEM. This allows an instruction from PMEM to be fetched at the same time as a read or write to both XMEM and YMEM, facilitating flexible, single-cycle multiply-accumulate (MAC), block multiplication, and data copy operations.

The CFX memory architecture and instruction set are designed to retrieve program instructions from PMEM and data from XMEM and YMEM. Although you can store data in PMEM, accessing that data might incur a delay if the memory block is in use, for example by the Instruction Bus to fetch the next instruction. Access to memory shared with other system processors might also be subject to delays. More information on both of these scenarios can be found in the *Hardware Reference Manual*.

In some instruction combinations that involve memory write operations, the CFX must buffer the write operation and then store the data to memory later. In most cases the write buffers are transparent to user applications, but in some situations—generally involving control registers with special behavior—the CFX’s write buffering is visible and your program must be aware of this. See the *Hardware Reference Manual* for details about the write buffers.

8.3 X DATA MEMORY

X memory (XMEM) is 24 bits wide with 16-bit addressing.

The system stack, normally referenced using the SP register, is located in X memory and grows upwards. XMEM also contains ROM look-up tables used by the math library firmware, and memory shared with other system processors. Access to shared memory might be subject to delays as described in the *Hardware Reference Manual*.

You can address XMEM using the X AGU indirect addressing registers (XP0..XP7), the stack pointer (SP), or an immediate address.

8.4 Y DATA MEMORY

Y memory (YMEM) is 24 bits wide with 16-bit addressing.

You can address YMEM using the Y AGU indirect addressing registers (YP0..YP3), or an immediate address.

8.5 XY DOUBLE DATA MEMORY

XY memory (XYMEM) refers to a 48-bit memory generated by combining XMEM and YMEM. XYMEM has 16-bit addressing. Operations that access XY memory access a word at the corresponding addresses in each of XMEM and YMEM; the low-order word (bits 0..23) is in X memory, and the high-order word (bits 24..47) is in Y memory.

You can address XYMEM using the X AGU indirect addressing registers (XP0..XP7), or an immediate address.

8.6 P MEMORY

Program memory (PMEM) is 32 bits wide with 16-bit addressing.

You can address PMEM for data using the X or Y AGU indirect addressing registers (XP0..XP7, YP0..YP3), or an immediate address. PMEM cannot be accessed for data at the same time as XMEM or YMEM.

PMEM contains program instructions in program RAM and ROM. Every instruction executed by the CFX must be fetched from this memory. The CFX has a small 16-word read-only instruction cache; when the cache is enabled, access to P memory while the CFX is fetching cached instructions from a loop does not result in a memory access conflict, and does not incur a delay. Otherwise, access to PMEM while the CFX is fetching a non-cached instruction from the same PMEM bank incurs a one-cycle delay. Refer to the *Hardware Reference Manual* for more information.

The interrupt vector table resides in PMEM. Each entry in this table is filled with an unconditional branch to the start of the associated ISR (i.e., a GOTO.0D instruction). See Section 7.7, “Handling Interrupts” on page 71 for details about handling interrupts.

Peripheral control registers are memory-mapped in PMEM. PMEM also contains memory shared with other system processors. Access to shared memory might be subject to delays if the memory is being accessed by another system processor. These delays are described in the *Hardware Reference Manual*.

9 Processing Modes

9.1 Introduction

This chapter describes the three processing modes of the CFX: Run, Sleep and Debug.

9.2 Run Mode

In Run mode, the CFX fetches and executes instructions normally, as described in Section 7.3, “Execution Pipeline” on page 52.

9.3 Sleep Mode

Use Sleep mode to save power. You can enter Sleep mode with one of the following instructions:

```
SLEEP
SLEEP restart-addr
```

The CFX enters a low-power mode with most of the gated clocks turned off. Both instructions complete and the CFX returns to Run mode when an interrupt is received, or a wakeup signal is received from an external source such as an external interrupt controller. The first form of the `SLEEP` instruction resumes execution at the next address after the `SLEEP` instruction. The second form resumes execution at the address specified in the instruction.

9.4 Debug Mode

The Debug mode allows the CFX to be controlled externally by a debugger or other application that needs to load, run and control CFX programs.

This page left blank for printing purposes.

10 CFX Instruction Set

10.1 INTRODUCTION

This chapter is intended for use as a reference manual for the CFX instruction set. It begins by organizing the instructions into functional groups so that you can find a specific instruction by what you intend to do. Next, it explains the syntax and use of instruction words which are composed of one or more instructions. Finally, each instruction is described in detail, including its operands, flags, execution time, and opcode. These are arranged in alphabetic order.

10.2 INSTRUCTION FUNCTION GROUPS

The CFX instruction set can be divided into the following function groups:

- Data Movement
  - General Data Movement
  - XY Data Movement
  - P Data Movement
- Stack Manipulation
- Control Flow and Looping
- Pointer Arithmetic
- Status Register Manipulation
- Logic
- Arithmetic
- Multiplication
- Miscellaneous

These groups are described in more detail below, where specific instruction mnemonics and operands are listed. The operand classes are explained in Section 10.4.4, “Operand Classes” on page 109.

10.2.1 Data Movement Instructions

Data movement instructions are used to move data between registers and memory. The data movement instructions are summarized in Table 24. Instructions that access double-width data in the XY memory bank and data in the P memory bank are listed separately.

TABLE 24: DATA MOVEMENT INSTRUCTIONS

Mnemonic and Operands	Type	Description
CLR X[single-mode(X)]	Move	Clear a word of X memory specified by an X pointer, with update
LOAD reg, X[imm16u]	Long	Load register direct from X memory
LOAD xmem-reg, X[single-mode(X)]	Move	Load register indirect from X memory

TABLE 24: DATA MOVEMENT INSTRUCTIONS (CONTINUED)

Mnemonic and Operands	Type	Description
LOAD ax-reg, X[dual-mode(X)]	X Move	Load A or X data register indirect from X memory
LOAD reg, Y[imm16u]	Long	Load register direct from Y memory
LOAD by-reg, Y[single-mode(Y)]	Move	Load B or Y data register indirect from Y memory
LOAD by-reg, Y[dual-mode(Y)]	Y Move	Load B or Y data register indirect from Y memory
LOAD reg, imm20s	Long	Load register with a value
LOAD d-reg, imm24s	Long	Load data register with a value
LOAD d-reg, imm5s	Move	Load data register with a value
LOAD d-reg, imm6s	Arith	Load data register with a value
LOAD l-reg, imm24s	Long	Load accumulator low part with a value
LOAD xstep-reg, imm5s	Move	Load X step register with a value
LOAD ystep-reg, imm5s	Move	Load Y step register with a value
LOAD ax-reg, imm2s	X Move	Load A or X data register with a value
LOAD by-reg, imm2s	Y Move	Load B or Y data register with a value
LOAD.DW ab-reg, imm24s	Long	Load accumulator with a double-width value
LOAD.DW ab-reg, imm5s	Move	Load accumulator with a double-width value
LOAD.DW d-reg, imm6s	Arith	Load data register with a double-width value
MOVE reg, reg	Move	Move register to register
MOVE ab-reg, x-reg	X Move	Move X data register to accumulator
MOVE xy-reg, a-reg	X Move	Move A accumulator to XY data register
MOVE ab-reg, y-reg	Y Move	Move Y data register to accumulator
MOVE xy-reg, b-reg	Y Move	Move B accumulator to XY data register
MOVE d-reg, d-reg	Arith	Move data register to data register
MOVE d-reg, d-reg, cond3	Arith	Move data register to data register, conditionally
MOVE.DW ab-reg, ab-reg	Move	Move double-width accumulator to double-width accumulator
MOVE.EX ab-reg, ab-reg	Move	Move extended accumulator to accumulator
MOVE.NX d-reg, d-reg	Arith	Move data register to data register with no extension
MOVE.NX a-reg, ab-reg	X Arith	Move accumulator to A accumulator with no extension
STORE X[imm16u], reg	Long	Store register direct to X memory

TABLE 24: DATA MOVEMENT INSTRUCTIONS (CONTINUED)

Mnemonic and Operands	Type	Description
STORE X[single-mode(X)], xmem-reg	Move	Store register indirect to X memory
STORE X[dual-mode(X)], a-reg	X Move	Store A accumulator indirect to X memory
STORE Y[imm16u], reg	Long	Store register direct to Y memory
STORE Y[single-mode(Y)], by-reg	Move	Store B or Y data register indirect to Y memory
STORE Y[dual-mode(Y)], b-reg	Y Move	Store B accumulator indirect to Y memory
STORE X[imm16u], imm8s	Long	Store a value direct to X memory
STORE Y[imm16u], imm8s	Long	Store a value direct to Y memory
SWAP d-reg, d-reg, cond3	Arith	Swap the contents of two data registers, conditionally

10.2.1.1 XY Data Movement Instructions

XY data movement instructions are used to move double-width data between the XY memory bank and the accumulators. These instructions are summarized in Table 25.

TABLE 25: XY DATA MOVEMENT INSTRUCTIONS

Mnemonic and Operands	Type	Description
LOAD.DW ab-reg, XY[imm16u]	Long	Load double-width accumulator direct from XY memory
LOAD.DW ab-reg, XY[single-mode(X)]	Move	Load double-width accumulator indirect via X pointer from XY memory
STORE.DW XY[imm16u], ab-reg	Long	Store double-width accumulator direct to XY memory
STORE.DW XY[single-mode(X)], ab-reg	Move	Store double-width accumulator indirect via X pointer to XY memory
STORE.DW XY[imm16u], imm8s	Long	Store a double-width immediate value direct to XY memory

10.2.1.2 P Data Movement Instructions

P data movement instructions are used to move data between registers and the P (program) memory bank. These instructions are summarized in Table 26.

TABLE 26: P DATA MOVEMENT INSTRUCTIONS

Mnemonic and Operands	Type	Description
LOAD reg, P[imm16u]	Long	Load register direct from P memory
LOAD reg, P[single-mode(X)]	Long	Load register indirect from P memory
LOAD reg, P[single-mode(Y)]	Long	Load register indirect from P memory

TABLE 26: P DATA MOVEMENT INSTRUCTIONS (CONTINUED)

Mnemonic and Operands	Type	Description
LOAD.EX ab-reg, P[imm16u]	Long	Load extended accumulator direct from P memory
LOAD.EX ab-reg, P[single-mode(X)]	Long	Load extended accumulator indirect via X pointer from P memory
LOAD.EX ab-reg, P[single-mode(Y)]	Move	Load extended accumulator indirect via Y pointer from P memory
STORE P[imm16u], reg	Long	Store register direct to P memory
STORE P[single-mode(X)], reg	Long	Store register indirect to P memory
STORE P[single-mode(Y)], reg	Long	Store register indirect to P memory
STORE P[imm16u], imm8s	Long	Store a value direct to P memory
STORE.EX P[imm16u], ab-reg	Long	Store extended accumulator direct to P memory
STORE.EX P[single-mode(X)], ab-reg	Long	Store extended accumulator indirect via X pointer to P memory
STORE.EX P[single-mode(Y)], ab-reg	Move	Store extended accumulator indirect via Y pointer to P memory

10.2.2 Stack Manipulation Instructions

Stack manipulation instructions are used to modify the stack pointer and move data to and from the stack. These instructions are summarized in Table 27.

TABLE 27: STACK MANIPULATION INSTRUCTIONS

Mnemonic and Operands	Type	Description
ADD SP, imm16	Long	Add a value to the stack pointer
ADD SP, imm7s	Move	Add a value to the stack pointer
ADD xp-reg, SP, imm16	Long	Add the stack pointer and a value, storing the result in an X pointer
ADD xp-reg, SP, imm6n	Move	Add the stack pointer and a value, storing the result in an X pointer
LOAD reg, X[stack-mode16]	Long	Load register from stack
LOAD stack-reg6, X[stack-mode6]	Move	Load register from stack
LOAD stack-reg4, X[stack-mode4]	Move	Load register from stack
STORE X[stack-mode16], reg	Long	Store register to stack
STORE X[stack-mode6], stack-reg6	Move	Store register to stack
STORE X[stack-mode4], stack-reg4	Move	Store register to stack
SUB SP, imm16	Long	Subtract a value from the stack pointer
SUB SP, imm7sn	Move	Subtract a value from the stack pointer

TABLE 27: STACK MANIPULATION INSTRUCTIONS (CONTINUED)

Mnemonic and Operands	Type	Description
SUB xp-reg, SP, imm16	Long	Subtract a value from the stack pointer, storing the result in an X pointer
SUB xp-reg, SP, imm6p	Move	Subtract a value from the stack pointer, storing the result in an X pointer

10.2.3 Control Flow and Looping Instructions

Control flow instructions are used to implement conditional logic and subroutines. Control flow instructions are summarized in Table 28.

Looping instructions are used to implement zero-overhead loops. Looping instructions are summarized in Table 29.

TABLE 28: CONTROL FLOW INSTRUCTIONS

Mnemonic and Operands	Type	Description
CALL imm16u	Synth	Call direct without delay slot
CALL xp-reg	Synth	Call indirect without delay slots
CALL.1D imm16u	Long	Call direct with one delay slot
CALL.2D xp-reg	Long	Call indirect with two delay slots
GOTO imm16u	Long	Branch direct without delay slot
GOTO imm16u, cond5	Long	Branch direct without delay slots, conditionally
GOTO xp-reg	Synth	Branch indirect without delay slots
GOTO xp-reg, cond5	Synth	Branch indirect without delay slots, conditionally
GOTO.0D imm16u	Long	Branch direct without delay slot
GOTO.0D imm16u, cond5	Long	Branch direct without delay slots, conditionally
GOTO.1D imm16u	Long	Branch direct with one delay slot
GOTO.2D imm16u, cond5	Long	Branch direct with two delay slots, conditionally
GOTO.2D xp-reg	Long	Branch indirect with two delay slots
GOTO.2D xp-reg, cond5	Long	Branch indirect with two delay slots, conditionally
INTERRUPT imm5u	Long	Software interrupt
RETURN	Synth	Return from subroutine without delay slots
RETURN.2D	Long	Return from subroutine with two delay slots
RETURNI	Long	Return from interrupt without delay slots

TABLE 28: CONTROL FLOW INSTRUCTIONS (CONTINUED)

Mnemonic and Operands	Type	Description
RETURNI.0D	Long	Return from interrupt without delay slots
RETURNI.2D	Long	Return from interrupt with two delay slots

TABLE 29: LOOPING INSTRUCTIONS

Mnemonic and Operands	Type	Description
ENDLOOP	Long	End loop
LOOP imm12u, imm16u	Synth	Loop with immediate count and no delay slots
LOOP d-reg, imm16u	Synth	Loop with data register count and no delay slots
LOOP.2D imm12u, imm16u	Long	Loop with immediate count and two delay slots
LOOP.2D d-reg, imm16u	Long	Loop with data register count and two delay slots

10.2.4 Pointer Arithmetic Instructions

Pointer arithmetic instructions are used to update the X and Y pointers based on immediate values or step registers. These instructions are summarized in Table 30.

TABLE 30: POINTER ARITHMETIC INSTRUCTIONS

Mnemonic and Operands	Type	Description
ADD xp-reg, xp-reg, xstep-reg	Move	Add an X pointer and an X step register, storing the result in an X pointer
ADD xp-reg, xp-reg, imm16	Long	Add a value to an X pointer, storing the result in an X pointer
ADD xp-reg, xp-reg, imm3s	Move	Add a value to an X pointer, storing the result in an X pointer
ADD yp-reg, yp-reg, ystep-reg	Move	Add a Y pointer and a Y step register, storing the result in a Y pointer
ADD yp-reg, yp-reg, imm3s	Move	Add a value to a Y pointer, storing the result in a Y pointer
SUB xp-reg, xp-reg, imm16	Long	Subtract a value from an X pointer, storing the result in an X pointer
SUB xp-reg, xp-reg, imm3sn	Move	Subtract a value from an X pointer, storing the result in an X pointer
SUB yp-reg, yp-reg, imm3sn	Move	Subtract a value from a Y pointer, storing the result in a Y pointer

TABLE 30: POINTER ARITHMETIC INSTRUCTIONS (CONTINUED)

Mnemonic and Operands	Type	Description
SUBSTEP xp-reg	Move	Subtract an X step register from the corresponding X pointer
SUBSTEP yp-reg	Move	Subtract a Y step register from the corresponding Y pointer

10.2.5 Status Register Manipulation Instructions

Status register manipulation instructions are used to quickly set or clear bits in the status register (SR). These instructions are summarized in Table 31.

TABLE 31: STATUS REGISTER INSTRUCTIONS

Mnemonic and Operands	Type	Description
BITCLR SR, imm4u	Long	Clear a bit in the status register
BITSET SR, imm4u	Long	Set a bit in the status register

10.2.6 Logic Instructions

Logic instructions implement individual bit manipulation and testing, and bitwise logical operations. These instructions are summarized in Table 32.

TABLE 32: LOGIC INSTRUCTIONS

Mnemonic and Operands	Type	Description
AND d-reg, d-reg, d-reg	Arith	Bitwise AND two data registers, storing the result in a data register
AND a-reg, a-reg, ab-reg	X Arith	Bitwise AND two accumulators, storing the result in an A accumulator
AND d-reg, d-reg, imm3s	Arith	Bitwise AND a data register and a value, storing the result in a data register
AND.DW d-reg, d-reg, imm3s	Arith	Bitwise AND a data register and a double-width value, storing the result in a data register
BITCLR d-reg, bit-index	Arith	Clear a bit in a data register
BITSET d-reg, bit-index	Arith	Set a bit in a data register
BITTGL d-reg, bit-index	Arith	Toggle a bit in a data register
BITTST d-reg, bit-index	Arith	Test a bit in a data register
NOT d-reg, d-reg	Arith	Bitwise NOT a data register, storing the result in a data register
OR d-reg, d-reg, d-reg	Arith	Bitwise OR two data registers, storing the result in a data register

TABLE 32: LOGIC INSTRUCTIONS (CONTINUED)

Mnemonic and Operands	Type	Description
OR a-reg, a-reg, ab-reg	X Arith	Bitwise OR an A accumulator and an accumulator, storing the result in an A accumulator
OR d-reg, d-reg, imm3s	Arith	Bitwise OR a data register and a value, storing the result in a data register
OR.DW d-reg, d-reg, imm3s	Arith	Bitwise OR a data register and a double-width value, storing the result in a data register
XOR d-reg, d-reg, d-reg	Arith	Bitwise XOR two data registers, storing the result in a data register
XOR d-reg, d-reg, imm3s	Arith	Bitwise XOR a data register and a value, storing the result in a data register
XOR.DW d-reg, d-reg, imm3s	Arith	Bitwise XOR a data register and a double-width value, storing the result in a data register

10.2.7 Arithmetic Instructions

Arithmetic instructions implement addition, subtraction, negation, division steps, base-2 logarithm and shift operations. These instructions are summarized in Table 33.

TABLE 33: ARITHMETIC INSTRUCTIONS

Mnemonic and Operands	Type	Description
ABS d-reg, d-reg	Arith	Calculate the absolute value of a data register
ABS a-reg, ab-reg	X Arith	Calculate the absolute value of an accumulator, placing the result in an A accumulator
ADD d-reg, d-reg, d-reg	Arith	Add two data registers, storing the result in a data register
ADD a-reg, a-reg, ab-reg	X Arith	Add two accumulators, storing the result in an A accumulator
ADD b-reg, b-reg, ab-reg	Y Arith	Add two accumulators, storing the result in a B accumulator
ADD d-reg, d-reg, imm3s	Arith	Add a value to a data register, storing the result in a data register
ADD d-reg, d-reg, cond3	Arith	Add a data register to a data register if the condition holds
ADD d-reg, imm3s, cond3	Arith	Add a value to a data register if the condition holds

TABLE 33: ARITHMETIC INSTRUCTIONS (CONTINUED)

Mnemonic and Operands	Type	Description
ADD.DW d-reg, d-reg, imm3s	Arith	Add a double-width value to a data register, storing the result in a data register
ADD.DW d-reg, imm3s, cond3	Arith	Add a double-width value to a data register, if the condition holds
ADD.FR d-reg, d-reg, imm3s	Arith	Add a fractional immediate value to a data register, storing the result in a data register
ADD.NX d-reg, d-reg, d-reg	Arith	Add two data registers with no extension, storing the result in a data register
ADD.NX a-reg, a-reg, ab-reg	X Arith	Add two accumulators with no extension, storing the result in an A accumulator
ADD.NX d-reg, d-reg, imm3s	Arith	Add a value to a data register with no extension, storing the result in a data register
ADD.NX.DW d-reg, d-reg, imm3s	Arith	Add a double-width value to a data register with no extension, storing the result in a data register
ADDSH d-reg, d-reg, d-reg	Arith	Add two data registers and shift right one bit, storing the result in a data register
ADDSH a-reg, a-reg, ab-reg	X Arith	Add two accumulators and shift right one bit, storing the result in an A accumulator
ADDSH b-reg, b-reg, ab-reg	Y Arith	Add two accumulators and shift right one bit, storing the result in a B accumulator
CMP d-reg, d-reg	Arith	Compare two data registers
CMP a-reg, ab-reg	X Arith	Compare two accumulators
CMP d-reg, imm3s	Arith	Compare a data register to a value
CMP.DW d-reg, imm3s	Arith	Compare a data register to a double-width value
CMPU.NX d-reg, d-reg	Arith	Compare two data registers as unsigned values with no extension
CMPU.NX a-reg, ab-reg	X Arith	Compare two accumulators as unsigned values with no extension
CMPU.NX d-reg, imm3s	Arith	Compare a data register to a value as unsigned values
CMPU.NX.DW d-reg, imm3s	Arith	Compare a data register to a double-width value as unsigned values
DIVST d-reg, d-reg, d-reg	Arith	Division step for dividing data register by data register, result to data register
LOG2ABS d-reg, d-reg	Arith	Calculate the logarithm base 2 of the absolute value of a data register, storing the result in a data register

TABLE 33: ARITHMETIC INSTRUCTIONS (CONTINUED)

Mnemonic and Operands	Type	Description
MAX d-reg, d-reg, d-reg	Arith	Determine the maximum value of two data registers, storing the result in a data register
MAX a-reg, a-reg, ab-reg	X Arith	Determine the maximum value of two accumulators, and store the result in an A accumulator
MIN d-reg, d-reg, d-reg	Arith	Determine the minimum value of two data registers, storing the result in a data register
MIN a-reg, a-reg, ab-reg	X Arith	Determine the minimum value of two accumulators, and store the result in an A accumulator
NEG d-reg, d-reg	Arith	Negate a data register, storing the result in a data register
NEG a-reg, ab-reg	X Arith	Negate an accumulator, storing the result in an A accumulator
NEG d-reg, d-reg, cond3	Arith	Negate a data register, storing the result in a data register, conditionally
NLOG2ABS d-reg, d-reg	Arith	Calculate the logarithm base 2 of the absolute value of a data register, negate the result, and store the result in a data register
NLOG2ABS a-reg, ab-reg	X Arith	Calculate the logarithm base 2 of the absolute value of an accumulator, negate the result, and store the result in an A accumulator
SHLL d-reg, d-reg, d-reg	Arith	Shift a data register left logically, by a count from a data register, storing the result in a data register
SHLL d-reg, d-reg, imm3u	Arith	Shift a data register left logically, by an immediate count, storing the result in a data register
SHLL d-reg, imm6u	Arith	Shift a data register left logically, by an immediate count
SHLL d-reg, d-reg, cond3	Arith	Shift a data register left logically, by a count from a data register, conditionally
SHLL d-reg, imm3u, cond3	Arith	Shift a data register left logically, by an immediate count, conditionally
SHLL.NX d-reg, d-reg, d-reg	Arith	Shift a data register left logically, with no extension, by a count from a data register, storing the result in a data register
SHLL.NX d-reg, d-reg, imm3u	Arith	Shift a data register left logically, with no extension, by an immediate count, storing the result in a data register

TABLE 33: ARITHMETIC INSTRUCTIONS (CONTINUED)

Mnemonic and Operands	Type	Description
SHLL.NX d-reg, imm6u	Arith	Shift a data register left logically, with no extension, by an immediate count
SHRA d-reg, d-reg, d-reg	Arith	Shift a data register right arithmetically, by a count from a data register, storing the result in a data register
SHRA d-reg, d-reg, imm3u	Arith	Shift a data register right arithmetically, by an immediate count, storing the result in a data register
SHRA d-reg, imm6u	Arith	Shift a data register right arithmetically, by an immediate count
SHRA d-reg, d-reg, cond3	Arith	Shift a data register right arithmetically, by a count from a data register, conditionally
SHRA d-reg, imm3u, cond3	Arith	Shift a data register right arithmetically, by an immediate count, conditionally
SHRL.NX d-reg, d-reg, d-reg	Arith	Shift a data register right logically, with no extension, by a count from a data register, storing the result in a data register
SHRL.NX d-reg, d-reg, imm3u	Arith	Shift a data register right logically, with no extension, by an immediate count, storing the result in a data register
SHRL.NX d-reg, imm6u	Arith	Shift a data register right logically, with no extension, by an immediate count
SUB d-reg, d-reg, d-reg	Arith	Subtract two data registers, storing the result in a data register
SUB a-reg, a-reg, ab-reg	X Arith	Subtract two accumulators, storing the result in an A accumulator
SUB b-reg, b-reg, ab-reg	Y Arith	Subtract two accumulators, storing the result in a B accumulator
SUB d-reg, d-reg, imm3sn	Arith	Subtract a value from a data register, storing the result in a data register
SUB d-reg, d-reg, cond3	Arith	Subtract a data register from a data register, conditionally
SUB d-reg, imm3sn, cond3	Arith	Subtract a value from a data register if the condition holds
SUB.DW d-reg, d-reg, imm3sn	Arith	Subtract a double-width value from a data register, storing the result in a data register
SUB.DW d-reg, imm3sn, cond3	Arith	Subtract a double-width value from a data register, if the condition holds
SUB.FR d-reg, d-reg, imm3s	Arith	Subtract a fractional immediate value from a data register, storing the result in a data register

TABLE 33: ARITHMETIC INSTRUCTIONS (CONTINUED)

Mnemonic and Operands	Type	Description
SUB.NX d-reg, d-reg, d-reg	Arith	Subtract two data registers with no extension, storing the result in a data register
SUB.NX a-reg, a-reg, ab-reg	X Arith	Subtract two accumulators with no extension, storing the result in an A accumulator
SUB.NX d-reg, d-reg, imm3sn	Arith	Subtract a value from a data register with no extension, storing the result in a data register
SUB.NX.DW d-reg, d-reg, imm3sn	Arith	Subtract a double-width value from a data register with no extension, storing the result in a data register
SUBSH d-reg, d-reg, d-reg	Arith	Subtract two data registers and shift right one bit, storing the result in a data register
SUBSH a-reg, a-reg, ab-reg	X Arith	Subtract two accumulators and shift right one bit, storing the result in an A accumulator
SUBSH b-reg, b-reg, ab-reg	Y Arith	Subtract two accumulators and shift right one bit, storing the result in a B accumulator

10.2.8 Multiplication Instructions

Multiplication instructions implement multiply and multiply-accumulate (MAC) operations with optional pre-addition or pre-subtraction. All multiplication instructions complete after 1 cycle, but the result is only available after one more cycle. These instructions are summarized in Table 34.

TABLE 34: MULTIPLICATION INSTRUCTIONS

Mnemonic and Operands	Type	Description
ADDMUL ab-reg, xy-reg, xy-reg, xy-reg	Arith	Add two XY data registers, multiply the result by a third XY data register, and store the result in an accumulator
ADDMULADD ab-reg, xy-reg, xy-reg, xy-reg	Arith	Add two XY data registers, multiply the result by a third XY data register, and add the result to an accumulator
ADDMULNEG ab-reg, xy-reg, xy-reg, xy-reg	Arith	Add two XY data registers, multiply the result by a third XY data register, negate the result and store it in an accumulator
ADDMULSUB ab-reg, xy-reg, xy-reg, xy-reg	Arith	Add two XY data registers, multiply the result by a third XY data register, and subtract the result from an accumulator

TABLE 34: MULTIPLICATION INSTRUCTIONS (CONTINUED)

Mnemonic and Operands	Type	Description
MUL ab-reg, xy-reg, xy-reg	Arith	Multiply two XY data registers, storing the result in an accumulator
MUL a-reg, x-reg, xy-reg	X Arith	Multiply two XY data registers, storing the result in an A accumulator
MUL b-reg, y-reg, xy-reg	Y Arith	Multiply two XY data registers, storing the result in a B accumulator
MUL.SU ab-reg, xy-reg, xy-reg	Arith	Multiply two XY data registers, one signed and one unsigned, storing the result in an accumulator
MUL.UU ab-reg, xy-reg, xy-reg	Arith	Multiply two XY data registers, both unsigned, storing the result in an accumulator
MULADD ab-reg, xy-reg, xy-reg	Arith	Multiply two XY data registers, and add the result to an accumulator
MULADD a-reg, x-reg, xy-reg	X Arith	Multiply two XY data registers, and add the result to an A accumulator
MULADD b-reg, y-reg, xy-reg	Y Arith	Multiply two XY data registers, and add the result to a B accumulator
MULADD.SU ab-reg, xy-reg, xy-reg	Arith	Multiply two XY data registers, one signed and one unsigned, and add the result to an accumulator
MULADD.UU ab-reg, xy-reg, xy-reg	Arith	Multiply two XY data registers, both unsigned, and add the result to an accumulator
MULNEG ab-reg, xy-reg, xy-reg	Arith	Multiply two XY data registers, negate the result and store it in an accumulator
MULNEG a-reg, x-reg, xy-reg	X Arith	Multiply two XY data registers, negate the result and store it in an A accumulator
MULNEG b-reg, y-reg, xy-reg	Y Arith	Multiply two XY data registers, negate the result and store it in a B accumulator
MULNEG.SU ab-reg, xy-reg, xy-reg	Arith	Multiply two XY data registers, one signed and one unsigned, negate the result and store it in an accumulator
MULNEG.UU ab-reg, xy-reg, xy-reg	Arith	Multiply two XY data registers, both unsigned, negate the result and store it in an accumulator
MULSUB ab-reg, xy-reg, xy-reg	Arith	Multiply two XY data registers, and subtract the result from an accumulator
MULSUB a-reg, x-reg, xy-reg	X Arith	Multiply two XY data registers, and subtract the result from an A accumulator
MULSUB b-reg, y-reg, xy-reg	Y Arith	Multiply two XY data registers, and subtract the result from a B accumulator

TABLE 34: MULTIPLICATION INSTRUCTIONS (CONTINUED)

Mnemonic and Operands	Type	Description
MULSUB.SU ab-reg, xy-reg, xy-reg	Arith	Multiply two XY data registers, one signed and one unsigned, and subtract the result from an accumulator
MULSUB.UU ab-reg, xy-reg, xy-reg	Arith	Multiply two XY data registers, both unsigned, and subtract the result from an accumulator
SUBMUL ab-reg, xy-reg, xy-reg, xy-reg	Arith	Subtract two XY data registers, multiply the result by a third XY data register, and store the result in an accumulator
SUBMULADD ab-reg, xy-reg, xy-reg, xy-reg	Arith	Subtract two XY data registers, multiply the result by a third XY data register, and add the result to an accumulator
SUBMULNEG ab-reg, xy-reg, xy-reg, xy-reg	Arith	Subtract two XY data registers, multiply the result by a third XY data register, negate the result and store it in an accumulator
SUBMULSUB ab-reg, xy-reg, xy-reg, xy-reg	Arith	Subtract two XY data registers, multiply the result by a third XY data register, and subtract the result from an accumulator

10.2.9 Miscellaneous Instructions

The functions in Table 35 do not fall into the groups described above.

TABLE 35: MISCELLANEOUS INSTRUCTIONS

Mnemonic and Operands	Type	Description
BREAKPOINT	Long	Breakpoint
NOP	Long	No operation
NOP	Move	No operation
NOP	Arith	No operation
NOP	X Arith	No operation
SLEEP	Long	Sleep
SLEEP imm16u	Long	Sleep with restart location

10.3 INSTRUCTION EXECUTION GROUPS AND PARALLELISM

There are three types of 32-bit instruction words; they are composed of instructions using the following syntax:

```
Long
Arithmetic  |||  Move
Move  >>>  Move
```

The *Arithmetic* and *Move* instructions execute in parallel, so *Long* and *Arithmetic-Move* instruction words execute in one cycle (for the most part; see instruction details), while *Move-Move* instruction words execute in two cycles (always).

Both the *Arithmetic* and *Move* instruction classes have single and dual forms. *Single Arithmetic* instructions use the X multiplier and/or ALU, while *Dual Arithmetic* instructions are composed of an *X Arithmetic* instruction that uses the X multiplier and/or ALU, and a *Y Arithmetic* instruction that uses the Y multiplier and/or ALU:

```
Single Arith         |||  Move
X Arith !! Y Arith   |||  Move
```

Single Move instructions can access all memories. Both X and Y buses and memories will be accessed simultaneously for XY memory accesses. *Dual Move* instructions are composed of an *X Move* instruction that accesses X memory or registers, and a *Y Move* instruction that accesses Y memory or registers:

```
Arithmetic    |||  Single Move
Arithmetic    |||  X Move !! Y Move
Single Move   >>>  Single Move
Single Move   >>>  X Move !! Y Move
```

All combinations of the different types are valid:

```
X Arith !! Y Arith   |||  X Move !! Y Move
X Move !! Y Move     >>>  Single Move
X Move !! Y Move     >>>  X Move !! Y Move
```

The *Single Arithmetic* instructions are summarized in alphabetical order in Table 36.

TABLE 36: SINGLE ARITHMETIC INSTRUCTIONS

Mnemonic and Operands	Description
ABS d-reg, d-reg	Calculate the absolute value of a data register
ADD d-reg, d-reg, d-reg	Add two data registers, storing the result in a data register
ADD d-reg, d-reg, imm3s	Add a value to a data register, storing the result in a data register
ADD d-reg, d-reg, cond3	Add a data register to a data register if the condition holds
ADD d-reg, imm3s, cond3	Add a value to a data register if the condition holds
ADD.DW d-reg, d-reg, imm3s	Add a double-width value to a data register, storing the result in a data register
ADD.DW d-reg, imm3s, cond3	Add a double-width value to a data register, if the condition holds
ADD.FR d-reg, d-reg, imm3s	Add a fractional immediate value to a data register, storing the result in a data register
ADD.NX d-reg, d-reg, d-reg	Add two data registers with no extension, storing the result in a data register

TABLE 36: SINGLE ARITHMETIC INSTRUCTIONS (CONTINUED)

Mnemonic and Operands	Description
ADD.NX d-reg, d-reg, imm3s	Add a value to a data register with no extension, storing the result in a data register
ADD.NX.DW d-reg, d-reg, imm3s	Add a double-width value to a data register with no extension, storing the result in a data register
ADDMUL ab-reg, xy-reg, xy-reg, xy-reg	Add two XY data registers, multiply the result by a third XY data register, and store the result in an accumulator
ADDMULADD ab-reg, xy-reg, xy-reg, xy-reg	Add two XY data registers, multiply the result by a third XY data register, and add the result to an accumulator
ADDMULNEG ab-reg, xy-reg, xy-reg, xy-reg	Add two XY data registers, multiply the result by a third XY data register, negate the result and store it in an accumulator
ADDMULSUB ab-reg, xy-reg, xy-reg, xy-reg	Add two XY data registers, multiply the result by a third XY data register, and subtract the result from an accumulator
ADDSH d-reg, d-reg, d-reg	Add two data registers and shift right one bit, storing the result in a data register
AND d-reg, d-reg, d-reg	Bitwise AND two data registers, storing the result in a data register
AND d-reg, d-reg, imm3s	Bitwise AND a data register and a value, storing the result in a data register
AND.DW d-reg, d-reg, imm3s	Bitwise AND a data register and a double-width value, storing the result in a data register
BITCLR d-reg, bit-index	Clear a bit in a data register
BITSET d-reg, bit-index	Set a bit in a data register
BITTGL d-reg, bit-index	Toggle a bit in a data register
BITTST d-reg, bit-index	Test a bit in a data register
CMP d-reg, d-reg	Compare two data registers
CMP d-reg, imm3s	Compare a data register to a value
CMP.DW d-reg, imm3s	Compare a data register to a double-width value
CMPU.NX d-reg, d-reg	Compare two data registers as unsigned values with no extension
CMPU.NX d-reg, imm3s	Compare a data register to a value as unsigned values
CMPU.NX.DW d-reg, imm3s	Compare a data register to a double-width value as unsigned values
DIVST d-reg, d-reg, d-reg	Division step for dividing data register by data register, result to data register
LOAD d-reg, imm6s	Load data register with a value
LOAD.DW d-reg, imm6s	Load data register with a double-width value

TABLE 36: SINGLE ARITHMETIC INSTRUCTIONS (CONTINUED)

Mnemonic and Operands	Description
LOG2ABS d-reg, d-reg	Calculate the logarithm base 2 of the absolute value of a data register, storing the result in a data register
MAX d-reg, d-reg, d-reg	Determine the maximum value of two data registers, storing the result in a data register
MIN d-reg, d-reg, d-reg	Determine the minimum value of two data registers, storing the result in a data register
MOVE d-reg, d-reg	Move data register to data register
MOVE d-reg, d-reg, cond3	Move data register to data register, conditionally
MOVE.NX d-reg, d-reg	Move data register to data register with no extension
MUL ab-reg, xy-reg, xy-reg	Multiply two XY data registers, storing the result in an accumulator
MUL.SU ab-reg, xy-reg, xy-reg	Multiply two XY data registers, one signed and one unsigned, storing the result in an accumulator
MUL.UU ab-reg, xy-reg, xy-reg	Multiply two XY data registers, both unsigned, storing the result in an accumulator
MULADD ab-reg, xy-reg, xy-reg	Multiply two XY data registers, and add the result to an accumulator
MULADD.SU ab-reg, xy-reg, xy-reg	Multiply two XY data registers, one signed and one unsigned, and add the result to an accumulator
MULADD.UU ab-reg, xy-reg, xy-reg	Multiply two XY data registers, both unsigned, and add the result to an accumulator
MULNEG ab-reg, xy-reg, xy-reg	Multiply two XY data registers, negate the result and store it in an accumulator
MULNEG.SU ab-reg, xy-reg, xy-reg	Multiply two XY data registers, one signed and one unsigned, negate the result and store it in an accumulator
MULNEG.UU ab-reg, xy-reg, xy-reg	Multiply two XY data registers, both unsigned, negate the result and store it in an accumulator
MULSUB ab-reg, xy-reg, xy-reg	Multiply two XY data registers, and subtract the result from an accumulator
MULSUB.SU ab-reg, xy-reg, xy-reg	Multiply two XY data registers, one signed and one unsigned, and subtract the result from an accumulator
MULSUB.UU ab-reg, xy-reg, xy-reg	Multiply two XY data registers, both unsigned, and subtract the result from an accumulator
NEG d-reg, d-reg	Negate a data register, storing the result in a data register
NEG d-reg, d-reg, cond3	Negate a data register, storing the result in a data register, conditionally

TABLE 36: SINGLE ARITHMETIC INSTRUCTIONS (CONTINUED)

Mnemonic and Operands	Description
NLOG2ABS d-reg, d-reg	Calculate the logarithm base 2 of the absolute value of a data register, negate the result, and store the result in a data register
NOP	No operation
NOT d-reg, d-reg	Bitwise NOT a data register, storing the result in a data register
OR d-reg, d-reg, d-reg	Bitwise OR two data registers, storing the result in a data register
OR d-reg, d-reg, imm3s	Bitwise OR a data register and a value, storing the result in a data register
OR.DW d-reg, d-reg, imm3s	Bitwise OR a data register and a double-width value, storing the result in a data register
SHLL d-reg, d-reg, d-reg	Shift a data register left logically, by a count from a data register, storing the result in a data register
SHLL d-reg, d-reg, imm3u	Shift a data register left logically, by an immediate count, storing the result in a data register
SHLL d-reg, imm6u	Shift a data register left logically, by an immediate count
SHLL d-reg, d-reg, cond3	Shift a data register left logically, by a count from a data register, conditionally
SHLL d-reg, imm3u, cond3	Shift a data register left logically, by an immediate count, conditionally
SHLL.NX d-reg, d-reg, d-reg	Shift a data register left logically, with no extension, by a count from a data register, storing the result in a data register
SHLL.NX d-reg, d-reg, imm3u	Shift a data register left logically, with no extension, by an immediate count, storing the result in a data register
SHLL.NX d-reg, imm6u	Shift a data register left logically, with no extension, by an immediate count
SHRA d-reg, d-reg, d-reg	Shift a data register right arithmetically, by a count from a data register, storing the result in a data register
SHRA d-reg, d-reg, imm3u	Shift a data register right arithmetically, by an immediate count, storing the result in a data register
SHRA d-reg, imm6u	Shift a data register right arithmetically, by an immediate count
SHRA d-reg, d-reg, cond3	Shift a data register right arithmetically, by a count from a data register, conditionally
SHRA d-reg, imm3u, cond3	Shift a data register right arithmetically, by an immediate count, conditionally

TABLE 36: SINGLE ARITHMETIC INSTRUCTIONS (CONTINUED)

Mnemonic and Operands	Description
SHRL.NX d-reg, d-reg, d-reg	Shift a data register right logically, with no extension, by a count from a data register, storing the result in a data register
SHRL.NX d-reg, d-reg, imm3u	Shift a data register right logically, with no extension, by an immediate count, storing the result in a data register
SHRL.NX d-reg, imm6u	Shift a data register right logically, with no extension, by an immediate count
SUB d-reg, d-reg, d-reg	Subtract two data registers, storing the result in a data register
SUB d-reg, d-reg, imm3sn	Subtract a value from a data register, storing the result in a data register
SUB d-reg, d-reg, cond3	Subtract a data register from a data register, conditionally
SUB d-reg, imm3sn, cond3	Subtract a value from a data register if the condition holds
SUB.DW d-reg, d-reg, imm3sn	Subtract a double-width value from a data register, storing the result in a data register
SUB.DW d-reg, imm3sn, cond3	Subtract a double-width value from a data register, if the condition holds
SUB.FR d-reg, d-reg, imm3s	Subtract a fractional immediate value from a data register, storing the result in a data register
SUB.NX d-reg, d-reg, d-reg	Subtract two data registers with no extension, storing the result in a data register
SUB.NX d-reg, d-reg, imm3sn	Subtract a value from a data register with no extension, storing the result in a data register
SUB.NX.DW d-reg, d-reg, imm3sn	Subtract a double-width value from a data register with no extension, storing the result in a data register
SUBMUL ab-reg, xy-reg, xy-reg, xy-reg	Subtract two XY data registers, multiply the result by a third XY data register, and store the result in an accumulator
SUBMULADD ab-reg, xy-reg, xy-reg, xy-reg	Subtract two XY data registers, multiply the result by a third XY data register, and add the result to an accumulator
SUBMULNEG ab-reg, xy-reg, xy-reg, xy-reg	Subtract two XY data registers, multiply the result by a third XY data register, negate the result and store it in an accumulator
SUBMULSUB ab-reg, xy-reg, xy-reg, xy-reg	Subtract two XY data registers, multiply the result by a third XY data register, and subtract the result from an accumulator
SUBSH d-reg, d-reg, d-reg	Subtract two data registers and shift right one bit, storing the result in a data register

TABLE 36: SINGLE ARITHMETIC INSTRUCTIONS (CONTINUED)

Mnemonic and Operands	Description
SWAP d-reg, d-reg, cond3	Swap the contents of two data registers, conditionally
XOR d-reg, d-reg, d-reg	Bitwise XOR two data registers, storing the result in a data register
XOR d-reg, d-reg, imm3s	Bitwise XOR a data register and a value, storing the result in a data register
XOR.DW d-reg, d-reg, imm3s	Bitwise XOR a data register and a double-width value, storing the result in a data register

The *X Arithmetic* instructions are summarized in alphabetical order in Table 37.

TABLE 37: X ARITHMETIC INSTRUCTIONS

Mnemonic and Operands	Description
ABS a-reg, ab-reg	Calculate the absolute value of an accumulator, placing the result in an A accumulator
ADD a-reg, a-reg, ab-reg	Add two accumulators, storing the result in an A accumulator
ADD.NX a-reg, a-reg, ab-reg	Add two accumulators with no extension, storing the result in an A accumulator
ADDSH a-reg, a-reg, ab-reg	Add two accumulators and shift right one bit, storing the result in an A accumulator
AND a-reg, a-reg, ab-reg	Bitwise AND two accumulators, storing the result in an A accumulator
CMP a-reg, ab-reg	Compare two accumulators
CMPU.NX a-reg, ab-reg	Compare two accumulators as unsigned values with no extension
MAX a-reg, a-reg, ab-reg	Determine the maximum value of two accumulators, and store the result in an A accumulator
MIN a-reg, a-reg, ab-reg	Determine the minimum value of two accumulators, and store the result in an A accumulator
MOVE.NX a-reg, ab-reg	Move accumulator to A accumulator with no extension
MUL a-reg, x-reg, xy-reg	Multiply two XY data registers, storing the result in an A accumulator
MULADD a-reg, x-reg, xy-reg	Multiply two XY data registers, and add the result to an A accumulator
MULNEG a-reg, x-reg, xy-reg	Multiply two XY data registers, negate the result and store it in an A accumulator
MULSUB a-reg, x-reg, xy-reg	Multiply two XY data registers, and subtract the result from an A accumulator

TABLE 37: X ARITHMETIC INSTRUCTIONS (CONTINUED)

Mnemonic and Operands	Description
NEG a-reg, ab-reg	Negate an accumulator, storing the result in an A accumulator
NLOG2ABS a-reg, ab-reg	Calculate the logarithm base 2 of the absolute value of an accumulator, negate the result, and store the result in an A accumulator
NOP	No operation
OR a-reg, a-reg, ab-reg	Bitwise OR an A accumulator and an accumulator, storing the result in an A accumulator
SUB a-reg, a-reg, ab-reg	Subtract two accumulators, storing the result in an A accumulator
SUB.NX a-reg, a-reg, ab-reg	Subtract two accumulators with no extension, storing the result in an A accumulator
SUBSH a-reg, a-reg, ab-reg	Subtract two accumulators and shift right one bit, storing the result in an A accumulator

The *Y Arithmetic* instructions are summarized in alphabetical order in Table 38.

TABLE 38: Y ARITHMETIC INSTRUCTIONS

Mnemonic and Operands	Description
ADD b-reg, b-reg, ab-reg	Add two accumulators, storing the result in a B accumulator
ADDSH b-reg, b-reg, ab-reg	Add two accumulators and shift right one bit, storing the result in a B accumulator
MUL b-reg, y-reg, xy-reg	Multiply two XY data registers, storing the result in a B accumulator
MULADD b-reg, y-reg, xy-reg	Multiply two XY data registers, and add the result to a B accumulator
MULNEG b-reg, y-reg, xy-reg	Multiply two XY data registers, negate the result and store it in a B accumulator
MULSUB b-reg, y-reg, xy-reg	Multiply two XY data registers, and subtract the result from a B accumulator
SUB b-reg, b-reg, ab-reg	Subtract two accumulators, storing the result in a B accumulator
SUBSH b-reg, b-reg, ab-reg	Subtract two accumulators and shift right one bit, storing the result in a B accumulator

The *Long* instructions are summarized in alphabetical order in Table 39.

TABLE 39: LONG INSTRUCTIONS

Mnemonic and Operands	Description
ADD SP, imm16	Add a value to the stack pointer
ADD xp-reg, SP, imm16	Add the stack pointer and a value, storing the result in an X pointer
ADD xp-reg, xp-reg, imm16	Add a value to an X pointer, storing the result in an X pointer
BITCLR SR, imm4u	Clear a bit in the status register
BITSET SR, imm4u	Set a bit in the status register
BREAKPOINT	Breakpoint
CALL.1D imm16u	Call direct with one delay slot
CALL.2D xp-reg	Call indirect with two delay slots
ENDLOOP	End loop
GOTO imm16u	Branch direct without delay slot
GOTO imm16u, cond5	Branch direct without delay slots, conditionally
GOTO.0D imm16u	Branch direct without delay slot
GOTO.0D imm16u, cond5	Branch direct without delay slots, conditionally
GOTO.1D imm16u	Branch direct with one delay slot
GOTO.2D imm16u, cond5	Branch direct with two delay slots, conditionally
GOTO.2D xp-reg	Branch indirect with two delay slots
GOTO.2D xp-reg, cond5	Branch indirect with two delay slots, conditionally
INTERRUPT imm5u	Software interrupt
LOAD reg, X[stack-mode16]	Load register from stack
LOAD reg, X[imm16u]	Load register direct from X memory
LOAD reg, Y[imm16u]	Load register direct from Y memory
LOAD reg, P[imm16u]	Load register direct from P memory
LOAD reg, P[single-mode(X)]	Load register indirect from P memory
LOAD reg, P[single-mode(Y)]	Load register indirect from P memory
LOAD reg, imm20s	Load register with a value
LOAD d-reg, imm24s	Load data register with a value
LOAD l-reg, imm24s	Load accumulator low part with a value
LOAD.DW ab-reg, XY[imm16u]	Load double-width accumulator direct from XY memory
LOAD.DW ab-reg, imm24s	Load accumulator with a double-width value
LOAD.EX ab-reg, P[imm16u]	Load extended accumulator direct from P memory
LOAD.EX ab-reg, P[single-mode(X)]	Load extended accumulator indirect via X pointer from P memory
LOOP.2D imm12u, imm16u	Loop with immediate count and two delay slots
LOOP.2D d-reg, imm16u	Loop with data register count and two delay slots

TABLE 39: LONG INSTRUCTIONS (CONTINUED)

Mnemonic and Operands	Description
NOP	No operation
RETURN.2D	Return from subroutine with two delay slots
RETURNI	Return from interrupt without delay slots
RETURNI.0D	Return from interrupt without delay slots
RETURNI.2D	Return from interrupt with two delay slots
SLEEP	Sleep
SLEEP imm16u	Sleep with restart location
STORE X[stack-mode16], reg	Store register to stack
STORE X[imm16u], reg	Store register direct to X memory
STORE Y[imm16u], reg	Store register direct to Y memory
STORE P[imm16u], reg	Store register direct to P memory
STORE P[single-mode(X)], reg	Store register indirect to P memory
STORE P[single-mode(Y)], reg	Store register indirect to P memory
STORE X[imm16u], imm8s	Store a value direct to X memory
STORE Y[imm16u], imm8s	Store a value direct to Y memory
STORE P[imm16u], imm8s	Store a value direct to P memory
STORE.DW XY[imm16u], ab-reg	Store double-width accumulator direct to XY memory
STORE.DW XY[imm16u], imm8s	Store a double-width immediate value direct to XY memory
STORE.EX P[imm16u], ab-reg	Store extended accumulator direct to P memory
STORE.EX P[single-mode(X)], ab-reg	Store extended accumulator indirect via X pointer to P memory
SUB SP, imm16	Subtract a value from the stack pointer
SUB xp-reg, SP, imm16	Subtract a value from the stack pointer, storing the result in an X pointer
SUB xp-reg, xp-reg, imm16	Subtract a value from an X pointer, storing the result in an X pointer

The *Single Move* instructions are summarized in alphabetical order in Table 40.

TABLE 40: SINGLE MOVE INSTRUCTIONS

Mnemonic and Operands	Description
ADD SP, imm7s	Add a value to the stack pointer
ADD xp-reg, SP, imm6n	Add the stack pointer and a value, storing the result in an X pointer
ADD xp-reg, xp-reg, xstep-reg	Add an X pointer and an X step register, storing the result in an X pointer

TABLE 40: SINGLE MOVE INSTRUCTIONS (CONTINUED)

Mnemonic and Operands	Description
ADD xp-reg, xp-reg, imm3s	Add a value to an X pointer, storing the result in an X pointer
ADD yp-reg, yp-reg, ystep-reg	Add a Y pointer and a Y step register, storing the result in a Y pointer
ADD yp-reg, yp-reg, imm3s	Add a value to a Y pointer, storing the result in a Y pointer
CLR X[single-mode(X)]	Clear a word of X memory specified by an X pointer, with update
LOAD stack-reg6, X[stack-mode6]	Load register from stack
LOAD stack-reg4, X[stack-mode4]	Load register from stack
LOAD xmem-reg, X[single-mode(X)]	Load register indirect from X memory
LOAD by-reg, Y[single-mode(Y)]	Load B or Y data register indirect from Y memory
LOAD d-reg, imm5s	Load data register with a value
LOAD xstep-reg, imm5s	Load X step register with a value
LOAD ystep-reg, imm5s	Load Y step register with a value
LOAD.DW ab-reg, XY[single-mode(X)]	Load double-width accumulator indirect via X pointer from XY memory
LOAD.DW ab-reg, imm5s	Load accumulator with a double-width value
LOAD.EX ab-reg, P[single-mode(Y)]	Load extended accumulator indirect via Y pointer from P memory
MOVE reg, reg	Move register to register
MOVE.DW ab-reg, ab-reg	Move double-width accumulator to double-width accumulator
MOVE.EX ab-reg, ab-reg	Move extended accumulator to accumulator
NOP	No operation
STORE X[stack-mode6], stack-reg6	Store register to stack
STORE X[stack-mode4], stack-reg4	Store register to stack
STORE X[single-mode(X)], xmem-reg	Store register indirect to X memory
STORE Y[single-mode(Y)], by-reg	Store B or Y data register indirect to Y memory
STORE.DW XY[single-mode(X)], ab-reg	Store double-width accumulator indirect via X pointer to XY memory
STORE.EX P[single-mode(Y)], ab-reg	Store extended accumulator indirect via Y pointer to P memory
SUB SP, imm7sn	Subtract a value from the stack pointer
SUB xp-reg, SP, imm6p	Subtract a value from the stack pointer, storing the result in an X pointer
SUB xp-reg, xp-reg, imm3sn	Subtract a value from an X pointer, storing the result in an X pointer
SUB yp-reg, yp-reg, imm3sn	Subtract a value from a Y pointer, storing the result in a Y pointer

TABLE 40: SINGLE MOVE INSTRUCTIONS (CONTINUED)

Mnemonic and Operands	Description
SUBSTEP xp-reg	Subtract an X step register from the corresponding X pointer
SUBSTEP yp-reg	Subtract a Y step register from the corresponding Y pointer

The *X Move* instructions are summarized in alphabetical order in Table 41.

TABLE 41: X MOVE INSTRUCTIONS

Mnemonic and Operands	Description
LOAD ax-reg, X[dual-mode(X)]	Load A or X data register indirect from X memory
LOAD ax-reg, imm2s	Load A or X data register with a value
MOVE ab-reg, x-reg	Move X data register to accumulator
MOVE xy-reg, a-reg	Move A accumulator to XY data register
STORE X[dual-mode(X)], a-reg	Store A accumulator indirect to X memory

The *Y Move* instructions are summarized in alphabetical order in Table 42.

TABLE 42: Y MOVE INSTRUCTIONS

Mnemonic and Operands	Description
LOAD by-reg, Y[dual-mode(Y)]	Load B or Y data register indirect from Y memory
LOAD by-reg, imm2s	Load B or Y data register with a value
MOVE ab-reg, y-reg	Move Y data register to accumulator
MOVE xy-reg, b-reg	Move B accumulator to XY data register
STORE Y[dual-mode(Y)], b-reg	Store B accumulator indirect to Y memory

10.4 INSTRUCTION DETAILS

Detailed descriptions of each instruction in the CFX instruction set begin with “ABS d-reg, d-reg” on page 119. Each instruction page provides a short description, operands, the operation that the instruction performs, the flags that are affected, a detailed description, the execution time, and an opcode. To avoid extraneous or unnecessarily repetitive information, the following subsections explain some common material before each instruction is described.

10.4.1 Instruction Types

The top right corner of each new instruction page shows its instruction type. Sometimes, abbreviations are used to save space.Table 43 shows the meanings of these instruction type abbreviations.

TABLE 43: INSTRUCTION TYPE ABBREVIATIONS

Abbreviation	Instruction Type
Long	*Long* instruction
Synth	No-delay-slot instruction synthesized by the assembler. A *Long* instruction followed by NOP(s) in the delay slot(s).
Move	*Single Move* instruction
X Move	*X Move* instruction
Y Move	*Y Move* instruction
Arith	*Single Arithmetic* instruction
X Arith	*X Arithmetic* instruction
Y Arith	*Y Arithmetic* instruction

10.4.2 Operation Descriptions

Each instruction provides a short description of the operation that the instruction performs. These descriptions are expressed in a sort of pseudo-code using C-style operators, order of operations, and functions. The following special functions are used in these descriptions:

signext(x) If *x* is an immediate value, this is the sign-extension of *x*, from its natural width to the width of other operands.

In instructions that end with `.NX`, this function first discards the upper 8 bits of the 56-bit value, *x*, then sign-extends this 48-bit value to 56 bits.

fractional(x) The fractional value corresponding to the 3-bit signed immediate value, *x*. See “ADD.FR d-reg, d-reg, imm3s” on page 140.

10.4.3 Instruction Encodings

Instruction words encode instructions in the combinations described in Chapter 3, “Instruction Types” on page 9. The upper one or two bits of the CFX instruction word indicate the type of the instruction word. The remaining bits are filled in with the encodings for the instructions and operands of the composed instructions.

The high two bits of a *Long* instruction word are `00`, as shown in Figure 12. For *Long* instructions, the instruction details show bits 0-29 of the instruction encoding.

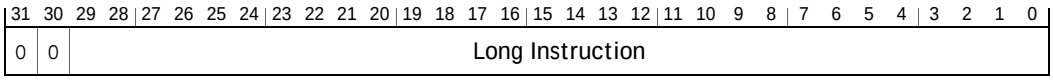


Figure 12: Long Instruction Encoding

The high two bits of a *Move-Move* instruction word are `01`, as shown in Figure 13. The *Move* instruction in bits 0-14 is executed in the first cycle, then the *Move* instruction in bits 15-29 is executed in the second cycle.

In the instruction details, the *Move* instruction encoding bits are numbered 0-14. For the second *Move* instruction in an instruction word, the encoding is shifted left 15 bits.

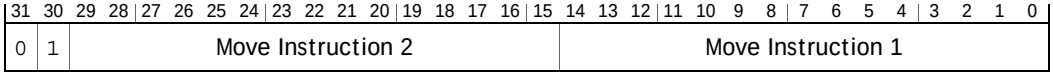


Figure 13: Move-Move Instruction Encoding

The high bit of an *Arithmetic-Move* instruction word is `1`, as shown in Figure 14. For *Arithmetic* instructions, the instruction details show bits 15-30 of the instruction encoding.

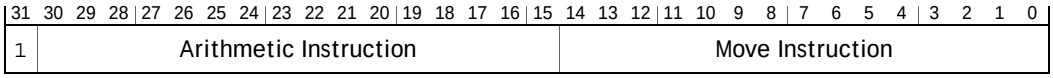


Figure 14: Arithmetic-Move Instruction Encoding

10.4.4 Operand Classes

The instruction details refer to instruction operands by class and name. Use the class to determine the allowed values for the operand. The descriptions and tables in this section map these values to the encoding in the instruction word. The name of the operand is used to find the width and placement of the field where the operand is encoded in the instruction word.

Immediate Operands

Most immediate values use the following notation:

- imm*N*s represents a signed immediate operand where *N* is the size in bits. The value can be between -2^{N-1} and $2^{N-1}-1$.
- imm*N*u represents an unsigned immediate operand where *N* is the size in bits. The value can be between 0 and $2^{N}-1$.
- imm*N*n represents a negative immediate operand where *N* is the size in bits. The value can be between -2^{N} and -1.
- imm*N* represents an immediate operand where *N* is the size in bits. The value can be signed or unsigned, and can be between -2^{N-1} and $2^{N}-1$.

For these immediate field types, the *N*-bit representation of the value is encoded into the field in the instruction.

Some immediate values use the following notation:

- imm*N*p represents a positive immediate operand where *N* is the size in bits. The value can be between 1 and 2^{N}.
- imm*N*sn represents a negated signed immediate operand where *N* is the size in bits. The value can be between $-2^{N-1}+1$ and 2^{N-1}.

For these immediate field types, the *N*-bit two's complement of the *N*-bit representation of the value is encoded into the field in the instruction.

bit-index Operand

The **bit-index** field type (used in single-bit operations) is an imm6u. For accumulator destinations, use a value in the range 0 to 55; the 6-bit representation is encoded into the field in the instruction. For XY data register destinations, use a value in the range 0 to 23; in this case, the assembler adds 24 to the value before the value is encoded into the instruction.

stack-mode16 Operand

The addressing mode **stack-mode16** is an indirect access to the stack in X memory via the stack pointer with a 16-bit signed offset. The syntax is "SP" or "SP + imm16u" or "SP - imm16u". The 16-bit immediate offset is stored in the field in the instruction.

stack-mode6 Operand

The addressing mode **stack-mode6** is an indirect access to the stack in X memory via the stack pointer with a 6-bit negative offset (-64..-1). The syntax is "SP - imm6n". The 6-bit representation of this negative offset is stored in the field in the instruction.

stack-mode4 Operand

The addressing mode **stack-mode4** is an indirect access to the stack in X memory via the stack pointer with a 4-bit negative offset (-16..-1). The syntax is "SP - imm4n". The 4-bit representation of this negative offset is stored in the field in the instruction.

single-mode(X) and single-mode(Y) Operands

The addressing modes **single-mode(X)** and **single-mode(Y)** are indirect accesses to X and Y memory, respectively, via X and Y pointers with possible pointer updates. These are the addressing modes available in *Single Move* instructions:

- `pointer-register`
- `pointer-register ++`
- `pointer-register --`
- `pointer-register += 2`
- `pointer-register -= 2`
- `pointer-register += step-register`
- `pointer-register -= step-register`
- `pointer-register +%= step-register`

Version 3 or greater of the CFX assembler supports the following synonyms for addressing modes:

- `pointer-register +` and `pointer-register += 1` are synonyms for `pointer-register ++`.
- `pointer-register -` and `pointer-register -= 1` are synonyms for `pointer-register --`.
- `pointer-register +!` is a synonym for `pointer-register += step-register`.
- `pointer-register -!` is a synonym for `pointer-register -= step-register`.
- `pointer-register +%!` is a synonym for `pointer-register +%= step-register`.

The pointer-register and step-register must match (e.g., `XP5 -= XSTEP5`). The single-mode(X) addressing mode is encoded into a 6-bit field shown in Figure 15. The single-mode(Y) addressing mode is encoded into a 5-bit field shown in Figure 16. The single addressing mode encoding is

shown in Table 44. The register encodings are shown in “xp-reg - X Pointers” on page 114 and “yp-reg - Y Pointers” on page 115.

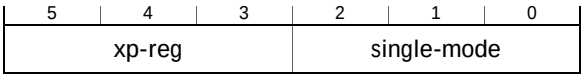


Figure 15: single-mode(X) Operand Encoding

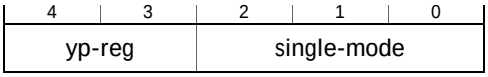


Figure 16: single-mode(Y) Operand Encoding

TABLE 44: “SINGLE-MODE” - ADDRESSING MODES FOR SINGLE MOVE INSTRUCTIONS

Addressing Mode	Encoding
`pointer-register`	`000`
`pointer-register ++`	`001`
`pointer-register --`	`010`
`pointer-register += 2`	`011`
`pointer-register -= 2`	`100`
`pointer-register += step-register`	`101`
`pointer-register -= step-register`	`110`
`pointer-register +%= step-register`	`111`

dual-mode(X) and dual-mode(Y) Operands

The addressing modes **dual-mode(X)** and **dual-mode(Y)** are also indirect accesses to X and Y memory, respectively, via X and Y pointers with possible pointer updates. These are the addressing modes available in *Dual Move* instructions:

- `pointer-register`
- `pointer-register ++`
- `pointer-register += step-register`
- `pointer-register +%= step-register`

Version 3 or greater of the CFX assembler supports the following synonyms for addressing modes:

- `pointer-register +` and `pointer-register += 1` are synonyms for `pointer-register ++`.
- `pointer-register +!` is a synonym for `pointer-register += step-register`.
- `pointer-register +%!` is a synonym for `pointer-register +%= step-register`.

Again, the pointer-register and step-register must match. Furthermore, for dual-mode(X), only the first four X pointers, `XP0` to `XP3`, may be used. The dual-mode(X) addressing mode is encoded into a 4-bit field shown in Figure 17. The dual-mode(Y) addressing mode is encoded into a 4-bit field shown in Figure 18. The single addressing mode encoding is shown in Table 45. The register

encodings are shown in “dual-xp-reg - X Pointers for Dual Move Instructions” on page 114 and “yp-reg - Y Pointers” on page 115.

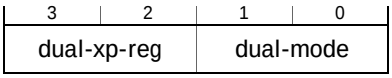


Figure 17: dual-mode(X) Operand Encoding

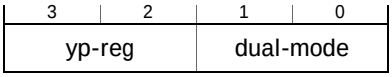


Figure 18: dual-mode(Y) Operand Encoding

TABLE 45: “DUAL-MODE” - ADDRESSING MODES FOR DUAL MOVE INSTRUCTIONS

Addressing Mode	Encoding
`pointer-register`	`00`
`pointer-register ++`	`01`
`pointer-register += step-register`	`10`
`pointer-register +%= step-register`	`11`

The remaining operand classes are used to specify registers or conditions. The values and encodings for these operand classes are summarized in the remaining tables in this section.

a-reg - A Accumulators

Register	Encoding
`A0`	`0`
`A1`	`1`

b-reg - B Accumulators

Register	Encoding
`B0`	`0`
`B1`	`1`

ab-reg - All Accumulators

Register	Encoding
`A0`	`00`
`A1`	`01`

Register	Encoding
B0	10
B1	11

x-reg - X Data Registers

Register	Encoding
X0	0
X1	1

y-reg - Y Data Registers

Register	Encoding
Y0	0
Y1	1

xy-reg - All XY Data Registers

Register	Encoding
X0	00
X1	01
Y0	10
Y1	11

ax-reg - X Side Data Registers

Register	Encoding
A0	00
A1	01
X0	10
X1	11

by-reg - Y Side Data Registers

Register	Encoding
B0	00
B1	01
Y0	10
Y1	11

d-reg - All Data Registers

Register	Encoding
A0	000
A1	001
B0	010
B1	011
X0	100
X1	101
Y0	110
Y1	111

l-reg - Accumulator Low Parts

Register	Encoding
A0.L	00
A1.L	01
B0.L	10
B1.L	11

xp-reg - X Pointers

Register	Encoding
XP0	000
XP1	001
XP2	010
XP3	011
XP4	100
XP5	101
XP6	110
XP7	111

dual-xp-reg - X Pointers for Dual Move Instructions

Register	Encoding
XP0	00
XP1	01
XP2	10
XP3	11

yp-reg - Y Pointers

Register	Encoding
YP0	00
YP1	01
YP2	10
YP3	11

xstep-reg - X Pointer Step Registers

Register	Encoding
XSTEP0	000
XSTEP1	001
XSTEP2	010
XSTEP3	011
XSTEP4	100
XSTEP5	101
XSTEP6	110
XSTEP7	111

ystep-reg - Y Pointer Step Registers

Register	Encoding
YSTEP0	00
YSTEP1	01
YSTEP2	10
YSTEP3	11

reg - All Registers

Register	Encoding	Register	Encoding	Register	Encoding	Register	Encoding
A0	000000	XSTEP0	010000	XCIRC0	100000	LR	110000
A1	000001	XSTEP1	010001	XCIRC1	100001	ILPC	110001
B0	000010	XSTEP2	010010	XCIRC2	100010	SR	110010
B1	000011	XSTEP3	010011	XCIRC3	100011	ILSR	110011
X0	000100	XSTEP4	010100	XCIRC4	100100	A0.L	110100
X1	000101	XSTEP5	010101	XCIRC5	100101	A1.L	110101
Y0	000110	XSTEP6	010110	XCIRC6	100110	B0.L	110110
Y1	000111	XSTEP7	010111	XCIRC7	100111	B1.L	110111

Register	Encoding	Register	Encoding	Register	Encoding	Register	Encoding
XP0	001000	YP0	011000	YCIRC0	101000	A0.H	111000
XP1	001001	YP1	011001	YCIRC1	101001	A1.H	111001
XP2	001010	YP2	011010	YCIRC2	101010	B0.H	111010
XP3	001011	YP3	011011	YCIRC3	101011	B1.H	111011
XP4	001100	YSTEP0	011100	LC	101100	A0.E	111100
XP5	001101	YSTEP1	011101	LSA	101101	A1.E	111101
XP6	001110	YSTEP2	011110	LEA	101110	B0.E	111110
XP7	001111	YSTEP3	011111	SP	101111	B1.E	111111

xmem-reg - for Indirect Load/Store to X Memory

Register	Encoding	Register	Encoding	Register	Encoding	Register	Encoding
A0	00000	XP0	01000	A0.H	10000	YP0	11000
A1	00001	XP1	01001	A1.H	10001	YP1	11001
B0	00010	XP2	01010	B0.H	10010	YP2	11010
B1	00011	XP3	01011	B1.H	10011	YP3	11011
X0	00100	XP4	01100	A0.L	10100	A0.E	11100
X1	00101	XP5	01101	A1.L	10101	A1.E	11101
Y0	00110	XP6	01110	B0.L	10110	B0.E	11110
Y1	00111	XP7	01111	B1.L	10111	B1.E	11111

stack-reg6 - for Load/Store to Stack, 6-Bit Offset

Register	Encoding	Register	Encoding	Register	Encoding	Register	Encoding
A0	00000	XP0	01000	LR	10000	A0.H	11000
A1	00001	XP1	01001	ILPC	10001	A1.H	11001
B0	00010	XP2	01010	SR	10010	B0.H	11010
B1	00011	XP3	01011	ILSR	10011	B1.H	11011
X0	00100	XP4	01100	A0.L	10100	A0.E	11100
X1	00101	XP5	01101	A1.L	10101	A1.E	11101
Y0	00110	XP6	01110	B0.L	10110	B0.E	11110
Y1	00111	XP7	01111	B1.L	10111	B1.E	11111

stack-reg4 - for Load/Store to Stack, 4-Bit Offset

Register	Encoding
YP0	000
YP1	001

Register	Encoding
YP2	010
YP3	011
LC	100
LSA	101
LEA	110
SP	111

cond5 - 5-Bit Condition Field

Conditions	Encoding	Conditions	Encoding
CARRY or X_CARRY	00000	Y_CARRY	10000
NO_CARRY or X_NO_CARRY	00001	Y_NO_CARRY	10001
EQ or X_EQ or ZERO or X_ZERO	00010	Y_EQ or Y_ZERO	10010
NE or X_NE or NOT_ZERO or X_NOT_ZERO	00011	Y_NE or Y_NOT_ZERO	10011
LT or X_LT or LTZ[1] or X_LTZ[1]	00100	Y_LT or Y_LTZ[1]	10100
GE or X_GE or GEZ[1] or X_GEZ[1]	00101	Y_GE or Y_GEZ[1]	10101
LE or X_LE or LEZ[1] or X_LEZ[1]	00110	Y_LE or Y_LEZ[1]	10110
GT or X_GT or GTZ[1] or X_GTZ[1]	00111	Y_GT or Y_GTZ[1]	10111
EXTEN or X_EXTEN	01000	Y_EXTEN	11000
NO_EXTEN or X_NO_EXTEN	01001	Y_NO_EXTEN	11001
OFLOW or X_OFLOW	01010	Y_OFLOW	11010
NO_OFLOW or X_NO_OFLOW	01011	Y_NO_OFLOW	11011
FULL	01100	SAT	11100
NOT_FULL	01101	NO_SAT	11101
EMPTY	01110	FALSE or 0[1]	11110
NOT_EMPTY	01111	TRUE or 1[1]	11111

1. This condition requires version 3 or greater of the CFX assembler.

Version 3 or greater of the CFX assembler supports using the "!" operator to select the logical complement of a condition.

cond3 - 3-Bit Condition Field

Conditions	Encoding
CARRY or X_CARRY	000
NO_CARRY or X_NO_CARRY	001
EQ or X_EQ or ZERO or X_ZERO	010
NE or X_NE or NOT_ZERO or X_NOT_ZERO	011
LT or X_LT or LTZ[1] or X_LTZ[1]	100
GE or X_GE or GEZ[1] or X_GEZ[1]	101
LE or X_LE or LEZ[1] or X_LEZ[1]	110
GT or X_GT or GTZ[1] or X_GTZ[1]	111

1. This condition requires version 3 or greater of the CFX assembler.

Version 3 or greater of the CFX assembler supports using the "!" operator to select the logical complement of a condition.

ABS d-reg, d-reg **Arith**

Calculate the absolute value of a data register

Operands `ABS dst-reg, src-reg`

Operation dst-reg = |src-reg|

Flags If Flag Updates are enabled, the X ALU status flags are affected as follows:

X Carry:	Not affected
X Negative:	Set according to ALU output
X Zero:	Set according to ALU output
X Overflow:	Set according to ALU output
X Extension:	Set according to ALU output

Description Calculate the absolute value of a data register, `src-reg`, and store the result in a data register, `dst-reg`.

Execution Time 1 cycle

Opcode

30	29	28	27	26	25	24	23 22 21	20	19	18	17 16 15
0	1	1	1	0	1	0	dst-reg	0	0	0	src-reg

ABS a-reg, ab-reg **X Arith**

Calculate the absolute value of an accumulator, placing the result in an A accumulator

Operands `ABS dst-reg, src-reg`

Operation dst-reg = |src-reg|

Flags If Flag Updates are enabled, the X ALU status flags are affected as follows:

X Carry:	Not affected
X Negative:	Set according to ALU output
X Zero:	Set according to ALU output
X Overflow:	Set according to ALU output
X Extension:	Set according to ALU output

Description Calculate the absolute value of an accumulator, `src-reg`, and store the result in an A accumulator, `dst-reg`.

Execution Time 1 cycle

Constraints The Y side instruction must not be `MUL*`.

The right-most source register on the X and Y sides must be opposite—one side using `a-reg` and the other side `b-reg`.

Opcode

ABS a-reg, a-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	1	0	1	1	1	dst	0	0	src	-	-	-	0	-	-

ABS a-reg, b-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	1	0	1	1	1	dst	1	0	src	-	-	-	0	-	-

Bits marked with "-" are determined by the Y side arithmetic instruction.

ADD SP, imm16

Long

Add a value to the stack pointer

Operands `ADD SP, data`

Operation SP = SP + data

Flags Not affected

Description Add a 16-bit immediate value, `data`, to the stack pointer (`SP`), and store the result back in `SP`.

Execution Time 1 cycle

Opcode

29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	0	1	1	0	1	0	0	0	0	data															

ADD SP, imm7s

Move

Add a value to the stack pointer

Operands `ADD SP, data`

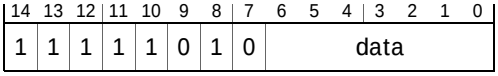
Operation SP = SP + signext(data)

Flags Not affected

Description Add a 7-bit signed immediate value, `data` (-64..63), to the stack pointer (`SP`), and store the result back in `SP`.

Execution Time 1 cycle

Opcode



14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	0	data						

If this instruction is the second move instruction in a Move-Move instruction word, the encoding is shifted left 15 bits.

ADD xp-reg, SP, imm16 **Long**

Add the stack pointer and a value, storing the result in an X pointer

Operands `ADD dst-reg, SP, data`

Operation dst-reg = SP + data

Flags Not affected

Description Add a 16-bit immediate value, `data`, to the stack pointer (`SP`), and store the result in an X pointer, `dst-reg`.

Execution Time 1 cycle

Opcode

29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	0	1	1	0	0	1	dst-reg			data															

ADD xp-reg, SP, imm6n

Move

Add the stack pointer and a value, storing the result in an X pointer

Operands `ADD dst-reg, SP, data`

Operation dst-reg = SP + data

Flags Not affected

Description Add a 6-bit negative immediate value, `data` (-64..-1), to the stack pointer (`SP`), and store the result in an X pointer, `dst-reg`.

Execution Time 1 cycle

Opcode

14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	dst-reg					data			

If this instruction is the second move instruction in a Move-Move instruction word, the encoding is shifted left 15 bits.

ADD xp-reg, xp-reg, xstep-reg

Move

Add an X pointer and an X step register, storing the result in an X pointer

Operands `ADD dst-reg, src-reg-1, src-reg-2`

Operation dst-reg = src-reg-1 + src-reg-2

Flags Not affected

Description Add an X pointer, `src-reg-1`, and any X step register, `src-reg-2`, and store the result in an X pointer, `dst-reg`.

Execution Time 1 cycle

Opcode

14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	dst-reg			src-reg-1			src-reg-2		

If this instruction is the second move instruction in a Move-Move instruction word, the encoding is shifted left 15 bits.

ADD xp-reg, xp-reg, imm16 **Long**

Add a value to an X pointer, storing the result in an X pointer

Operands `ADD dst-reg, src-reg, data`

Operation dst-reg = src-reg + data

Flags Not affected

Description Add a 16-bit immediate value, `data`, to an X pointer, `src-reg`, and store the result in an X pointer, `dst-reg`.

Execution Time 1 cycle

Opcode

29	28	27	26	25	24	23	22	21 20	19 18 17 16	15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0
1	1	1	0	1	0	1	0	dst-reg	src-reg	data

ADD xp-reg, xp-reg, imm3s

Move

Add a value to an X pointer, storing the result in an X pointer

Operands `ADD dst-reg, src-reg, data`

Operation dst-reg = src-reg + signext(data)

Flags Not affected

Description Add a 3-bit signed immediate value, `data` (-4..3), to an X pointer, `src-reg`, and store the result in an X pointer, `dst-reg`.

Execution Time 1 cycle

Opcode

14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	0	dst-reg			src-reg		data			

If this instruction is the second move instruction in a Move-Move instruction word, the encoding is shifted left 15 bits.

ADD yp-reg, yp-reg, ystep-reg

Move

Add a Y pointer and a Y step register, storing the result in a Y pointer

Operands `ADD dst-reg, src-reg-1, src-reg-2`

Operation dst-reg = src-reg-1 + src-reg-2

Flags Not affected

Description Add a Y pointer, `src-reg-1`, and any Y step register, `src-reg-2`, and store the result in a Y pointer, `dst-reg`.

Execution Time 1 cycle

Opcode

14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	0	0	0	dst		src1		src2	

If this instruction is the second move instruction in a Move-Move instruction word, the encoding is shifted left 15 bits.

ADD yp-reg, yp-reg, imm3s

Move

Add a value to a Y pointer, storing the result in a Y pointer

Operands `ADD dst-reg, src-reg, data`

Operation dst-reg = src-reg + signext(data)

Flags Not affected

Description Add a 3-bit signed immediate value, `data` (-4..3), to a Y pointer, `src-reg`, and store the result in a Y pointer, `dst-reg`.

Execution Time 1 cycle

Opcode

14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	1	dst		src	data			

If this instruction is the second move instruction in a Move-Move instruction word, the encoding is shifted left 15 bits.

ADD d-reg, d-reg, d-reg **Arith**

Add two data registers, storing the result in a data register

Operands `ADD dst-reg, src-reg-1, src-reg-2`

Operation dst-reg = src-reg-1 + src-reg-2

Flags If Flag Updates are enabled, the X ALU status flags are affected as follows:

X Carry:	Set according to ALU output
X Negative:	Set according to ALU output
X Zero:	Set according to ALU output
X Overflow:	Set according to ALU output
X Extension:	Set according to ALU output

Description Add two data registers, `src-reg-1` and `src-reg-2`, and store the result in a data register, `dst-reg`.

Execution Time 1 cycle

Opcode

30	29	28	27	26	25	24	23 22 21	20 19 18	17 16 15
0	1	0	0	0	0	0	dst-reg	src-reg-1	src-reg-2

ADD a-reg, a-reg, ab-reg **X Arith**

Add two accumulators, storing the result in an A accumulator

Operands `ADD dst-reg, src-reg-1, src-reg-2`

Operation dst-reg = src-reg-1 + src-reg-2

Flags If Flag Updates are enabled, the X ALU status flags are affected as follows:

X Carry:	Set according to ALU output
X Negative:	Set according to ALU output
X Zero:	Set according to ALU output
X Overflow:	Set according to ALU output
X Extension:	Set according to ALU output

Description Add an A accumulator, `src-reg-1`, and an A or B accumulator, `src-reg-2`, and store the result in an A accumulator, `dst-reg`.

Execution Time 1 cycle

Constraints When the Y side instruction is not `MUL*`, the right-most source register on the X and Y sides must be opposite—one side using `a-reg` and the other side `b-reg`.

Opcode

Y side instruction is not MUL*

ADD a-reg, a-reg, a-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	1	0	0	0	0	dst	0	s1	s2	-	-	-	0	-	-

ADD a-reg, a-reg, b-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	1	0	0	0	0	dst	1	s1	s2	-	-	-	0	-	-

Y side instruction is MUL*

ADD a-reg, a-reg, a-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	0	1	0	0	dst	0	s1	s2	-	-	-	-	-	-

ADD a-reg, a-reg, b-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	0	1	0	0	dst	1	s1	s2	-	-	-	-	-	-

Bits marked with “-” are determined by the Y side arithmetic instruction.

ADD b-reg, b-reg, ab-reg **Y Arith**

Add two accumulators, storing the result in a B accumulator

Operands `ADD dst-reg, src-reg-1, src-reg-2`

Operation dst-reg = src-reg-1 + src-reg-2

Flags If Flag Updates are enabled, the Y ALU status flags are affected as follows:

Y Carry:	Set according to ALU output
Y Negative:	Set according to ALU output
Y Zero:	Set according to ALU output
Y Overflow:	Set according to ALU output
Y Extension:	Set according to ALU output

Description Add a B accumulator, `src-reg-1`, and an A or B accumulator, `src-reg-2`, and store the result in a B accumulator, `dst-reg`.

Execution Time 1 cycle

Constraints When the X side instruction is not `MUL*`, the right-most source register on the X and Y sides must be opposite—one side using `a-reg` and the other side `b-reg`.

Opcode

X side instruction is not MUL*

ADD b-reg, b-reg, a-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	1	0	-	-	-	-	1	-	-	0	0	dst	-	s1	s2

ADD b-reg, b-reg, b-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	1	0	-	-	-	-	0	-	-	0	0	dst	-	s1	s2

X side instruction is MUL*

ADD b-reg, b-reg, a-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	0	0	-	-	-	-	-	-	0	0	dst	1	s1	s2

ADD b-reg, b-reg, b-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	0	0	-	-	-	-	-	-	0	0	dst	0	s1	s2

Bits marked with "-" are determined by the X side arithmetic instruction.

ADD d-reg, d-reg, imm3s **Arith**

Add a value to a data register, storing the result in a data register

Operands `ADD dst-reg, src-reg, data`

Operation dst-reg = src-reg + signext(data)

Flags If Flag Updates are enabled, the X ALU status flags are affected as follows:

X Carry:	Set according to ALU output
X Negative:	Set according to ALU output
X Zero:	Set according to ALU output
X Overflow:	Set according to ALU output
X Extension:	Set according to ALU output

Description Add a 3-bit signed immediate value, `data` (-4..3), to a data register, `src-reg`, and store the result in a data register, `dst-reg`.

Execution Time 1 cycle

Opcode

30	29	28	27	26	25	24	23 22 21	20 19 18	17 16 15
0	0	1	0	0	0	0	dst-reg	src-reg	data

ADD d-reg, d-reg, cond3 **Arith**

Add a data register to a data register if the condition holds

Operands `ADD dst-reg, src-reg, condition`

Operation if condition holds, dst-reg = dst-reg + src-reg

Flags Not affected

Description If the condition holds, add a data register, `src-reg`, to another data register, `dst-reg`, storing the result back in `dst-reg`.

Execution Time 1 cycle

Remarks Version 3 or greater of the CFX assembler supports the following alternate form for this conditional instruction:

```
ADD d-reg, d-reg if cond3
```

Opcode

30	29	28	27	26	25	24	23 22 21	20 19 18	17 16 15
0	1	1	1	1	1	0	dst-reg	cond	src-reg

ADD d-reg, imm3s, cond3 **Arith**

Add a value to a data register if the condition holds

Operands `ADD dst-reg, data, condition`

Operation if condition holds, dst-reg = dst-reg + signext(data)

Flags Not affected

Description If the condition holds, add a 3-bit signed immediate value, `data` (-4..3), to a data register, `dst-reg`, and store the result back in `dst-reg`.

Execution Time 1 cycle

Remarks Version 3 or greater of the CFX assembler supports the following alternate form for this conditional instruction:

```
ADD d-reg, imm3s if cond3
```

Opcode

30	29	28	27	26	25	24	23 22 21	20 19 18	17 16 15
0	1	0	1	1	0	0	dst-reg	cond	data

ADD.DW d-reg, d-reg, imm3s **Arith**

Add a double-width value to a data register, storing the result in a data register

Operands `ADD.DW dst-reg, src-reg, data`

Operation dst-reg = src-reg + (signext(data) >> 24)

Flags If Flag Updates are enabled, the X ALU status flags are affected as follows:

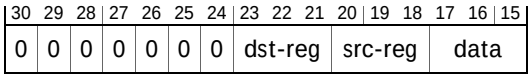
X Carry:	Set according to ALU output
X Negative:	Set according to ALU output
X Zero:	Set according to ALU output
X Overflow:	Set according to ALU output
X Extension:	Set according to ALU output

Description Add a 3-bit signed immediate value, interpreted as a double-width value (aligned with the low register of an accumulator, or the 24-bit word below an XY data register), to a data register, `src-reg`, and store the result in a data register, `dst-reg`.

Execution Time 1 cycle

Remarks Double-width arithmetic is not useful when both `src-reg` and `dst-reg` are XY data registers.

Opcode



30	29	28	27	26	25	24	23 22 21 20	19 18 17 16	15
0	0	0	0	0	0	0	dst-reg	src-reg	data

ADD.DW d-reg, imm3s, cond3 **Arith**

Add a double-width value to a data register, if the condition holds

Operands `ADD.DW dst-reg, data, condition`

Operation if condition holds, dst-reg = dst-reg + (signext(data) >> 24)

Flags Not affected

Description If the condition holds, add a 3-bit signed immediate value, interpreted as a double-width value (aligned with the low register of an accumulator, or the 24-bit word below an XY data register), to a data register, `dst-reg`, and store the result back in `dst-reg`.

Execution Time 1 cycle

Remarks Double-width arithmetic is not useful when `dst-reg` is an XY data register.

Version 3 or greater of the CFX assembler supports the following alternate form for this conditional instruction:

```
ADD.DW d-reg, imm3s if cond3
```

Opcode

30	29	28	27	26	25	24	23 22 21	20 19 18	17 16 15
0	1	0	1	1	0	1	dst-reg	cond	data

ADD.FR d-reg, d-reg, imm3s **Arith**

Add a fractional immediate value to a data register, storing the result in a data register

Operands `ADD.FR dst-reg, src-reg, data`

Operation dst-reg = src-reg + fractional(data)

Flags If Flag Updates are enabled, the X ALU status flags are affected as follows:

X Carry:	Set according to ALU output
X Negative:	Set according to ALU output
X Zero:	Set according to ALU output
X Overflow:	Set according to ALU output
X Extension:	Set according to ALU output

Description Add a fractional immediate value, `data`, to a data register, `src-reg`, and store the result in a data register, `dst-reg`.

The 3-bit signed immediate value is interpreted as follows:

Imm Value	Fract Value	Hex Value Added
-4	-1	0xFF800000000000
-3	$-3/4$	0xFFA00000000000
-2	$-1/2$	0xFFC00000000000
-1	$-1/4$	0xFFE00000000000
1	$+1/4$	0x00200000000000
2	$+1/2$	0x00400000000000
3	$+3/4$	0x00600000000000
0	+1	0x00800000000000

Execution Time 1 cycle

Opcode

30	29	28	27	26	25	24	23 22 21 20	19 18 17 16	15
0	1	1	0	1	1	0	dst-reg	src-reg	data

ADD.NX d-reg, d-reg, d-reg **Arith**

Add two data registers with no extension, storing the result in a data register

Operands `ADD.NX dst-reg, src-reg-1, src-reg-2`

Operation dst-reg = signext(src-reg-1 + src-reg-2)

Flags If Flag Updates are enabled, the X ALU status flags are affected as follows:

X Carry:	Not affected
X Negative:	Set according to ALU output
X Zero:	Set according to ALU output
X Overflow:	Set to 0
X Extension:	Not affected

Description Add two data registers, `src-reg-1` and `src-reg-2`, and store the result in a data register, `dst-reg`.

Before the result is written to the destination register, the upper 8 bits are overwritten with the sign extension from bit 47 of the result.

Execution Time 1 cycle

Opcode

30	29	28	27	26	25	24	23 22 21 20	19 18 17 16	15
0	1	0	0	0	0	1	dst-reg	src-reg-1	src-reg-2

ADD.NX a-reg, a-reg, ab-reg **X Arith**

Add two accumulators with no extension, storing the result in an A accumulator

Operands `ADD.NX dst-reg, src-reg-1, src-reg-2`

Operation dst-reg = signext(src-reg-1 + src-reg-2)

Flags If Flag Updates are enabled, the X ALU status flags are affected as follows:

X Carry:	Not affected
X Negative:	Set according to ALU output
X Zero:	Set according to ALU output
X Overflow:	Set to 0
X Extension:	Not affected

Description Add an A accumulator, `src-reg-1`, and an A or B accumulator, `src-reg-2`, and store the result in an A accumulator, `dst-reg`.

Before the result is written to the destination register, the upper 8 bits are overwritten with the sign extension from bit 47 of the result.

Execution Time 1 cycle

Constraints The Y side instruction must not be `MUL*`.

The right-most source register on the X and Y sides must be opposite—one side using `a-reg` and the other side `b-reg`.

Opcode

ADD.NX a-reg, a-reg, a-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	1	0	0	0	0	dst	0	s1	s2	-	-	-	1	-	-

ADD.NX a-reg, a-reg, b-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	1	0	0	0	0	dst	1	s1	s2	-	-	-	1	-	-

Bits marked with "-" are determined by the Y side arithmetic instruction.

ADD.NX d-reg, d-reg, imm3s **Arith**

Add a value to a data register with no extension, storing the result in a data register

Operands `ADD.NX dst-reg, src-reg, data`

Operation dst-reg = signext(src-reg + signext(data))

Flags If Flag Updates are enabled, the X ALU status flags are affected as follows:

X Carry:	Not affected
X Negative:	Set according to ALU output
X Zero:	Set according to ALU output
X Overflow:	Set to 0
X Extension:	Not affected

Description Add a 3-bit signed immediate value, `data` (-4..3), to a data register, `src-reg`, and store the result in a data register, `dst-reg`.

Before the result is written to the destination register, the upper 8 bits are overwritten with the sign extension from bit 47 of the result.

Execution Time 1 cycle

Opcode

30	29	28	27	26	25	24	23 22 21 20	19 18 17 16	15
0	0	1	0	0	0	1	dst-reg	src-reg	data

ADD.NX.DW d-reg, d-reg, imm3s **Arith**

Add a double-width value to a data register with no extension, storing the result in a data register

Operands `ADD.NX.DW dst-reg, src-reg, data`

Operation dst-reg = signext(src-reg + (signext(data) >> 24))

Flags If Flag Updates are enabled, the X ALU status flags are affected as follows:

X Carry:	Not affected
X Negative:	Set according to ALU output
X Zero:	Set according to ALU output
X Overflow:	Set to 0
X Extension:	Not affected

Description Add a 3-bit signed immediate value, interpreted as a double-width value (aligned with the low register of an accumulator, or the 24-bit word below an XY data register), to a data register, `src-reg`, and store the result in a data register, `dst-reg`.

Before the result is written to the destination register, the upper 8 bits are overwritten with the sign extension from bit 47 of the result.

Execution Time 1 cycle

Remarks Double-width arithmetic is not useful when both `src-reg` and `dst-reg` are XY data registers.

Opcode

30	29	28	27	26	25	24	23 22 21	20 19 18	17 16 15
0	0	0	0	0	0	1	dst-reg	src-reg	data

ADDMUL ab-reg, xy-reg, xy-reg, xy-reg **Arith**

Add two XY data registers, multiply the result by a third XY data register, and store the result in an accumulator

Operands `ADDMUL dst-reg, src-reg-1, src-reg-2, src-reg-3`

Operation dst-reg = 0 + src-reg-1 * (src-reg-2 + src-reg-3)

Flags If Flag Updates are enabled, the X ALU status flags are affected when the ALU operation completes (after the next clock cycle), as follows:

X Carry:	Not affected
X Negative:	Set according to result of adding 0 to product
X Zero:	Set according to result of adding 0 to product
X Overflow:	Set to 0
X Extension:	Set according to result of adding 0 to product

Description Add two XY data registers, `src-reg-2` and `src-reg-3`, multiply by a third XY data register, `src-reg-1`, and write the result to an accumulator, `dst-reg`.

Execution Time 2 cycles, 1 pipelined (ALU operation)

Constraints The three source registers must be one of four combinations:

src-reg-1	src-reg-2	src-reg-3
x-reg	x-reg	y-reg
x-reg	y-reg	y-reg
y-reg	x-reg	y-reg
y-reg	x-reg	x-reg

Opcode

ADDMUL ab-reg, x-reg, x-reg, y-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	1	0	0	1	s1	s2	s3	0	0	0	1	0	dst	

ADDMUL ab-reg, x-reg, y-reg, y-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	1	0	0	1	s1	s2	s3	0	1	0	1	0	dst	

ADDMUL ab-reg, y-reg, x-reg, y-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	1	0	0	1	s1	s2	s3	1	0	0	1	0	dst	

ADDMUL ab-reg, y-reg, x-reg, x-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	1	0	0	1	s1	s2	s3	1	1	0	1	0	dst	

ADDMULADD ab-reg, xy-reg, xy-reg, xy-reg **Arith**

Add two XY data registers, multiply the result by a third XY data register, and add the result to an accumulator

Operands `ADDMULADD dst-reg, src-reg-1, src-reg-2, src-reg-3`

Operation dst-reg = dst-reg + src-reg-1 * (src-reg-2 + src-reg-3)

Flags If Flag Updates are enabled, the X ALU status flags are affected when the ALU operation completes (after the next clock cycle), as follows:

X Carry:	Set according to result of adding `dst-reg` to product
X Negative:	Set according to result of adding `dst-reg` to product
X Zero:	Set according to result of adding `dst-reg` to product
X Overflow:	Set according to result of adding `dst-reg` to product
X Extension:	Set according to result of adding `dst-reg` to product

Description Add two XY data registers, `src-reg-2` and `src-reg-3`, multiply by a third XY data register, `src-reg-1`, add an accumulator, `dst-reg`, and write the result back to `dst-reg`.

Execution Time 2 cycles, 1 pipelined (ALU operation)

Constraints The three source registers must be one of four combinations:

src-reg-1	src-reg-2	src-reg-3
x-reg	x-reg	y-reg
x-reg	y-reg	y-reg
y-reg	x-reg	y-reg
y-reg	x-reg	x-reg

Opcode

ADDMULADD ab-reg, x-reg, x-reg, y-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	1	0	0	1	s1	s2	s3	0	0	0	0	0	dst	

ADDMULADD ab-reg, x-reg, y-reg, y-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	1	0	0	1	s1	s2	s3	0	1	0	0	0	dst	

ADDMULADD ab-reg, y-reg, x-reg, y-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	1	0	0	1	s1	s2	s3	1	0	0	0	0	dst	

ADDMULADD ab-reg, y-reg, x-reg, x-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	1	0	0	1	s1	s2	s3	1	1	0	0	0	dst	

ADDMULNEG ab-reg, xy-reg, xy-reg, xy-reg **Arith**

Add two XY data registers, multiply the result by a third XY data register, negate the result and store it in an accumulator

Operands `ADDMULNEG dst-reg, src-reg-1, src-reg-2, src-reg-3`

Operation dst-reg = 0 - src-reg-1 * (src-reg-2 + src-reg-3)

Flags If Flag Updates are enabled, the X ALU status flags are affected when the ALU operation completes (after the next clock cycle), as follows:

X Carry:	Not affected
X Negative:	Set according to result of subtracting product from 0
X Zero:	Set according to result of subtracting product from 0
X Overflow:	Set to 0
X Extension:	Set according to result of subtracting product from 0

Description Add two XY data registers, `src-reg-2` and `src-reg-3`, multiply by a third XY data register, `src-reg-1`, negate the result and write it to an accumulator, `dst-reg`.

Execution Time 2 cycles, 1 pipelined (ALU operation)

Constraints The three source registers must be one of four combinations:

src-reg-1	src-reg-2	src-reg-3
x-reg	x-reg	y-reg
x-reg	y-reg	y-reg
y-reg	x-reg	y-reg
y-reg	x-reg	x-reg

Opcode

ADDMULNEG ab-reg, x-reg, x-reg, y-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	1	0	0	1	s1	s2	s3	0	0	0	1	1	dst	

ADDMULNEG ab-reg, x-reg, y-reg, y-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	1	0	0	1	s1	s2	s3	0	1	0	1	1	dst	

ADDMULNEG ab-reg, y-reg, x-reg, y-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	1	0	0	1	s1	s2	s3	1	0	0	1	1	dst	

ADDMULNEG ab-reg, y-reg, x-reg, x-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	1	0	0	1	s1	s2	s3	1	1	0	1	1	dst	

ADDMULSUB ab-reg, xy-reg, xy-reg, xy-reg **Arith**

Add two XY data registers, multiply the result by a third XY data register, and subtract the result from an accumulator

Operands `ADDMULSUB dst-reg, src-reg-1, src-reg-2, src-reg-3`

Operation dst-reg = dst-reg - src-reg-1 * (src-reg-2 + src-reg-3)

Flags If Flag Updates are enabled, the X ALU status flags are affected when the ALU operation completes (after the next clock cycle), as follows:

X Carry:	Set according to result of subtracting product from `dst-reg`
X Negative:	Set according to result of subtracting product from `dst-reg`
X Zero:	Set according to result of subtracting product from `dst-reg`
X Overflow:	Set according to result of subtracting product from `dst-reg`
X Extension:	Set according to result of subtracting product from `dst-reg`

Description Add two XY data registers, `src-reg-2` and `src-reg-3`, multiply by a third XY data register, `src-reg-1`, subtract the result from an accumulator, `dst-reg`, and write the result back to `dst-reg`.

Execution Time 2 cycles, 1 pipelined (ALU operation)

Constraints The three source registers must be one of four combinations:

src-reg-1	src-reg-2	src-reg-3
x-reg	x-reg	y-reg
x-reg	y-reg	y-reg
y-reg	x-reg	y-reg
y-reg	x-reg	x-reg

Opcode

ADDMULSUB ab-reg, x-reg, x-reg, y-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	1	0	0	1	s1	s2	s3	0	0	0	0	1	dst	

ADDMULSUB ab-reg, x-reg, y-reg, y-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	1	0	0	1	s1	s2	s3	0	1	0	0	1	dst	

ADDMULSUB ab-reg, y-reg, x-reg, y-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	1	0	0	1	s1	s2	s3	1	0	0	0	1	dst	

ADDMULSUB ab-reg, y-reg, x-reg, x-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	1	0	0	1	s1	s2	s3	1	1	0	0	1	dst	

ADDSH d-reg, d-reg, d-reg **Arith**

Add two data registers and shift right one bit, storing the result in a data register

Operands `ADDSH dst-reg, src-reg-1, src-reg-2`

Operation dst-reg = (src-reg-1 + src-reg-2) >> 1

Flags If Flag Updates are enabled, the X ALU status flags are affected as follows:

X Carry:	Set according to the ALU output of `src-reg-1 + src-reg-2`
X Negative:	Set according to ALU output
X Zero:	Set according to ALU output
X Overflow:	Set according to the ALU output of `src-reg-1 + src-reg-2`
X Extension:	Set according to ALU output

Note that the X Carry and X Overflow flags are updated based on the addition, not the shift operation.

Description Add two data registers, `src-reg-1` and `src-reg-2`, and perform an arithmetic shift of one bit to the right before storing the result in a data register, `dst-reg`.

Execution Time 1 cycle

Opcode

30	29	28	27	26	25	24	23 22 21	20 19 18	17 16 15
0	1	0	1	1	1	0	dst-reg	src-reg-1	src-reg-2

ADDSH a-reg, a-reg, ab-reg **X Arith**

Add two accumulators and shift right one bit, storing the result in an A accumulator

Operands `ADDSH dst-reg, src-reg-1, src-reg-2`

Operation dst-reg = (src-reg-1 + src-reg-2) >> 1

Flags If Flag Updates are enabled, the X ALU status flags are affected as follows:

X Carry:	Set according to the ALU output of `src-reg-1 + src-reg-2`
X Negative:	Set according to ALU output
X Zero:	Set according to ALU output
X Overflow:	Set according to the ALU output of `src-reg-1 + src-reg-2`
X Extension:	Set according to ALU output

Note that the X Carry and X Overflow flags are updated based on the addition, not the shift operation.

Description Add an A accumulator, `src-reg-1`, and an A or B accumulator, `src-reg-2`, and perform an arithmetic shift of one bit to the right before storing the result in an A accumulator, `dst-reg`.

Execution Time 1 cycle

Constraints When the Y side instruction is not `MUL*`, the right-most source register on the X and Y sides must be opposite—one side using `a-reg` and the other side `b-reg`.

Opcode

Y side instruction is not MUL*

ADDSH a-reg, a-reg, a-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	1	0	1	0	1	dst	0	s1	s2	-	-	-	0	-	-

ADDSH a-reg, a-reg, b-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	1	0	1	0	1	dst	1	s1	s2	-	-	-	0	-	-

Y side instruction is MUL*

ADDSH a-reg, a-reg, a-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	0	1	1	0	dst	0	s1	s2	-	-	-	-	-	-

ADDSH a-reg, a-reg, b-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	0	1	1	0	dst	1	s1	s2	-	-	-	-	-	-

Bits marked with "-" are determined by the Y side arithmetic instruction.

ADDSH b-reg, b-reg, ab-reg **Y Arith**

Add two accumulators and shift right one bit, storing the result in a B accumulator

Operands `ADDSH dst-reg, src-reg-1, src-reg-2`

Operation dst-reg = (src-reg-1 + src-reg-2) >> 1

Flags If Flag Updates are enabled, the Y ALU status flags are affected as follows:

Y Carry:	Set according to the ALU output of `src-reg-1 + src-reg-2`
Y Negative:	Set according to ALU output
Y Zero:	Set according to ALU output
Y Overflow:	Set according to the ALU output of `src-reg-1 + src-reg-2`
Y Extension:	Set according to ALU output

Note that the Y Carry and Y Overflow flags are updated based on the addition, not the shift operation.

Description Add a B accumulator, `src-reg-1`, and an A or B accumulator, `src-reg-2`, and perform an arithmetic shift of one bit to the right before storing the result in a B accumulator, `dst-reg`.

Execution Time 1 cycle

Constraints When the X side instruction is not `MUL*`, the right-most source register on the X and Y sides must be opposite—one side using `a-reg` and the other side `b-reg`.

Opcode

X side instruction is not MUL*

ADDSH b-reg, b-reg, a-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	1	0	-	-	-	-	1	-	-	1	0	dst	-	s1	s2

ADDSH b-reg, b-reg, b-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	1	0	-	-	-	-	0	-	-	1	0	dst	-	s1	s2

X side instruction is MUL*

ADDSH b-reg, b-reg, a-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	0	0	-	-	-	-	-	-	1	0	dst	1	s1	s2

ADDSH b-reg, b-reg, b-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	0	0	-	-	-	-	-	-	1	0	dst	0	s1	s2

Bits marked with "-" are determined by the X side arithmetic instruction.

AND d-reg, d-reg, d-reg **Arith**

Bitwise AND two data registers, storing the result in a data register

Operands `AND dst-reg, src-reg-1, src-reg-2`

Operation dst-reg = src-reg-1 AND src-reg-2

Flags If Flag Updates are enabled, the X ALU status flags are affected as follows:

X Carry:	Not affected
X Negative:	Set according to ALU output
X Zero:	Set according to ALU output
X Overflow:	Set to 0
X Extension:	Set according to ALU output

Description Bitwise AND two data registers, `src-reg-1` and `src-reg-2`, and store the result in a data register, `dst-reg`.

Execution Time 1 cycle

Opcode

30	29	28	27	26	25	24	23 22 21	20 19 18	17 16 15
0	1	0	1	0	0	0	dst-reg	src-reg-1	src-reg-2

AND a-reg, a-reg, ab-reg **X Arith**

Bitwise AND two accumulators, storing the result in an A accumulator

Operands `AND dst-reg, src-reg-1, src-reg-2`

Operation dst-reg = src-reg-1 AND src-reg-2

Flags If Flag Updates are enabled, the X ALU status flags are affected as follows:

X Carry:	Not affected
X Negative:	Set according to ALU output
X Zero:	Set according to ALU output
X Overflow:	Set to 0
X Extension:	Set according to ALU output

Description Bitwise AND an A accumulator, `src-reg-1`, and an A or B accumulator, `src-reg-2`, and store the result in an A accumulator, `dst-reg`.

Execution Time 1 cycle

Constraints The Y side instruction must not be `MUL*`.

The right-most source register on the X and Y sides must be opposite—one side using `a-reg` and the other side `b-reg`.

Opcode

AND a-reg, a-reg, a-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	1	0	1	0	0	dst	0	s1	s2	-	-	-	0	-	-

AND a-reg, a-reg, b-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	1	0	1	0	0	dst	1	s1	s2	-	-	-	0	-	-

Bits marked with "-" are determined by the Y side arithmetic instruction.

AND d-reg, d-reg, imm3s **Arith**

Bitwise AND a data register and a value, storing the result in a data register

Operands `AND dst-reg, src-reg, data`

Operation dst-reg = src-reg AND signext(data)

Flags If Flag Updates are enabled, the X ALU status flags are affected as follows:

X Carry:	Not affected
X Negative:	Set according to ALU output
X Zero:	Set according to ALU output
X Overflow:	Set to 0
X Extension:	Set according to ALU output

Description Bitwise AND a data register, `src-reg`, and a 3-bit signed immediate value, `data` (-4..3), and store the result in a data register, `dst-reg`.

Execution Time 1 cycle

Opcode

30	29	28	27	26	25	24	23 22 21	20 19 18	17 16 15
0	0	1	1	0	0	0	dst-reg	src-reg	data

AND.DW d-reg, d-reg, imm3s **Arith**

Bitwise AND a data register and a double-width value, storing the result in a data register

Operands `AND.DW dst-reg, src-reg, data`

Operation dst-reg = src-reg AND (signext(data) >> 24)

Flags If Flag Updates are enabled, the X ALU status flags are affected as follows:

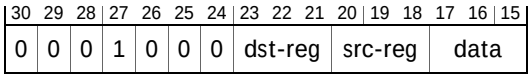
X Carry:	Not affected
X Negative:	Set according to ALU output
X Zero:	Set according to ALU output
X Overflow:	Set to 0
X Extension:	Set according to ALU output

Description Bitwise AND a data register, `src-reg`, and a 3-bit signed immediate value, `data` (-4..3), interpreted as a double-width value (aligned with the low register of an accumulator, or the 24-bit word below an XY data register), and store the result in a data register, `dst-reg`.

Execution Time 1 cycle

Remarks Double-width logic is not useful when `dst-reg` is an XY data register.

Opcode



30	29	28	27	26	25	24	23 22 21 20	19 18 17 16	15
0	0	0	1	0	0	0	dst-reg	src-reg	data

BITCLR SR, imm4u **Long**

Clear a bit in the status register

Operands `BITCLR SR, index`

Operation SR = SR AND NOT(1 << index)

Flags Not affected

Description Clear the bit specified by `index` in the `SR` register.

Execution Time 1 cycle

Opcode

29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	0	1	1	0	1	1	0	0	0	0	0	0	0	0	1	1	0	0	0	0	0	index			

BITCLR d-reg, bit-index **Arith**

Clear a bit in a data register

Operands `BITCLR dst-reg, index`

Operation dst-reg = dst-reg AND NOT(1 << index)

Flags If Flag Updates are enabled, the X ALU status flags are affected as follows:

X Carry:	Set to the original value of the modified bit
X Negative:	Not affected
X Zero:	Not affected
X Overflow:	Not affected
X Extension:	Not affected

Description Clear the bit specified by `index` in a data register, `dst-reg`.

Execution Time 1 cycle

Constraints If `dst-reg` is an accumulator, `index` must be between 0 and 55, where bits 0-23 are in the low accumulator register, bits 24-47 are in the high accumulator register, and bits 48-55 are in the accumulator extension.

If `dst-reg` is an XY data register, `index` must be between 0 and 23.

Opcode

30	29	28	27	26	25	24	23 22 21 20	19 18 17 16 15
0	0	1	1	1	1	0	dst-reg	index

If `dst-reg` is an XY data register, `index+24` is encoded in the instruction word.

BITSET SR, imm4u **Long**

Set a bit in the status register

Operands `BITSET SR, index`

Operation SR = SR OR (1 << index)

Flags Not affected

Description Set the bit specified by `index` in the `SR` register.

Execution Time 1 cycle

3 cycles if setting the `IE` bit in `SR`

Opcode

29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	0	1	1	0	1	1	0	0	0	0	0	0	0	0	1	0	1	0	0	0	0	index			

BITSET d-reg, bit-index **Arith**

Set a bit in a data register

Operands `BITSET dst-reg, index`

Operation dst-reg = dst-reg OR (1 << index)

Flags If Flag Updates are enabled, the X ALU status flags are affected as follows:

X Carry:	Set to the original value of the modified bit
X Negative:	Not affected
X Zero:	Not affected
X Overflow:	Not affected
X Extension:	Not affected

Description Set the bit specified by `index` in a data register, `dst-reg`.

Execution Time 1 cycle

Constraints If `dst-reg` is an accumulator, `index` must be between 0 and 55, where bits 0-23 are in the low accumulator register, bits 24-47 are in the high accumulator register, and bits 48-55 are in the accumulator extension.

If `dst-reg` is an XY data register, `index` must be between 0 and 23.

Opcode

30	29	28	27	26	25	24	23 22 21 20	19 18 17 16 15
0	0	1	1	1	0	1	dst-reg	index

If `dst-reg` is an XY data register, `index+24` is encoded in the instruction word.

BITTGL d-reg, bit-index **Arith**

Toggle a bit in a data register

Operands `BITTGL dst-reg, index`

Operation dst_reg = dst-reg XOR (1 << index)

Flags If Flag Updates are enabled, the X ALU status flags are affected as follows:

X Carry:	Set to the original value of the modified bit
X Negative:	Not affected
X Zero:	Not affected
X Overflow:	Not affected
X Extension:	Not affected

Description Toggle the bit specified by `index` in a data register, `dst-reg`.

Execution Time 1 cycle

Constraints If `dst-reg` is an accumulator, `index` must be between 0 and 55, where bits 0-23 are in the low accumulator register, bits 24-47 are in the high accumulator register, and bits 48-55 are in the accumulator extension.

If `dst-reg` is an XY data register, `index` must be between 0 and 23.

Opcode

30	29	28	27	26	25	24	23 22 21 20	19 18 17 16 15
0	0	1	1	1	1	1	dst-reg	index

If `dst-reg` is an XY data register, `index+24` is encoded in the instruction word.

BITTST d-reg, bit-index **Arith**

Test a bit in a data register

Operands `BITTST dst-reg, index`

Operation X Carry flag = (dst-reg >> index) AND 1

Flags The X ALU status flags are affected as follows:

X Carry:	Set to the value of the specified bit
X Negative:	Not affected
X Zero:	Not affected
X Overflow:	Not affected
X Extension:	Not affected

Description Set the X Carry flag to the value of the bit specified by `index` in a data register, `dst-reg`.

The X Carry flag is updated regardless of whether the Flag Updates mode is enabled.

Execution Time 1 cycle

Constraints If `dst-reg` is an accumulator, `index` must be between 0 and 55, where bits 0-23 are in the low accumulator register, bits 24-47 are in the high accumulator register, and bits 48-55 are in the accumulator extension.

If `dst-reg` is an XY data register, `index` must be between 0 and 23.

Opcode

30	29	28	27	26	25	24	23 22 21 20	19 18 17 16 15
0	0	1	1	1	0	0	dst-reg	index

If `dst-reg` is an XY data register, `index+24` is encoded in the instruction word.

BREAKPOINT **Long**

Breakpoint

Operands `BREAKPOINT`

Operation if software breakpoints are enabled, performs the following:

- Halt the DSP
- Enter debug mode

Flags Not affected

Description Halts the DSP for debugging if software breakpoints are enabled through the debug port.

If software breakpoints are not enabled, this instruction is treated as a `NOP`.

Execution Time 3 cycles

Opcode

29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	0	1	1	0	1	1	0	0	0	0	0	0	0	1	0	1	0	0	0	0	0	0	0	0	0

CALL imm16u **Synth**

Call direct without delay slot

Operands `CALL addr`

Operation LR = PC + 2

PC = addr

Flags Not affected

Description Call a subroutine specified by an immediate address or label, `addr`, putting the return address in the Link Register (`LR`). The delay slot is filled with a `NOP` instruction.

Execution Time 2 cycles

Remarks This instruction is synthesized by the assembler as follows:

```
CALL.1D addr
NOP
```

Opcode

See CALL.1D imm16u and NOP.

CALL xp-reg **Synth**

Call indirect without delay slots

Operands `CALL src-reg`

Operation LR = PC + 2

PC = src-reg

Flags Not affected

Description Call a subroutine at the address in an X pointer, `src-reg`, putting the return address in the Link Register (`LR`). The delay slots are filled with `NOP` instructions.

Execution Time 3 cycles

Remarks This instruction is synthesized by the assembler as follows:

```
CALL.2D src-reg
NOP >>> NOP
```

Opcode

See CALL.2D xp-reg and NOP.

CALL.1D imm16u **Long**

Call direct with one delay slot

Operands `CALL.1D addr`

Operation LR = PC + 2

PC = addr

Flags Not affected

Description Call a subroutine specified by an immediate address or label, `addr`, putting the return address in the Link Register (`LR`).

This instruction has one delay slot. The instruction word immediately after this instruction in memory is executed before the subroutine is executed, and the subroutine will return to the instruction word after the delay slot. The instruction word in the delay slot must be an *Arithmetic-Move* or non-control-flow single-cycle *Long* instruction. See Section 7.5.5, “Program Flow Delay Slot Restrictions” on page 62 for more information.

Execution Time 2 cycles, 1 pipelined (delay slot)

Opcode

29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	1	0	0	0	0	0	0	0	addr															

CALL.2D xp-reg **Long**

Call indirect with two delay slots

Operands `CALL.2D src-reg`

Operation LR = PC + 2 or 3

PC = src-reg

Flags Not affected

Description Call a subroutine at the address in an X pointer, `src-reg`, putting the return address in the Link Register (`LR`).

This instruction has two delay slots. One or two instruction words immediately following this instruction in memory are executed before the subroutine is executed, and the subroutine will return to the next instruction word after the delay slots.

The delay slots can be filled in one of two ways:

- One *Move-Move* instruction that executes in exactly 2 cycles. In this case, `PC` + 2 will be stored in `LR`.
- Two *Arithmetic-Move* or non-control-flow single-cycle *Long* instructions. In this case, `PC` + 3 will be stored in `LR`.

See Section 7.5.5, “Program Flow Delay Slot Restrictions” on page 62 for more information.

Execution Time 3 cycles, 2 pipelined (delay slots)

Remarks The value of `PC` + 2 is stored in `LR` if the two delay slot cycles are filled with one *Move-Move* instruction word. Otherwise, with two instruction words in the delay slots, the return address is `PC` + 3.

Opcode

29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	0	1	1	0	1	1	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	src-reg		

CLR X[single-mode(X)] **Move**

Clear a word of X memory specified by an X pointer, with update

Operands `CLR X[ addr ]`

Operation X[addr] = 0

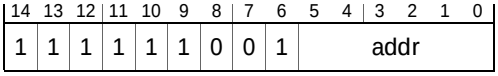
update addr

Flags Not affected

Description Clear a word of X memory specified by an X pointer in `addr`, then update the X pointer as specified in `addr`.

Execution Time 1 cycle

Opcode



If this instruction is the second move instruction in a Move-Move instruction word, the encoding is shifted left 15 bits.

CMP d-reg, d-reg **Arith**

Compare two data registers

Operands `CMP src-reg-1, src-reg-2`

Operation src-reg-1 - src-reg-2

update X ALU status flags

Flags The X ALU status flags are affected as follows:

X Carry:	If Flag Updates are enabled, set according to ALU output of subtraction
X Negative:	Set according to ALU output of subtraction
X Zero:	Set according to ALU output of subtraction
X Overflow:	Set according to ALU output of subtraction
X Extension:	Set according to ALU output of subtraction

Description Compare two data registers, `src-reg-1` and `src-reg-2`, by subtracting the second from the first and updating the X ALU status flags based on the result. The X Carry flag is only updated when Flag Updates mode is enabled; the other flags are always updated.

Execution Time 1 cycle

Opcode

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
0	1	0	0	0	1	0	0	0	0	src-reg-1			src-reg-2		

CMP a-reg, ab-reg

X Arith

Compare two accumulators

Operands

```
CMP src-reg-1, src-reg-2
```

Operation

src-reg-1 - src-reg-2

update X ALU status flags

Flags

The X ALU status flags are affected as follows:

X Carry:	If Flag Updates are enabled, set according to ALU output of subtraction
X Negative:	Set according to ALU output of subtraction
X Zero:	Set according to ALU output of subtraction
X Overflow:	Set according to ALU output of subtraction
X Extension:	Set according to ALU output of subtraction

Description

Compare two accumulators, `src-reg-1` and `src-reg-2`, by subtracting the second from the first and updating the X ALU status flags based on the result. The X Carry flag is only updated when Flag Updates mode is enabled; the other flag updates are always performed.

Execution Time

1 cycle

Constraints

The Y side instruction must not be `MUL*`.

The right-most source register on the X and Y sides must be opposite—one side using `a-reg` and the other side `b-reg`.

Opcode

CMP a-reg, a-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	1	0	0	0	1	0	0	s1	s2	-	-	-	0	-	-

CMP a-reg, b-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	1	0	0	0	1	0	1	s1	s2	-	-	-	0	-	-

Bits marked with "-" are determined by the Y side arithmetic instruction.

CMP d-reg, imm3s

Arith

Compare a data register to a value

Operands `CMP src-reg, data`

Operation src-reg - signext(data)

update X ALU status flags

Flags The X ALU status flags are affected as follows:

X Carry:	If Flag Updates are enabled, set according to ALU output of subtraction
X Negative:	Set according to ALU output of subtraction
X Zero:	Set according to ALU output of subtraction
X Overflow:	Set according to ALU output of subtraction
X Extension:	Set according to ALU output of subtraction

Description Compare a data register, `src-reg`, to a 3-bit signed immediate value, `data` (-4..3), by subtracting the value from the register and updating the X ALU status flags based on the result. The X Carry flag is only updated when Flag Updates mode is enabled; the other flag updates are always performed.

Execution Time 1 cycle

Opcode

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
0	0	1	0	0	1	0	0	0	0	src-reg			data		

CMP.DW d-reg, imm3s **Arith**

Compare a data register to a double-width value

Operands `CMP.DW src-reg, data`

Operation src-reg - (signext(data) >> 24)

update X ALU status flags

Flags The X ALU status flags are affected as follows:

X Carry:	If Flag Updates are enabled, set according to ALU output of subtraction
X Negative:	Set according to ALU output of subtraction
X Zero:	Set according to ALU output of subtraction
X Overflow:	Set according to ALU output of subtraction
X Extension:	Set according to ALU output of subtraction

Description Compare a data register, `src-reg`, to a 3-bit signed immediate value, interpreted as a double-width value (aligned with the low register of an accumulator, or the 24-bit word below an XY data register), by subtracting the value from the register and updating the X ALU status flags based on the result. The X Carry flag is only updated when Flag Updates mode is enabled; the other flag updates are always performed.

Execution Time 1 cycle

Remarks Double-width comparison is not useful when `src-reg` is an XY data register.

Opcode

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
0	0	0	0	0	1	0	0	0	0	src-reg			data		

CMPU.NX d-reg, d-reg **Arith**

Compare two data registers as unsigned values with no extension

Operands `CMPU.NX src-reg-1, src-reg-2`

Operation signext(src-reg-1 - src-reg-2)

update X ALU status flags

Flags The X ALU status flags are affected as follows:

X Carry:	If Flag Updates are enabled, set according to ALU output of subtraction
X Negative:	Set according to ALU output of subtraction
X Zero:	Set according to ALU output of subtraction
X Overflow:	Set to 0
X Extension:	Not affected

Description Compare two data registers, `src-reg-1` and `src-reg-2`, by subtracting the second from the first, overwriting the upper 8 bits of the result with the sign extension from bit 47 of the result, and updating the X ALU status flags based on the result. The X Carry flag is only updated when Flag Updates mode is enabled; the other flag updates are always performed.

Execution Time 1 cycle

Opcode

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
0	1	0	0	0	1	1	0	0	0	src-reg-1			src-reg-2		

CMPU.NX a-reg, ab-reg **X Arith**

Compare two accumulators as unsigned values with no extension

Operands `CMPU.NX src-reg-1, src-reg-2`

Operation signext(src-reg-1 - src-reg-2)

update X ALU status flags

Flags The X ALU status flags are affected as follows:

X Carry:	If Flag Updates are enabled, set according to ALU output of subtraction
X Negative:	Set according to ALU output of subtraction
X Zero:	Set according to ALU output of subtraction
X Overflow:	Set to 0
X Extension:	Not affected

Description Compare two accumulators, `src-reg-1` and `src-reg-2`, by subtracting the second from the first, overwriting the upper 8 bits of the result with the sign extension from bit 47 of the result, and updating the X ALU status flags based on the result. The X Carry flag is only updated when Flag Updates mode is enabled; the other flag updates are always performed.

Execution Time 1 cycle

Constraints The Y side instruction must not be `MUL*`.

The right-most source register on the X and Y sides must be opposite—one side using `a-reg` and the other side `b-reg`.

Opcode

CMPU.NX a-reg, a-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	1	0	0	0	1	0	0	s1	s2	-	-	-	1	-	-

CMPU.NX a-reg, b-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	1	0	0	0	1	0	1	s1	s2	-	-	-	1	-	-

Bits marked with "-" are determined by the Y side arithmetic instruction.

CMPU.NX d-reg, imm3s

Arith

Compare a data register to a value as unsigned values

Operands `CMPU.NX src-reg, data`

Operation signext(src-reg - signext(data))

update X ALU status flags

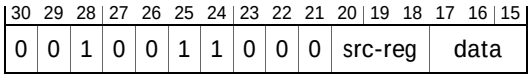
Flags The X ALU status flags are affected as follows:

X Carry:	If Flag Updates are enabled, set according to ALU output of subtraction
X Negative:	Set according to ALU output of subtraction
X Zero:	Set according to ALU output of subtraction
X Overflow:	Set to 0
X Extension:	Not affected

Description Compare a data register, `src-reg`, to a 3-bit signed immediate value, `data` (-4..3), by subtracting the value from the register, overwriting the upper 8 bits of the result with the sign extension from bit 47 of the result, and updating the X ALU status flags based on the result. The X Carry flag is only updated when Flag Updates mode is enabled; the other flag updates are always performed.

Execution Time 1 cycle

Opcode



30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
0	0	1	0	0	1	1	0	0	0	src-reg			data		

CMPU.NX.DW d-reg, imm3s **Arith**

Compare a data register to a double-width value as unsigned values

Operands `CMPU.NX.DW src-reg, data`

Operation signext(src-reg - (signext(data) >> 24))

update X ALU status flags

Flags The X ALU status flags are affected as follows:

X Carry:	If Flag Updates are enabled, set according to ALU output of subtraction
X Negative:	Set according to ALU output of subtraction
X Zero:	Set according to ALU output of subtraction
X Overflow:	Set to 0
X Extension:	Not affected

Description Compare a data register, `src-reg`, to a 3-bit signed immediate value, `data` (-4..3), interpreted as a double-width value (aligned with the low register of an accumulator, or the 24-bit word below an XY data register), by subtracting the value from the register, overwriting the upper 8 bits of the result with the sign extension from bit 47 of the result, and updating the X ALU status flags based on the result. The X Carry flag is only updated when Flag Updates mode is enabled; the other flag updates are always performed.

Execution Time 1 cycle

Remarks Double-width comparison is not useful when `src-reg` is an XY data register.

Opcode

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
0	0	0	0	0	1	1	0	0	0	src-reg			data		

DIVST d-reg, d-reg, d-reg **Arith**

Division step for dividing data register by data register, result to data register

Operands `DIVST dst-reg, src-reg-1, src-reg-2`

Operation generate 1 quotient bit of src-reg-1 / src-reg-2

Flags The X ALU status flags are affected as follows:

X Carry:	Not affected
X Negative:	Set according to the ALU output of the addition or subtraction step in the division operation
X Zero:	Not affected
X Overflow:	Not affected
X Extension:	Not affected

Description `DIVST` implements one iteration of a standard non-restoring binary division algorithm. The X Negative flag is updated as part of the logic, regardless of whether the Flag Updates mode is enabled.

See Section 4.9, “Miscellaneous Operations” on page 34 for more information.

Execution Time 1 cycle

Opcode

30	29	28	27	26	25	24	23 22 21	20 19 18	17 16 15
0	1	1	0	1	0	0	dst-reg	src-reg-1	src-reg-2

ENDLOOP

Long

End loop

Operands ENDLOOP

Operation Pop loop stack

Flags Not affected

Description End a hardware loop before the count has reached 0 by popping one entry from the loop stack.

Execution Time 1 cycle

Opcode

29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	0	1	1	0	1	1	0	0	0	0	0	0	0	1	0	0	1	0	0	0	0	0	0	0	0

GOTO imm16u **Long**

Branch direct without delay slot

Operands `GOTO addr`

Operation PC = addr

Flags Not affected

Description Branch to the specified address or label, `addr`. This instruction has no delay slots.

Execution Time 2 cycles

Remarks This instruction is a synonym for:

```
GOTO.0D addr
```

Opcode

See GOTO.0D imm16u.

GOTO imm16u, cond5 **Long**

Branch direct without delay slots, conditionally

Operands `GOTO addr, condition`

Operation if condition holds then PC = addr

Flags Not affected

Description Branch to the specified address or label, `addr`, if the condition holds. This instruction has no delay slots.

Execution Time 3 cycles

1 cycle if branch not taken

Remarks This instruction is a synonym for:

```
GOTO.0D addr, condition
```

Version 3 or greater of the CFX assembler supports the following alternate form for this conditional instruction:

```
GOTO imm16u if cond5
```

Opcode

See GOTO.0D imm16u, cond5.

GOTO xp-reg **Synth**

Branch indirect without delay slots

Operands `GOTO src-reg`

Operation PC = src-reg

Flags Not affected

Description Branch to the address in the X pointer, `src-reg`. The delay slots are filled with `NOP` instructions.

Execution Time 3 cycles

Remarks This instruction is synthesized by the assembler as follows:

```
GOTO.2D addr
NOP >>> NOP
```

Opcode

See GOTO.2D xp-reg and NOP.

GOTO xp-reg, cond5 **Synth**

Branch indirect without delay slots, conditionally

Operands `GOTO src-reg, condition`

Operation if condition holds then PC = src-reg

Flags Not affected

Description Branch to the address in the X pointer, `src-reg`, if the condition holds. The delay slots are filled with `NOP` instructions.

Execution Time 3 cycles

Remarks This instruction is synthesized by the assembler as follows:

```
GOTO.2D addr, condition
NOP >>> NOP
```

Version 3 or greater of the CFX assembler supports the following alternate form for this conditional instruction:

```
GOTO xp-reg if cond5
```

Opcode

See GOTO.2D xp-reg, cond5 and NOP.

GOTO.0D imm16u **Long**

Branch direct without delay slot

Operands `GOTO.0D addr`

Operation PC = addr

Flags Not affected

Description Branch to the specified address or label, `addr`. This instruction has no delay slots.

Execution Time 2 cycles

Opcode

29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	0	0	1	0	0	0	0	0	addr															

GOTO.0D imm16u, cond5 **Long**

Branch direct without delay slots, conditionally

Operands `GOTO.0D addr, condition`

Operation if condition holds then PC = addr

Flags Not affected

Description Branch to the specified address or label, `addr`, if the condition holds. This instruction has no delay slots.

Execution Time 3 cycles

1 cycle if branch not taken

Remarks Version 3 or greater of the CFX assembler supports the following alternate form for this conditional instruction:

```
GOTO.0D imm16u if cond5
```

Opcode

29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	0	1	1	condition					addr															

GOTO.1D imm16u **Long**

Branch direct with one delay slot

Operands `GOTO.1D addr`

Operation PC = addr

Flags Not affected

Description Branch to the specified address or label, `addr`.

This instruction has one delay slot. The instruction word immediately after this instruction in memory is executed before the branch is executed. The instruction word in the delay slot must be an *Arithmetic-Move* or non-control-flow single-cycle *Long* instruction. See Section 7.5.5, “Program Flow Delay Slot Restrictions” on page 62 for more information.

Execution Time 2 cycles, 1 pipelined (delay slot)

Opcode

29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	0	0	0	0	0	0	0	0	addr															

GOTO.2D imm16u, cond5 **Long**

Branch direct with two delay slots, conditionally

Operands `GOTO.2D addr, condition`

Operation if condition holds then PC = addr

Flags Not affected

Description Branch to the specified address or label, `addr`, if the condition holds.

This instruction has two delay slots. One or two instruction words immediately following this instruction in memory are executed before the branch is executed, whether the condition is true or not.

The delay slots can be filled in one of two ways:

- One *Move-Move* instruction that executes in exactly 2 cycles.
- Two *Arithmetic-Move* or non-control-flow single-cycle *Long* instructions.

See Section 7.5.5, “Program Flow Delay Slot Restrictions” on page 62 for more information.

Execution Time 3 cycles, 2 pipelined (delay slots)

Remarks Version 3 or greater of the CFX assembler supports the following alternate form for this conditional instruction:

```
GOTO.2D imm16u if cond5
```

Opcode

29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	0	1	0	condition					addr															

GOTO.2D xp-reg **Long**

Branch indirect with two delay slots

Operands `GOTO.2D src-reg`

Operation PC = src-reg

Flags Not affected

Description Branch to the address in the X pointer, `src-reg`.

This instruction has two delay slots. One or two instruction words immediately following this instruction in memory are executed before the branch is executed.

The delay slots can be filled in one of two ways:

- One *Move-Move* instruction that executes in exactly 2 cycles.
- Two *Arithmetic-Move* or non-control-flow single-cycle *Long* instructions.

See Section 7.5.5, “Program Flow Delay Slot Restrictions” on page 62 for more information.

Execution Time 3 cycles, 2 pipelined (delay slots)

Opcode

29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	0	1	1	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	addr		

GOTO.2D xp-reg, cond5 **Long**

Branch indirect with two delay slots, conditionally

Operands `GOTO.2D src-reg, condition`

Operation if condition holds then PC = src-reg

Flags Not affected

Description Branch to the address in the X pointer, `src-reg`, if the condition holds.

This instruction has two delay slots. One or two instruction words immediately following this instruction in memory are executed before the branch is executed, whether the condition is true or not.

The delay slots can be filled in one of two ways:

- One *Move-Move* instruction that executes in exactly 2 cycles.
- Two *Arithmetic-Move* or non-control-flow single-cycle *Long* instructions.

See Section 7.5.5, “Program Flow Delay Slot Restrictions” on page 62 for more information.

Execution Time 3 cycles, 2 pipelined (delay slots)

Remarks Version 3 or greater of the CFX assembler supports the following alternate form for this conditional instruction:

```
GOTO.2D xp-reg if cond5
```

Opcode

29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	0	1	1	0	1	1	0	0	0	0	0	0	0	0	0	0	1	condition				addr			

INTERRUPT imm5u

Long

Software interrupt

Operands `INTERRUPT vector-number`

Operation ILPC = PC + 1

ILSR = SR

PC = vector-table-base + vector-number

Flags Not affected

Description Vector to the specified software interrupt handler, storing the current status register (`SR`) and program counter (`PC`) in the interrupt-link status register (`ILSR`) and interrupt-link program counter (`ILPC`) respectively.

Execution Time 2 cycles

Opcode

29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	vector-number				

LOAD reg, X[stack-mode16]

Long

Load register from stack

Operands `LOAD register, X[ addr ]`

Operation register = X[addr]

Flags Not affected

Description Load a value from the stack, at a 16-bit offset from the stack pointer, `SP`, into a register, `register`.

Execution Time 1 cycle

Opcode

29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	0	register						0	addr															

LOAD stack-reg6, X[stack-mode6] **Move**

Load register from stack

Operands `LOAD register, X[ addr ]`

Operation register = X[addr]

Flags Not affected

Description Load a value from the stack, at a 6-bit negative offset (-64..-1) from the stack pointer, `SP`, into a register, `register`.

Execution Time 1 cycle

Opcode

14	13	12	11 10 9 8	7	6 5 4	3 2 1 0
1	0	1	register	0	addr	

If this instruction is the second move instruction in a Move-Move instruction word, the encoding is shifted left 15 bits.

LOAD stack-reg4, X[stack-mode4]

Move

Load register from stack

Operands `LOAD register, X[ addr ]`

Operation register = X[addr]

Flags Not affected

Description Load a value from the stack, at a negative 4-bit offset (-16..-1) from the stack pointer, `SP`, into a register, `register`.

Execution Time 1 cycle

Opcode

14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	register			0	addr			

If this instruction is the second move instruction in a Move-Move instruction word, the encoding is shifted left 15 bits.

LOAD reg, X[imm16u]

Long

Load register direct from X memory

Operands `LOAD register, X[ addr ]`

Operation register = X[addr]

Flags Not affected

Description Load a value at a 16-bit address, `addr`, in X memory into a register, `register`.

Execution Time 1 cycle

Opcode

29	28	27	26	25	24	23	22 21 20	19 18 17 16	15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0
1	1	1	0	0	0	0	register	0	addr

LOAD xmem-reg, X[single-mode(X)] **Move**

Load register indirect from X memory

Operands `LOAD register, X[ addr ]`

Operation register = X[addr]

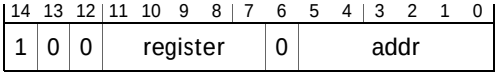
update addr

Flags Not affected

Description Load a value from X memory into a register, `register`. The address, `addr`, specifies an X pointer and an optional update. The X pointer is updated after the value is loaded.

Execution Time 1 cycle

Opcode



If this instruction is the second move instruction in a Move-Move instruction word, the encoding is shifted left 15 bits.

LOAD ax-reg, X[dual-mode(X)]

X Move

Load A or X data register indirect from X memory

Operands `LOAD register, X[ addr ]`

Operation register = X[addr]

update addr

Flags Not affected

Description Load a value from X memory into an A accumulator or X data register, `register`. The address, `addr`, specifies an X pointer and an optional update. The X pointer is updated after the value is loaded.

Execution Time 1 cycle

Constraints Only the first four X pointers, `XP0` to `XP3`, can be used in dual-mode(X).

Opcode

14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	reg	addr					-	-	-	-	-	-	-

If this instruction is the second move instruction in a Move-Move instruction word, the encoding is shifted left 15 bits.

Bits marked with "-" are determined by the Y side move instruction.

LOAD reg, Y[imm16u]

Long

Load register direct from Y memory

Operands `LOAD register, Y[ addr ]`

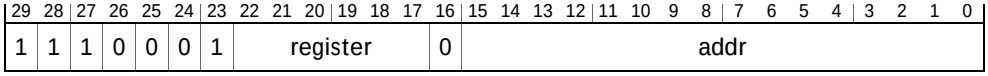
Operation register = Y[addr]

Flags Not affected

Description Load a value at a 16-bit address, `addr`, in Y memory into a register, `register`.

Execution Time 1 cycle

Opcode



29	28	27	26	25	24	23	22 21 20	19 18 17 16	15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0
1	1	1	0	0	0	1	register	0	addr

LOAD by-reg, Y[single-mode(Y)]

Move

Load B or Y data register indirect from Y memory

Operands `LOAD register, Y[ addr ]`

Operation register = Y[addr]

update addr

Flags Not affected

Description Load a value from Y memory into a B accumulator or Y data register, `register`. The address, `addr`, specifies a Y pointer and an optional update. The Y pointer is updated after the value is loaded.

Execution Time 1 cycle

Opcode

14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	0	1	0	reg		0		addr			

If this instruction is the second move instruction in a Move-Move instruction word, the encoding is shifted left 15 bits.

LOAD by-reg, Y[dual-mode(Y)] **Y Move**

Load B or Y data register indirect from Y memory

Operands `LOAD register, Y[ addr ]`

Operation register = Y[addr]

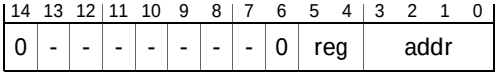
update addr

Flags Not affected

Description Load a value from Y memory into a B accumulator or Y data register, `register`. The address, `addr`, specifies a Y pointer and an optional update. The Y pointer is updated after the value is loaded.

Execution Time 1 cycle

Opcode



14	13	12	11	10	9	8	7	6	5 4	3 2 1 0
0	-	-	-	-	-	-	-	0	reg	addr

If this instruction is the second move instruction in a Move-Move instruction word, the encoding is shifted left 15 bits.

Bits marked with "-" are determined by the X side move instruction.

LOAD reg, P[imm16u] **Long**

Load register direct from P memory

Operands `LOAD register, P[ addr ]`

Operation register = P[addr]

Flags Not affected

Description Load a 24-bit value at a 16-bit address, `addr`, in P memory into a register, `register`.

Execution Time 1 cycle

Opcode

29	28	27	26	25	24	23	22 21 20	19 18 17 16	15 14 13 12	11 10 9 8	7 6 5 4	3 2 1 0
1	1	1	0	0	1	1	register	0	addr			

LOAD reg, P[single-mode(X)] **Long**

Load register indirect from P memory

Operands `LOAD register, P[ addr ]`

Operation register = P[addr]

update addr

Flags Not affected

Description Load a 24-bit value from P memory into a register, `register`. The address, `addr`, specifies an X pointer and an optional update. The X pointer is updated after the value is loaded.

Execution Time 1 cycle

Opcode

29	28	27	26	25	24	23	22 21 20	19 18 17 16	15 14 13 12	11 10 9 8	7 6 5 4	3 2 1 0
0	1	1	0	0	0	1	register	0	0 0 0 0	0 0 0 0	0 0	addr

LOAD reg, P[single-mode(Y)] **Long**

Load register indirect from P memory

Operands `LOAD register, P[ addr ]`

Operation register = P[addr]

update addr

Flags Not affected

Description Load a 24-bit value from P memory into a register, `register`. The address, `addr`, specifies a Y pointer and an optional update. The Y pointer is updated after the value is loaded.

Execution Time 1 cycle

Opcode

29	28	27	26	25	24	23	22 21 20	19 18 17 16	15 14 13 12	11 10 9 8	7 6 5 4	3 2 1 0
0	1	1	0	0	0	1	register	0	0 0 0 0	0 0 0 0	0 1 0	addr

LOAD reg, imm20s

Long

Load register with a value

Operands `LOAD register, data`

Operation register = signext(data)

Flags Not affected

Description Load a 20-bit signed immediate value, `data`, into a register, `register`.

Execution Time 1 cycle

Opcode

29	28	27	26	25 24	23 22 21 20	19 18 17 16	15 14 13 12	11 10 9 8	7 6 5 4	3 2 1 0
1	1	0	0	register		data				

LOAD d-reg, imm24s **Long**

Load data register with a value

Operands `LOAD register, data`

Operation register = signext(data)

Flags Not affected

Description Load a 24-bit signed immediate value, `data`, into a register, `register`.

Execution Time 1 cycle

Opcode

29	28	27	26 25 24	23 22 21 20 19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0
0	0	1	register	data

LOAD d-reg, imm5s

Move

Load data register with a value

Operands `LOAD register, data`

Operation register = signext(data)

Flags Not affected

Description Load a 5-bit signed immediate value, `data`, into a data register, `register`.

Execution Time 1 cycle

Opcode

14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	1	0	register				data			

If this instruction is the second move instruction in a Move-Move instruction word, the encoding is shifted left 15 bits.

LOAD d-reg, imm6s **Arith**

Load data register with a value

Operands `LOAD register, data`

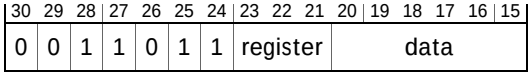
Operation register = signext(data)

Flags Not affected

Description Load a 6-bit signed immediate value, `data`, into a data register, `register`.

Execution Time 1 cycle

Opcode



30	29	28	27	26	25	24	23 22 21	20 19 18 17 16 15
0	0	1	1	0	1	1	register	data

LOAD l-reg, imm24s

Long

Load accumulator low part with a value

Operands `LOAD register, data`

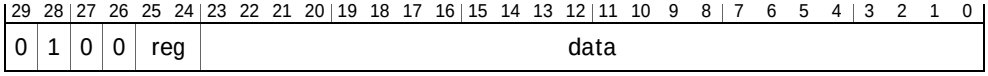
Operation register = data

Flags Not affected

Description Load a 24-bit signed immediate value, `data`, into the low part of an accumulator, `register`.

Execution Time 1 cycle

Opcode



29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	reg		data																							

LOAD xstep-reg, imm5s **Move**

Load X step register with a value

Operands `LOAD register, data`

Operation register = signext(data)

Flags Not affected

Description Load a 5-bit signed immediate value, `data`, into an X step register, `register`.

Execution Time 1 cycle

Opcode

14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	1	1	register				data			

If this instruction is the second move instruction in a Move-Move instruction word, the encoding is shifted left 15 bits.

LOAD ystep-reg, imm5s

Move

Load Y step register with a value

Operands `LOAD register, data`

Operation register = signext(data)

Flags Not affected

Description Load a 5-bit signed immediate value, `data`, into a Y step register, `register`.

Execution Time 1 cycle

Opcode

14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	0	0	reg			data			

If this instruction is the second move instruction in a Move-Move instruction word, the encoding is shifted left 15 bits.

LOAD ax-reg, imm2s

X Move

Load A or X data register with a value

Operands `LOAD register, data`

Operation register = signext(data)

Flags Not affected

Description Load a 2-bit signed immediate value, `data` (-2..1), into an A accumulator or X data register, `register`.

Execution Time 1 cycle

Opcode

14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	reg		data		0	-	-	-	-	-	-	-

If this instruction is the second move instruction in a Move-Move instruction word, the encoding is shifted left 15 bits.

Bits marked with "-" are determined by the Y side move instruction.

LOAD by-reg, imm2s

Y Move

Load B or Y data register with a value

Operands `LOAD register, data`

Operation register = signext(data)

Flags Not affected

Description Load a 2-bit signed immediate value, `data` (-2..1), into a B accumulator or Y data register, `register`.

Execution Time 1 cycle

Opcode

14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	-	-	-	-	-	-	-	1	1	reg		data		0

If this instruction is the second move instruction in a Move-Move instruction word, the encoding is shifted left 15 bits.

Bits marked with "-" are determined by the X side move instruction.

LOAD.DW ab-reg, XY[imm16u]

Long

Load double-width accumulator direct from XY memory

Operands `LOAD.DW accumulator, XY[ addr ]`

Operation accumulator.e = 0xFF * Y[addr][23]

accumulator.h = Y[addr]

accumulator.l = X[addr]

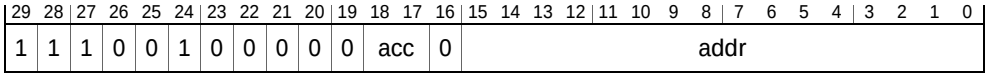
Flags Not affected

Description Load a 48-bit value at a 16-bit address, `addr`, in XY memory into an accumulator, `accumulator`.

The 48-bit value is sign-extended to 56 bits.

Execution Time 1 cycle

Opcode



29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	0	1	0	0	0	0	0	acc		0	addr															

LOAD.DW ab-reg, XY[single-mode(X)]

Move

Load double-width accumulator indirect via X pointer from XY memory

Operands

```
LOAD.DW accumulator, XY[ addr ]
```

Operation

accumulator.e = 0xFF * Y[addr][23]

accumulator.h = Y[addr]

accumulator.l = X[addr]

update addr

Flags

Not affected

Description

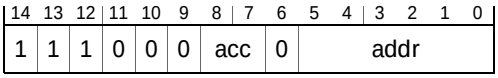
Load a 48-bit value from XY memory into an accumulator, `accumulator`. The address, `addr`, specifies an X pointer and an optional update. The X pointer is updated after the value is loaded.

The 48-bit value is sign-extended to 56 bits.

Execution Time

1 cycle

Opcode



If this instruction is the second move instruction in a Move-Move instruction word, the encoding is shifted left 15 bits.

LOAD.DW ab-reg, imm24s **Long**

Load accumulator with a double-width value

Operands `LOAD.DW accumulator, data`

Operation accumulator.e = 0xFF * data[23]

accumulator.h = 0xFFFFFF * data[23]

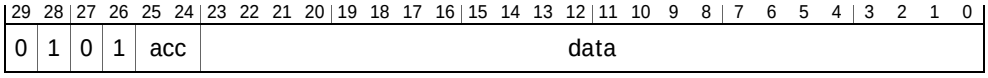
accumulator.l = data

Flags Not affected

Description Load a 24-bit signed immediate value, `data`, interpreted as a double-width value (aligned with the low register of the accumulator), into an accumulator, `accumulator`.

Execution Time 1 cycle

Opcode



LOAD.DW ab-reg, imm5s

Move

Load accumulator with a double-width value

Operands `LOAD.DW accumulator, data`

Operation accumulator.e = 0xFF * data[4]

accumulator.h = 0xFFFFFF * data[4]

accumulator.l = signext(data)

Flags Not affected

Description Load a 5-bit signed immediate value, `data`, interpreted as a double-width value (aligned with the low register of the accumulator), into an accumulator, `accumulator`.

Execution Time 1 cycle

Opcode

14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	0	1	acc			data			

If this instruction is the second move instruction in a Move-Move instruction word, the encoding is shifted left 15 bits.

LOAD.DW d-reg, imm6s **Arith**

Load data register with a double-width value

Operands `LOAD.DW register, data`

Operation register = signext(data) >> 24

Flags Not affected

Description Load a 6-bit signed immediate value, `data`, interpreted as a double-width value (aligned with the low register of the accumulator), into a data register, `register`.

Execution Time 1 cycle

Remarks Double-width load immediate is not useful when `register` is an XY data register.

Opcode

30	29	28	27	26	25	24	23 22 21 20	19 18 17 16 15
0	0	0	1	0	1	1	register	data

LOAD.EX ab-reg, P[imm16u]

Long

Load extended accumulator direct from P memory

Operands `LOAD.EX accumulator, P[ addr ]`

Operation accumulator.e = P[addr][31..24]

accumulator.h = P[addr][23..0]

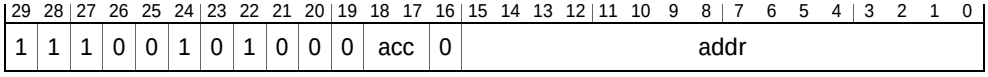
accumulator.l = 0

Flags Not affected

Description Load a 32-bit value at a 16-bit address, `addr`, in P memory into an accumulator, `accumulator`.

Execution Time 1 cycle

Opcode



29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	0	1	0	1	0	0	0	acc		0	addr															

LOAD.EX ab-reg, P[single-mode(X)] **Long**

Load extended accumulator indirect via X pointer from P memory

Operands `LOAD.EX accumulator, P[ addr ]`

Operation accumulator.e = P[addr][31..24]

accumulator.h = P[addr][23..0]

accumulator.l = 0

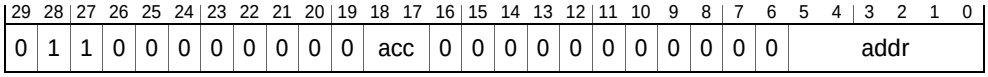
update addr

Flags Not affected

Description Load a 32-bit value from P memory into an accumulator, `accumulator`. The address, `addr`, specifies an X pointer and an optional update. The X pointer is updated after the value is loaded.

Execution Time 1 cycle

Opcode



29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	0	0	0	0	0	0	0	acc		0	0	0	0	0	0	0	0	0	0	0	addr					

LOAD.EX ab-reg, P[single-mode(Y)]

Move

Load extended accumulator indirect via Y pointer from P memory

Operands `LOAD.EX accumulator, P[ addr ]`

Operation accumulator.e = P[addr][31..24]

accumulator.h = P[addr][23..0]

accumulator.l = 0

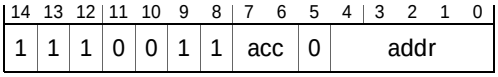
update addr

Flags Not affected

Description Load a 32-bit value from P memory into an accumulator, `accumulator`. The address, `addr`, specifies a Y pointer and an optional update. The Y pointer is updated after the value is loaded.

Execution Time 1 cycle

Opcode



If this instruction is the second move instruction in a Move-Move instruction word, the encoding is shifted left 15 bits.

LOG2ABS d-reg, d-reg **Arith**

Calculate the logarithm base 2 of the absolute value of a data register, storing the result in a data register

Operands `LOG2ABS dst-reg, src-reg`

Operation dst-reg = log2(|src-reg|)

Flags If Flag Updates are enabled, the X ALU status flags are affected as follows:

X Carry:	Not affected
X Negative:	Set according to ALU output
X Zero:	Set according to ALU output
X Overflow:	Set to 0
X Extension:	Not affected

Description Calculate the logarithm base 2 of the absolute value of a data register, `src-reg`, and store the result in a data register, `dst-reg`.

Execution Time 1 cycle

Opcode

30	29	28	27	26	25	24	23 22 21	20	19	18	17 16 15
0	1	1	0	1	1	1	dst-reg	0	0	0	src-reg

LOOP imm12u, imm16u **Synth**

Loop with immediate count and no delay slots

Operands `LOOP count, loop-end-addr`

Operation push (PC+2, loop-end-addr, count) on the loop stack

Flags Not affected

Description Start a loop with a 12-bit unsigned immediate loop count, `count`. The loop is initiated by pushing the end address, start address, and count onto the loop stack. The start address is `PC` + 2. The loop body is then executed *count* times, where the value 0 is interpreted as 4096.

Execution Time 3 cycles

Constraints Nested loops cannot have the same end address.

Remarks This instruction is synthesized by the assembler as follows:

```
LOOP.2D count, loop-end-addr
NOP >>> NOP
```

Opcode

See LOOP.2D imm12u, imm16u and NOP.

LOOP d-reg, imm16u **Synth**

Loop with data register count and no delay slots

Operands `LOOP count, loop-end-addr`

Operation push (PC+2, loop-end-addr, count) on the loop stack

Flags Not affected

Description Start a loop with a loop count from a data register, `count`. The loop is initiated by pushing the end address, start address, and count onto the loop stack. The count is taken from the bottom 12 bits of an XY data register, or the bottom 12 bits of the *high* part of an accumulator. The start address is `PC` + 2. The loop body is then executed *count* times, where the value 0 is interpreted as 4096.

Execution Time 3 cycles

Constraints Nested loops cannot have the same end address.

Remarks This instruction is synthesized by the assembler as follows:

```
LOOP.2D count, loop-end-addr
NOP >>> NOP
```

Opcode

See LOOP.2D d-reg, imm16u and NOP.

LOOP.2D imm12u, imm16u **Long**

Loop with immediate count and two delay slots

Operands `LOOP.2D count, loop-end-addr`

Operation push (PC+2/3, loop-end-addr, count) on the loop stack

Flags Not affected

Description Start a loop with a 12-bit unsigned immediate loop count, `count`. The loop is initiated by pushing the end address, start address, and count onto the loop stack. The start address is `PC` + 2 if the two delay slot cycles are filled with one (*Move-Move*) instruction. Otherwise, with two instructions in the delay slots, the start address is `PC` + 3. The loop body is then executed *count* times, where the value 0 is interpreted as 4096. The instructions in the delay slots are not part of the loop body.

The two delay slots can be filled in one of two ways:

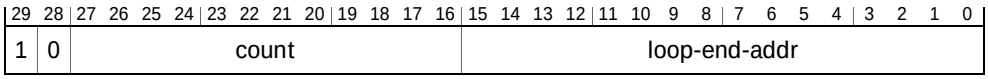
- One *Move-Move* instruction that executes in exactly 2 cycles.
- Two *Arithmetic-Move* or non-control-flow single-cycle *Long* instructions.

See Section 7.5.5, “Program Flow Delay Slot Restrictions” on page 62 for more information.

Execution Time 3 cycles, 2 pipelined (delay slots)

Constraints Nested loops cannot have the same end address.

Opcode



LOOP.2D d-reg, imm16u **Long**

Loop with data register count and two delay slots

Operands `LOOP.2D count, loop-end-addr`

Operation push (PC+2/3, loop-end-addr, count) on the loop stack

Flags Not affected

Description Start a loop with a loop count from a data register, `count`. The loop is initiated by pushing the end address, start address, and count onto the loop stack. The count is taken from the bottom 12 bits of an XY data register or the bottom 12 bits of the *high* part of an accumulator. The start address is `PC` + 2 if the two delay slot cycles are filled with one (*Move-Move*) instruction. Otherwise, with two instructions in the delay slots, the start address is `PC` + 3. The loop body is then executed *count* times, where the value 0 is interpreted as 4096. The instructions in the delay slots are not part of the loop body.

The two delay slots can be filled in one of two ways:

- One *Move-Move* instruction that executes in exactly 2 cycles.
- Two *Arithmetic-Move* or non-control-flow single-cycle *Long* instructions.

See Section 7.5.5, “Program Flow Delay Slot Restrictions” on page 62 for more information.

Execution Time 3 cycles, 2 pipelined (delay slots)

Constraints Nested loops cannot have the same end address.

Opcode

29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	0	1	1	0	0	0	count			loop-end-addr															

MAX d-reg, d-reg, d-reg **Arith**

Determine the maximum value of two data registers, storing the result in a data register

Operands `MAX dst-reg, src-reg-1, src-reg-2`

Operation dst-reg = max(src-reg-1, src-reg-2)

Flags If Flag Updates are enabled, the X ALU status flags are affected as follows:

X Carry:	Set according to the ALU output of `src-reg-1 - src-reg-2`
X Negative:	Set according to the ALU output of `src-reg-1 - src-reg-2`
X Zero:	Set according to the ALU output of `src-reg-1 - src-reg-2`
X Overflow:	Set according to the ALU output of `src-reg-1 - src-reg-2`
X Extension:	Set according to the ALU output of `src-reg-1 - src-reg-2`

Description Calculate the maximum value of two data registers, `src-reg-1` and `src-reg-2`, and store the result in a data register, `dst-reg`.

Execution Time 1 cycle

Opcode

30	29	28	27	26	25	24	23 22 21	20 19 18	17 16 15
0	1	1	0	0	1	0	dst-reg	src-reg-1	src-reg-2

MAX a-reg, a-reg, ab-reg **X Arith**

Determine the maximum value of two accumulators, and store the result in an A accumulator

Operands `MAX dst-reg, src-reg-1, src-reg-2`

Operation dst-reg = max(src-reg-1, src-reg-2)

Flags If Flag Updates are enabled, the X ALU status flags are affected as follows:

X Carry:	Set according to the ALU output of `src-reg-1 - src-reg-2`
X Negative:	Set according to the ALU output of `src-reg-1 - src-reg-2`
X Zero:	Set according to the ALU output of `src-reg-1 - src-reg-2`
X Overflow:	Set according to the ALU output of `src-reg-1 - src-reg-2`
X Extension:	Set according to the ALU output of `src-reg-1 - src-reg-2`

Description Calculate the maximum value of two accumulators, `src-reg-1` and `src-reg-2`, and store the result in an A accumulator, `dst-reg`.

Execution Time 1 cycle

Constraints The Y side instruction must not be `MUL*`.

The right-most source register on the X and Y sides must be opposite—one side using `a-reg` and the other side `b-reg`.

Opcode

MAX a-reg, a-reg, a-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	1	0	0	1	1	dst	0	s1	s2	-	-	-	0	-	-

MAX a-reg, a-reg, b-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	1	0	0	1	1	dst	1	s1	s2	-	-	-	0	-	-

Bits marked with "-" are determined by the Y side arithmetic instruction.

MIN d-reg, d-reg, d-reg **Arith**

Determine the minimum value of two data registers, storing the result in a data register

Operands `MIN dst-reg, src-reg-1, src-reg-2`

Operation dst-reg = min(src-reg-1, src-reg-2)

Flags If Flag Updates are enabled, the X ALU status flags are affected as follows:

X Carry:	Set according to the ALU output of `src-reg-1 - src-reg-2`
X Negative:	Set according to the ALU output of `src-reg-1 - src-reg-2`
X Zero:	Set according to the ALU output of `src-reg-1 - src-reg-2`
X Overflow:	Set according to the ALU output of `src-reg-1 - src-reg-2`
X Extension:	Set according to the ALU output of `src-reg-1 - src-reg-2`

Description Calculate the minimum value of two data registers, `src-reg-1` and `src-reg-2`, and store the result in a data register, `dst-reg`.

Execution Time 1 cycle

Opcode

30	29	28	27	26	25	24	23 22 21	20 19 18	17 16 15
0	1	1	0	0	1	1	dst-reg	src-reg-1	src-reg-2

MIN a-reg, a-reg, ab-reg **X Arith**

Determine the minimum value of two accumulators, and store the result in an A accumulator

Operands `MIN dst-reg, src-reg-1, src-reg-2`

Operation dst-reg = min(src-reg-1, src-reg-2)

Flags If Flag Updates are enabled, the X ALU status flags are affected as follows:

X Carry:	Set according to the ALU output of `src-reg-1 - src-reg-2`
X Negative:	Set according to the ALU output of `src-reg-1 - src-reg-2`
X Zero:	Set according to the ALU output of `src-reg-1 - src-reg-2`
X Overflow:	Set according to the ALU output of `src-reg-1 - src-reg-2`
X Extension:	Set according to the ALU output of `src-reg-1 - src-reg-2`

Description Calculate the minimum value of two accumulators, `src-reg-1` and `src-reg-2`, and store the result in an A accumulator, `dst-reg`.

Execution Time 1 cycle

Constraints The Y side instruction must not be `MUL*`.

The right-most source register on the X and Y sides must be opposite—one side using `a-reg` and the other side `b-reg`.

Opcode

MIN a-reg, a-reg, a-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	1	0	0	1	1	dst	0	s1	s2	-	-	-	1	-	-

MIN a-reg, a-reg, b-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	1	0	0	1	1	dst	1	s1	s2	-	-	-	1	-	-

Bits marked with "-" are determined by the Y side arithmetic instruction.

MOVE reg, reg

Move

Move register to register

Operands `MOVE dst-reg, src-reg`

Operation dst-reg = src-reg

Flags If saturation is enabled and `src-reg` is an accumulator, these status flags are updated:

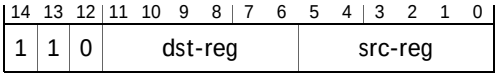
Saturated: Set if saturation occurs

Description Move the contents of a register, `src-reg`, to a register, `dst-reg`.

If `src-reg` is a 56-bit accumulator, the value from the accumulator undergoes rounding and saturation if they are enabled.

Execution Time 1 cycle

Opcode



If this instruction is the second move instruction in a Move-Move instruction word, the encoding is shifted left 15 bits.

MOVE ab-reg, x-reg

X Move

Move X data register to accumulator

Operands `MOVE dst-reg, src-reg`

Operation dst-reg.e = 0xFF * src-reg[23]

dst-reg.h = src-reg

dst-reg.l = 0

Flags Not affected

Description Move the contents of an X data register, `src-reg`, to an accumulator, `dst-reg`.

Execution Time 1 cycle

Constraints The destination register can only be a `b-reg` when the Y side instruction does not have a `b-reg` as a destination register.

Opcode

MOVE a-reg, x-reg

14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	dst	src	0	1	1	-	-	-	-	-	-	-

MOVE b-reg, x-reg

14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	dst	src	1	1	1	-	-	-	-	-	-	-

If this instruction is the second move instruction in a Move-Move instruction word, the encoding is shifted left 15 bits.

Bits marked with "-" are determined by the Y side move instruction.

MOVE xy-reg, a-reg

X Move

Move A accumulator to XY data register

Operands `MOVE dst-reg, src-reg`

Operation dst-reg = src-reg.h

Flags If saturation is enabled, these status flags are updated:

Saturated: Set if saturation occurs

Description Move the contents of an A accumulator, `src-reg`, to an XY data register, `dst-reg`.

If rounding and saturation are enabled, they are performed on the value from `src-reg`.

Execution Time 1 cycle

Constraints The destination register can only be a `y-reg` when the Y side instruction does not have a `y-reg` as a destination register.

Opcode

MOVE x-reg, a-reg

14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	dst	src	0	0	1	-	-	-	-	-	-	-

MOVE y-reg, a-reg

14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	dst	src	1	0	1	-	-	-	-	-	-	-

If this instruction is the second move instruction in a Move-Move instruction word, the encoding is shifted left 15 bits.

Bits marked with "-" are determined by the Y side move instruction.

MOVE ab-reg, y-reg

Y Move

Move Y data register to accumulator

Operands `MOVE dst-reg, src-reg`

Operation dst-reg.e = 0xFF * src-reg[23]

dst-reg.h = src-reg

dst-reg.l = 0

Flags Not affected

Description Move the contents of a Y data register, `src-reg`, to an accumulator, `dst-reg`.

Execution Time 1 cycle

Constraints The destination register can only be an `a-reg` when the X side instruction does not have an `a-reg` as a destination register.

Opcode

MOVE a-reg, y-reg

14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	-	-	-	-	-	-	-	1	1	dst	src	1	1	1

MOVE b-reg, y-reg

14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	-	-	-	-	-	-	-	1	1	dst	src	0	1	1

If this instruction is the second move instruction in a Move-Move instruction word, the encoding is shifted left 15 bits.

Bits marked with "-" are determined by the X side move instruction.

MOVE xy-reg, b-reg

Y Move

Move B accumulator to XY data register

Operands `MOVE dst-reg, src-reg`

Operation dst-reg = src-reg.h

Flags If saturation is enabled, these status flags are updated:

Saturated: Set if saturation occurs

Description Move the contents of a B accumulator, `src-reg`, to an XY data register, `dst-reg`.

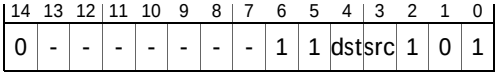
If rounding and saturation are enabled, they are performed on the value from `src-reg`.

Execution Time 1 cycle

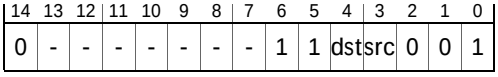
Constraints The destination register can only be an `x-reg` when the X side instruction does not have an `x-reg` as a destination register.

Opcode

MOVE x-reg, b-reg



MOVE y-reg, b-reg



If this instruction is the second move instruction in a Move-Move instruction word, the encoding is shifted left 15 bits.

Bits marked with "-" are determined by the X side move instruction.

MOVE d-reg, d-reg **Arith**

Move data register to data register

Operands `MOVE dst-reg, src-reg`

Operation dst-reg = src-reg

Flags If Flag Updates are enabled, the X ALU status flags are affected as follows:

X Carry:	Not affected
X Negative:	Set according to value in the source register
X Zero:	Set according to value in the source register
X Overflow:	Set to 0
X Extension:	Set according to value in the source register

Description Move the contents of a data register, `src-reg`, to a data register, `dst-reg`.

Between accumulators, this instruction moves the entire 56-bit value of the accumulator.

Rounding and saturation do not occur on the value from `src-reg`.

Execution Time 1 cycle

Remarks This instruction is a synonym for "`XOR.DW dst-reg, src-reg, 0`".

This instruction requires version 3 or greater of the CFX assembler.

Note that this instruction updates the flags when Flag Updates is enabled, while other data move instructions do not.

Opcode

See XOR.DW d-reg, d-reg, imm3s.

MOVE d-reg, d-reg, cond3 **Arith**

Move data register to data register, conditionally

Operands `MOVE dst-reg, src-reg, condition`

Operation if condition holds, dst-reg = src-reg

Flags Not affected

Description If the condition holds, move the contents of a data register, `src-reg`, to a data register, `dst-reg`.

Between accumulators, this instruction moves the entire 56-bit value of the accumulator.

Rounding and saturation do not occur on the value from `src-reg`.

Execution Time 1 cycle

Remarks Version 3 or greater of the CFX assembler supports the following alternate form for this conditional instruction:

```
MOVE d-reg, d-reg if cond3
```

Opcode

30	29	28	27	26	25	24	23 22 21	20 19 18	17 16 15
0	1	1	1	1	0	1	dst-reg	cond	src-reg

MOVE.DW ab-reg, ab-reg

Move

Move double-width accumulator to double-width accumulator

Operands `MOVE.DW dst-reg, src-reg`

Operation dst-reg.e = 0xFF * src-reg.h[23]

dst-reg.h = src-reg.h

dst-reg.l = src-reg.l

Flags If saturation is enabled, these status flags are updated:

Saturated: Set if saturation occurs

Description Move the double-width contents of an accumulator, `src-reg`, to an accumulator, `dst-reg`.

Before the result is written to the destination register, the upper 8 bits are overwritten with the sign extension from bit 47 of the source register.

If enabled, saturation is performed on the value from `src-reg`.

Execution Time 1 cycle

Opcode

14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	0	1	0	0	0	dst		src	

If this instruction is the second move instruction in a Move-Move instruction word, the encoding is shifted left 15 bits.

MOVE.EX ab-reg, ab-reg

Move

Move extended accumulator to accumulator

Operands `MOVE.EX dst-reg, src-reg`

Operation dst-reg.e = src-reg.e

dst-reg.h = src-reg.h

dst-reg.l = 0

Flags Not affected

Description Move the extended value of an accumulator, `src-reg`, to an accumulator, `dst-reg`.

The lower 24 bits of the destination are overwritten with 0.

If enabled, rounding is performed on the value from `src-reg`.

Execution Time 1 cycle

Opcode

14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	0	1	0	0	1	dst		src	

If this instruction is the second move instruction in a Move-Move instruction word, the encoding is shifted left 15 bits.

MOVE.NX d-reg, d-reg

Arith

Move data register to data register with no extension

Operands `MOVE.NX dst-reg, src-reg`

Operation dst-reg.e = 0xFF * src-reg.h[23]

dst-reg.h = src-reg.h

dst-reg.l = src-reg.l

Flags Not affected

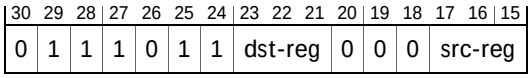
Description Move the contents of a data register, `src-reg`, to a data register, `dst-reg`.

Before the result is written to the destination register, the upper 8 bits are overwritten with the sign extension from bit 47 of the source register.

Rounding and saturation do not occur on the value from `src-reg`.

Execution Time 1 cycle

Opcode



30	29	28	27	26	25	24	23 22 21	20	19	18	17 16 15
0	1	1	1	0	1	1	dst-reg	0	0	0	src-reg

MOVE.NX a-reg, ab-reg

X Arith

Move accumulator to A accumulator with no extension

Operands `MOVE.NX dst-reg, src-reg`

Operation dst-reg.e = 0xFF * src-reg.h[23]

dst-reg.h = src-reg.h

dst-reg.l = src-reg.l

Flags Not affected

Description Move Accumulator to A Accumulator with No Extension

Move the contents of an accumulator, `src-reg`, to an A accumulator, `dst-reg`.

Before the result is written to the destination register, the upper 8 bits are overwritten with the sign extension from bit 47 of the source register.

Rounding and saturation do not occur on the value from `src-reg`.

Execution Time 1 cycle

Constraints The Y side instruction must not be `MUL*`.

The right-most source register on the X and Y sides must be opposite—one side using `a-reg` and the other side `b-reg`.

Opcode

MOVE.NX a-reg, a-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	1	0	1	1	1	dst	0	0	src	-	-	-	1	-	-

MOVE.NX a-reg, b-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	1	0	1	1	1	dst	1	0	src	-	-	-	1	-	-

Bits marked with "-" are determined by the Y side arithmetic instruction.

MUL ab-reg, xy-reg, xy-reg **Arith**

Multiply two XY data registers, storing the result in an accumulator

Operands `MUL dst-reg, src-reg-1, src-reg-2`

Operation dst-reg = 0 + src-reg-1 * src-reg-2

Flags If Flag Updates are enabled, the X ALU status flags are affected when the ALU operation completes (after the next clock cycle), as follows:

X Carry:	Not affected
X Negative:	Set according to result of adding 0 to product
X Zero:	Set according to result of adding 0 to product
X Overflow:	Set to 0
X Extension:	Set according to result of adding 0 to product

Description Multiply two XY data registers, `src-reg-1` and `src-reg-2`, and write the result to an accumulator, `dst-reg`.

Execution Time 2 cycles, 1 pipelined (ALU operation)

Opcode

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	1	0	0	0	0	0	src1		src2	1	0	dst		

MUL a-reg, x-reg, xy-reg **X Arith**

Multiply two XY data registers, storing the result in an A accumulator

Operands `MUL dst-reg, src-reg-1, src-reg-2`

Operation dst-reg = 0 + src-reg-1 * src-reg-2

Flags If Flag Updates are enabled, the X ALU status flags are affected when the ALU operation completes (after the next clock cycle), as follows:

X Carry:	Not affected
X Negative:	Set according to result of adding 0 to product
X Zero:	Set according to result of adding 0 to product
X Overflow:	Set to 0
X Extension:	Set according to result of adding 0 to product

Description Multiply two XY data registers, `src-reg-1` and `src-reg-2`, and write the result to an A accumulator, `dst-reg`.

Execution Time 2 cycles, 1 pipelined (ALU operation)

Constraints When the Y side instruction is `MUL*`, the right-most source register on the X and Y sides must be opposite—one side using `x-reg` and the other side `y-reg`.

Opcode

Y side instruction is not MUL*

MUL a-reg, x-reg, x-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	0	0	0	s1	s2	1	0	dst	-	-	-	-	-	-

MUL a-reg, x-reg, y-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	0	0	1	s1	s2	1	0	dst	-	-	-	-	-	-

Y side instruction is MUL*

MUL a-reg, x-reg, x-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	1	1	1	0	dst	0	s1	s2	-	-	-	0	-	-

MUL a-reg, x-reg, y-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	1	1	1	0	dst	1	s1	s2	-	-	-	0	-	-

Bits marked with “-” are determined by the Y side arithmetic instruction.

MUL b-reg, y-reg, xy-reg

Y Arith

Multiply two XY data registers, storing the result in a B accumulator

Operands `MUL dst-reg, src-reg-1, src-reg-2`

Operation dst-reg = 0 + src-reg-1 * src-reg-2

Flags If Flag Updates are enabled, the Y ALU status flags are affected when the ALU operation completes (after the next clock cycle), as follows:

Y Carry:	Not affected
Y Negative:	Set according to result of adding 0 to product
Y Zero:	Set according to result of adding 0 to product
Y Overflow:	Set to 0
Y Extension:	Set according to result of adding 0 to product

Description Multiply two XY data registers, `src-reg-1` and `src-reg-2`, and write the result to a B accumulator, `dst-reg`.

Execution Time 2 cycles, 1 pipelined (ALU operation)

Constraints When the X side instruction is `MUL*`, the right-most source register on the X and Y sides must be opposite—one side using `x-reg` and the other side `y-reg`.

Opcode

X side instruction is not MUL* or NOP

MUL b-reg, y-reg, x-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	0	1	-	-	-	-	-	-	1	s1	s2	1	0	dst

MUL b-reg, y-reg, y-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	0	1	-	-	-	-	-	-	0	s1	s2	1	0	dst

X side instruction is MUL* or NOP

MUL b-reg, y-reg, x-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	1	1	-	-	-	1	-	-	1	0	dst	-	s1	s2

MUL b-reg, y-reg, y-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	1	1	-	-	-	0	-	-	1	0	dst	-	s1	s2

Bits marked with “-” are determined by the X side arithmetic instruction.

MUL.SU ab-reg, xy-reg, xy-reg **Arith**

Multiply two XY data registers, one signed and one unsigned, storing the result in an accumulator

Operands `MUL.SU dst-reg, src-reg-1, src-reg-2`

Operation dst-reg = 0 + src-reg-1 * src-reg-2

Flags If Flag Updates are enabled, the X ALU status flags are affected when the ALU operation completes (after the next clock cycle), as follows:

X Carry:	Not affected
X Negative:	Set according to result of adding 0 to product
X Zero:	Set according to result of adding 0 to product
X Overflow:	Set to 0
X Extension:	Set according to result of adding 0 to product

Description Multiply two XY data registers, `src-reg-1` and `src-reg-2`, and write the result to an accumulator, `dst-reg`.

The multiplication treats `src-reg-1` as a signed value and `src-reg-2` as an unsigned value.

Execution Time 2 cycles, 1 pipelined (ALU operation)

Opcode

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	1	0	0	0	1	0	src1		src2	1	0	dst		

MUL.UU ab-reg, xy-reg, xy-reg **Arith**

Multiply two XY data registers, both unsigned, storing the result in an accumulator

Operands `MUL.UU dst-reg, src-reg-1, src-reg-2`

Operation dst-reg = 0 + src-reg-1 * src-reg-2

Flags If Flag Updates are enabled, the X ALU status flags are affected when the ALU operation completes (after the next clock cycle), as follows:

X Carry:	Not affected
X Negative:	Set according to result of adding 0 to product
X Zero:	Set according to result of adding 0 to product
X Overflow:	Set to 0
X Extension:	Set according to result of adding 0 to product

Description Multiply two XY data registers, `src-reg-1` and `src-reg-2`, and write the result to an accumulator, `dst-reg`.

The multiplication treats `src-reg-1` and `src-reg-2` as unsigned values.

Execution Time 2 cycles, 1 pipelined (ALU operation)

Opcode

30	29	28	27	26	25	24	23	22 21	20	19	18	17 16	15
1	0	1	0	0	0	0	1	src1	src2	1	0	dst	

MULADD ab-reg, xy-reg, xy-reg **Arith**

Multiply two XY data registers, and add the result to an accumulator

Operands `MULADD dst-reg, src-reg-1, src-reg-2`

Operation dst-reg = dst-reg + src-reg-1 * src-reg-2

Flags If Flag Updates are enabled, the X ALU status flags are affected when the ALU operation completes (after the next clock cycle), as follows:

X Carry:	Set according to result of adding `dst-reg` to product
X Negative:	Set according to result of adding `dst-reg` to product
X Zero:	Set according to result of adding `dst-reg` to product
X Overflow:	Set according to result of adding `dst-reg` to product
X Extension:	Set according to result of adding `dst-reg` to product

Description Multiply two XY data registers, `src-reg-1` and `src-reg-2`, add an accumulator, `dst-reg`, and write the result back to `dst-reg`.

Execution Time 2 cycles, 1 pipelined (ALU operation)

Opcode

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	1	0	0	0	0	0	src1		src2	0	0	dst		

MULADD a-reg, x-reg, xy-reg **X Arith**

Multiply two XY data registers, and add the result to an A accumulator

Operands `MULADD dst-reg, src-reg-1, src-reg-2`

Operation dst-reg = dst-reg + src-reg-1 * src-reg-2

Flags If Flag Updates are enabled, the X ALU status flags are affected when the ALU operation completes (after the next clock cycle), as follows:

X Carry:	Set according to result of adding `dst-reg` to product
X Negative:	Set according to result of adding `dst-reg` to product
X Zero:	Set according to result of adding `dst-reg` to product
X Overflow:	Set according to result of adding `dst-reg` to product
X Extension:	Set according to result of adding `dst-reg` to product

Description Multiply two XY data registers, `src-reg-1` and `src-reg-2`, add an A accumulator, `dst-reg`, and write the result back to `dst-reg`.

Execution Time 2 cycles, 1 pipelined (ALU operation)

Constraints When the Y side instruction is `MUL*`, the right-most source register on the X and Y sides must be opposite—one side using `x-reg` and the other side `y-reg`.

Opcode

Y side instruction is not MUL*

MULADD a-reg, x-reg, x-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	0	0	0	s1	s2	0	0	dst	-	-	-	-	-	-

MULADD a-reg, x-reg, y-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	0	0	1	s1	s2	0	0	dst	-	-	-	-	-	-

Y side instruction is MUL*

MULADD a-reg, x-reg, x-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	1	1	0	0	dst	0	s1	s2	-	-	-	0	-	-

MULADD a-reg, x-reg, y-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	1	1	0	0	dst	1	s1	s2	-	-	-	0	-	-

Bits marked with "-" are determined by the Y side arithmetic instruction.

MULADD b-reg, y-reg, xy-reg **Y Arith**

Multiply two XY data registers, and add the result to a B accumulator

Operands `MULADD dst-reg, src-reg-1, src-reg-2`

Operation dst-reg = dst-reg + src-reg-1 * src-reg-2

Flags If Flag Updates are enabled, the Y ALU status flags are affected when the ALU operation completes (after the next clock cycle), as follows:

Y Carry:	Set according to result of adding `dst-reg` to product
Y Negative:	Set according to result of adding `dst-reg` to product
Y Zero:	Set according to result of adding `dst-reg` to product
Y Overflow:	Set according to result of adding `dst-reg` to product
Y Extension:	Set according to result of adding `dst-reg` to product

Description Multiply two XY data registers, `src-reg-1` and `src-reg-2`, add a B accumulator, `dst-reg`, and write the result back to `dst-reg`.

Execution Time 2 cycles, 1 pipelined (ALU operation)

Constraints When the X side instruction is `MUL*`, the right-most source register on the X and Y sides must be opposite—one side using `x-reg` and the other side `y-reg`.

Opcode

X side instruction is not MUL* or NOP

MULADD b-reg, y-reg, x-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	0	1	-	-	-	-	-	-	1	s1	s2	0	0	dst

MULADD b-reg, y-reg, y-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	0	1	-	-	-	-	-	-	0	s1	s2	0	0	dst

X side instruction is MUL* or NOP

MULADD b-reg, y-reg, x-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	1	1	-	-	-	1	-	-	0	0	dst	-	s1	s2

MULADD b-reg, y-reg, y-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	1	1	-	-	-	0	-	-	0	0	dst	-	s1	s2

Bits marked with "-" are determined by the X side arithmetic instruction.

MULADD.SU ab-reg, xy-reg, xy-reg **Arith**

Multiply two XY data registers, one signed and one unsigned, and add the result to an accumulator

Operands `MULADD.SU dst-reg, src-reg-1, src-reg-2`

Operation dst-reg = dst-reg + src-reg-1 * src-reg-2

Flags If Flag Updates are enabled, the X ALU status flags are affected when the ALU operation completes (after the next clock cycle), as follows:

X Carry:	Set according to result of adding `dst-reg` to product
X Negative:	Set according to result of adding `dst-reg` to product
X Zero:	Set according to result of adding `dst-reg` to product
X Overflow:	Set according to result of adding `dst-reg` to product
X Extension:	Set according to result of adding `dst-reg` to product

Description Multiply two XY data registers, `src-reg-1` and `src-reg-2`, add an accumulator, `dst-reg`, and write the result back to `dst-reg`.

The multiplication treats `src-reg-1` as a signed value and `src-reg-2` as an unsigned value.

Execution Time 2 cycles, 1 pipelined (ALU operation)

Opcode

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	1	0	0	0	1	0	src1		src2	0	0	dst		

MULADD.UU ab-reg, xy-reg, xy-reg **Arith**

Multiply two XY data registers, both unsigned, and add the result to an accumulator

Operands `MULADD.UU dst-reg, src-reg-1, src-reg-2`

Operation dst-reg = dst-reg + src-reg-1 * src-reg-2

Flags If Flag Updates are enabled, the X ALU status flags are affected when the ALU operation completes (after the next clock cycle), as follows:

X Carry:	Set according to result of adding `dst-reg` to product
X Negative:	Set according to result of adding `dst-reg` to product
X Zero:	Set according to result of adding `dst-reg` to product
X Overflow:	Set according to result of adding `dst-reg` to product
X Extension:	Set according to result of adding `dst-reg` to product

Description Multiply two XY data registers, `src-reg-1` and `src-reg-2`, add an accumulator, `dst-reg`, and write the result back to `dst-reg`.

The multiplication treats `src-reg-1` and `src-reg-2` as unsigned values.

Execution Time 2 cycles, 1 pipelined (ALU operation)

Opcode

30	29	28	27	26	25	24	23	22 21	20 19	18	17	16 15
1	0	1	0	0	0	0	1	src1	src2	0	0	dst

MULNEG ab-reg, xy-reg, xy-reg **Arith**

Multiply two XY data registers, negate the result and store it in an accumulator

Operands `MULNEG dst-reg, src-reg-1, src-reg-2`

Operation dst-reg = 0 - src-reg-1 * src-reg-2

Flags If Flag Updates are enabled, the X ALU status flags are affected when the ALU operation completes (after the next clock cycle), as follows:

X Carry:	Not affected
X Negative:	Set according to result of subtracting product from 0
X Zero:	Set according to result of subtracting product from 0
X Overflow:	Set to 0
X Extension:	Set according to result of subtracting product from 0

Description Multiply two XY data registers, `src-reg-1` and `src-reg-2`, negate the result and write it to an accumulator, `dst-reg`.

Execution Time 2 cycles, 1 pipelined (ALU operation)

Opcode

30	29	28	27	26	25	24	23	22 21	20 19	18	17	16 15
1	0	1	0	0	0	0	0	src1	src2	1	1	dst

MULNEG a-reg, x-reg, xy-reg **X Arith**

Multiply two XY data registers, negate the result and store it in an A accumulator

Operands `MULNEG dst-reg, src-reg-1, src-reg-2`

Operation dst-reg = 0 - src-reg-1 * src-reg-2

Flags If Flag Updates are enabled, the X ALU status flags are affected when the ALU operation completes (after the next clock cycle), as follows:

X Carry:	Not affected
X Negative:	Set according to result of subtracting product from 0
X Zero:	Set according to result of subtracting product from 0
X Overflow:	Set to 0
X Extension:	Set according to result of subtracting product from 0

Description Multiply two XY data registers, `src-reg-1` and `src-reg-2`, negate the result and write it to an A accumulator, `dst-reg`.

Execution Time 2 cycles, 1 pipelined (ALU operation)

Constraints When the Y side instruction is `MUL*`, the right-most source register on the X and Y sides must be opposite—one side using `x-reg` and the other side `y-reg`.

Opcode

Y side instruction is not MUL*

MULNEG a-reg, x-reg, x-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	0	0	0	s1	s2	1	1	dst	-	-	-	-	-	-

MULNEG a-reg, x-reg, y-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	0	0	1	s1	s2	1	1	dst	-	-	-	-	-	-

Y side instruction is MUL*

MULNEG a-reg, x-reg, x-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	1	1	1	1	dst	0	s1	s2	-	-	-	0	-	-

MULNEG a-reg, x-reg, y-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	1	1	1	1	dst	1	s1	s2	-	-	-	0	-	-

Bits marked with “-” are determined by the Y side arithmetic instruction.

MULNEG b-reg, y-reg, xy-reg **Y Arith**

Multiply two XY data registers, negate the result and store it in a B accumulator

Operands `MULNEG dst-reg, src-reg-1, src-reg-2`

Operation dst-reg = 0 - src-reg-1 * src-reg-2

Flags If Flag Updates are enabled, the Y ALU status flags are affected when the ALU operation completes (after the next clock cycle), as follows:

Y Carry:	Not affected
Y Negative:	Set according to result of subtracting product from 0
Y Zero:	Set according to result of subtracting product from 0
Y Overflow:	Set to 0
Y Extension:	Set according to result of subtracting product from 0

Description Multiply two XY data registers, `src-reg-1` and `src-reg-2`, negate the result and write it to a B accumulator, `dst-reg`.

Execution Time 2 cycles, 1 pipelined (ALU operation)

Constraints When the X side instruction is `MUL*`, the right-most source register on the X and Y sides must be opposite—one side using `x-reg` and the other side `y-reg`.

Opcode

X side instruction is not MUL* or NOP

MULNEG b-reg, y-reg, x-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	0	1	-	-	-	-	-	-	1	s1	s2	1	1	dst

MULNEG b-reg, y-reg, y-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	0	1	-	-	-	-	-	-	0	s1	s2	1	1	dst

X side instruction is MUL* or NOP

MULNEG b-reg, y-reg, x-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	1	1	-	-	-	1	-	-	1	1	dst	-	s1	s2

MULNEG b-reg, y-reg, y-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	1	1	-	-	-	0	-	-	1	1	dst	-	s1	s2

Bits marked with “-” are determined by the X side arithmetic instruction.

MULNEG.SU ab-reg, xy-reg, xy-reg **Arith**

Multiply two XY data registers, one signed and one unsigned, negate the result and store it in an accumulator

Operands `MULNEG.SU dst-reg, src-reg-1, src-reg-2`

Operation dst-reg = 0 - src-reg-1 * src-reg-2

Flags If Flag Updates are enabled, the X ALU status flags are affected when the ALU operation completes (after the next clock cycle), as follows:

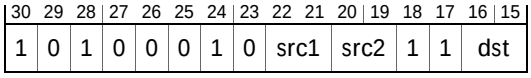
X Carry:	Not affected
X Negative:	Set according to result of subtracting product from 0
X Zero:	Set according to result of subtracting product from 0
X Overflow:	Set to 0
X Extension:	Set according to result of subtracting product from 0

Description Multiply two XY data registers, `src-reg-1` and `src-reg-2`, negate the result and write it to an accumulator, `dst-reg`.

The multiplication treats `src-reg-1` as a signed value and `src-reg-2` as an unsigned value.

Execution Time 2 cycles, 1 pipelined (ALU operation)

Opcode



30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	1	0	0	0	1	0	src1		src2	1	1	dst		

MULNEG.UU ab-reg, xy-reg, xy-reg **Arith**

Multiply two XY data registers, both unsigned, negate the result and store it in an accumulator

Operands `MULNEG.UU dst-reg, src-reg-1, src-reg-2`

Operation dst-reg = 0 - src-reg-1 * src-reg-2

Flags If Flag Updates are enabled, the X ALU status flags are affected when the ALU operation completes (after the next clock cycle), as follows:

X Carry:	Not affected
X Negative:	Set according to result of subtracting product from 0
X Zero:	Set according to result of subtracting product from 0
X Overflow:	Set to 0
X Extension:	Set according to result of subtracting product from 0

Description Multiply two XY data registers, `src-reg-1` and `src-reg-2`, negate the result and write it to an accumulator, `dst-reg`.

The multiplication treats `src-reg-1` and `src-reg-2` as unsigned values.

Execution Time 2 cycles, 1 pipelined (ALU operation)

Opcode

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	1	0	0	0	0	1	src1		src2		1	1	dst	

MULSUB ab-reg, xy-reg, xy-reg **Arith**

Multiply two XY data registers, and subtract the result from an accumulator

Operands `MULSUB dst-reg, src-reg-1, src-reg-2`

Operation dst-reg = dst-reg - src-reg-1 * src-reg-2

Flags If Flag Updates are enabled, the X ALU status flags are affected when the ALU operation completes (after the next clock cycle), as follows:

X Carry:	Set according to result of subtracting product from `dst-reg`
X Negative:	Set according to result of subtracting product from `dst-reg`
X Zero:	Set according to result of subtracting product from `dst-reg`
X Overflow:	Set according to result of subtracting product from `dst-reg`
X Extension:	Set according to result of subtracting product from `dst-reg`

Description Multiply two XY data registers, `src-reg-1` and `src-reg-2`, subtract the result from an accumulator, `dst-reg`, and write the result back to `dst-reg`.

Execution Time 2 cycles, 1 pipelined (ALU operation)

Opcode

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	1	0	0	0	0	0	src1		src2	0	1	dst		

MULSUB a-reg, x-reg, xy-reg **X Arith**

Multiply two XY data registers, and subtract the result from an A accumulator

Operands `MULSUB dst-reg, src-reg-1, src-reg-2`

Operation dst-reg = dst-reg - src-reg-1 * src-reg-2

Flags If Flag Updates are enabled, the X ALU status flags are affected when the ALU operation completes (after the next clock cycle), as follows:

X Carry:	Set according to result of subtracting product from `dst-reg`
X Negative:	Set according to result of subtracting product from `dst-reg`
X Zero:	Set according to result of subtracting product from `dst-reg`
X Overflow:	Set according to result of subtracting product from `dst-reg`
X Extension:	Set according to result of subtracting product from `dst-reg`

Description Multiply two XY data registers, `src-reg-1` and `src-reg-2`, subtract the result from an A accumulator, `dst-reg`, and write the result back to `dst-reg`.

Execution Time 2 cycles, 1 pipelined (ALU operation)

Constraints When the Y side instruction is `MUL*`, the right-most source register on the X and Y sides must be opposite—one side using `x-reg` and the other side `y-reg`.

Opcode

Y side instruction is not MUL*

MULSUB a-reg, x-reg, x-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	0	0	0	s1	s2	0	1	dst	-	-	-	-	-	-

MULSUB a-reg, x-reg, y-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	0	0	1	s1	s2	0	1	dst	-	-	-	-	-	-

Y side instruction is MUL*

MULSUB a-reg, x-reg, x-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	1	1	0	1	dst	0	s1	s2	-	-	-	0	-	-

MULSUB a-reg, x-reg, y-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	1	1	0	1	dst	1	s1	s2	-	-	-	0	-	-

Bits marked with "-" are determined by the Y side arithmetic instruction.

MULSUB b-reg, y-reg, xy-reg **Y Arith**

Multiply two XY data registers, and subtract the result from a B accumulator

Operands `MULSUB dst-reg, src-reg-1, src-reg-2`

Operation dst-reg = dst-reg - src-reg-1 * src-reg-2

Flags If Flag Updates are enabled, the Y ALU status flags are affected when the ALU operation completes (after the next clock cycle), as follows:

Y Carry:	Set according to result of subtracting product from `dst-reg`
Y Negative:	Set according to result of subtracting product from `dst-reg`
Y Zero:	Set according to result of subtracting product from `dst-reg`
Y Overflow:	Set according to result of subtracting product from `dst-reg`
Y Extension:	Set according to result of subtracting product from `dst-reg`

Description Multiply two XY data registers, `src-reg-1` and `src-reg-2`, subtract the result from a B accumulator, `dst-reg`, and write the result back to `dst-reg`.

Execution Time 2 cycles, 1 pipelined (ALU operation)

Constraints When the X side instruction is `MUL*`, the right-most source register on the X and Y sides must be opposite—one side using `x-reg` and the other side `y-reg`.

Opcode

X side instruction is not MUL* or NOP

MULSUB b-reg, y-reg, x-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	0	1	-	-	-	-	-	-	1	s1	s2	0	1	dst

MULSUB b-reg, y-reg, y-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	0	1	-	-	-	-	-	-	0	s1	s2	0	1	dst

X side instruction is MUL* or NOP

MULSUB b-reg, y-reg, x-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	1	1	-	-	-	1	-	-	0	1	dst	-	s1	s2

MULSUB b-reg, y-reg, y-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	1	1	-	-	-	0	-	-	0	1	dst	-	s1	s2

Bits marked with "-" are determined by the X side arithmetic instruction.

MULSUB.SU ab-reg, xy-reg, xy-reg **Arith**

Multiply two XY data registers, one signed and one unsigned, and subtract the result from an accumulator

Operands `MULSUB.SU dst-reg, src-reg-1, src-reg-2`

Operation dst-reg = dst-reg - src-reg-1 * src-reg-2

Flags If Flag Updates are enabled, the X ALU status flags are affected when the ALU operation completes (after the next clock cycle), as follows:

X Carry:	Set according to result of subtracting product from `dst-reg`
X Negative:	Set according to result of subtracting product from `dst-reg`
X Zero:	Set according to result of subtracting product from `dst-reg`
X Overflow:	Set according to result of subtracting product from `dst-reg`
X Extension:	Set according to result of subtracting product from `dst-reg`

Description Multiply two XY data registers, `src-reg-1` and `src-reg-2`, subtract the result from an accumulator, `dst-reg`, and write the result back to `dst-reg`.

The multiplication treats `src-reg-1` as a signed value and `src-reg-2` as an unsigned value.

Execution Time 2 cycles, 1 pipelined (ALU operation)

Opcode

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	1	0	0	0	1	0	src1		src2	0	1	dst		

MULSUB.UU ab-reg, xy-reg, xy-reg **Arith**

Multiply two XY data registers, both unsigned, and subtract the result from an accumulator

Operands `MULSUB.UU dst-reg, src-reg-1, src-reg-2`

Operation dst-reg = dst-reg - src-reg-1 * src-reg-2

Flags If Flag Updates are enabled, the X ALU status flags are affected when the ALU operation completes (after the next clock cycle), as follows:

X Carry:	Set according to result of subtracting product from `dst-reg`
X Negative:	Set according to result of subtracting product from `dst-reg`
X Zero:	Set according to result of subtracting product from `dst-reg`
X Overflow:	Set according to result of subtracting product from `dst-reg`
X Extension:	Set according to result of subtracting product from `dst-reg`

Description Multiply two XY data registers, `src-reg-1` and `src-reg-2`, subtract the result from an accumulator, `dst-reg`, and write the result back to `dst-reg`.

The multiplication treats `src-reg-1` and `src-reg-2` as unsigned values.

Execution Time 2 cycles, 1 pipelined (ALU operation)

Opcode

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	1	0	0	0	0	1	src1		src2	0	1	dst		

NEG d-reg, d-reg **Arith**

Negate a data register, storing the result in a data register

Operands `NEG dst-reg, src-reg`

Operation dst-reg = -src-reg

Flags If Flag Updates are enabled, the X ALU status flags are affected as follows:

X Carry:	Not affected
X Negative:	Set according to ALU output
X Zero:	Set according to ALU output
X Overflow:	Set according to ALU output
X Extension:	Set according to ALU output

Description Negate a data register, `src-reg`, and store the result in a data register, `dst-reg`.

Execution Time 1 cycle

Opcode

30	29	28	27	26	25	24	23 22 21	20	19	18	17 16 15
0	1	1	1	0	0	0	dst-reg	0	0	0	src-reg

NEG a-reg, ab-reg **X Arith**

Negate an accumulator, storing the result in an A accumulator

Operands `NEG dst-reg, src-reg`

Operation dst-reg = -src-reg

Flags If Flag Updates are enabled, the X ALU status flags are affected as follows:

X Carry:	Not affected
X Negative:	Set according to ALU output
X Zero:	Set according to ALU output
X Overflow:	Set according to ALU output
X Extension:	Set according to ALU output

Description Negate an accumulator, `src-reg`, and store the result in an A accumulator, `dst-reg`.

Execution Time 1 cycle

Constraints The Y side instruction must not be `MUL*`.

The right-most source register on the X and Y sides must be opposite—one side using `a-reg` and the other side `b-reg`.

Opcode

NEG a-reg, a-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	1	0	1	1	0	dst	0	0	src	-	-	-	0	-	-

NEG a-reg, b-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	1	0	1	1	0	dst	1	0	src	-	-	-	0	-	-

Bits marked with "-" are determined by the Y side arithmetic instruction.

NEG d-reg, d-reg, cond3 **Arith**

Negate a data register, storing the result in a data register, conditionally

Operands `NEG dst-reg, src-reg, condition`

Operation if condition holds, dst-reg = -src-reg

Flags Not affected

Description If the condition holds, negate a data register, `src-reg`, and store the result in a data register, `dst-reg`.

Execution Time 1 cycle

Remarks Version 3 or greater of the CFX assembler supports the following alternate form for this conditional instruction:

```
NEG d-reg, d-reg if cond3
```

Opcode

30	29	28	27	26	25	24	23 22 21	20 19 18 17	16 15
0	1	1	1	1	1	1	dst-reg	cond	src-reg

NLOG2ABS d-reg, d-reg **Arith**

Calculate the logarithm base 2 of the absolute value of a data register, negate the result, and store the result in a data register

Operands `NLOG2ABS dst-reg, src-reg`

Operation dst-reg = -log2(|src-reg|)

Flags If Flag Updates are enabled, the X ALU status flags are affected as follows:

X Carry:	Not affected
X Negative:	Set according to ALU output
X Zero:	Set according to ALU output
X Overflow:	Set to 0
X Extension:	Not affected

Description Calculate the logarithm base 2 of the absolute value of a data register, `src-reg`, negate the result and store it in a data register, `dst-reg`.

Execution Time 1 cycle

Opcode

30	29	28	27	26	25	24	23 22 21	20	19	18	17 16 15
0	1	1	1	0	0	1	dst-reg	0	0	0	src-reg

NLOG2ABS a-reg, ab-reg **X Arith**

Calculate the logarithm base 2 of the absolute value of an accumulator, negate the result, and store the result in an A accumulator

Operands `NLOG2ABS dst-reg, src-reg`

Operation dst-reg = -log2(|src-reg|)

Flags If Flag Updates are enabled, the X ALU status flags are affected as follows:

X Carry:	Not affected
X Negative:	Set according to ALU output
X Zero:	Set according to ALU output
X Overflow:	Set to 0
X Extension:	Not affected

Description Calculate the logarithm base 2 of the absolute value of an accumulator, `src-reg`, negate the result and store it in an A accumulator, `dst-reg`.

Execution Time 1 cycle

Constraints The Y side instruction must not be `MUL*`.

The right-most source register on the X and Y sides must be opposite—one side using `a-reg` and the other side `b-reg`.

Opcode

NLOG2ABS a-reg, a-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	1	0	1	1	0	dst	0	0	src	-	-	-	1	-	-

NLOG2ABS a-reg, b-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	1	0	1	1	0	dst	1	0	src	-	-	-	1	-	-

Bits marked with "-" are determined by the Y side arithmetic instruction.

NOP

Long

No operation

Operands NOP

Operation no operation

Flags Not affected

Description Does nothing.

Execution Time 1 cycle

Opcode

29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

NOP

Move

No operation

Operands NOP

Operation no operation

Flags Not affected

Description Does nothing.

Execution Time 1 cycle

Opcode

14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	0	1	0	1	0	1	1	0	0

If this instruction is the second move instruction in a Move-Move instruction word, the encoding is shifted left 15 bits.

NOP

Arith

No operation

Operands NOP

Operation no operation

Flags Not affected

Description Does nothing.

Execution Time 1 cycle

Opcode

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
0	1	1	0	1	0	1	0	0	0	0	0	0	0	0	0

NOP

X Arith

No operation

Operands NOP

Operation no operation

Flags Not affected

Description Does nothing.

Execution Time 1 cycle

Constraints The Y side instruction must be MUL*.

Opcode

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	1	1	0	0	0	-	0	0	-	-	-	1	-	-

Bits marked with "-" are determined by the Y side arithmetic instruction.

NOT d-reg, d-reg **Arith**

Bitwise NOT a data register, storing the result in a data register

Operands `NOT dst-reg, src-reg`

Operation dst-reg = NOT src-reg

Flags If Flag Updates are enabled, the X ALU status flags are affected as follows:

X Carry:	Not affected
X Negative:	Set according to ALU output
X Zero:	Set according to ALU output
X Overflow:	Set to 0
X Extension:	Set according to ALU output

Description Bitwise NOT a data register, `src-reg`, and store the result in a data register, `dst-reg`.

Execution Time 1 cycle

Remarks This instruction is a synonym for "`XOR.DW dst-reg, src-reg, -1`".

This instruction requires version 3 or greater of the CFX assembler.

Opcode

See XOR.DW d-reg, d-reg, imm3s.

OR d-reg, d-reg, d-reg **Arith**

Bitwise OR two data registers, storing the result in a data register

Operands `OR dst-reg, src-reg-1, src-reg-2`

Operation dst-reg = src-reg-1 OR src-reg-2

Flags If Flag Updates are enabled, the X ALU status flags are affected as follows:

X Carry:	Not affected
X Negative:	Set according to ALU output
X Zero:	Set according to ALU output
X Overflow:	Set to 0
X Extension:	Set according to ALU output

Description Bitwise OR two data registers, `src-reg-1` and `src-reg-2`, and store the result in a data register, `dst-reg`.

Execution Time 1 cycle

Opcode

30	29	28	27	26	25	24	23 22 21	20 19 18	17 16 15
0	1	0	1	0	0	1	dst-reg	src-reg-1	src-reg-2

OR a-reg, a-reg, ab-reg **X Arith**

Bitwise OR an A accumulator and an accumulator, storing the result in an A accumulator

Operands `OR dst-reg, src-reg-1, src-reg-2`

Operation dst-reg = src-reg-1 OR src-reg-2

Flags If Flag Updates are enabled, the X ALU status flags are affected as follows:

X Carry:	Not affected
X Negative:	Set according to ALU output
X Zero:	Set according to ALU output
X Overflow:	Set to 0
X Extension:	Set according to ALU output

Description Bitwise OR an A accumulator, `src-reg-1`, and an A or B accumulator, `src-reg-2`, and store the result in an A accumulator, `dst-reg`.

Execution Time 1 cycle

Constraints The Y side instruction must not be `MUL*`.

The right-most source register on the X and Y sides must be opposite—one side using `a-reg` and the other side `b-reg`.

Opcode

OR a-reg, a-reg, a-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	1	0	1	0	0	dst	0	s1	s2	-	-	-	1	-	-

OR a-reg, a-reg, b-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	1	0	1	0	0	dst	1	s1	s2	-	-	-	1	-	-

Bits marked with "-" are determined by the Y side arithmetic instruction.

OR d-reg, d-reg, imm3s **Arith**

Bitwise OR a data register and a value, storing the result in a data register

Operands `OR dst-reg, src-reg, data`

Operation dst-reg = src-reg OR signext(data)

Flags If Flag Updates are enabled, the X ALU status flags are affected as follows:

X Carry:	Not affected
X Negative:	Set according to ALU output
X Zero:	Set according to ALU output
X Overflow:	Set to 0
X Extension:	Set according to ALU output

Description Bitwise OR a data register, `src-reg`, and a 3-bit signed immediate value, `data` (-4..3), and store the result in a data register, `dst-reg`.

Execution Time 1 cycle

Opcode

30	29	28	27	26	25	24	23 22 21	20 19 18	17 16 15
0	0	1	1	0	0	1	dst-reg	src-reg	data

OR.DW d-reg, d-reg, imm3s **Arith**

Bitwise OR a data register and a double-width value, storing the result in a data register

Operands `OR.DW dst-reg, src-reg, data`

Operation dst-reg = src-reg OR (signext(data) >> 24)

Flags If Flag Updates are enabled, the X ALU status flags are affected as follows:

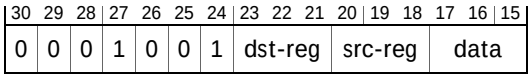
X Carry:	Not affected
X Negative:	Set according to ALU output
X Zero:	Set according to ALU output
X Overflow:	Set to 0
X Extension:	Set according to ALU output

Description Bitwise OR a data register, `src-reg`, and a 3-bit signed immediate value, `data` (-4..3), interpreted as a double-width value (aligned with the low register of an accumulator, or the 24-bit word below an XY data register), and store the result in a data register, `dst-reg`.

Execution Time 1 cycle

Remarks Double-width logic is not useful when `dst-reg` is an XY data register.

Opcode



30	29	28	27	26	25	24	23 22 21 20	19 18 17 16	15
0	0	0	1	0	0	1	dst-reg	src-reg	data

RETURN **Synth**

Return from subroutine without delay slots

Operands `RETURN`

Operation PC = LR

Flags Not affected

Description Return from a subroutine, to the return address found in the Link Register (`LR`). The delay slots are filled with a `NOP` instruction.

Execution Time 3 cycles

Remarks This instruction is synthesized by the assembler as follows:

```
RETURN.2D
NOP >>> NOP
```

Opcode

See RETURN.2D and NOP.

RETURN.2D

Long

Return from subroutine with two delay slots

Operands `RETURN.2D`

Operation PC = LR

Flags Not affected

Description Return from a subroutine, to the return address found in the Link Register (`LR`).

This instruction has two delay slots. One or two instruction words immediately following this instruction in memory are executed before returning from the subroutine.

The delay slots can be filled in one of two ways:

- One *Move-Move* instruction that executes in exactly 2 cycles.
- Two *Arithmetic-Move* or non-control-flow single-cycle *Long* instructions.

See Section 7.5.5, “Program Flow Delay Slot Restrictions” on page 62 for more information.

Execution Time 3 cycles, 2 pipelined (delay slots)

Opcode

29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	0	1	1	0	1	1	0	0	0	0	0	0	0	0	0	1	1	0	0	0	0	0	0	0	0

RETURNI **Long**

Return from interrupt without delay slots

Operands `RETURNI`

Operation SR = ILSR

PC = ILPC

Flags Not affected

Description Return from an interrupt, restoring the status register (`SR`) from the interrupt-link status register (`ILSR`), and the program counter (`PC`) from the interrupt-link program counter (`ILPC`). This instruction has no delay slots.

Execution Time 3 cycles

Remarks This instruction is a synonym for:

```
RETURNI.0D
```

Opcode

See RETURNI.0D.

RETURNI.0D

Long

Return from interrupt without delay slots

Operands `RETURNI.0D`

Operation SR = ILSR

PC = ILPC

Flags Not affected

Description Return from an interrupt, restoring the status register (`SR`) from the interrupt-link status register (`ILSR`), and the program counter (`PC`) from the interrupt-link program counter (`ILPC`). This instruction has no delay slots.

Execution Time 3 cycles

Opcode

29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	0	1	1	0	1	1	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0

RETURNI.2D **Long**

Return from interrupt with two delay slots

Operands `RETURNI.2D`

Operation SR = ILSR

PC = ILPC

Flags Not affected

Description Return from an interrupt, restoring the status register (`SR`) from the interrupt-link status register (`ILSR`), and the program counter (`PC`) from the interrupt-link program counter (`ILPC`).

This instruction has two delay slots. One or two instruction words immediately following this instruction in memory are executed before returning from the interrupt.

The delay slots can be filled in one of two ways:

- One *Move-Move* instruction that executes in exactly 2 cycles.
- Two *Arithmetic-Move* or non-control-flow single-cycle *Long* instructions.

See Section 7.7.3, “RETURNI.2D Delay Slot Restrictions” on page 73 for more information.

The PCU internally disables the Flag Updates mode during the delay slots of `RETURNI.2D`, regardless of the setting of the mode bit in the status register (`SR`).

Execution Time 3 cycles, 2 pipelined (delay slots)

Opcode

29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	0	1	1	0	1	1	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0

SHLL d-reg, d-reg, d-reg **Arith**

Shift a data register left logically, by a count from a data register, storing the result in a data register

Operands `SHLL dst-reg, src-reg-1, src-reg-2`

Operation if src-reg-2 ≥ 0, dst-reg = src-reg-1 << src-reg-2

if src-reg-2 < 0, dst-reg = src-reg-1 >> |src-reg-2|

Flags If Flag Updates are enabled, the X ALU status flags are affected as follows:

X Carry:	Set to the last bit shifted out, or to 0 if the count is zero
X Negative:	Set according to ALU output
X Zero:	Set according to ALU output
X Overflow:	If the shift count is positive, set according to whether the sign bit changes during the shift operation; otherwise, set to 0
X Extension:	Set according to ALU output

Description Shift a data register, `src-reg-1`, left by a signed shift count specified in the low 7 bits of a data register, `src-reg-2`, and store the result in a data register, `dst-reg`.

If the shift count is positive, this instruction performs a logical shift left where 0 is shifted into the low-order bits.

If the shift count is negative, this instruction performs an arithmetic shift right where the high-order bits are filled with the sign bit.

If `src-reg-2` is an accumulator, the shift count is bits 0-6 of the high part of the accumulator. Otherwise, the shift count is bits 0-6 of `src-reg-2`.

Execution Time 1 cycle

Constraints The shift count must be in the range -56 to 56.

Opcode

30	29	28	27	26	25	24	23 22 21 20	19 18 17 16 15
0	1	0	0	1	0	0	dst-reg	src-reg-1	src-reg-2

SHLL d-reg, d-reg, imm3u **Arith**

Shift a data register left logically, by an immediate count, storing the result in a data register

Operands `SHLL dst-reg, src-reg, data`

Operation dst-reg = src-reg << data

Flags If Flag Updates are enabled, the X ALU status flags are affected as follows:

X Carry:	Set to the last bit shifted out, or to 0 if the count is zero
X Negative:	Set according to ALU output
X Zero:	Set according to ALU output
X Overflow:	Set according to whether the sign bit changes during the shift operation
X Extension:	Set according to ALU output

Description Shift a data register, `src-reg`, left by a shift count specified by a 3-bit unsigned immediate value, `data`, and store the result in a data register, `dst-reg`.

This instruction performs a logical shift left where 0 is shifted into the low-order bits.

Execution Time 1 cycle

Opcode

30	29	28	27	26	25	24	23 22 21	20 19 18	17 16 15
0	0	1	0	1	0	0	dst-reg	src-reg	data

SHLL d-reg, imm6u

Arith

Shift a data register left logically, by an immediate count

Operands `SHLL dst-reg, data`

Operation dst-reg = dst-reg << data

Flags If Flag Updates are enabled, the X ALU status flags are affected as follows:

X Carry:	Set to the last bit shifted out, or to 0 if the count is zero
X Negative:	Set according to ALU output
X Zero:	Set according to ALU output
X Overflow:	Set according to whether the sign bit changes during the shift operation
X Extension:	Set according to ALU output

Description Shift a data register, `dst-reg`, left by a shift count specified by a 6-bit unsigned immediate value, `data`, and store the result back in `dst-reg`.

This instruction performs a logical shift left where 0 is shifted into the low-order bits.

Execution Time 1 cycle

Constraints The shift count must be in the range 0 to 56.

Opcode

30	29	28	27	26	25	24	23 22 21 20	19 18 17 16 15
0	0	0	0	1	0	0	dst-reg	data

SHLL d-reg, d-reg, cond3 **Arith**

Shift a data register left logically, by a count from a data register, conditionally

Operands `SHLL dst-reg, src-reg, condition`

Operation if condition holds,

if src-reg ≥ 0, dst-reg = dst-reg << src-reg

if src-reg < 0, dst-reg = dst-reg >> |src-reg|

Flags Not affected

Description If the condition holds, shift a data register, `dst-reg`, left by a signed shift count specified in the low 7 bits of a data register, `src-reg`, and store the result back in `dst-reg`.

If the shift count is positive, this instruction performs a logical shift left where 0 is shifted into the low-order bits.

If the shift count is negative, this instruction performs an arithmetic shift right where the high-order bits are filled with the sign bit.

If `src-reg` is an accumulator, the shift count is bits 0-6 of the high part of the accumulator. Otherwise, the shift count is bits 0-6 of `src-reg`.

Execution Time 1 cycle

Constraints The shift count must be in the range -56 to 56.

Remarks Version 3 or greater of the CFX assembler supports the following alternate form for this conditional instruction:

```
SHLL d-reg, d-reg if cond3
```

Opcode

30	29	28	27	26	25	24	23 22 21	20 19 18	17 16 15
0	0	0	1	1	0	0	dst-reg	cond	src-reg

SHLL d-reg, imm3u, cond3 **Arith**

Shift a data register left logically, by an immediate count, conditionally

Operands `SHLL dst-reg, data, condition`

Operation if condition holds, dst-reg = dst-reg << data

Flags Not affected

Description If the condition holds, shift a data register, `dst-reg`, left by a shift count specified by a 3-bit unsigned immediate value, `data`, and store the result back in `dst-reg`.

This instruction performs a logical shift left where 0 is shifted into the low-order bits.

Execution Time 1 cycle

Remarks Version 3 or greater of the CFX assembler supports the following alternate form for this conditional instruction:

```
SHLL d-reg, imm3u if cond3
```

Opcode

30	29	28	27	26	25	24	23 22 21	20 19 18	17 16 15
0	0	0	1	1	0	1	dst-reg	cond	data

SHLL.NX d-reg, d-reg, d-reg **Arith**

Shift a data register left logically, with no extension, by a count from a data register, storing the result in a data register

Operands `SHLL.NX dst-reg, src-reg-1, src-reg-2`

Operation if src-reg-2 ≥ 0, dst-reg = signext(src-reg-1 << src-reg-2)

if src-reg-2 < 0, dst-reg = signext(src-reg-1 >> |src-reg-2|)

Flags If Flag Updates are enabled, the X ALU status flags are affected as follows:

X Carry:	Not affected
X Negative:	Set according to ALU output
X Zero:	Set according to ALU output
X Overflow:	Set to 0
X Extension:	Not affected

Description Shift a data register, `src-reg-1`, left by a signed shift count specified in the low 7 bits of a data register, `src-reg-2`, and store the result in a data register, `dst-reg`.

If `src-reg-1` is an accumulator, the extension (most significant 8 bits) are treated as 0 before performing the shift.

Before the result is written to the destination register, the upper 8 bits are overwritten with the sign extension from bit 47 of the result.

If the shift count is positive, this instruction performs a logical shift left where 0 is shifted into the low-order bits.

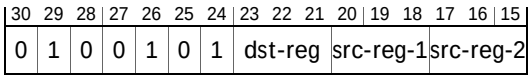
If the shift count is negative, this instruction performs a logical shift right where 0 is shifted into the high-order bits.

If `src-reg-2` is an accumulator, the shift count is bits 0-6 of the high part of the accumulator. Otherwise, the shift count is bits 0-6 of `src-reg-2`.

Execution Time 1 cycle

Constraints The shift count must be in the range -56 to 56.

Opcode



30	29	28	27	26	25	24	23 22 21 20	19 18 17 16	15
0	1	0	0	1	0	1	dst-reg	src-reg-1	src-reg-2

SHLL.NX d-reg, d-reg, imm3u **Arith**

Shift a data register left logically, with no extension, by an immediate count, storing the result in a data register

Operands `SHLL.NX dst-reg, src-reg, data`

Operation dst-reg = signext(src-reg << data)

Flags If Flag Updates are enabled, the X ALU status flags are affected as follows:

X Carry:	Not affected
X Negative:	Set according to ALU output
X Zero:	Set according to ALU output
X Overflow:	Set to 0
X Extension:	Not affected

Description Shift a data register, `src-reg`, left by a shift count specified by a 3-bit unsigned immediate value, `data`, and store the result in a data register, `dst-reg`.

Before the result is written to the destination register, the upper 8 bits are overwritten with the sign extension from bit 47 of the result.

This instruction performs a logical shift left where 0 is shifted into the low-order bits.

Execution Time 1 cycle

Opcode

30	29	28	27	26	25	24	23 22 21	20 19 18	17 16 15
0	0	1	0	1	0	1	dst-reg	src-reg	data

SHLL.NX d-reg, imm6u **Arith**

Shift a data register left logically, with no extension, by an immediate count

Operands `SHLL.NX dst-reg, data`

Operation dst-reg = signext(dst-reg << data)

Flags If Flag Updates are enabled, the X ALU status flags are affected as follows:

X Carry:	Not affected
X Negative:	Set according to ALU output
X Zero:	Set according to ALU output
X Overflow:	Set to 0
X Extension:	Not affected

Description Shift a data register, `dst-reg`, left by a shift count specified by a 6-bit unsigned immediate value, `data`, and store the result back in `dst-reg`.

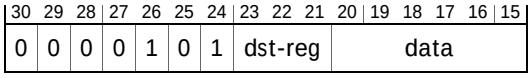
Before the result is written to the destination register, the upper 8 bits are overwritten with the sign extension from bit 47 of the result.

This instruction performs a logical shift left where 0 is shifted into the low-order bits.

Execution Time 1 cycle

Constraints The shift count must be in the range 0 to 56.

Opcode



30	29	28	27	26	25	24	23 22 21 20	19 18 17 16 15
0	0	0	0	1	0	1	dst-reg	data

SHRA d-reg, d-reg, d-reg **Arith**

Shift a data register right arithmetically, by a count from a data register, storing the result in a data register

Operands

```
SHRA dst-reg, src-reg-1, src-reg-2
```

Operation

if src-reg-2 ≥ 0, dst-reg = src-reg-1 >> src-reg-2

if src-reg-2 < 0, dst-reg = src-reg-1 << |src-reg-2|

Flags

If Flag Updates are enabled, the X ALU status flags are affected as follows:

X Carry:	Set to the last bit shifted out, or to 0 if the count is zero
X Negative:	Set according to ALU output
X Zero:	Set according to ALU output
X Overflow:	If the shift count is negative, set according to whether the sign bit changes during the shift operation; otherwise, set to 0
X Extension:	Set according to ALU output

Description

Shift a data register, `src-reg-1`, right by a signed shift count specified in the low 7 bits of a data register, `src-reg-2`, and store the result in a data register, `dst-reg`.

If the shift count is positive, this instruction performs an arithmetic shift right where the high-order bits are filled with the sign bit.

If the shift count is negative, this instruction performs a logical shift left where 0 is shifted into the low-order bits.

If `src-reg-2` is an accumulator, the shift count is bits 0-6 of the high part of the accumulator. Otherwise, the shift count is bits 0-6 of `src-reg-2`.

Execution Time

1 cycle

Constraints

The shift count must be in the range -56 to 56.

Opcode

30	29	28	27	26	25	24	23 22 21	20 19 18	17 16 15
0	1	0	0	1	1	0	dst-reg	src-reg-1	src-reg-2

SHRA d-reg, d-reg, imm3u **Arith**

Shift a data register right arithmetically, by an immediate count, storing the result in a data register

Operands `SHRA dst-reg, src-reg, data`

Operation dst-reg = src-reg >> data

Flags If Flag Updates are enabled, the X ALU status flags are affected as follows:

X Carry:	Set to the last bit shifted out, or to 0 if the count is zero
X Negative:	Set according to ALU output
X Zero:	Set according to ALU output
X Overflow:	Set to 0
X Extension:	Set according to ALU output

Description Shift a data register, `src-reg`, right by a shift count specified by a 3-bit unsigned immediate value, `data`, and store the result in a data register, `dst-reg`.

This instruction performs an arithmetic shift right where the high-order bits are filled with the sign bit.

Execution Time 1 cycle

Opcode

30	29	28	27	26	25	24	23 22 21	20 19 18	17 16 15
0	0	1	0	1	1	0	dst-reg	src-reg	data

SHRA d-reg, imm6u **Arith**

Shift a data register right arithmetically, by an immediate count

Operands `SHRA dst-reg, data`

Operation dst-reg = dst-reg >> data

Flags If Flag Updates are enabled, the X ALU status flags are affected as follows:

X Carry:	Set to the last bit shifted out, or to 0 if the count is zero
X Negative:	Set according to ALU output
X Zero:	Set according to ALU output
X Overflow:	Set to 0
X Extension:	Set according to ALU output

Description Shift a data register, `dst-reg`, right by a shift count specified by a 6-bit unsigned immediate value, `data`, and store the result back in `dst-reg`.

This instruction performs an arithmetic shift right where the high-order bits are filled with the sign bit.

Execution Time 1 cycle

Constraints The shift count must be in the range 0 to 56.

Opcode

30	29	28	27	26	25	24	23 22 21	20 19 18 17 16 15
0	0	0	0	1	1	0	dst-reg	data

SHRA d-reg, d-reg, cond3 **Arith**

Shift a data register right arithmetically, by a count from a data register, conditionally

Operands `SHRA dst-reg, src-reg, condition`

Operation if condition holds,

if src-reg ≥ 0, dst-reg = dst-reg >> src-reg

if src-reg < 0, dst-reg = dst-reg << |src-reg|

Flags Not affected

Description If the condition holds, shift a data register, `dst-reg`, right by a signed shift count specified in the low 7 bits of a data register, `src-reg`, and store the result back in `dst-reg`.

If the shift count is positive, this instruction performs an arithmetic shift right where the high-order bits are filled with the sign bit.

If the shift count is negative, this instruction performs a logical shift left where 0 is shifted into the low-order bits.

If `src-reg` is an accumulator, the shift count is bits 0-6 of the high part of the accumulator. Otherwise, the shift count is bits 0-6 of `src-reg`.

Execution Time 1 cycle

Constraints The shift count must be in the range -56 to 56.

Remarks Version 3 or greater of the CFX assembler supports the following alternate form for this conditional instruction:

```
SHRA d-reg, d-reg if cond3
```

Opcode

30	29	28	27	26	25	24	23 22 21	20 19 18 17	16 15
0	0	0	1	1	1	0	dst-reg	cond	src-reg

SHRA d-reg, imm3u, cond3 **Arith**

Shift a data register right arithmetically, by an immediate count, conditionally

Operands `SHRA dst-reg, data, condition`

Operation if condition holds, dst-reg = dst-reg >> data

Flags Not affected

Description If the condition holds, shift a data register, `dst-reg`, right by a shift count specified by a 3-bit unsigned immediate value, `data`, and store the result back in `dst-reg`.

This instruction performs an arithmetic shift right where the high-order bits are filled with the sign bit.

Execution Time 1 cycle

Remarks Version 3 or greater of the CFX assembler supports the following alternate form for this conditional instruction:

```
SHRA d-reg, imm3u if cond3
```

Opcode

30	29	28	27	26	25	24	23 22 21	20 19 18	17 16 15
0	0	0	1	1	1	1	dst-reg	cond	data

SHRL.NX d-reg, d-reg, d-reg **Arith**

Shift a data register right logically, with no extension, by a count from a data register, storing the result in a data register

Operands `SHRL.NX dst-reg, src-reg-1, src-reg-2`

Operation if src-reg-2 ≥ 0, dst-reg = signext(src-reg-1 >> src-reg-2)

if src-reg-2 < 0, dst-reg = signext(src-reg-1 << |src-reg-2|)

Flags If Flag Updates are enabled, the X ALU status flags are affected as follows:

X Carry:	Not affected
X Negative:	Set according to ALU output
X Zero:	Set according to ALU output
X Overflow:	Set to 0
X Extension:	Not affected

Description Shift a data register, `src-reg-1`, right by a signed shift count specified in the low 7 bits of a data register, `src-reg-2`, and store the result in a data register, `dst-reg`.

If `src-reg-1` is an accumulator, the extension (most significant 8 bits) are treated as 0 before performing the shift.

Before the result is written to the destination register, the upper 8 bits are overwritten with the sign extension from bit 47 of the result.

If the shift count is positive, this instruction performs a logical shift right where 0 is shifted into the high-order bits.

If the shift count is negative, this instruction performs a logical shift left where 0 is shifted into the low-order bits.

Execution Time 1 cycle

Constraints The shift count must be in the range -56 to 56.

Opcode

30	29	28	27	26	25	24	23 22 21	20 19 18 17 16	15
0	1	0	0	1	1	1	dst-reg	src-reg-1	src-reg-2

SHRL.NX d-reg, d-reg, imm3u **Arith**

Shift a data register right logically, with no extension, by an immediate count, storing the result in a data register

Operands `SHRL.NX dst-reg, src-reg, data`

Operation dst-reg = signext(src-reg >> data)

Flags If Flag Updates are enabled, the X ALU status flags are affected as follows:

X Carry:	Not affected
X Negative:	Set according to ALU output
X Zero:	Set according to ALU output
X Overflow:	Set to 0
X Extension:	Not affected

Description Shift a data register, `src-reg`, right by a shift count specified by a 3-bit unsigned immediate value, `data`, and store the result in a data register, `dst-reg`.

If `src-reg` is an accumulator, the extension (most significant 8 bits) are treated as 0 before performing the shift.

Before the result is written to the destination register, the upper 8 bits are overwritten with the sign extension from bit 47 of the result.

This instruction performs a logical shift right where 0 is shifted into the high-order bits.

Execution Time 1 cycle

Opcode

30	29	28	27	26	25	24	23 22 21	20 19 18	17 16 15
0	0	1	0	1	1	1	dst-reg	src-reg	data

SHRL.NX d-reg, imm6u **Arith**

Shift a data register right logically, with no extension, by an immediate count

Operands `SHRL.NX dst-reg, data`

Operation dst-reg = signext(dst-reg >> data)

Flags If Flag Updates are enabled, the X ALU status flags are affected as follows:

X Carry:	Not affected
X Negative:	Set according to ALU output
X Zero:	Set according to ALU output
X Overflow:	Set to 0
X Extension:	Not affected

Description Shift a data register, `dst-reg`, right by a shift count specified by a 6-bit unsigned immediate value, `data`, and store the result back in `dst-reg`.

If `dst-reg` is an accumulator, the extension (most significant 8 bits) are treated as 0 before performing the shift.

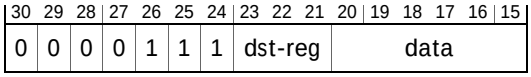
Before the result is written to the destination register, the upper 8 bits are overwritten with the sign extension from bit 47 of the result.

This instruction performs a logical shift right where 0 is shifted into the high-order bits.

Execution Time 1 cycle

Constraints The shift count must be in the range 0 to 56.

Opcode



30	29	28	27	26	25	24	23 22 21	20 19 18 17 16 15
0	0	0	0	1	1	1	dst-reg	data

SLEEP

Long

Sleep

Operands SLEEP

Operation enter Sleep mode

wait for enabled interrupt

Flags Not affected

Description The CFX enters Sleep mode. Sleep mode is exited (and this instruction completes) when an enabled interrupt is received.

Execution Time 3 cycles

Opcode

29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

SLEEP imm16u

Long

Sleep with restart location

Operands `SLEEP restart-addr`

Operation enter Sleep mode

wait for enabled interrupt

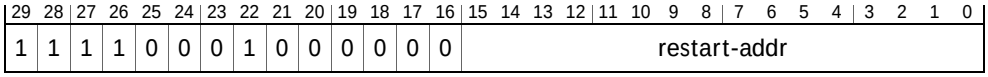
PC = restart-addr

Flags Not affected

Description The CFX enters Sleep mode. Sleep mode is exited when an enabled interrupt is received. The program counter (`PC`) is loaded with `restart-addr` and execution continues at that address.

Execution Time 3 cycles

Opcode



29	28	27	26	25	24	23	22	21	20	19	18	17	16	15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0
1	1	1	1	0	0	0	1	0	0	0	0	0	0	restart-addr

STORE X[stack-mode16], reg

Long

Store register to stack

Operands `STORE X[ addr ], register`

Operation X[addr] = register

Flags If saturation is enabled and `register` is an accumulator, these status flags are updated:

Saturated: Set if saturation occurs

Description Store a value from a register, `register`, to the stack, at a 16-bit offset from the stack pointer, `SP`.

If `register` is a 56-bit accumulator, the value from `register` undergoes rounding and saturation if they are enabled.

Execution Time 1 cycle

Opcode

29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	0	register						1	addr															

STORE X[stack-mode6], stack-reg6

Move

Store register to stack

Operands `STORE X[ addr ], register`

Operation X[addr] = register

Flags If saturation is enabled and `register` is an accumulator, these status flags are updated:

Saturated: Set if saturation occurs

Description Store a value from a register, `register`, to the stack, at a 6-bit negative offset (-64..-1) from the stack pointer, `SP`.

If `register` is a 56-bit accumulator, the value from `register` undergoes rounding and saturation if they are enabled.

Execution Time 1 cycle

Opcode

14	13	12	11 10 9 8	7	6 5 4 3 2 1 0
1	0	1	register	1	addr

If this instruction is the second move instruction in a Move-Move instruction word, the encoding is shifted left 15 bits.

STORE X[stack-mode4], stack-reg4 **Move**

Store register to stack

Operands `STORE X[ addr ], register`

Operation X[addr] = register

Flags Not affected

Description Store a value from a register, `register`, to the stack, at a 4-bit negative offset (-16..-1) from the stack pointer, `SP`.

Execution Time 1 cycle

Opcode

14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	register			1	addr			

If this instruction is the second move instruction in a Move-Move instruction word, the encoding is shifted left 15 bits.

STORE X[imm16u], reg **Long**

Store register direct to X memory

Operands `STORE X[ addr ], register`

Operation X[addr] = register

Flags If saturation is enabled and `register` is an accumulator, these status flags are updated:

Saturated: Set if saturation occurs

Description Store a value from a register, `register`, to a 16-bit address, `addr`, in X memory.

If `register` is a 56-bit accumulator, the value from `register` undergoes rounding and saturation if they are enabled.

Execution Time 1 cycle

Opcode

29	28	27	26	25	24	23	22 21 20	19 18 17 16	15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0
1	1	1	0	0	0	0	register	1	addr

STORE X[single-mode(X)], xmem-reg

Move

Store register indirect to X memory

Operands `STORE X[ addr ], register`

Operation X[addr] = register

update addr

Flags If saturation is enabled and `register` is an accumulator, these status flags are updated:

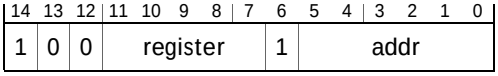
Saturated: Set if saturation occurs

Description Store a value from a register, `register`, to X memory. The address, `addr`, specifies an X pointer and an optional update. The X pointer is updated after the value is stored.

If `register` is a 56-bit accumulator, the value from `register` undergoes rounding and saturation if they are enabled.

Execution Time 1 cycle

Opcode



If this instruction is the second move instruction in a Move-Move instruction word, the encoding is shifted left 15 bits.

STORE X[dual-mode(X)], a-reg

X Move

Store A accumulator indirect to X memory

Operands

```
STORE X[ addr ], accumulator
```

Operation

X[addr] = accumulator

update addr

Flags

If saturation is enabled, these status flags are updated:

Saturated: Set if saturation occurs

Description

Store a value from an A accumulator, `accumulator`, to X memory. The address, `addr`, specifies an X pointer and an optional update. The X pointer is updated after the value is stored.

If rounding and saturation are enabled, they are performed on the value from `accumulator`.

Execution Time

1 cycle

Constraints

Only the first four X pointers, `XP0` to `XP3`, can be used in dual-mode(X).

Opcode

14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	ac	addr			-	-	-	-	-	-	-	-

If this instruction is the second move instruction in a Move-Move instruction word, the encoding is shifted left 15 bits.

Bits marked with "-" are determined by the Y side move instruction.

STORE Y[imm16u], reg **Long**

Store register direct to Y memory

Operands `STORE Y[ addr ], register`

Operation Y[addr] = register

Flags If saturation is enabled and `register` is an accumulator, these status flags are updated:

Saturated: Set if saturation occurs

Description Store a value from a register, `register`, to a 16-bit address, `addr`, in Y memory.

If `register` is a 56-bit accumulator, the value from `register` undergoes rounding and saturation if they are enabled.

Execution Time 1 cycle

Opcode

29	28	27	26	25	24	23	22 21 20	19 18 17 16	15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0
1	1	1	0	0	0	1	register	1	addr

STORE Y[single-mode(Y)], by-reg

Move

Store B or Y data register indirect to Y memory

Operands `STORE Y[ addr ], register`

Operation Y[addr] = register

update addr

Flags If saturation is enabled and `register` is an accumulator, these status flags are updated:

Saturated: Set if saturation occurs

Description Store a value from a B accumulator or Y data register, `register` to Y memory. The address, `addr`, specifies a Y pointer and an optional update. The Y pointer is updated after the value is stored.

If `register` is a 56-bit accumulator, the value from `register` undergoes rounding and saturation if they are enabled.

Execution Time 1 cycle

Opcode

14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	0	1	0	reg		1	addr				

If this instruction is the second move instruction in a Move-Move instruction word, the encoding is shifted left 15 bits.

STORE Y[dual-mode(Y)], b-reg

Y Move

Store B accumulator indirect to Y memory

Operands

```
STORE Y[ addr ], accumulator
```

Operation

Y[addr] = accumulator

update addr

Flags

If saturation is enabled, these status flags are updated:

Saturated: Set if saturation occurs

Description

Store a value from a B accumulator, `accumulator`, to Y memory. The address, `addr`, specifies a Y pointer and an optional update. The Y pointer is updated after the value is stored.

If rounding and saturation are enabled, they are performed on the value from `accumulator`.

Execution Time

1 cycle

Opcode

14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	-	-	-	-	-	-	-	1	0	ac	addr			

If this instruction is the second move instruction in a Move-Move instruction word, the encoding is shifted left 15 bits.

Bits marked with "-" are determined by the X side move instruction.

STORE P[imm16u], reg **Long**

Store register direct to P memory

Operands `STORE P[ addr ], register`

Operation P[addr] = signext(register)

Flags If saturation is enabled and `register` is an accumulator, these status flags are updated:

Saturated: Set if saturation occurs

Description Store a value from a register, `register`, to a 16-bit address, `addr`, in P memory.

If `register` is a 56-bit accumulator, the value from `register` undergoes rounding and saturation if they are enabled.

The 24-bit single-width value is sign-extended to 32 bits and written to P memory.

Execution Time 1 cycle

Opcode

29	28	27	26	25	24	23	22 21 20	19 18 17 16	15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0
1	1	1	0	0	1	1	register	1	addr

STORE P[single-mode(X)], reg **Long**

Store register indirect to P memory

Operands `STORE P[ addr ], register`

Operation P[addr] = signext(register)

update addr

Flags If saturation is enabled and `register` is an accumulator, these status flags are updated:

Saturated: Set if saturation occurs

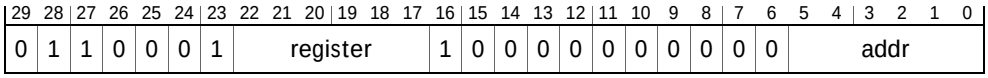
Description Store a value from a register, `register`, to P memory. The address, `addr`, specifies an X pointer and an optional update. The X pointer is updated after the value is stored.

If `register` is a 56-bit accumulator, the value from `register` undergoes rounding and saturation if they are enabled.

The 24-bit single-width value is sign-extended to 32 bits and written to P memory.

Execution Time 1 cycle

Opcode



STORE P[single-mode(Y)], reg **Long**

Store register indirect to P memory

Operands `STORE P[ addr ], register`

Operation P[addr] = signext(register)

update addr

Flags If saturation is enabled and `register` is an accumulator, these status flags are updated:

Saturated: Set if saturation occurs

Description Store a value from a register, `register`, to P memory. The address, `addr`, specifies a Y pointer and an optional update. The Y pointer is updated after the value is stored.

If `register` is a 56-bit accumulator, the value from `register` undergoes rounding and saturation if they are enabled.

The 24-bit single-width value is sign-extended to 32 bits and written to P memory.

Execution Time 1 cycle

Opcode

29	28	27	26	25	24	23	22 21 20	19 18 17 16	15	14	13	12	11	10	9	8	7	6	5	4	3 2 1 0
0	1	1	0	0	0	1	register		1	0	0	0	0	0	0	0	0	0	1	0	addr

STORE X[imm16u], imm8s **Long**

Store a value direct to X memory

Operands `STORE X[ addr ], data`

Operation X[addr] = signext(data)

Flags Not affected

Description Store an 8-bit signed immediate value, `data`, to a 16-bit address, `addr`, in X memory.

Execution Time 1 cycle

Opcode

29	28	27	26	25	24	23 22 21 20	19 18 17 16	15 14 13 12	11 10 9 8	7 6 5 4	3 2 1 0
1	1	0	1	0	0	data		addr			

STORE Y[imm16u], imm8s **Long**

Store a value direct to Y memory

Operands `STORE Y[ addr ], data`

Operation Y[addr] = signext(data)

Flags Not affected

Description Store an 8-bit signed immediate value, `data`, to a 16-bit address, `addr`, in Y memory.

Execution Time 1 cycle

Opcode

29	28	27	26	25	24	23 22 21 20	19 18 17 16	15 14 13 12	11 10 9 8	7 6 5 4	3 2 1 0
1	1	0	1	0	1	data		addr			

STORE P[imm16u], imm8s **Long**

Store a value direct to P memory

Operands `STORE P[ addr ], data`

Operation P[addr] = signext(data)

Flags Not affected

Description Store an 8-bit signed immediate value, `data`, to a 16-bit address, `addr`, in P memory.

Execution Time 1 cycle

Opcode

29	28	27	26	25	24	23 22 21 20	19 18 17 16	15 14 13 12	11 10 9 8	7 6 5 4	3 2 1 0
1	1	0	1	1	1	data		addr			

STORE.DW XY[imm16u], ab-reg **Long**

Store double-width accumulator direct to XY memory

Operands `STORE.DW XY[ addr ], accumulator`

Operation Y[addr] = accumulator.h

X[addr] = accumulator.l

Flags If saturation is enabled, these status flags are updated:

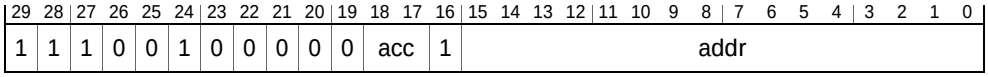
Saturated: Set if saturation occurs

Description Store a 48-bit value from an accumulator, `accumulator`, to a 16-bit address, `addr`, in XY memory.

If enabled, saturation is performed on the value from `accumulator`.

Execution Time 1 cycle

Opcode



STORE.DW XY[single-mode(X)], ab-reg

Move

Store double-width accumulator indirect via X pointer to XY memory

Operands `STORE.DW XY[ addr ], accumulator`

Operation Y[addr] = accumulator.h

X[addr] = accumulator.l

update addr

Flags If saturation is enabled, these status flags are updated:

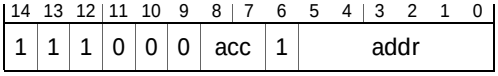
Saturated: Set if saturation occurs

Description Store a 48-bit value from an accumulator, `accumulator`, to XY memory. The address, `addr`, specifies an X pointer and an optional update. The X pointer is updated after the value is stored.

If enabled, saturation is performed on the value from `accumulator`.

Execution Time 1 cycle

Opcode



If this instruction is the second move instruction in a Move-Move instruction word, the encoding is shifted left 15 bits.

STORE.DW XY[imm16u], imm8s **Long**

Store a double-width immediate value direct to XY memory

Operands `STORE.DW XY[ addr ], data`

Operation Y[addr] = 0xFFFFFF * data[7]

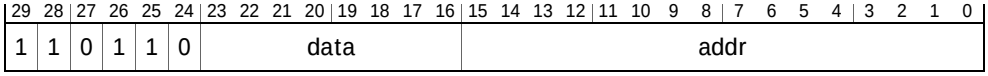
X[addr] = signext(data)

Flags Not affected

Description Store an 8-bit immediate value, `data`, interpreted as a double-width (48-bit) value, to a 16-bit address, `addr`, in XY memory.

Execution Time 1 cycle

Opcode



29	28	27	26	25	24	23 22 21 20	19 18 17 16	15 14 13 12	11 10 9 8	7 6 5 4	3 2 1 0
1	1	0	1	1	0	data		addr			

STORE.EX P[imm16u], ab-reg **Long**

Store extended accumulator direct to P memory

Operands `STORE.EX P[ addr ], accumulator`

Operation P[addr] = accumulator.e <<24 | accumulator.h

Flags Not affected

Description Store a 32-bit value from an accumulator, `accumulator`, to a 16-bit address, `addr`, in P memory.

If enabled, rounding is performed on the value from `accumulator`.

Execution Time 1 cycle

Opcode

29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	0	1	0	1	0	0	0	acc		1	addr															

STORE.EX P[single-mode(X)], ab-reg **Long**

Store extended accumulator indirect via X pointer to P memory

Operands `STORE.EX P[ addr ], accumulator`

Operation P[addr] = accumulator.e <<24 | accumulator.h

update addr

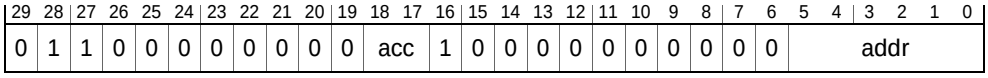
Flags Not affected

Description Store a 32-bit value from an accumulator, `accumulator`, to P memory. The address, `addr`, specifies an X pointer and an optional update. The X pointer is updated after the value is loaded.

If enabled, rounding is performed on the value from `accumulator`.

Execution Time 1 cycle

Opcode



29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	0	0	0	0	0	0	0	acc		1	0	0	0	0	0	0	0	0	0	0	addr					

STORE.EX P[single-mode(Y)], ab-reg

Move

Store extended accumulator indirect via Y pointer to P memory

Operands

```
STORE.EX P[ addr ], accumulator
```

Operation

P[addr] = accumulator.e <<24 | accumulator.h

update addr

Flags

Not affected

Description

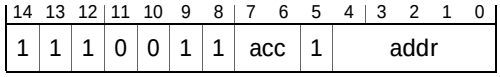
Store a 32-bit value from an accumulator, `accumulator`, to P memory. The address, `addr`, specifies a Y pointer and an optional update. The Y pointer is updated after the value is loaded.

If enabled, rounding is performed on the value from `accumulator`.

Execution Time

1 cycle

Opcode



14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	0	1	1	acc		1		addr			

If this instruction is the second move instruction in a Move-Move instruction word, the encoding is shifted left 15 bits.

SUB SP, imm16 **Long**

Subtract a value from the stack pointer

Operands `SUB SP, data`

Operation SP = SP - data

Flags Not affected

Description Subtract a 16-bit immediate value, `data`, from the stack pointer (`SP`), and store the result back in `SP`.

Execution Time 1 cycle

Remarks This instruction is a synonym for "`ADD SP, -data`".

This instruction requires version 3 or greater of the CFX assembler.

Opcode

See ADD SP, imm16.

SUB SP, imm7sn

Move

Subtract a value from the stack pointer

Operands `SUB SP, data`

Operation SP = SP - signext(data)

Flags Not affected

Description Subtract a 7-bit signed immediate value, `data` (-63..64), from the stack pointer (`SP`), and store the result back in `SP`.

Execution Time 1 cycle

Remarks This instruction is a synonym for "`ADD SP, -data`".

This instruction requires version 3 or greater of the CFX assembler.

Opcode

See ADD SP, imm7s.

SUB xp-reg, SP, imm16 **Long**

Subtract a value from the stack pointer, storing the result in an X pointer

Operands `SUB dst-reg, SP, data`

Operation dst-reg = SP - data

Flags Not affected

Description Subtract a 16-bit immediate value, `data`, from the stack pointer (`SP`), and store the result in an X pointer, `dst-reg`.

Execution Time 1 cycle

Remarks This instruction is a synonym for "`ADD dst-reg, SP, -data`".

This instruction requires version 3 or greater of the CFX assembler.

Opcode

See ADD xp-reg, SP, imm16.

SUB xp-reg, SP, imm6p

Move

Subtract a value from the stack pointer, storing the result in an X pointer

Operands `SUB dst-reg, SP, data`

Operation dst-reg = SP - data

Flags Not affected

Description Subtract a 6-bit positive immediate value, `data` (1..64), from the stack pointer (`SP`), and store the result in an X pointer, `dst-reg`.

Execution Time 1 cycle

Remarks This instruction is a synonym for "`ADD dst-reg, SP, -data`".

This instruction requires version 3 or greater of the CFX assembler.

Opcode

See ADD xp-reg, SP, imm6n.

SUB xp-reg, xp-reg, imm16 **Long**

Subtract a value from an X pointer, storing the result in an X pointer

Operands `SUB dst-reg, src-reg, data`

Operation dst-reg = src-reg - data

Flags Not affected

Description Subtract a 16-bit immediate value, `data`, from an X pointer, `src-reg`, and store the result in an X pointer, `dst-reg`.

Execution Time 1 cycle

Remarks This instruction is a synonym for "`ADD dst-reg, src-reg, -data`".

This instruction requires version 3 or greater of the CFX assembler.

Opcode

See ADD xp-reg, xp-reg, imm16.

SUB xp-reg, xp-reg, imm3sn

Move

Subtract a value from an X pointer, storing the result in an X pointer

Operands `SUB dst-reg, src-reg, data`

Operation dst-reg = src-reg - signext(data)

Flags Not affected

Description Subtract a 3-bit signed immediate value, `data` (-3..4), from an X pointer, `src-reg`, and store the result in an X pointer, `dst-reg`.

Execution Time 1 cycle

Remarks This instruction is a synonym for "`ADD dst-reg, src-reg, -data`".

This instruction requires version 3 or greater of the CFX assembler.

Opcode

See ADD xp-reg, xp-reg, imm3s.

SUB yp-reg, yp-reg, imm3sn

Move

Subtract a value from a Y pointer, storing the result in a Y pointer

Operands `SUB dst-reg, src-reg, data`

Operation dst-reg = src-reg - signext(data)

Flags Not affected

Description Subtract a 3-bit signed immediate value, `data` (-3..4), from a Y pointer, `src-reg`, and store the result in a Y pointer, `dst-reg`.

Execution Time 1 cycle

Remarks This instruction is a synonym for "`ADD dst-reg, src-reg, -data`".

This instruction requires version 3 or greater of the CFX assembler.

Opcode

See ADD yp-reg, yp-reg, imm3s.

SUB d-reg, d-reg, d-reg **Arith**

Subtract two data registers, storing the result in a data register

Operands `SUB dst-reg, src-reg-1, src-reg-2`

Operation dst-reg = src-reg-1 - src-reg-2

Flags If Flag Updates are enabled, the X ALU status flags are affected as follows:

X Carry:	Set according to ALU output
X Negative:	Set according to ALU output
X Zero:	Set according to ALU output
X Overflow:	Set according to ALU output
X Extension:	Set according to ALU output

Description Subtract a data register, `src-reg-2`, from a data register, `src-reg-1`, and store the result in a data register, `dst-reg`.

Execution Time 1 cycle

Opcode

30	29	28	27	26	25	24	23 22 21	20 19 18	17 16 15
0	1	1	0	0	0	0	dst-reg	src-reg-1	src-reg-2

SUB a-reg, a-reg, ab-reg **X Arith**

Subtract two accumulators, storing the result in an A accumulator

Operands `SUB dst-reg, src-reg-1, src-reg-2`

Operation dst-reg = src-reg-1 - src-reg-2

Flags If Flag Updates are enabled, the X ALU status flags are affected as follows:

X Carry:	Set according to ALU output
X Negative:	Set according to ALU output
X Zero:	Set according to ALU output
X Overflow:	Set according to ALU output
X Extension:	Set according to ALU output

Description Subtract an A or B accumulator, `src-reg-2`, from an A accumulator, `src-reg-1`, and store the result in an A accumulator, `dst-reg`.

Execution Time 1 cycle

Constraints When the Y side instruction is not `MUL*`, the right-most source register on the X and Y sides must be opposite—one side using `a-reg` and the other side `b-reg`.

Opcode

Y side instruction is not MUL*

SUB a-reg, a-reg, a-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	1	0	0	1	0	dst	0	s1	s2	-	-	-	0	-	-

SUB a-reg, a-reg, b-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	1	0	0	1	0	dst	1	s1	s2	-	-	-	0	-	-

Y side instruction is MUL*

SUB a-reg, a-reg, a-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	0	1	0	1	dst	0	s1	s2	-	-	-	-	-	-

SUB a-reg, a-reg, b-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	0	1	0	1	dst	1	s1	s2	-	-	-	-	-	-

Bits marked with "-" are determined by the Y side arithmetic instruction.

SUB b-reg, b-reg, ab-reg

Y Arith

Subtract two accumulators, storing the result in a B accumulator

Operands `SUB dst-reg, src-reg-1, src-reg-2`

Operation dst-reg = src-reg-1 - src-reg-2

Flags If Flag Updates are enabled, the Y ALU status flags are affected as follows:

Y Carry:	Set according to ALU output
Y Negative:	Set according to ALU output
Y Zero:	Set according to ALU output
Y Overflow:	Set according to ALU output
Y Extension:	Set according to ALU output

Description Subtract an A or B accumulator, `src-reg-2`, from a B accumulator, `src-reg-1`, and store the result in a B accumulator, `dst-reg`.

Execution Time 1 cycle

Constraints When the X side instruction is not `MUL*`, the right-most source register on the X and Y sides must be opposite—one side using `a-reg` and the other side `b-reg`.

Opcode

X side instruction is not MUL*

SUB b-reg, b-reg, a-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	1	0	-	-	-	-	1	-	-	0	1	dst	-	s1	s2

SUB b-reg, b-reg, b-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	1	0	-	-	-	-	0	-	-	0	1	dst	-	s1	s2

X side instruction is MUL*

SUB b-reg, b-reg, a-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	0	0	-	-	-	-	-	-	0	1	dst	1	s1	s2

SUB b-reg, b-reg, b-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	0	0	-	-	-	-	-	-	0	1	dst	0	s1	s2

Bits marked with "-" are determined by the X side arithmetic instruction.

SUB d-reg, d-reg, imm3sn **Arith**

Subtract a value from a data register, storing the result in a data register

Operands `SUB dst-reg, src-reg, data`

Operation dst-reg = src-reg - signext(data)

Flags If Flag Updates are enabled, the X ALU status flags are affected as follows:

X Carry:	Set according to ALU output
X Negative:	Set according to ALU output
X Zero:	Set according to ALU output
X Overflow:	Set according to ALU output
X Extension:	Set according to ALU output

Description Subtract a 3-bit signed immediate value, `data` (-3..4), from a data register, `src-reg`, and store the result in a data register, `dst-reg`.

Execution Time 1 cycle

Remarks This instruction is a synonym for "`ADD dst-reg, src-reg, -data`".

This instruction requires version 3 or greater of the CFX assembler.

Opcode

See ADD d-reg, d-reg, imm3s.

SUB d-reg, d-reg, cond3 **Arith**

Subtract a data register from a data register, conditionally

Operands `SUB dst-reg, src-reg, condition`

Operation if condition holds, dst-reg = dst-reg - src-reg

Flags Not affected

Description If the condition holds, subtract a data register, `src-reg`, from a data register, `dst-reg`, and store the result back in `dst-reg`.

Execution Time 1 cycle

Remarks Version 3 or greater of the CFX assembler supports the following alternate form for this conditional instruction:

```
SUB d-reg, d-reg if cond3
```

Opcode

30	29	28	27	26	25	24	23 22 21	20 19 18 17	16 15
0	1	1	1	1	0	0	dst-reg	cond	src-reg

SUB d-reg, imm3sn, cond3 **Arith**

Subtract a value from a data register if the condition holds

Operands `SUB dst-reg, data, condition`

Operation if condition holds, dst-reg = dst-reg - signext(data)

Flags Not affected

Description If the condition holds, subtract a 3-bit signed immediate value, `data` (-3..4), from a data register, `dst-reg`, and store the result back in `dst-reg`.

Execution Time 1 cycle

Remarks This instruction is a synonym for "`ADD dst-reg, -data, condition`".

This instruction requires version 3 or greater of the CFX assembler.

Version 3 or greater of the CFX assembler supports the following alternate form for this conditional instruction:

```
SUB d-reg, imm3sn if cond3
```

Opcode

See ADD d-reg, imm3s, cond3.

SUB.DW d-reg, d-reg, imm3sn **Arith**

Subtract a double-width value from a data register, storing the result in a data register

Operands `SUB.DW dst-reg, src-reg, data`

Operation dst-reg = src-reg - (signext(data) >> 24)

Flags If Flag Updates are enabled, the X ALU status flags are affected as follows:

X Carry:	Set according to ALU output
X Negative:	Set according to ALU output
X Zero:	Set according to ALU output
X Overflow:	Set according to ALU output
X Extension:	Set according to ALU output

Description Subtract a 3-bit signed immediate value, interpreted as a double-width value (aligned with the low register of an accumulator, or the 24-bit word below an XY data register), from a data register, `src-reg`, and store the result in a data register, `dst-reg`.

Execution Time 1 cycle

Remarks Double-width arithmetic is not useful when both `src-reg` and `dst-reg` are XY data registers.

This instruction is a synonym for "`ADD.DW dst-reg, src-reg, -data`".

This instruction requires version 3 or greater of the CFX assembler.

Opcode

See ADD.DW d-reg, d-reg, imm3s.

SUB.DW d-reg, imm3sn, cond3 **Arith**

Subtract a double-width value from a data register, if the condition holds

Operands `SUB.DW dst-reg, data, condition`

Operation if condition holds, dst-reg = dst-reg - (signext(data) >> 24)

Flags Not affected

Description If the condition holds, subtract a 3-bit signed immediate value, interpreted as a double-width value (aligned with the low register of an accumulator, or the 24-bit word below an XY data register), from a data register, `dst-reg`, and store the result back in `dst-reg`.

Execution Time 1 cycle

Remarks Double-width arithmetic is not useful when `dst-reg` is an XY data register.

This instruction is a synonym for "`ADD.DW dst-reg, -data, condition`".

This instruction requires version 3 or greater of the CFX assembler.

Version 3 or greater of the CFX assembler supports the following alternate form for this conditional instruction:

```
SUB.DW d-reg, imm3sn if cond3
```

Opcode

See ADD.DW d-reg, imm3s, cond3.

SUB.FR d-reg, d-reg, imm3s **Arith**

Subtract a fractional immediate value from a data register, storing the result in a data register

Operands `SUB.FR dst-reg, src-reg, data`

Operation dst-reg = src-reg - fractional(data)

Flags If Flag Updates are enabled, the X ALU status flags are affected as follows:

X Carry:	Set according to ALU output
X Negative:	Set according to ALU output
X Zero:	Set according to ALU output
X Overflow:	Set according to ALU output
X Extension:	Set according to ALU output

Description Subtract a fractional immediate value, `data`, from a data register, `src-reg`, and store the result in a data register, `dst-reg`.

The 3-bit signed immediate value is interpreted as follows:

Imm Value	Fract Value	Hex Value Subtracted
-4	-1	0xFF800000000000
-3	$-3/4$	0xFFA00000000000
-2	$-1/2$	0xFFC00000000000
-1	$-1/4$	0xFFE00000000000
1	$+1/4$	0x00200000000000
2	$+1/2$	0x00400000000000
3	$+3/4$	0x00600000000000
0	+1	0x00800000000000

Execution Time 1 cycle

Remarks If `data == 0` this instruction is a synonym for "`ADD.FR dst-reg, src-reg, -4`".

If `data == -4` this instruction is a synonym for "`ADD.FR dst-reg, src-reg, 0`".

Otherwise this instruction is a synonym for "`ADD.FR dst-reg, src-reg, -data`".

This instruction requires version 3 or greater of the CFX assembler.

Opcode

See ADD.FR d-reg, d-reg, imm3s.

SUB.NX d-reg, d-reg, d-reg **Arith**

Subtract two data registers with no extension, storing the result in a data register

Operands `SUB.NX dst-reg, src-reg-1, src-reg-2`

Operation dst-reg = signext(src-reg-1 - src-reg-2)

Flags If Flag Updates are enabled, the X ALU status flags are affected as follows:

X Carry:	Not affected
X Negative:	Set according to ALU output
X Zero:	Set according to ALU output
X Overflow:	Set to 0
X Extension:	Not affected

Description Subtract a data register, `src-reg-2`, from a data register, `src-reg-1`, and store the result in a data register, `dst-reg`.

Before the result is written to the destination register, the upper 8 bits are overwritten with the sign extension from bit 47 of the result.

Execution Time 1 cycle

Opcode

30	29	28	27	26	25	24	23 22 21 20	19 18 17 16	15
0	1	1	0	0	0	1	dst-reg	src-reg-1	src-reg-2

SUB.NX a-reg, a-reg, ab-reg **X Arith**

Subtract two accumulators with no extension, storing the result in an A accumulator

Operands `SUB.NX dst-reg, src-reg-1, src-reg-2`

Operation dst-reg = signext(src-reg-1 - src-reg-2)

Flags If Flag Updates are enabled, the X ALU status flags are affected as follows:

X Carry:	Not affected
X Negative:	Set according to ALU output
X Zero:	Set according to ALU output
X Overflow:	Set to 0
X Extension:	Not affected

Description Subtract an A or B accumulator, `src-reg-2`, from an A accumulator, `src-reg-1`, and store the result in an A accumulator, `dst-reg`.

Before the result is written to the destination register, the upper 8 bits are overwritten with the sign extension from bit 47 of the result.

Execution Time 1 cycle

Constraints The Y side instruction must not be `MUL*`.

The right-most source register on the X and Y sides must be opposite—one side using `a-reg` and the other side `b-reg`.

Opcode

SUB.NX a-reg, a-reg, a-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	1	0	0	1	0	dst	0	s1	s2	-	-	-	1	-	-

SUB.NX a-reg, a-reg, b-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	1	0	0	1	0	dst	1	s1	s2	-	-	-	1	-	-

Bits marked with "-" are determined by the Y side arithmetic instruction.

SUB.NX d-reg, d-reg, imm3sn **Arith**

Subtract a value from a data register with no extension, storing the result in a data register

Operands

```
SUB.NX dst-reg, src-reg, data
```

Operation

dst-reg = signext(src-reg - signext(data))

Flags

If Flag Updates are enabled, the X ALU status flags are affected as follows:

X Carry:	Not affected
X Negative:	Set according to ALU output
X Zero:	Set according to ALU output
X Overflow:	Set to 0
X Extension:	Not affected

Description

Subtract a 3-bit signed immediate value, `data` (-3..4), from a data register, `src-reg`, and store the result in a data register, `dst-reg`.

Before the result is written to the destination register, the upper 8 bits are overwritten with the sign extension from bit 47 of the result.

Execution Time

1 cycle

Remarks

This instruction is a synonym for "`ADD.NX dst-reg, src-reg, -data`".

This instruction requires version 3 or greater of the CFX assembler.

Opcode

See ADD.NX d-reg, d-reg, imm3s.

SUB.NX.DW d-reg, d-reg, imm3sn **Arith**

Subtract a double-width value from a data register with no extension, storing the result in a data register

Operands `SUB.NX.DW dst-reg, src-reg, data`

Operation dst-reg = signext(src-reg - (signext(data) >> 24))

Flags If Flag Updates are enabled, the X ALU status flags are affected as follows:

X Carry:	Not affected
X Negative:	Set according to ALU output
X Zero:	Set according to ALU output
X Overflow:	Set to 0
X Extension:	Not affected

Description Subtract a 3-bit signed immediate value, interpreted as a double-width value (aligned with the low register of an accumulator, or the 24-bit word below an XY data register), from a data register, `src-reg`, and store the result in a data register, `dst-reg`.

Before the result is written to the destination register, the upper 8 bits are overwritten with the sign extension from bit 47 of the result.

Execution Time 1 cycle

Remarks Double-width arithmetic is not useful when both `src-reg` and `dst-reg` are XY data registers.

This instruction is a synonym for "`ADD.NX.DW dst-reg, src-reg, -data`".

This instruction requires version 3 or greater of the CFX assembler.

Opcode

See ADD.NX.DW d-reg, d-reg, imm3s.

SUBMUL ab-reg, xy-reg, xy-reg, xy-reg **Arith**

Subtract two XY data registers, multiply the result by a third XY data register, and store the result in an accumulator

Operands `SUBMUL dst-reg, src-reg-1, src-reg-2, src-reg-3`

Operation dst-reg = 0 + src-reg-1 * (src-reg-2 - src-reg-3)

Flags If Flag Updates are enabled, the X ALU status flags are affected when the ALU operation completes (after the next clock cycle), as follows:

X Carry:	Not affected
X Negative:	Set according to result of adding 0 to product
X Zero:	Set according to result of adding 0 to product
X Overflow:	Set to 0
X Extension:	Set according to result of adding 0 to product

Description Subtract an XY data register, `src-reg-3`, from an XY data register, `src-reg-2`, multiply by a third XY data register, `src-reg-1`, and write the result to an accumulator, `dst-reg`.

Execution Time 2 cycles, 1 pipelined (ALU operation)

Constraints The three source registers must be one of four combinations:

src-reg-1	src-reg-2	src-reg-3
x-reg	x-reg	y-reg
x-reg	y-reg	y-reg
y-reg	x-reg	y-reg
y-reg	x-reg	x-reg

Opcode

SUBMUL ab-reg, x-reg, x-reg, y-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	1	0	0	1	s1	s2	s3	0	0	1	1	0	dst	

SUBMUL ab-reg, x-reg, y-reg, y-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	1	0	0	1	s1	s2	s3	0	1	1	1	0	dst	

SUBMUL ab-reg, y-reg, x-reg, y-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	1	0	0	1	s1	s2	s3	1	0	1	1	0	dst	

SUBMUL ab-reg, y-reg, x-reg, x-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	1	0	0	1	s1	s2	s3	1	1	1	1	0	dst	

SUBMULADD ab-reg, xy-reg, xy-reg, xy-reg **Arith**

Subtract two XY data registers, multiply the result by a third XY data register, and add the result to an accumulator

Operands `SUBMULADD dst-reg, src-reg-1, src-reg-2, src-reg-3`

Operation dst-reg = dst-reg + src-reg-1 * (src-reg-2 - src-reg-3)

Flags If Flag Updates are enabled, the X ALU status flags are affected when the ALU operation completes (after the next clock cycle), as follows:

X Carry:	Set according to result of adding `dst-reg` to product
X Negative:	Set according to result of adding `dst-reg` to product
X Zero:	Set according to result of adding `dst-reg` to product
X Overflow:	Set according to result of adding `dst-reg` to product
X Extension:	Set according to result of adding `dst-reg` to product

Description Subtract an XY data register, `src-reg-3`, from an XY data register, `src-reg-2`, multiply by a third XY data register, `src-reg-1`, add an accumulator, `dst-reg`, and write the result back to `dst-reg`.

Execution Time 2 cycles, 1 pipelined (ALU operation)

Constraints The three source registers must be one of four combinations:

src-reg-1	src-reg-2	src-reg-3
x-reg	x-reg	y-reg
x-reg	y-reg	y-reg
y-reg	x-reg	y-reg
y-reg	x-reg	x-reg

Opcode

SUBMULADD ab-reg, x-reg, x-reg, y-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	1	0	0	1	s1	s2	s3	0	0	1	0	0	dst	

SUBMULADD ab-reg, x-reg, y-reg, y-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	1	0	0	1	s1	s2	s3	0	1	1	0	0	dst	

SUBMULADD ab-reg, y-reg, x-reg, y-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	1	0	0	1	s1	s2	s3	1	0	1	0	0	dst	

SUBMULADD ab-reg, y-reg, x-reg, x-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	1	0	0	1	s1	s2	s3	1	1	1	0	0	dst	

SUBMULNEG ab-reg, xy-reg, xy-reg, xy-reg **Arith**

Subtract two XY data registers, multiply the result by a third XY data register, negate the result and store it in an accumulator

Operands `SUBMULNEG dst-reg, src-reg-1, src-reg-2, src-reg-3`

Operation dst-reg = 0 - src-reg-1 * (src-reg-2 - src-reg-3)

Flags If Flag Updates are enabled, the X ALU status flags are affected when the ALU operation completes (after the next clock cycle), as follows:

X Carry:	Not affected
X Negative:	Set according to result of subtracting product from 0
X Zero:	Set according to result of subtracting product from 0
X Overflow:	Set to 0
X Extension:	Set according to result of subtracting product from 0

Description Subtract an XY data register, `src-reg-3`, from an XY data register, `src-reg-2`, multiply by a third XY data register, `src-reg-1`, negate the result and write it to an accumulator, `dst-reg`.

Execution Time 2 cycles, 1 pipelined (ALU operation)

Constraints The three source registers must be one of four combinations:

src-reg-1	src-reg-2	src-reg-3
x-reg	x-reg	y-reg
x-reg	y-reg	y-reg
y-reg	x-reg	y-reg
y-reg	x-reg	x-reg

Opcode

SUBMULNEG ab-reg, x-reg, x-reg, y-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	1	0	0	1	s1	s2	s3	0	0	1	1	1	dst	

SUBMULNEG ab-reg, x-reg, y-reg, y-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	1	0	0	1	s1	s2	s3	0	1	1	1	1	dst	

SUBMULNEG ab-reg, y-reg, x-reg, y-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	1	0	0	1	s1	s2	s3	1	0	1	1	1	dst	

SUBMULNEG ab-reg, y-reg, x-reg, x-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	1	0	0	1	s1	s2	s3	1	1	1	1	1	dst	

SUBMULSUB ab-reg, xy-reg, xy-reg, xy-reg **Arith**

Subtract two XY data registers, multiply the result by a third XY data register, and subtract the result from an accumulator

Operands `SUBMULSUB dst-reg, src-reg-1, src-reg-2, src-reg-3`

Operation dst-reg = dst-reg - src-reg-1 * (src-reg-2 - src-reg-3)

Flags If Flag Updates are enabled, the X ALU status flags are affected when the ALU operation completes (after the next clock cycle), as follows:

X Carry:	Set according to result of subtracting product from `dst-reg`
X Negative:	Set according to result of subtracting product from `dst-reg`
X Zero:	Set according to result of subtracting product from `dst-reg`
X Overflow:	Set according to result of subtracting product from `dst-reg`
X Extension:	Set according to result of subtracting product from `dst-reg`

Description Subtract an XY data register, `src-reg-3`, from an XY data register, `src-reg-2`, multiply by a third XY data register, `src-reg-1`, subtract the result from an accumulator, `dst-reg`, and write the result back to `dst-reg`.

Execution Time 2 cycles, 1 pipelined (ALU operation)

Constraints The three source registers must be one of four combinations:

src-reg-1	src-reg-2	src-reg-3
x-reg	x-reg	y-reg
x-reg	y-reg	y-reg
y-reg	x-reg	y-reg
y-reg	x-reg	x-reg

Opcode

SUBMULSUB ab-reg, x-reg, x-reg, y-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	1	0	0	1	s1	s2	s3	0	0	1	0	1	dst	

SUBMULSUB ab-reg, x-reg, y-reg, y-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	1	0	0	1	s1	s2	s3	0	1	1	0	1	dst	

SUBMULSUB ab-reg, y-reg, x-reg, y-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	1	0	0	1	s1	s2	s3	1	0	1	0	1	dst	

SUBMULSUB ab-reg, y-reg, x-reg, x-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	1	0	0	1	s1	s2	s3	1	1	1	0	1	dst	

SUBSH d-reg, d-reg, d-reg **Arith**

Subtract two data registers and shift right one bit, storing the result in a data register

Operands `SUBSH dst-reg, src-reg-1, src-reg-2`

Operation dst-reg = (src-reg-1 - src-reg-2) >> 1

Flags If Flag Updates are enabled, the X ALU status flags are affected as follows:

X Carry:	Set according to the ALU output of `src-reg-1 - src-reg-2`
X Negative:	Set according to ALU output
X Zero:	Set according to ALU output
X Overflow:	Set according to the ALU output of `src-reg-1 - src-reg-2`
X Extension:	Set according to ALU output

Note that the X Carry and X Overflow flags are updated based on the subtraction, not the shift operation.

Description Subtract a data register, `src-reg-2`, from a data register, `src-reg-1`, and perform an arithmetic shift of one bit to the right before storing the result in a data register, `dst-reg`.

Execution Time 1 cycle

Opcode

30	29	28	27	26	25	24	23 22 21	20 19 18	17 16 15
0	1	0	1	1	1	1	dst-reg	src-reg-1	src-reg-2

SUBSH a-reg, a-reg, ab-reg **X Arith**

Subtract two accumulators and shift right one bit, storing the result in an A accumulator

Operands `SUBSH dst-reg, src-reg-1, src-reg-2`

Operation dst-reg = (src-reg-1 - src-reg-2) >> 1

Flags If Flag Updates are enabled, the X ALU status flags are affected as follows:

X Carry:	Set according to the ALU output of `src-reg-1 - src-reg-2`
X Negative:	Set according to ALU output
X Zero:	Set according to ALU output
X Overflow:	Set according to the ALU output of `src-reg-1 - src-reg-2`
X Extension:	Set according to ALU output

Note that the X Carry and X Overflow flags are updated based on the subtraction, not the shift operation.

Description Subtract an A or B accumulator, `src-reg-2`, from an A accumulator, `src-reg-1`, and perform an arithmetic shift of one bit to the right before storing the result in an A accumulator, `dst-reg`.

Execution Time 1 cycle

Constraints When the Y side instruction is not `MUL*`, the right-most source register on the X and Y sides must be opposite—one side using `a-reg` and the other side `b-reg`.

Opcode

Y side instruction is not MUL*

SUBSH a-reg, a-reg, a-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	1	0	1	0	1	dst	0	s1	s2	-	-	-	1	-	-

SUBSH a-reg, a-reg, b-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	1	0	1	0	1	dst	1	s1	s2	-	-	-	1	-	-

Y side instruction is MUL*

SUBSH a-reg, a-reg, a-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	0	1	1	1	dst	0	s1	s2	-	-	-	-	-	-

SUBSH a-reg, a-reg, b-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	0	1	1	1	dst	1	s1	s2	-	-	-	-	-	-

Bits marked with "-" are determined by the Y side arithmetic instruction.

SUBSH b-reg, b-reg, ab-reg **Y Arith**

Subtract two accumulators and shift right one bit, storing the result in a B accumulator

Operands `SUBSH dst-reg, src-reg-1, src-reg-2`

Operation dst-reg = (src-reg-1 - src-reg-2) >> 1

Flags If Flag Updates are enabled, the Y ALU status flags are affected as follows:

Y Carry:	Set according to the ALU output of `src-reg-1 - src-reg-2`
Y Negative:	Set according to ALU output
Y Zero:	Set according to ALU output
Y Overflow:	Set according to the ALU output of `src-reg-1 - src-reg-2`
Y Extension:	Set according to ALU output

Note that the Y Carry and Y Overflow flags are updated based on the subtraction, not the shift operation.

Description Subtract an A or B accumulator, `src-reg-2`, from a B accumulator, `src-reg-1`, and perform an arithmetic shift of one bit to the right before storing the result in a B accumulator, `dst-reg`.

Execution Time 1 cycle

Constraints When the X side instruction is not `MUL*`, the right-most source register on the X and Y sides must be opposite—one side using `a-reg` and the other side `b-reg`.

Opcode

X side instruction is not MUL*

SUBSH b-reg, b-reg, a-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	1	0	-	-	-	-	1	-	-	1	1	dst	-	s1	s2

SUBSH b-reg, b-reg, b-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	1	0	-	-	-	-	0	-	-	1	1	dst	-	s1	s2

X side instruction is MUL*

SUBSH b-reg, b-reg, a-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	0	0	-	-	-	-	-	-	1	1	dst	1	s1	s2

SUBSH b-reg, b-reg, b-reg

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
1	0	0	0	-	-	-	-	-	-	1	1	dst	0	s1	s2

Bits marked with "-" are determined by the X side arithmetic instruction.

SUBSTEP xp-reg **Move**

Subtract an X step register from the corresponding X pointer

Operands `SUBSTEP dst-reg`

Operation dst-reg = dst-reg - xstep-reg

(xstep-reg is chosen based on which dst-reg is specified)

Flags Not affected

Description Subtract the corresponding X step register from the specified X pointer, `dst-reg`.

Execution Time 1 cycle

Opcode

14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	0	1	0	1	0	0	dst-reg		

If this instruction is the second move instruction in a Move-Move instruction word, the encoding is shifted left 15 bits.

SUBSTEP yp-reg

Move

Subtract a Y step register from the corresponding Y pointer

Operands `SUBSTEP dst-reg`

Operation dst-reg = dst-reg - ystep-reg

(ystep-reg is chosen based on which dst-reg is specified)

Flags Not affected

Description Subtract the corresponding Y step register from the specified Y pointer, `dst-reg`.

Execution Time 1 cycle

Opcode

14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	0	1	0	1	0	1	0	dst	

If this instruction is the second move instruction in a Move-Move instruction word, the encoding is shifted left 15 bits.

SWAP d-reg, d-reg, cond3 **Arith**

Swap the contents of two data registers, conditionally

Operands `SWAP reg-1, reg-2, condition`

Operation if condition holds, then

temp = reg-1

reg-1 = reg-2

reg-2 = temp

Flags Not affected

Description If the condition holds, swap the contents of two data registers, `reg-1` and `reg-2`.

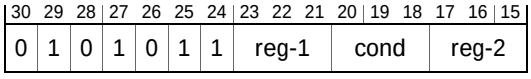
Execution Time 1 cycle

Constraints The registers, `reg-1` and `reg-2`, must be in different register files; for example, it is possible to swap `A0` with `B0`, or `X0` with `Y1`, but not `A0` with `A1`, or `X0` with `X1`.

Remarks Version 3 or greater of the CFX assembler supports the following alternate form for this conditional instruction:

```
SWAP d-reg, d-reg if cond3
```

Opcode



30	29	28	27	26	25	24	23 22 21	20 19 18 17	16 15
0	1	0	1	0	1	1	reg-1	cond	reg-2

XOR d-reg, d-reg, d-reg **Arith**

Bitwise XOR two data registers, storing the result in a data register

Operands `XOR dst-reg, src-reg-1, src-reg-2`

Operation dst-reg = src-reg-1 XOR src-reg-2

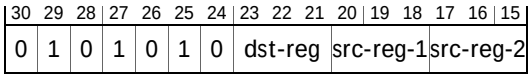
Flags If Flag Updates are enabled, the X ALU status flags are affected as follows:

X Carry:	Not affected
X Negative:	Set according to ALU output
X Zero:	Set according to ALU output
X Overflow:	Set to 0
X Extension:	Set according to ALU output

Description Bitwise XOR two data registers, `src-reg-1` and `src-reg-2`, and store the result in a data register, `dst-reg`.

Execution Time 1 cycle

Opcode



30	29	28	27	26	25	24	23 22 21 20	19 18 17 16	15
0	1	0	1	0	1	0	dst-reg	src-reg-1	src-reg-2

XOR d-reg, d-reg, imm3s **Arith**

Bitwise XOR a data register and a value, storing the result in a data register

Operands `XOR dst-reg, src-reg, data`

Operation dst-reg = src-reg XOR signext(data)

Flags If Flag Updates are enabled, the X ALU status flags are affected as follows:

X Carry:	Not affected
X Negative:	Set according to ALU output
X Zero:	Set according to ALU output
X Overflow:	Set to 0
X Extension:	Set according to ALU output

Description Bitwise XOR a data register, `src-reg`, and a 3-bit signed immediate value, `data` (-4..3), and store the result in a data register, `dst-reg`.

Execution Time 1 cycle

Opcode

30	29	28	27	26	25	24	23 22 21	20 19 18	17 16 15
0	0	1	1	0	1	0	dst-reg	src-reg	data

XOR.DW d-reg, d-reg, imm3s **Arith**

Bitwise XOR a data register and a double-width value, storing the result in a data register

Operands `XOR.DW dst-reg, src-reg, data`

Operation dst-reg = src-reg XOR (signext(data) >> 24)

Flags If Flag Updates are enabled, the X ALU status flags are affected as follows:

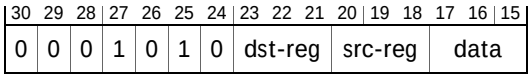
X Carry:	Not affected
X Negative:	Set according to ALU output
X Zero:	Set according to ALU output
X Overflow:	Set to 0
X Extension:	Set according to ALU output

Description Bitwise XOR a data register, `src-reg`, and a 3-bit signed immediate value, `data` (-4..3), interpreted as a double-width value (aligned with the low register of an accumulator, or the 24-bit word below an XY data register), and store the result in a data register, `dst-reg`.

Execution Time 1 cycle

Remarks Double-width logic is not useful when `dst-reg` is an XY data register.

Opcode



30	29	28	27	26	25	24	23 22 21	20 19 18	17 16 15
0	0	0	1	0	1	0	dst-reg	src-reg	data

This page left blank for printing purposes.

A Instruction Set Summary

A.1 INTRODUCTION

Table 46 on page 377 summarizes the instruction set. The following section identifies the information in each column of the summary table.

A.2 EXPLANATION OF COLUMNS IN THE SUMMARY TABLE

The columns in the summary table provide the following information:

Instruction The instruction with operands

In some cases, more than one instruction and opcode are given for the same or similar instructions. There are two reasons for this:

1. A different opcode is used for multiplication and dual arithmetic instructions depending on the combination of operand registers used.
2. A different opcode is used for dual arithmetic instructions depending on whether the other half of the dual arithmetic instruction is a multiplication or ALU instruction.

T The type of instruction:

Symbol	Instruction Type
L	Long
M	Single Move
XM	X Move
YM	Y Move
A	Single Arithmetic
XA	X Arithmetic
YA	Y Arithmetic
S	Synthesized

31..0 The opcode

Bits marked with field names are determined by the encoding of the corresponding field. Some fields have been abbreviated. For example, src-reg-1 might be abbreviated src1 or s1.

Bits marked with "-" in *X* and *Y Move* and *Arithmetic* instructions are determined by the other side of the *Move* or *Arithmetic* instruction.

Empty bits are determined by the instruction type and/or the other half of the instruction for *Move* and *Arithmetic* instructions.

The opcode for synthesized instructions is marked “synth”, indicating that the opcode is synthesized from one or more other instruction opcodes. Consult the instruction description in Chapter 10, “CFX Instruction Set” on page 83 for details.

#C The total number of cycles required to execute the instruction

Pipelined ALU cycles are indicated with “, 1A”, and pipelined branch delay slots are indicated with “, 1D” and “, 2D”.

The `BITSET SR, imm4u` instruction takes 3 cycles if setting the `IE` bit. This is indicated with “, 3I”.

The `GOTO imm16u, cond5` and `GOTO.0D imm16u, cond5` instructions take only 1 cycle if the branch is not taken. This is indicated with “, 1N”.

Operation The action performed by the core for the given instruction.

B, C, N, Z, O, E, S How the instruction affects the status flags in the `SR` register.

The “B” column specifies whether the “X” or “Y” status flags are affected.

The other columns can contain one of the following symbols:

Symbol	Flag Update
	The flag is not affected.
0	The flag is set to 0.
Δ	The flag is set according to the instruction.
F0	If Flag Updates are enabled, the flag is set to 0.
FΔ	If Flag Updates are enabled, the flag is set according to the instruction.

The details of the flag update depend on the instruction:

Instruction	Details
Comparison (`CMP*`)	The flags are updated based on the result of the subtraction, `src-reg-1 - src-reg-2`.
Bit Manipulation (`BITSET`, `BITCLR`, `BITTGL`)	The Carry flag is updated with the original value of the modified bit

Instruction	Details
Shift Instructions (SHLL*, SHRL*, SHRA*)	The Carry flag is set to the last bit shifted out, or 0 if the shift count is zero. The Overflow flag is set based on whether the sign bit changes during the shift operation, and is always set to 0 for right shifts where the shift count is positive, or left shifts where the shift count is negative. All other bits are set based on the 56-bit result of the shift operation.
Arithmetic and Logic Instructions	The flags are updated based on the 56-bit result of the instruction operation.
Add/Subtract and Shift (ADDSH, SUBSH)	The Carry and Overflow flags are updated based on the 56-bit result of the addition or subtraction, before the shift operation. All other bits are set based on the 56-bit result of the shift operation.
Multiplication Instructions (*MUL*)	The flags are always updated during the second cycle of the multiplication instruction, based on the result of the ALU operation (Add, Subtract, Add-Zero or Negate). For multiplication instructions that involve a pre-addition or subtraction (ADDMUL*, SUBMUL*), the flags are only updated based on the ALU operation, not the pre-addition or subtraction.
Extreme Value Instructions (MIN, MAX)	The flags are updated based on the result of the subtraction, src-reg-1 - src-reg-2.
Division (DIVST)	The X Negative flag is updated based on the result of the addition or subtraction within the division-step operation.
MOVE and STORE Instructions	If saturation is enabled and the source register is an accumulator, the Saturated flag is set if saturation occurs.

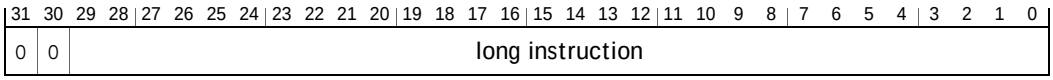
A.3 SUMMARY OF INSTRUCTION TYPES

Instruction words contain instructions in three possible combinations. The upper one or two bits of the instruction word indicate which combination it is.

Long Instruction Word

A *Long* instruction word contains a single *Long* instruction:

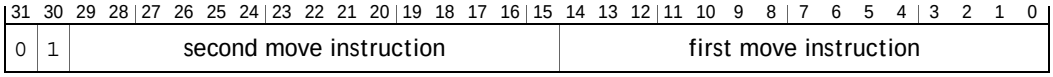
<long instruction>



Move-Move Instruction Word

A *Move-Move* instruction word contains two *Move* instructions that are executed sequentially:

```
<first move instruction> >>> <second move instruction>
```

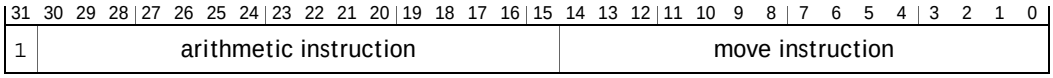


In the summary table, the *Move* instruction opcodes are shown in bits 0-14. For the second *Move* instruction in an instruction word, the opcode is shifted left 15 bits.

Arithmetic-Move Instruction Word

An *Arithmetic-Move* instruction word contains an *Arithmetic* instruction and a *Move* instruction that are executed in parallel:

```
<arithmetic instruction> ||| <move instruction>
```



A.4 Alphabetical Instruction Summary Table

Table 46: Instruction Set Summary

Instruction	T	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	#C	Operation	B	C	N	Z	O	E	S
ABS d-reg, d-reg	A		0	1	1	1	0	1	0	dst			0	0	0	src																			1	dst-reg = |src-reg|	X		FΔ	FΔ	FΔ	FΔ	
ABS a-reg, a-reg	XA		1	1	0	1	1	1	d	0	0	s	-	-	-	0	-	-																1	dst-reg = |src-reg|	X		FΔ	FΔ	FΔ	FΔ	
ABS a-reg, b-reg			1	1	0	1	1	1	d	1	0	s	-	-	-	0	-	-																									
ADD SP, imm16	L			1	1	1	1	0	0	1	1	0	1	0	0	0	0	data																1	SP = SP + data							
ADD SP, imm7s	M																		1	1	1	1	1	0	1	0	data						1	SP = SP + signext(data)							
ADD xp-reg, SP, imm16	L			1	1	1	1	0	0	1	1	0	0	1	dst			data																1	dst-reg = SP + data							
ADD xp-reg, SP, imm6n	M																		1	1	1	0	1	0	dst				data				1	dst-reg = SP + data							
ADD xp-reg, xp-reg, xstep-reg	M																		1	1	1	0	1	1	dst				src1			src2	1	dst-reg = src-reg-1 + src-reg-2							
ADD xp-reg, xp-reg, imm16	L			1	1	1	0	1	0	1	0	dst			src			data																1	dst-reg = src-reg + data							
ADD xp-reg, xp-reg, imm3s	M																		1	1	1	1	0	0	dst				src			data	1	dst-reg = src-reg + signext(data)							
ADD yp-reg, yp-reg, ystep-reg	M																		1	1	1	1	1	1	0	0	0	dst		src 1		src 2	1	dst-reg = src-reg-1 + src-reg-2							
ADD yp-reg, yp-reg, imm3s	M																		1	1	1	1	1	0	1	1	dst		src		data		1	dst-reg = src-reg + signext(data)							
ADD d-reg, d-reg, d-reg	A		0	1	0	0	0	0	0	dst			src1			src2																	1	dst-reg = src-reg-1 + src-reg-2	X	FΔ	FΔ	FΔ	FΔ	FΔ	
ADD a-reg, a-reg, a-reg (Y side instruction is not MUL*)	XA		1	1	0	0	0	0	d	0	s 1	s 2	-	-	-	0	-	-															1	dst-reg = src-reg-1 + src-reg-2	X	FΔ	FΔ	FΔ	FΔ	FΔ	
ADD a-reg, a-reg, b-reg (Y side instruction is not MUL*)			1	1	0	0	0	0	d	1	s 1	s 2	-	-	-	0	-	-																									
ADD a-reg, a-reg, a-reg (Y side instruction is MUL*)			1	0	0	1	0	0	d	0	s 1	s 2	-	-	-	-	-	-																									
ADD a-reg, a-reg, b-reg (Y side instruction is MUL*)			1	0	0	1	0	0	d	1	s 1	s 2	-	-	-	-	-	-																									
ADD b-reg, b-reg, a-reg (X side instruction is not MUL*)	YA		1	1	0	-	-	-	-	1	-	-	0	0	d	-	s 1	s 2																1	dst-reg = src-reg-1 + src-reg-2	Y	FΔ	FΔ	FΔ	FΔ	FΔ	
ADD b-reg, b-reg, b-reg (X side instruction is not MUL*)			1	1	0	-	-	-	-	0	-	-	0	0	d	-	s 1	s 2																									
ADD b-reg, b-reg, a-reg (X side instruction is MUL*)			1	0	0	0	-	-	-	-	-	-	0	0	d	1	s 1	s 2																									
ADD b-reg, b-reg, b-reg (X side instruction is MUL*)			1	0	0	0	-	-	-	-	-	-	0	0	d	0	s 1	s 2																									

TABLE 46: INSTRUCTION SET SUMMARY (CONTINUED)

Instruction	T	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	#C	Operation	B	C	N	Z	O	E	S
ADD d-reg, d-reg, imm3s	A		0	0	1	0	0	0	0	dst			src			data																			1	dst-reg = src-reg + signext(data)	X	FΔ	FΔ	FΔ	FΔ	FΔ	
ADD d-reg, d-reg, cond3	A		0	1	1	1	1	1	0	dst			cond			src																			1	if condition holds, dst-reg = dst-reg + src-reg							
ADD d-reg, imm3s, cond3	A		0	1	0	1	1	0	0	dst			cond			data																			1	if condition holds, dst-reg = dst-reg + signext(data)							
ADD.DW d-reg, d-reg, imm3s	A		0	0	0	0	0	0	0	dst			src			data																			1	dst-reg = src-reg + (signext(data) >> 24)	X	FΔ	FΔ	FΔ	FΔ	FΔ	
ADD.DW d-reg, imm3s, cond3	A		0	1	0	1	1	0	1	dst			cond			data																			1	if condition holds, dst-reg = dst-reg + (signext(data) >> 24)							
ADD.FR d-reg, d-reg, imm3s	A		0	1	1	0	1	1	0	dst			src			data																			1	dst-reg = src-reg + fractional(data)	X	FΔ	FΔ	FΔ	FΔ	FΔ	
ADD.NX d-reg, d-reg, d-reg	A		0	1	0	0	0	0	1	dst			src1			src2																			1	dst-reg = signext(src-reg-1 + src-reg-2)	X		FΔ	FΔ	F0		
ADD.NX a-reg, a-reg, a-reg	XA		1	1	0	0	0	0	d	0	s1	s2	-	-	-	1	-	-																	1	dst-reg = signext(src-reg-1 + src-reg-2)	X		FΔ	FΔ	F0		
ADD.NX a-reg, a-reg, b-reg			1	1	0	0	0	0	d	1	s1	s2	-	-	-	1	-	-																									
ADD.NX d-reg, d-reg, imm3s	A		0	0	1	0	0	0	1	dst			src			data																			1	dst-reg = signext(src-reg + signext(data))	X		FΔ	FΔ	F0		
ADD.NX.DW d-reg, d-reg, imm3s	A		0	0	0	0	0	0	1	dst			src			data																			1	dst-reg = signext(src-reg + (signext(data) >> 24))	X		FΔ	FΔ	F0		
ADDMUL ab-reg, x-reg, x-reg, y-reg	A		1	0	1	0	0	1	s1	s2	s3	0	0	0	1	0	dst																		2, 1A	dst-reg = 0 + src-reg-1 * (src-reg-2 + src-reg-3)	X		FΔ	FΔ	F0	FΔ	
ADDMUL ab-reg, x-reg, y-reg, y-reg			1	0	1	0	0	1	s1	s2	s3	0	1	0	1	0	dst																									
ADDMUL ab-reg, y-reg, x-reg, y-reg			1	0	1	0	0	1	s1	s2	s3	1	0	0	1	0	dst																									
ADDMUL ab-reg, y-reg, x-reg, x-reg			1	0	1	0	0	1	s1	s2	s3	1	1	0	1	0	dst																									
ADDMULADD ab-reg, x-reg, x-reg, y-reg	A		1	0	1	0	0	1	s1	s2	s3	0	0	0	0	0	dst																		2, 1A	dst-reg = dst-reg + src-reg-1 * (src-reg-2 + src-reg-3)	X	FΔ	FΔ	FΔ	FΔ	FΔ	
ADDMULADD ab-reg, x-reg, y-reg, y-reg			1	0	1	0	0	1	s1	s2	s3	0	1	0	0	0	dst																									
ADDMULADD ab-reg, y-reg, x-reg, y-reg			1	0	1	0	0	1	s1	s2	s3	1	0	0	0	0	dst																									
ADDMULADD ab-reg, y-reg, x-reg, x-reg			1	0	1	0	0	1	s1	s2	s3	1	1	0	0	0	dst																									

TABLE 46: INSTRUCTION SET SUMMARY (CONTINUED)

Instruction	T	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	#C	Operation	B	C	N	Z	O	E	S
ADDMULNEG ab-reg, x-reg, x-reg, y-reg	A		1	0	1	0	0	1	s1	s2	s3	0	0	0	1	1	dst																	2, 1A	dst-reg = 0 - src-reg-1 * (src-reg-2 + src-reg-3)	X		FΔ	FΔ	F0	FΔ	
ADDMULNEG ab-reg, x-reg, y-reg, y-reg			1	0	1	0	0	1	s1	s2	s3	0	1	0	1	1	dst																									
ADDMULNEG ab-reg, y-reg, x-reg, y-reg			1	0	1	0	0	1	s1	s2	s3	1	0	0	1	1	dst																									
ADDMULNEG ab-reg, y-reg, x-reg, x-reg			1	0	1	0	0	1	s1	s2	s3	1	1	0	1	1	dst																									
ADDMULSUB ab-reg, x-reg, x-reg, y-reg	A		1	0	1	0	0	1	s1	s2	s3	0	0	0	0	1	dst																	2, 1A	dst-reg = dst-reg - src-reg-1 * (src-reg-2 + src-reg-3)	X	FΔ	FΔ	FΔ	FΔ	FΔ	
ADDMULSUB ab-reg, x-reg, y-reg, y-reg			1	0	1	0	0	1	s1	s2	s3	0	1	0	0	1	dst																									
ADDMULSUB ab-reg, y-reg, x-reg, y-reg			1	0	1	0	0	1	s1	s2	s3	1	0	0	0	1	dst																									
ADDMULSUB ab-reg, y-reg, x-reg, x-reg			1	0	1	0	0	1	s1	s2	s3	1	1	0	0	1	dst																									
ADDSH d-reg, d-reg, d-reg	A		0	1	0	1	1	1	0	dst			src1			src2																	1	dst-reg = (src-reg-1 + src-reg-2) >> 1	X	FΔ	FΔ	FΔ	FΔ	FΔ	
ADDSH a-reg, a-reg, a-reg (Y side instruction is not MUL*)	XA		1	1	0	1	0	1	d	0	s1	s2	-	-	-	0	-	-															1	dst-reg = (src-reg-1 + src-reg-2) >> 1	X	FΔ	FΔ	FΔ	FΔ	FΔ	
ADDSH a-reg, a-reg, b-reg (Y side instruction is not MUL*)			1	1	0	1	0	1	d	1	s1	s2	-	-	-	0	-	-																									
ADDSH a-reg, a-reg, a-reg (Y side instruction is MUL*)			1	0	0	1	1	0	d	0	s1	s2	-	-	-	-	-	-																									
ADDSH a-reg, a-reg, b-reg (Y side instruction is MUL*)			1	0	0	1	1	0	d	1	s1	s2	-	-	-	-	-	-																									
ADDSH b-reg, b-reg, a-reg (X side instruction is not MUL*)	YA		1	1	0	-	-	-	-	1	-	-	1	0	d	-	s1	s2															1	dst-reg = (src-reg-1 + src-reg-2) >> 1	Y	FΔ	FΔ	FΔ	FΔ	FΔ	
ADDSH b-reg, b-reg, b-reg (X side instruction is not MUL*)			1	1	0	-	-	-	-	0	-	-	1	0	d	-	s1	s2																									
ADDSH b-reg, b-reg, a-reg (X side instruction is MUL*)			1	0	0	0	-	-	-	-	-	-	1	0	d	1	s1	s2																									
ADDSH b-reg, b-reg, b-reg (X side instruction is MUL*)			1	0	0	0	-	-	-	-	-	-	1	0	d	0	s1	s2																									
AND d-reg, d-reg, d-reg	A		0	1	0	1	0	0	0	dst			src1			src2																	1	dst-reg = src-reg-1 AND src-reg-2	X		FΔ	FΔ	F0	FΔ	

TABLE 46: INSTRUCTION SET SUMMARY (CONTINUED)

Instruction	T	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	#C	Operation	B	C	N	Z	O	E	S
AND a-reg, a-reg, a-reg	XA		1	1	0	1	0	0	d	0	s1	s2	-	-	-	0	-	-																1	dst-reg = src-reg-1 AND src-reg-2	X		FΔ	FΔ	F0	FΔ	
AND a-reg, a-reg, b-reg			1	1	0	1	0	0	d	1	s1	s2	-	-	-	0	-	-																									
AND d-reg, d-reg, imm3s	A		0	0	1	1	0	0	0	dst			src			data																		1	dst-reg = src-reg AND signext(data)	X		FΔ	FΔ	F0	FΔ	
AND.DW d-reg, d-reg, imm3s	A		0	0	0	1	0	0	0	dst			src			data																		1	dst-reg = src-reg AND (signext(data) >> 24)	X		FΔ	FΔ	F0	FΔ	
BITCLR SR, imm4u	L			1	1	1	1	0	0	1	1	0	1	1	0	0	0	0	0	0	0	0	1	1	0	0	0	0	0	index				1	SR = SR AND NOT(1 << index)							
BITCLR d-reg, bit-index	A		0	0	1	1	1	1	0	dst			index																					1	dst-reg = dst-reg AND NOT(1 << index)	X	FΔ					
BITSET SR, imm4u	L			1	1	1	1	0	0	1	1	0	1	1	0	0	0	0	0	0	0	0	1	0	1	0	0	0	0	index				1, 3I	SR = SR OR (1 << index)							
BITSET d-reg, bit-index	A		0	0	1	1	1	0	1	dst			index																					1	dst-reg = dst-reg OR (1 << index)	X	FΔ					
BITTGL d-reg, bit-index	A		0	0	1	1	1	1	1	dst			index																					1	dst_reg = dst-reg XOR (1 << index)	X	FΔ					
BITTST d-reg, bit-index	A		0	0	1	1	1	0	0	dst			index																					1	X Carry flag = (dst-reg >> index) AND 1	X	Δ					
BREAKPOINT	L			1	1	1	1	0	0	1	1	0	1	1	0	0	0	0	0	0	0	0	1	0	1	0	0	0	0	0	0	0	0	3	if software breakpoints are enabled, performs the following: • Halt the DSP • Enter debug mode							
CALL imm16u	S	synth																															2	LR = PC + 2 PC = addr							
CALL xp-reg	S	synth																															3	LR = PC + 2 PC = src-reg							
CALL.1D imm16u	L			1	1	1	0	1	1	1	0	0	0	0	0	0	0	addr																2, 1D	LR = PC + 2 PC = addr							
CALL.2D xp-reg	L			1	1	1	1	0	0	1	1	0	1	1	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	src			3, 2D	LR = PC + 2 or 3 PC = src-reg							
CLR X[single-mode(X)]	M																		1	1	1	1	1	1	0	0	1	addr					1	X[addr] = 0 update addr							
CMP d-reg, d-reg	A		0	1	0	0	0	1	0	0	0	0	src1			src2																		1	src-reg-1 - src-reg-2 update X ALU status flags	X	FΔ	Δ	Δ	Δ	Δ	
CMP a-reg, a-reg	XA		1	1	0	0	0	1	0	0	s1	s2	-	-	-	0	-	-																1	src-reg-1 - src-reg-2 update X ALU status flags	X	FΔ	Δ	Δ	Δ	Δ	
CMP a-reg, b-reg			1	1	0	0	0	1	0	1	s1	s2	-	-	-	0	-	-																									
CMP d-reg, imm3s	A		0	0	1	0	0	1	0	0	0	0	src			data																		1	src-reg - signext(data) update X ALU status flags	X	FΔ	Δ	Δ	Δ	Δ	
CMP.DW d-reg, imm3s	A		0	0	0	0	0	1	0	0	0	0	src			data																		1	src-reg - (signext(data) >> 24) update X ALU status flags	X	FΔ	Δ	Δ	Δ	Δ	

TABLE 46: INSTRUCTION SET SUMMARY (CONTINUED)

Instruction	T	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	#C	Operation	B	C	N	Z	O	E	S
CMPU.NX d-reg, d-reg	A		0	1	0	0	0	1	1	0	0	0	src1			src2																		1	signext(src-reg-1 - src-reg-2) update X ALU status flags	X	FΔ	Δ	Δ	0		
CMPU.NX a-reg, a-reg	XA		1	1	0	0	0	1	0	0	s1	s2	-	-	-	1	-	-																1	signext(src-reg-1 - src-reg-2) update X ALU status flags	X	FΔ	Δ	Δ	0		
CMPU.NX a-reg, b-reg			1	1	0	0	0	1	0	1	s1	s2	-	-	-	1	-	-																									
CMPU.NX d-reg, imm3s	A		0	0	1	0	0	1	1	0	0	0	src			data																		1	signext(src-reg - signext(data)) update X ALU status flags	X	FΔ	Δ	Δ	0		
CMPU.NX.DW d-reg, imm3s	A		0	0	0	0	0	1	1	0	0	0	src			data																		1	signext(src-reg - (signext(data) >> 24)) update X ALU status flags	X	FΔ	Δ	Δ	0		
DIVST d-reg, d-reg, d-reg	A		0	1	1	0	1	0	0	dst			src1			src2																		1	generate 1 quotient bit of src-reg-1 / src-reg-2	X		Δ				
ENDLOOP	L			1	1	1	1	0	0	1	1	0	1	1	0	0	0	0	0	0	0	1	0	0	1	0	0	0	0	0	0	0	1	Pop loop stack							
GOTO imm16u	L	synth																															2	PC = addr							
GOTO imm16u, cond5	L	synth																															3, 1N	if condition holds then PC = addr							
GOTO xp-reg	S	synth																															3	PC = src-reg							
GOTO xp-reg, cond5	S	synth																															3	if condition holds then PC = src-reg							
GOTO.0D imm16u	L			1	1	1	0	1	1	0	0	1	0	0	0	0	0	addr																2	PC = addr							
GOTO.0D imm16u, cond5	L			1	1	1	0	1	1	0	1	1	condition					addr																3, 1N	if condition holds then PC = addr							
GOTO.1D imm16u	L			1	1	1	0	1	1	0	0	0	0	0	0	0	0	addr																2, 1D	PC = addr							
GOTO.2D imm16u, cond5	L			1	1	1	0	1	1	0	1	0	condition					addr																3, 2D	if condition holds then PC = addr							
GOTO.2D xp-reg	L			1	1	1	1	0	0	1	1	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	addr			3, 2D	PC = src-reg							
GOTO.2D xp-reg, cond5	L			1	1	1	1	0	0	1	1	0	1	1	0	0	0	0	0	0	0	0	0	0	0	1	condition				addr			3, 2D	if condition holds then PC = src-reg							
INTERRUPT imm5u	L			1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	vector-num				2	ILPC = PC + 1 ILSR = SR PC = vector-table-base + vector-number							
LOAD reg, X[stack-mode16]	L			1	1	1	0	1	0	0	register					0	addr																	1	register = X[addr]							
LOAD stack-reg6, X[stack-mode6]	M																		1	0	1	register				0	addr						1	register = X[addr]							
LOAD stack-reg4, X[stack-mode4]	M																		1	1	1	1	1	1	1	reg			0	addr			1	register = X[addr]							
LOAD reg, X[imm16u]	L			1	1	1	0	0	0	0	register					0	addr																	1	register = X[addr]							
LOAD xmem-reg, X[single-mode(X)]	M																		1	0	0	register				0	addr						1	register = X[addr] update addr							
LOAD ax-reg, X[dual-mode(X)]	XM																		0	0	reg	addr				-	-	-	-	-	-	-	1	register = X[addr] update addr							

TABLE 46: INSTRUCTION SET SUMMARY (CONTINUED)

Instruction	T	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	#C	Operation	B	C	N	Z	O	E	S
LOAD reg, Y[imm16u]	L			1	1	1	0	0	0	1	register						0	addr																1	register = Y[addr]							
LOAD by-reg, Y[single-mode(Y)]	M																	1	1	1	0	0	1	0	reg		0	addr						1	register = Y[addr] update addr							
LOAD by-reg, Y[dual-mode(Y)]	YM																	0	-	-	-	-	-	-	-	0	reg		addr					1	register = Y[addr] update addr							
LOAD reg, P[imm16u]	L			1	1	1	0	0	1	1	register						0	addr																1	register = P[addr]							
LOAD reg, P[single-mode(X)]	L			0	1	1	0	0	0	1	register						0	0	0	0	0	0	0	0	0	0	0	addr						1	register = P[addr] update addr							
LOAD reg, P[single-mode(Y)]	L			0	1	1	0	0	0	1	register						0	0	0	0	0	0	0	0	0	0	1	0	addr						1	register = P[addr] update addr							
LOAD reg, imm20s	L			1	1	0	0	register				data																						1	register = signext(data)							
LOAD d-reg, imm24s	L			0	0	1	reg			data																								1	register = signext(data)							
LOAD d-reg, imm5s	M																	1	1	1	1	0	1	0	reg			data						1	register = signext(data)							
LOAD d-reg, imm6s	A		0	0	1	1	0	1	1	reg		data																						1	register = signext(data)							
LOAD l-reg, imm24s	L			0	1	0	0	reg		data																								1	register = data							
LOAD xstep-reg, imm5s	M																	1	1	1	1	0	1	1	reg			data						1	register = signext(data)							
LOAD ystep-reg, imm5s	M																	1	1	1	1	1	0	0	0	reg		data						1	register = signext(data)							
LOAD ax-reg, imm2s	XM																	0	1	1	reg		dat		0	-	-	-	-	-	-	-	1	register = signext(data)							
LOAD by-reg, imm2s	YM																	0	-	-	-	-	-	-	-	1	1	reg		dat		0		1	register = signext(data)							
LOAD.DW ab-reg, XY[imm16u]	L			1	1	1	0	0	1	0	0	0	0	0	0	acc	0	addr																1	accumulator.e = 0xFF * Y[addr][23] accumulator.h = Y[addr] accumulator.l = X[addr]							
LOAD.DW ab-reg, XY[single-mode(X)]	M																	1	1	1	0	0	0	acc	0	addr								1	accumulator.e = 0xFF * Y[addr][23] accumulator.h = Y[addr] accumulator.l = X[addr] update addr							
LOAD.DW ab-reg, imm24s	L			0	1	0	1	acc		data																								1	accumulator.e = 0xFF * data[23] accumulator.h = 0xFFFFFF * data[23] accumulator.l = data							
LOAD.DW ab-reg, imm5s	M																	1	1	1	1	1	0	0	1	acc		data						1	accumulator.e = 0xFF * data[4] accumulator.h = 0xFFFFFF * data[4] accumulator.l = signext(data)							
LOAD.DW d-reg, imm6s	A		0	0	0	1	0	1	1	reg		data																						1	register = signext(data) >> 24							
LOAD.EX ab-reg, P[imm16u]	L			1	1	1	0	0	1	0	1	0	0	0	acc	0	addr																	1	accumulator.e = P[addr][31..24] accumulator.h = P[addr][23..0] accumulator.l = 0							

TABLE 46: INSTRUCTION SET SUMMARY (CONTINUED)

Instruction	T	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	#C	Operation	B	C	N	Z	O	E	S
LOAD.EX ab-reg, P[single-mode(X)]	L			0	1	1	0	0	0	0	0	0	0	0	acc		0	0	0	0	0	0	0	0	0	0	addr							1	accumulator.e = P[addr][31..24] accumulator.h = P[addr][23..0] accumulator.l = 0 update addr							
LOAD.EX ab-reg, P[single-mode(Y)]	M																		1	1	1	0	0	1	1	acc		0	addr				1	accumulator.e = P[addr][31..24] accumulator.h = P[addr][23..0] accumulator.l = 0 update addr							
LOG2ABS d-reg, d-reg	A		0	1	1	0	1	1	1	dst			0	0	0	src																		1	dst-reg = log2(|src-reg|)	X		FΔ	FΔ	F0		
LOOP imm12u, imm16u	S	synth																																3	push (PC+2, loop-end-addr, count) on the loop stack							
LOOP d-reg, imm16u	S	synth																																3	push (PC+2, loop-end-addr, count) on the loop stack							
LOOP.2D imm12u, imm16u	L			1	0	count											loop-end-addr																	3, 2D	push (PC+2/3, loop-end-addr, count) on the loop stack							
LOOP.2D d-reg, imm16u	L			1	1	1	1	0	0	1	1	0	0	0	count		loop-end-addr																		3, 2D	push (PC+2/3, loop-end-addr, count) on the loop stack							
MAX d-reg, d-reg, d-reg	A		0	1	1	0	0	1	0	dst			src1			src2																		1	dst-reg = max(src-reg-1, src-reg-2)	X	FΔ	FΔ	FΔ	FΔ	FΔ	
MAX a-reg, a-reg, a-reg	XA		1	1	0	0	1	1	d	0	s1	s2	-	-	-	0	-	-																1	dst-reg = max(src-reg-1, src-reg-2)	X	FΔ	FΔ	FΔ	FΔ	FΔ	
MAX a-reg, a-reg, b-reg			1	1	0	0	1	1	d	1	s1	s2	-	-	-	0	-	-																									
MIN d-reg, d-reg, d-reg	A		0	1	1	0	0	1	1	dst			src1			src2																		1	dst-reg = min(src-reg-1, src-reg-2)	X	FΔ	FΔ	FΔ	FΔ	FΔ	
MIN a-reg, a-reg, a-reg	XA		1	1	0	0	1	1	d	0	s1	s2	-	-	-	1	-	-																1	dst-reg = min(src-reg-1, src-reg-2)	X	FΔ	FΔ	FΔ	FΔ	FΔ	
MIN a-reg, a-reg, b-reg			1	1	0	0	1	1	d	1	s1	s2	-	-	-	1	-	-																									
MOVE reg, reg	M																		1	1	0	dst-reg					src-reg							1	dst-reg = src-reg							Δ
MOVE a-reg, x-reg	XM																		0	1	1	d	s	0	1	1	-	-	-	-	-	-	-	1	dst-reg.e = 0xFF * src-reg[23]							
MOVE b-reg, x-reg																			0	1	1	d	s	1	1	1	-	-	-	-	-	-	-		dst-reg.h = src-reg dst-reg.l = 0							
MOVE x-reg, a-reg	XM																		0	1	1	d	s	0	0	1	-	-	-	-	-	-	-	1	dst-reg = src-reg.h							Δ
MOVE y-reg, a-reg																			0	1	1	d	s	1	0	1	-	-	-	-	-	-	-										
MOVE a-reg, y-reg	YM																		0	-	-	-	-	-	-	-	1	1	d	s	1	1	1	1	dst-reg.e = 0xFF * src-reg[23]							
MOVE b-reg, y-reg																			0	-	-	-	-	-	-	-	1	1	d	s	0	1	1		dst-reg.h = src-reg dst-reg.l = 0							
MOVE x-reg, b-reg	YM																		0	-	-	-	-	-	-	-	1	1	d	s	1	0	1	1	dst-reg = src-reg.h							Δ
MOVE y-reg, b-reg																			0	-	-	-	-	-	-	-	1	1	d	s	0	0	1										
MOVE d-reg, d-reg	A	synth																																1	dst-reg = src-reg	X		FΔ	FΔ	F0	FΔ	

TABLE 46: INSTRUCTION SET SUMMARY (CONTINUED)

Instruction	T	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	#C	Operation	B	C	N	Z	O	E	S
MOVE d-reg, d-reg, cond3	A		0	1	1	1	1	0	1	dst			cond			src																			1	if condition holds, dst-reg = src-reg							
MOVE.DW ab-reg, ab-reg	M																		1	1	1	1	1	1	0	1	0	0	0	dst		src		1	dst-reg.e = 0xFF * src-reg.h[23] dst-reg.h = src-reg.h dst-reg.l = src-reg.l							Δ
MOVE.EX ab-reg, ab-reg	M																		1	1	1	1	1	1	0	1	0	0	1	dst		src		1	dst-reg.e = src-reg.e dst-reg.h = src-reg.h dst-reg.l = 0							
MOVE.NX d-reg, d-reg	A		0	1	1	1	0	1	1	dst			0	0	0	src																			1	dst-reg.e = 0xFF * src-reg.h[23] dst-reg.h = src-reg.h dst-reg.l = src-reg.l							
MOVE.NX a-reg, a-reg	XA		1	1	0	1	1	1	d	0	0	s	-	-	-	1	-	-																1	dst-reg.e = 0xFF * src-reg.h[23] dst-reg.h = src-reg.h dst-reg.l = src-reg.l							
MOVE.NX a-reg, b-reg			1	1	0	1	1	1	d	1	0	s	-	-	-	1	-	-																									
MUL ab-reg, xy-reg, xy-reg	A		1	0	1	0	0	0	0	0	src 1		src 2		1	0	dst																		2, 1A	dst-reg = 0 + src-reg-1 * src-reg-2	X		FΔ	FΔ	F0	FΔ	
MUL a-reg, x-reg, x-reg (Y side instruction is not MUL*)	XA		1	0	0	0	0	s 1	s 2	1	0	d	-	-	-	-	-	-																	2, 1A	dst-reg = 0 + src-reg-1 * src-reg-2	X		FΔ	FΔ	F0	FΔ	
MUL a-reg, x-reg, y-reg (Y side instruction is not MUL*)			1	0	0	0	1	s 1	s 2	1	0	d	-	-	-	-	-	-																									
MUL a-reg, x-reg, x-reg (Y side instruction is MUL*)			1	0	1	1	1	0	d	0	s 1	s 2	-	-	-	0	-	-																									
MUL a-reg, x-reg, y-reg (Y side instruction is MUL*)			1	0	1	1	1	0	d	1	s 1	s 2	-	-	-	0	-	-																									
MUL b-reg, y-reg, x-reg (X side instruction is not MUL* or NOP)	YA		1	0	0	1	-	-	-	-	-	-	1	s 1	s 2	1	0	d																	2, 1A	dst-reg = 0 + src-reg-1 * src-reg-2	Y		FΔ	FΔ	F0	FΔ	
MUL b-reg, y-reg, y-reg (X side instruction is not MUL* or NOP)			1	0	0	1	-	-	-	-	-	-	0	s 1	s 2	1	0	d																									
MUL b-reg, y-reg, x-reg (X side instruction is MUL* or NOP)			1	0	1	1	-	-	-	1	-	-	1	0	d	-	s 1	s 2																									
MUL b-reg, y-reg, y-reg (X side instruction is MUL* or NOP)			1	0	1	1	-	-	-	0	-	-	1	0	d	-	s 1	s 2																									
MUL.SU ab-reg, xy-reg, xy-reg	A		1	0	1	0	0	0	1	0	src 1		src 2		1	0	dst																		2, 1A	dst-reg = 0 + src-reg-1 * src-reg-2	X		FΔ	FΔ	F0	FΔ	

TABLE 46: INSTRUCTION SET SUMMARY (CONTINUED)

Instruction	T	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	#C	Operation	B	C	N	Z	O	E	S
MUL.UU ab-reg, xy-reg, xy-reg	A		1	0	1	0	0	0	0	1	src 1		src 2		1	0	dst																	2, 1A	dst-reg = 0 + src-reg-1 * src-reg-2	X		FΔ	FΔ	F0	FΔ	
MULADD ab-reg, xy-reg, xy-reg	A		1	0	1	0	0	0	0	0	src 1		src 2		0	0	dst																	2, 1A	dst-reg = dst-reg + src-reg-1 * src-reg-2	X	FΔ	FΔ	FΔ	FΔ	FΔ	
MULADD a-reg, x-reg, x-reg (Y side instruction is not MUL*)	XA		1	0	0	0	0	s 1	s 2	0	0	d	-	-	-	-	-																2, 1A	dst-reg = dst-reg + src-reg-1 * src-reg-2	X	FΔ	FΔ	FΔ	FΔ	FΔ	
MULADD a-reg, x-reg, y-reg (Y side instruction is not MUL*)			1	0	0	0	1	s 1	s 2	0	0	d	-	-	-	-	-																									
MULADD a-reg, x-reg, x-reg (Y side instruction is MUL*)			1	0	1	1	0	0	d	0	s 1	s 2	-	-	-	0	-	-																									
MULADD a-reg, x-reg, y-reg (Y side instruction is MUL*)			1	0	1	1	0	0	d	1	s 1	s 2	-	-	-	0	-	-																									
MULADD b-reg, y-reg, x-reg (X side instruction is not MUL* or NOP)	YA		1	0	0	1	-	-	-	-	-	-	1	s 1	s 2	0	0	d																2, 1A	dst-reg = dst-reg + src-reg-1 * src-reg-2	Y	FΔ	FΔ	FΔ	FΔ	FΔ	
MULADD b-reg, y-reg, y-reg (X side instruction is not MUL* or NOP)			1	0	0	1	-	-	-	-	-	-	0	s 1	s 2	0	0	d																									
MULADD b-reg, y-reg, x-reg (X side instruction is MUL* or NOP)			1	0	1	1	-	-	-	1	-	-	0	0	d	-	s 1	s 2																									
MULADD b-reg, y-reg, y-reg (X side instruction is MUL* or NOP)			1	0	1	1	-	-	-	0	-	-	0	0	d	-	s 1	s 2																									
MULADD.SU ab-reg, xy-reg, xy-reg	A		1	0	1	0	0	0	1	0	src 1		src 2		0	0	dst																	2, 1A	dst-reg = dst-reg + src-reg-1 * src-reg-2	X	FΔ	FΔ	FΔ	FΔ	FΔ	
MULADD.UU ab-reg, xy-reg, xy-reg	A		1	0	1	0	0	0	0	1	src 1		src 2		0	0	dst																	2, 1A	dst-reg = dst-reg + src-reg-1 * src-reg-2	X	FΔ	FΔ	FΔ	FΔ	FΔ	
MULNEG ab-reg, xy-reg, xy-reg	A		1	0	1	0	0	0	0	0	src 1		src 2		1	1	dst																	2, 1A	dst-reg = 0 - src-reg-1 * src-reg-2	X		FΔ	FΔ	F0	FΔ	

TABLE 46: INSTRUCTION SET SUMMARY (CONTINUED)

Instruction	T	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	#C	Operation	B	C	N	Z	O	E	S
MULNEG a-reg, x-reg, x-reg (Y side instruction is not MUL*)	XA		1	0	0	0	0	s1	s2	1	1	d	-	-	-	-	-	-																2, 1A	dst-reg = 0 - src-reg-1 * src-reg-2	X		FΔ	FΔ	F0	FΔ	
MULNEG a-reg, x-reg, y-reg (Y side instruction is not MUL*)			1	0	0	0	1	s1	s2	1	1	d	-	-	-	-	-	-																									
MULNEG a-reg, x-reg, x-reg (Y side instruction is MUL*)			1	0	1	1	1	1	d	0	s1	s2	-	-	-	0	-	-																									
MULNEG a-reg, x-reg, y-reg (Y side instruction is MUL*)			1	0	1	1	1	1	d	1	s1	s2	-	-	-	0	-	-																									
MULNEG b-reg, y-reg, x-reg (X side instruction is not MUL* or NOP)	YA		1	0	0	1	-	-	-	-	-	-	1	s1	s2	1	1	d																	2, 1A	dst-reg = 0 - src-reg-1 * src-reg-2	Y		FΔ	FΔ	F0	FΔ	
MULNEG b-reg, y-reg, y-reg (X side instruction is not MUL* or NOP)			1	0	0	1	-	-	-	-	-	-	0	s1	s2	1	1	d																									
MULNEG b-reg, y-reg, x-reg (X side instruction is MUL* or NOP)			1	0	1	1	-	-	-	1	-	-	1	1	d	-	s1	s2																									
MULNEG b-reg, y-reg, y-reg (X side instruction is MUL* or NOP)			1	0	1	1	-	-	-	0	-	-	1	1	d	-	s1	s2																									
MULNEG.SU ab-reg, xy-reg, xy-reg	A		1	0	1	0	0	0	1	0	src1		src2			1	1	dst																2, 1A	dst-reg = 0 - src-reg-1 * src-reg-2	X		FΔ	FΔ	F0	FΔ	
MULNEG.UU ab-reg, xy-reg, xy-reg	A		1	0	1	0	0	0	0	1	src1		src2			1	1	dst																2, 1A	dst-reg = 0 - src-reg-1 * src-reg-2	X		FΔ	FΔ	F0	FΔ	
MULSUB ab-reg, xy-reg, xy-reg	A		1	0	1	0	0	0	0	0	src1		src2			0	1	dst																2, 1A	dst-reg = dst-reg - src-reg-1 * src-reg-2	X	FΔ	FΔ	FΔ	FΔ	FΔ	
MULSUB a-reg, x-reg, x-reg (Y side instruction is not MUL*)	XA		1	0	0	0	0	s1	s2	0	1	d	-	-	-	-	-	-																2, 1A	dst-reg = dst-reg - src-reg-1 * src-reg-2	X	FΔ	FΔ	FΔ	FΔ	FΔ	
MULSUB a-reg, x-reg, y-reg (Y side instruction is not MUL*)			1	0	0	0	1	s1	s2	0	1	d	-	-	-	-	-	-																									
MULSUB a-reg, x-reg, x-reg (Y side instruction is MUL*)			1	0	1	1	0	1	d	0	s1	s2	-	-	-	0	-	-																									
MULSUB a-reg, x-reg, y-reg (Y side instruction is MUL*)			1	0	1	1	0	1	d	1	s1	s2	-	-	-	0	-	-																									

TABLE 46: INSTRUCTION SET SUMMARY (CONTINUED)

Instruction	T	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	#C	Operation	B	C	N	Z	O	E	S
MULSUB b-reg, y-reg, x-reg (X side instruction is not MUL* or NOP)	YA		1	0	0	1	-	-	-	-	-	-	1	s1	s2	0	1	d																2, 1A	dst-reg = dst-reg - src-reg-1 * src-reg-2	Y	FΔ	FΔ	FΔ	FΔ	FΔ	
MULSUB b-reg, y-reg, y-reg (X side instruction is not MUL* or NOP)			1	0	0	1	-	-	-	-	-	-	0	s1	s2	0	1	d																									
MULSUB b-reg, y-reg, x-reg (X side instruction is MUL* or NOP)			1	0	1	1	-	-	-	1	-	-	0	1	d	-	s1	s2																									
MULSUB b-reg, y-reg, y-reg (X side instruction is MUL* or NOP)			1	0	1	1	-	-	-	0	-	-	0	1	d	-	s1	s2																									
MULSUB.SU ab-reg, xy-reg, xy-reg	A		1	0	1	0	0	0	1	0	src1		src2			0	1	dst																2, 1A	dst-reg = dst-reg - src-reg-1 * src-reg-2	X	FΔ	FΔ	FΔ	FΔ	FΔ	
MULSUB.UU ab-reg, xy-reg, xy-reg	A		1	0	1	0	0	0	0	1	src1		src2			0	1	dst																2, 1A	dst-reg = dst-reg - src-reg-1 * src-reg-2	X	FΔ	FΔ	FΔ	FΔ	FΔ	
NEG d-reg, d-reg	A		0	1	1	1	0	0	0	dst			0	0	0	src																		1	dst-reg = -src-reg	X		FΔ	FΔ	FΔ	FΔ	
NEG a-reg, a-reg	XA		1	1	0	1	1	0	d	0	0	s	-	-	-	0	-	-																1	dst-reg = -src-reg	X		FΔ	FΔ	FΔ	FΔ	
NEG a-reg, b-reg			1	1	0	1	1	0	d	1	0	s	-	-	-	0	-	-																									
NEG d-reg, d-reg, cond3	A		0	1	1	1	1	1	1	dst			cond			src																		1	if condition holds, dst-reg = -src-reg							
NLOG2ABS d-reg, d-reg	A		0	1	1	1	0	0	1	dst			0	0	0	src																		1	dst-reg = -log2(|src-reg|)	X		FΔ	FΔ	F0		
NLOG2ABS a-reg, a-reg	XA		1	1	0	1	1	0	d	0	0	s	-	-	-	1	-	-																1	dst-reg = -log2(|src-reg|)	X		FΔ	FΔ	F0		
NLOG2ABS a-reg, b-reg			1	1	0	1	1	0	d	1	0	s	-	-	-	1	-	-																									
NOP	L			0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	no operation							
NOP	M																		1	1	1	1	1	1	0	1	0	1	0	1	1	0	0	1	no operation							
NOP	A		0	1	1	0	1	0	1	0	0	0	0	0	0	0	0	0																1	no operation							
NOP	XA		1	0	1	1	0	0	0	-	0	0	-	-	-	1	-	-																1	no operation							
NOT d-reg, d-reg	A	synth																																1	dst-reg = NOT src-reg	X		FΔ	FΔ	F0	FΔ	
OR d-reg, d-reg, d-reg	A		0	1	0	1	0	0	1	dst			src1			src2																		1	dst-reg = src-reg-1 OR src-reg-2	X		FΔ	FΔ	F0	FΔ	
OR a-reg, a-reg, a-reg	XA		1	1	0	1	0	0	d	0	s1	s2	-	-	-	1	-	-																1	dst-reg = src-reg-1 OR src-reg-2	X		FΔ	FΔ	F0	FΔ	
OR a-reg, a-reg, b-reg			1	1	0	1	0	0	d	1	s1	s2	-	-	-	1	-	-																									
OR d-reg, d-reg, imm3s	A		0	0	1	1	0	0	1	dst			src			data																		1	dst-reg = src-reg OR signext(data)	X		FΔ	FΔ	F0	FΔ	
OR.DW d-reg, d-reg, imm3s	A		0	0	0	1	0	0	1	dst			src			data																		1	dst-reg = src-reg OR (signext(data) >> 24)	X		FΔ	FΔ	F0	FΔ	
RETURN	S	synth																																3	PC = LR							
RETURN.2D	L			1	1	1	1	0	0	1	1	0	1	1	0	0	0	0	0	0	0	0	0	1	1	0	0	0	0	0	0	0	0	3, 2D	PC = LR							

TABLE 46: INSTRUCTION SET SUMMARY (CONTINUED)

Instruction	T	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	#C	Operation	B	C	N	Z	O	E	S
RETURNI	L	synth																																3	SR = ILSR PC = ILPC							
RETURNI.0D	L			1	1	1	1	0	0	1	1	0	1	1	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	3	SR = ILSR PC = ILPC							
RETURNI.2D	L			1	1	1	1	0	0	1	1	0	1	1	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	3, 2D	SR = ILSR PC = ILPC							
SHLL d-reg, d-reg, d-reg	A		0	1	0	0	1	0	0	dst			src1			src2																		1	if src-reg-2 ≥ 0, dst-reg = src-reg-1 << src-reg-2 if src-reg-2 < 0, dst-reg = src-reg-1 >> |src-reg-2|	X	FΔ	FΔ	FΔ	FΔ	FΔ	
SHLL d-reg, d-reg, imm3u	A		0	0	1	0	1	0	0	dst			src			data																		1	dst-reg = src-reg << data	X	FΔ	FΔ	FΔ	FΔ	FΔ	
SHLL d-reg, imm6u	A		0	0	0	0	1	0	0	dst			data																					1	dst-reg = dst-reg << data	X	FΔ	FΔ	FΔ	FΔ	FΔ	
SHLL d-reg, d-reg, cond3	A		0	0	0	1	1	0	0	dst			cond			src																		1	if condition holds, if src-reg ≥ 0, dst-reg = dst-reg << src-reg if src-reg < 0, dst-reg = dst-reg >> |src-reg|							
SHLL d-reg, imm3u, cond3	A		0	0	0	1	1	0	1	dst			cond			data																		1	if condition holds, dst-reg = dst-reg << data							
SHLL.NX d-reg, d-reg, d-reg	A		0	1	0	0	1	0	1	dst			src1			src2																		1	if src-reg-2 ≥ 0, dst-reg = signext(src-reg-1 << src-reg-2) if src-reg-2 < 0, dst-reg = signext(src-reg-1 >> |src-reg-2|)	X		FΔ	FΔ	F0		
SHLL.NX d-reg, d-reg, imm3u	A		0	0	1	0	1	0	1	dst			src			data																		1	dst-reg = signext(src-reg << data)	X		FΔ	FΔ	F0		
SHLL.NX d-reg, imm6u	A		0	0	0	0	1	0	1	dst			data																					1	dst-reg = signext(dst-reg << data)	X		FΔ	FΔ	F0		
SHRA d-reg, d-reg, d-reg	A		0	1	0	0	1	1	0	dst			src1			src2																		1	if src-reg-2 ≥ 0, dst-reg = src-reg-1 >> src-reg-2 if src-reg-2 < 0, dst-reg = src-reg-1 << |src-reg-2|	X	FΔ	FΔ	FΔ	FΔ	FΔ	
SHRA d-reg, d-reg, imm3u	A		0	0	1	0	1	1	0	dst			src			data																		1	dst-reg = src-reg >> data	X	FΔ	FΔ	FΔ	F0	FΔ	
SHRA d-reg, imm6u	A		0	0	0	0	1	1	0	dst			data																					1	dst-reg = dst-reg >> data	X	FΔ	FΔ	FΔ	F0	FΔ	
SHRA d-reg, d-reg, cond3	A		0	0	0	1	1	1	0	dst			cond			src																		1	if condition holds, if src-reg ≥ 0, dst-reg = dst-reg >> src-reg if src-reg < 0, dst-reg = dst-reg << |src-reg|							
SHRA d-reg, imm3u, cond3	A		0	0	0	1	1	1	1	dst			cond			data																		1	if condition holds, dst-reg = dst-reg >> data							
SHRL.NX d-reg, d-reg, d-reg	A		0	1	0	0	1	1	1	dst			src1			src2																		1	if src-reg-2 ≥ 0, dst-reg = signext(src-reg-1 >> src-reg-2) if src-reg-2 < 0, dst-reg = signext(src-reg-1 << |src-reg-2|)	X		FΔ	FΔ	F0		

TABLE 46: INSTRUCTION SET SUMMARY (CONTINUED)

Instruction	T	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	#C	Operation	B	C	N	Z	O	E	S
SHRL.NX d-reg, d-reg, imm3u	A		0	0	1	0	1	1	1	dst			src			data																			1	dst-reg = signext(src-reg >> data)	X		FΔ	FΔ	F0		
SHRL.NX d-reg, imm6u	A		0	0	0	0	1	1	1	dst			data																						1	dst-reg = signext(dst-reg >> data)	X		FΔ	FΔ	F0		
SLEEP	L			1	1	1	1	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	3	enter Sleep mode wait for enabled interrupt							
SLEEP imm16u	L			1	1	1	1	0	0	0	1	0	0	0	0	0	0	restart-addr																	3	enter Sleep mode wait for enabled interrupt PC = restart-addr							
STORE X[stack-mode16], reg	L			1	1	1	0	1	0	0	register						1	addr																	1	X[addr] = register							Δ
STORE X[stack-mode6], stack-reg6	M																	1	0	1	register						1	addr						1	X[addr] = register							Δ
STORE X[stack-mode4], stack-reg4	M																	1	1	1	1	1	1	1	reg				1	addr				1	X[addr] = register							
STORE X[imm16u], reg	L			1	1	1	0	0	0	0	register						1	addr																	1	X[addr] = register							Δ
STORE X[single-mode(X)], xmem-reg	M																	1	0	0	register						1	addr						1	X[addr] = register update addr							Δ
STORE X[dual-mode(X)], a-reg	XM																	0	1	0	a	addr			-	-	-	-	-	-	-			1	X[addr] = accumulator update addr							Δ
STORE Y[imm16u], reg	L			1	1	1	0	0	0	1	register						1	addr																	1	Y[addr] = register							Δ
STORE Y[single-mode(Y)], by-reg	M																	1	1	1	0	0	1	0	reg				1	addr				1	Y[addr] = register update addr							Δ
STORE Y[dual-mode(Y)], b-reg	YM																	0	-	-	-	-	-	-	-	1	0	a	addr					1	Y[addr] = accumulator update addr							Δ
STORE P[imm16u], reg	L			1	1	1	0	0	1	1	register						1	addr																	1	P[addr] = signext(register)							Δ
STORE P[single-mode(X)], reg	L			0	1	1	0	0	0	1	register						1	0	0	0	0	0	0	0	0	0	addr								1	P[addr] = signext(register) update addr							Δ
STORE P[single-mode(Y)], reg	L			0	1	1	0	0	0	1	register						1	0	0	0	0	0	0	0	0	1	0	addr							1	P[addr] = signext(register) update addr							Δ
STORE X[imm16u], imm8s	L			1	1	0	1	0	0	data								addr																	1	X[addr] = signext(data)							
STORE Y[imm16u], imm8s	L			1	1	0	1	0	1	data								addr																	1	Y[addr] = signext(data)							
STORE P[imm16u], imm8s	L			1	1	0	1	1	1	data								addr																	1	P[addr] = signext(data)							
STORE.DW XY[imm16u], ab-reg	L			1	1	1	0	0	1	0	0	0	0	0	acc		1	addr																	1	Y[addr] = accumulator.h X[addr] = accumulator.l							Δ
STORE.DW XY[single-mode(X)], ab-reg	M																	1	1	1	0	0	0	acc		1	addr							1	Y[addr] = accumulator.h X[addr] = accumulator.l update addr							Δ

TABLE 46: INSTRUCTION SET SUMMARY (CONTINUED)

Instruction	T	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	#C	Operation	B	C	N	Z	O	E	S
STORE.DW XY[imm16u], imm8s	L			1	1	0	1	1	0	data								addr																1	Y[addr] = 0xFFFFFF * data[7] X[addr] = signext(data)							
STORE.EX P[imm16u], ab-reg	L			1	1	1	0	0	1	0	1	0	0	0	acc		1	addr																1	P[addr] = accumulator.e <<24 | accumulator.h							
STORE.EX P[single-mode(X)], ab-reg	L			0	1	1	0	0	0	0	0	0	0	0	acc		1	0	0	0	0	0	0	0	0	0	0	addr						1	P[addr] = accumulator.e <<24 | accumulator.h update addr							
STORE.EX P[single-mode(Y)], ab-reg	M																	1	1	1	0	0	1	1	acc		1	addr						1	P[addr] = accumulator.e <<24 | accumulator.h update addr							
SUB SP, imm16	L	synth																																1	SP = SP - data							
SUB SP, imm7sn	M	synth																																1	SP = SP - signext(data)							
SUB xp-reg, SP, imm16	L	synth																																1	dst-reg = SP - data							
SUB xp-reg, SP, imm6p	M	synth																																1	dst-reg = SP - data							
SUB xp-reg, xp-reg, imm16	L	synth																																1	dst-reg = src-reg - data							
SUB xp-reg, xp-reg, imm3sn	M	synth																																1	dst-reg = src-reg - signext(data)							
SUB yp-reg, yp-reg, imm3sn	M	synth																																1	dst-reg = src-reg - signext(data)							
SUB d-reg, d-reg, d-reg	A		0	1	1	0	0	0	0	dst			src1			src2																	1	dst-reg = src-reg-1 - src-reg-2	X	FΔ	FΔ	FΔ	FΔ	FΔ	
SUB a-reg, a-reg, a-reg (Y side instruction is not MUL*)	XA		1	1	0	0	1	0	d	0	s1	s2	-	-	-	0	-	-															1	dst-reg = src-reg-1 - src-reg-2	X	FΔ	FΔ	FΔ	FΔ	FΔ	
SUB a-reg, a-reg, b-reg (Y side instruction is not MUL*)			1	1	0	0	1	0	d	1	s1	s2	-	-	-	0	-	-																									
SUB a-reg, a-reg, a-reg (Y side instruction is MUL*)			1	0	0	1	0	1	d	0	s1	s2	-	-	-	-	-	-																									
SUB a-reg, a-reg, b-reg (Y side instruction is MUL*)			1	0	0	1	0	1	d	1	s1	s2	-	-	-	-	-	-																									
SUB b-reg, b-reg, a-reg (X side instruction is not MUL*)	YA		1	1	0	-	-	-	-	1	-	-	0	1	d	-	s1	s2																1	dst-reg = src-reg-1 - src-reg-2	Y	FΔ	FΔ	FΔ	FΔ	FΔ	
SUB b-reg, b-reg, b-reg (X side instruction is not MUL*)			1	1	0	-	-	-	-	0	-	-	0	1	d	-	s1	s2																									
SUB b-reg, b-reg, a-reg (X side instruction is MUL*)			1	0	0	0	-	-	-	-	-	-	0	1	d	1	s1	s2																									
SUB b-reg, b-reg, b-reg (X side instruction is MUL*)			1	0	0	0	-	-	-	-	-	-	0	1	d	0	s1	s2																									

TABLE 46: INSTRUCTION SET SUMMARY (CONTINUED)

Instruction	T	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	#C	Operation	B	C	N	Z	O	E	S
SUB d-reg, d-reg, imm3sn	A	synth																																1	dst-reg = src-reg - signext(data)	X	FΔ	FΔ	FΔ	FΔ	FΔ	
SUB d-reg, d-reg, cond3	A		0	1	1	1	1	0	0	dst			cond			src																		1	if condition holds, dst-reg = dst-reg - src-reg							
SUB d-reg, imm3sn, cond3	A	synth																																1	if condition holds, dst-reg = dst-reg - signext(data)							
SUB.DW d-reg, d-reg, imm3sn	A	synth																																1	dst-reg = src-reg - (signext(data) >> 24)	X	FΔ	FΔ	FΔ	FΔ	FΔ	
SUB.DW d-reg, imm3sn, cond3	A	synth																																1	if condition holds, dst-reg = dst-reg - (signext(data) >> 24)							
SUB.FR d-reg, d-reg, imm3s	A	synth																																1	dst-reg = src-reg - fractional(data)	X	FΔ	FΔ	FΔ	FΔ	FΔ	
SUB.NX d-reg, d-reg, d-reg	A		0	1	1	0	0	0	1	dst			src1			src2																		1	dst-reg = signext(src-reg-1 - src-reg-2)	X		FΔ	FΔ	F0		
SUB.NX a-reg, a-reg, a-reg	XA		1	1	0	0	1	0	d	0	s1	s2	-	-	-	1	-	-																1	dst-reg = signext(src-reg-1 - src-reg-2)	X		FΔ	FΔ	F0		
SUB.NX a-reg, a-reg, b-reg			1	1	0	0	1	0	d	1	s1	s2	-	-	-	1	-	-																									
SUB.NX d-reg, d-reg, imm3sn	A	synth																																1	dst-reg = signext(src-reg - signext(data))	X		FΔ	FΔ	F0		
SUB.NX.DW d-reg, d-reg, imm3sn	A	synth																																1	dst-reg = signext(src-reg - (signext(data) >> 24))	X		FΔ	FΔ	F0		
SUBMUL ab-reg, x-reg, x-reg, y-reg	A		1	0	1	0	0	1	s1	s2	s3	0	0	1	1	0	dst																	2, 1A	dst-reg = 0 + src-reg-1 * (src-reg-2 - src-reg-3)	X		FΔ	FΔ	F0	FΔ	
SUBMUL ab-reg, x-reg, y-reg, y-reg			1	0	1	0	0	1	s1	s2	s3	0	1	1	1	0	dst																									
SUBMUL ab-reg, y-reg, x-reg, y-reg			1	0	1	0	0	1	s1	s2	s3	1	0	1	1	0	dst																									
SUBMUL ab-reg, y-reg, x-reg, x-reg			1	0	1	0	0	1	s1	s2	s3	1	1	1	1	0	dst																									
SUBMULADD ab-reg, x-reg, x-reg, y-reg	A		1	0	1	0	0	1	s1	s2	s3	0	0	1	0	0	dst																	2, 1A	dst-reg = dst-reg + src-reg-1 * (src-reg-2 - src-reg-3)	X	FΔ	FΔ	FΔ	FΔ	FΔ	
SUBMULADD ab-reg, x-reg, y-reg, y-reg			1	0	1	0	0	1	s1	s2	s3	0	1	1	0	0	dst																									
SUBMULADD ab-reg, y-reg, x-reg, y-reg			1	0	1	0	0	1	s1	s2	s3	1	0	1	0	0	dst																									
SUBMULADD ab-reg, y-reg, x-reg, x-reg			1	0	1	0	0	1	s1	s2	s3	1	1	1	0	0	dst																									

TABLE 46: INSTRUCTION SET SUMMARY (CONTINUED)

Instruction	T	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	#C	Operation	B	C	N	Z	O	E	S
SUBMULNEG ab-reg, x-reg, x-reg, y-reg	A		1	0	1	0	0	1	s1	s2	s3	0	0	1	1	1	dst																	2, 1A	dst-reg = 0 - src-reg-1 * (src-reg-2 - src-reg-3)	X		FΔ	FΔ	F0	FΔ	
SUBMULNEG ab-reg, x-reg, y-reg, y-reg			1	0	1	0	0	1	s1	s2	s3	0	1	1	1	1	dst																									
SUBMULNEG ab-reg, y-reg, x-reg, y-reg			1	0	1	0	0	1	s1	s2	s3	1	0	1	1	1	dst																									
SUBMULNEG ab-reg, y-reg, x-reg, x-reg			1	0	1	0	0	1	s1	s2	s3	1	1	1	1	1	dst																									
SUBMULSUB ab-reg, x-reg, x-reg, y-reg	A		1	0	1	0	0	1	s1	s2	s3	0	0	1	0	1	dst																	2, 1A	dst-reg = dst-reg - src-reg-1 * (src-reg-2 - src-reg-3)	X	FΔ	FΔ	FΔ	FΔ	FΔ	
SUBMULSUB ab-reg, x-reg, y-reg, y-reg			1	0	1	0	0	1	s1	s2	s3	0	1	1	0	1	dst																									
SUBMULSUB ab-reg, y-reg, x-reg, y-reg			1	0	1	0	0	1	s1	s2	s3	1	0	1	0	1	dst																									
SUBMULSUB ab-reg, y-reg, x-reg, x-reg			1	0	1	0	0	1	s1	s2	s3	1	1	1	0	1	dst																									
SUBSH d-reg, d-reg, d-reg	A		0	1	0	1	1	1	1	dst			src1			src2																	1	dst-reg = (src-reg-1 - src-reg-2) >> 1	X	FΔ	FΔ	FΔ	FΔ	FΔ	
SUBSH a-reg, a-reg, a-reg (Y side instruction is not MUL*)	XA		1	1	0	1	0	1	d	0	s1	s2	-	-	-	1	-	-															1	dst-reg = (src-reg-1 - src-reg-2) >> 1	X	FΔ	FΔ	FΔ	FΔ	FΔ	
SUBSH a-reg, a-reg, b-reg (Y side instruction is not MUL*)			1	1	0	1	0	1	d	1	s1	s2	-	-	-	1	-	-																									
SUBSH a-reg, a-reg, a-reg (Y side instruction is MUL*)			1	0	0	1	1	1	d	0	s1	s2	-	-	-	-	-	-																									
SUBSH a-reg, a-reg, b-reg (Y side instruction is MUL*)			1	0	0	1	1	1	d	1	s1	s2	-	-	-	-	-	-																									
SUBSH b-reg, b-reg, a-reg (X side instruction is not MUL*)	YA		1	1	0	-	-	-	-	1	-	-	1	1	d	-	s1	s2																1	dst-reg = (src-reg-1 - src-reg-2) >> 1	Y	FΔ	FΔ	FΔ	FΔ	FΔ	
SUBSH b-reg, b-reg, b-reg (X side instruction is not MUL*)			1	1	0	-	-	-	-	0	-	-	1	1	d	-	s1	s2																									
SUBSH b-reg, b-reg, a-reg (X side instruction is MUL*)			1	0	0	0	-	-	-	-	-	-	1	1	d	1	s1	s2																									
SUBSH b-reg, b-reg, b-reg (X side instruction is MUL*)			1	0	0	0	-	-	-	-	-	-	1	1	d	0	s1	s2																									
SUBSTEP xp-reg	M																		1	1	1	1	1	1	0	1	0	1	0	0	dst		1	dst-reg = dst-reg - xstep-reg (xstep-reg is chosen based on which dst-reg is specified)							

Table 46: Instruction Set Summary (Continued)

Instruction	T	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	#C	Operation	B	C	N	Z	O	E	S
SUBSTEP yp-reg	M																		1	1	1	1	1	1	0	1	0	1	0	1	0	dst		1	dst-reg = dst-reg - ystep-reg (ystep-reg is chosen based on which dst-reg is specified)							
SWAP d-reg, d-reg, cond3	A		0	1	0	1	0	1	1	reg-1			cond			reg-2																		1	if condition holds, then temp = reg-1 reg-1 = reg-2 reg-2 = temp							
XOR d-reg, d-reg, d-reg	A		0	1	0	1	0	1	0	dst			src1			src2																		1	dst-reg = src-reg-1 XOR src-reg-2	X		FΔ	FΔ	F0	FΔ	
XOR d-reg, d-reg, imm3s	A		0	0	1	1	0	1	0	dst			src			data																		1	dst-reg = src-reg XOR signext(data)	X		FΔ	FΔ	F0	FΔ	
XOR.DW d-reg, d-reg, imm3s	A		0	0	0	1	0	1	0	dst			src			data																		1	dst-reg = src-reg XOR (signext(data) >> 24)	X		FΔ	FΔ	F0	FΔ	

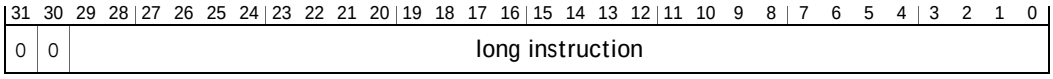
This page left blank for printing purposes.

B Opcode Map

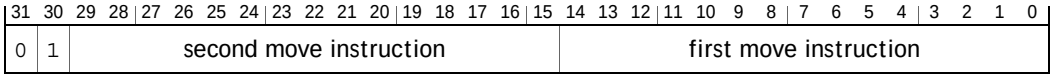
The instruction opcodes, in binary order, are summarized by type in Section B.1, “Long Instructions” on page 396, Section B.2, “Move Instructions” on page 398, and Section B.3, “Arithmetic Instructions” on page 400. These tables can be used to decode instruction opcodes.

First, identify the instruction word type:

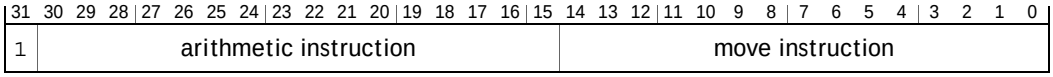
Long Instruction Word



Move-Move Instruction Word



Arithmetic-Move Instruction Word



Then decode the remaining bits based on the instruction types.

B.1 LONG INSTRUCTIONS

Long instructions are found only in *Long* instruction words. The *Long* instruction opcode occupies bits 0-29.

TABLE 47: LONG INSTRUCTIONS

29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Instruction
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	NOP
0	0	1	register			data																								LOAD d-reg, imm24s
0	1	0	0	reg		data																								LOAD I-reg, imm24s
0	1	0	1	acc		data																								LOAD.DW ab-reg, imm24s
0	1	1	0	0	0	0	0	0	0	0	acc		0	0	0	0	0	0	0	0	0	0	0	addr						LOAD.EX ab-reg, P[single-mode(X)]
0	1	1	0	0	0	0	0	0	0	0	acc		1	0	0	0	0	0	0	0	0	0	0	addr						STORE.EX P[single-mode(X)], ab-reg
0	1	1	0	0	0	1	register						0	0	0	0	0	0	0	0	0	0	0	addr						LOAD reg, P[single-mode(X)]
0	1	1	0	0	0	1	register						0	0	0	0	0	0	0	0	0	0	1	0	addr					LOAD reg, P[single-mode(Y)]
0	1	1	0	0	0	1	register						1	0	0	0	0	0	0	0	0	0	0	addr						STORE P[single-mode(X)], reg
0	1	1	0	0	0	1	register						1	0	0	0	0	0	0	0	0	0	1	0	addr					STORE P[single-mode(Y)], reg
1	0	count												loop-end-addr																LOOP.2D imm12u, imm16u
1	1	0	0	register						data																				LOAD reg, imm20s
1	1	0	1	0	0	data								addr																STORE X[imm16u], imm8s
1	1	0	1	0	1	data								addr																STORE Y[imm16u], imm8s
1	1	0	1	1	0	data								addr																STORE.DW XY[imm16u], imm8s
1	1	0	1	1	1	data								addr																STORE P[imm16u], imm8s
1	1	1	0	0	0	0	register						0	addr																LOAD reg, X[imm16u]
1	1	1	0	0	0	0	register						1	addr																STORE X[imm16u], reg
1	1	1	0	0	0	1	register						0	addr																LOAD reg, Y[imm16u]
1	1	1	0	0	0	1	register						1	addr																STORE Y[imm16u], reg
1	1	1	0	0	1	0	0	0	0	0	acc		0	addr																LOAD.DW ab-reg, XY[imm16u]
1	1	1	0	0	1	0	0	0	0	0	acc		1	addr																STORE.DW XY[imm16u], ab-reg
1	1	1	0	0	1	0	1	0	0	0	acc		0	addr																LOAD.EX ab-reg, P[imm16u]
1	1	1	0	0	1	0	1	0	0	0	acc		1	addr																STORE.EX P[imm16u], ab-reg
1	1	1	0	0	1	1	register						0	addr																LOAD reg, P[imm16u]
1	1	1	0	0	1	1	register						1	addr																STORE P[imm16u], reg
1	1	1	0	1	0	0	register						0	addr																LOAD reg, X[stack-mode16]
1	1	1	0	1	0	0	register						1	addr																STORE X[stack-mode16], reg
1	1	1	0	1	0	1	0	dst-reg			src-reg			data																ADD xp-reg, xp-reg, imm16
1	1	1	0	1	1	0	0	0	0	0	0	0	0	addr																GOTO.1D imm16u
1	1	1	0	1	1	0	0	1	0	0	0	0	0	addr																GOTO.0D imm16u

TABLE 47: LONG INSTRUCTIONS (CONTINUED)

29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Instruction
1	1	1	0	1	1	0	1	0	condition					addr																GOTO.2D imm16u, cond5
1	1	1	0	1	1	0	1	1	condition					addr																GOTO.0D imm16u, cond5
1	1	1	0	1	1	1	0	0	0	0	0	0	0	addr																CALL.1D imm16u
1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	vector-number					INTERRUPT imm5u
1	1	1	1	0	0	0	1	0	0	0	0	0	0	restart-addr																SLEEP imm16u
1	1	1	1	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	SLEEP
1	1	1	1	0	0	1	1	0	0	0	count			loop-end-addr																LOOP.2D d-reg, imm16u
1	1	1	1	0	0	1	1	0	0	1	dst-reg			data																ADD xp-reg, SP, imm16
1	1	1	1	0	0	1	1	0	1	0	0	0	0	data																ADD SP, imm16
1	1	1	1	0	0	1	1	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	addr			GOTO.2D xp-reg
1	1	1	1	0	0	1	1	0	1	1	0	0	0	0	0	0	0	0	0	0	1	condition					addr			GOTO.2D xp-reg, cond5
1	1	1	1	0	0	1	1	0	1	1	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	src-reg			CALL.2D xp-reg
1	1	1	1	0	0	1	1	0	1	1	0	0	0	0	0	0	0	0	0	1	1	0	0	0	0	0	0	0	0	RETURN.2D
1	1	1	1	0	0	1	1	0	1	1	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	RETURNI.0D
1	1	1	1	0	0	1	1	0	1	1	0	0	0	0	0	0	0	0	1	0	1	0	0	0	0	index				BITSET SR, imm4u
1	1	1	1	0	0	1	1	0	1	1	0	0	0	0	0	0	0	0	1	1	0	0	0	0	0	index				BITCLR SR, imm4u
1	1	1	1	0	0	1	1	0	1	1	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	RETURNI.2D
1	1	1	1	0	0	1	1	0	1	1	0	0	0	0	0	0	0	0	1	0	0	1	0	0	0	0	0	0	0	ENDLOOP
1	1	1	1	0	0	1	1	0	1	1	0	0	0	0	0	0	0	0	1	0	1	0	0	0	0	0	0	0	0	BREAKPOINT

B.2 MOVE INSTRUCTIONS

Move instructions are found in *Move-Move* instruction words and *Arithmetic-Move* instruction words. In *Move-Move* instruction words, there are two *Move* instruction opcodes in bits 0-14 and 15-29.

In *X* and *Y Move* instructions, a "-" in a bit column indicates a bit that is determined by the other half of the *Dual Move* instruction.

TABLE 48: MOVE INSTRUCTIONS

29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	Instruction
14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
0	-	-	-	-	-	-	-	0	reg		addr				LOAD by-reg, Y[dual-mode(Y)]
0	-	-	-	-	-	-	-	1	0	ac	addr				STORE Y[dual-mode(Y)], b-reg
0	-	-	-	-	-	-	-	1	1	reg		data		0	LOAD by-reg, imm2s
0	-	-	-	-	-	-	-	1	1	dst	src	0	0	1	MOVE y-reg, b-reg
0	-	-	-	-	-	-	-	1	1	dst	src	0	1	1	MOVE b-reg, y-reg
0	-	-	-	-	-	-	-	1	1	dst	src	1	0	1	MOVE x-reg, b-reg
0	-	-	-	-	-	-	-	1	1	dst	src	1	1	1	MOVE a-reg, y-reg
0	0	reg		addr				-	-	-	-	-	-	-	LOAD ax-reg, X[dual-mode(X)]
0	1	0	ac	addr				-	-	-	-	-	-	-	STORE X[dual-mode(X)], a-reg
0	1	1	reg		data		0	-	-	-	-	-	-	-	LOAD ax-reg, imm2s
0	1	1	dst	src	0	0	1	-	-	-	-	-	-	-	MOVE x-reg, a-reg
0	1	1	dst	src	0	1	1	-	-	-	-	-	-	-	MOVE a-reg, x-reg
0	1	1	dst	src	1	0	1	-	-	-	-	-	-	-	MOVE y-reg, a-reg
0	1	1	dst	src	1	1	1	-	-	-	-	-	-	-	MOVE b-reg, x-reg
1	0	0	register					0	addr						LOAD xmem-reg, X[single-mode(X)]
1	0	0	register					1	addr						STORE X[single-mode(X)], xmem-reg
1	0	1	register					0	addr						LOAD stack-reg6, X[stack-mode6]
1	0	1	register					1	addr						STORE X[stack-mode6], stack-reg6
1	1	0	dst-reg						src-reg						MOVE reg, reg
1	1	1	0	0	0	acc		0	addr						LOAD.DW ab-reg, XY[single-mode(X)]
1	1	1	0	0	0	acc		1	addr						STORE.DW XY[single-mode(X)], ab-reg
1	1	1	0	0	1	0	reg		0	addr					LOAD by-reg, Y[single-mode(Y)]
1	1	1	0	0	1	0	reg		1	addr					STORE Y[single-mode(Y)], by-reg
1	1	1	0	0	1	1	acc		0	addr					LOAD.EX ab-reg, P[single-mode(Y)]
1	1	1	0	0	1	1	acc		1	addr					STORE.EX P[single-mode(Y)], ab-reg
1	1	1	0	1	0	dst-reg			data						ADD xp-reg, SP, imm6n
1	1	1	0	1	1	dst-reg			src-reg-1			src-reg-2			ADD xp-reg, xp-reg, xstep-reg

TABLE 48: MOVE INSTRUCTIONS (CONTINUED)

29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	Instruction
14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
1	1	1	1	0	0	dst-reg			src-reg			data			ADD xp-reg, xp-reg, imm3s
1	1	1	1	0	1	0	register			data					LOAD d-reg, imm5s
1	1	1	1	0	1	1	register			data					LOAD xstep-reg, imm5s
1	1	1	1	1	0	0	0	reg		data					LOAD ystep-reg, imm5s
1	1	1	1	1	0	0	1	acc		data					LOAD.DW ab-reg, imm5s
1	1	1	1	1	0	1	0	data							ADD SP, imm7s
1	1	1	1	1	0	1	1	dst		src		data			ADD yp-reg, yp-reg, imm3s
1	1	1	1	1	1	0	0	0	dst		src1		src2		ADD yp-reg, yp-reg, ystep-reg
1	1	1	1	1	1	0	0	1	addr						CLR X[single-mode(X)]
1	1	1	1	1	1	0	1	0	0	0	dst		src		MOVE.DW ab-reg, ab-reg
1	1	1	1	1	1	0	1	0	0	1	dst		src		MOVE.EX ab-reg, ab-reg
1	1	1	1	1	1	0	1	0	1	0	0	dst-reg			SUBSTEP xp-reg
1	1	1	1	1	1	0	1	0	1	0	1	0	dst		SUBSTEP yp-reg
1	1	1	1	1	1	0	1	0	1	0	1	1	0	0	NOP
1	1	1	1	1	1	1	register			0	addr				LOAD stack-reg4, X[stack-mode4]
1	1	1	1	1	1	1	register			1	addr				STORE X[stack-mode4], stack-reg4

B.3 ARITHMETIC INSTRUCTIONS

Arithmetic instructions are found only in *Arithmetic-Move* instruction words. The *Arithmetic* instruction opcode occupies bits 15-30.

In *X* and *Y Arithmetic* instructions, a "-" in a bit column indicates a bit that is determined by the other half of the *Dual Arithmetic* instruction.

TABLE 49: ARITHMETIC INSTRUCTIONS

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	Instruction
0	0	0	0	0	0	0	dst-reg			src-reg			data			ADD.DW d-reg, d-reg, imm3s
0	0	0	0	0	0	1	dst-reg			src-reg			data			ADD.NX.DW d-reg, d-reg, imm3s
0	0	0	0	0	1	0	0	0	0	src-reg			data			CMP.DW d-reg, imm3s
0	0	0	0	0	1	1	0	0	0	src-reg			data			CMPU.NX.DW d-reg, imm3s
0	0	0	0	1	0	0	dst-reg			data						SHLL d-reg, imm6u
0	0	0	0	1	0	1	dst-reg			data						SHLL.NX d-reg, imm6u
0	0	0	0	1	1	0	dst-reg			data						SHRA d-reg, imm6u
0	0	0	0	1	1	1	dst-reg			data						SHRL.NX d-reg, imm6u
0	0	0	1	0	0	0	dst-reg			src-reg			data			AND.DW d-reg, d-reg, imm3s
0	0	0	1	0	0	1	dst-reg			src-reg			data			OR.DW d-reg, d-reg, imm3s
0	0	0	1	0	1	0	dst-reg			src-reg			data			XOR.DW d-reg, d-reg, imm3s
0	0	0	1	0	1	1	register			data						LOAD.DW d-reg, imm6s
0	0	0	1	1	0	0	dst-reg			cond			src-reg			SHLL d-reg, d-reg, cond3
0	0	0	1	1	0	1	dst-reg			cond			data			SHLL d-reg, imm3u, cond3
0	0	0	1	1	1	0	dst-reg			cond			src-reg			SHRA d-reg, d-reg, cond3
0	0	0	1	1	1	1	dst-reg			cond			data			SHRA d-reg, imm3u, cond3
0	0	1	0	0	0	0	dst-reg			src-reg			data			ADD d-reg, d-reg, imm3s
0	0	1	0	0	0	1	dst-reg			src-reg			data			ADD.NX d-reg, d-reg, imm3s
0	0	1	0	0	1	0	0	0	0	src-reg			data			CMP d-reg, imm3s
0	0	1	0	0	1	1	0	0	0	src-reg			data			CMPU.NX d-reg, imm3s
0	0	1	0	1	0	0	dst-reg			src-reg			data			SHLL d-reg, d-reg, imm3u
0	0	1	0	1	0	1	dst-reg			src-reg			data			SHLL.NX d-reg, d-reg, imm3u
0	0	1	0	1	1	0	dst-reg			src-reg			data			SHRA d-reg, d-reg, imm3u
0	0	1	0	1	1	1	dst-reg			src-reg			data			SHRL.NX d-reg, d-reg, imm3u
0	0	1	1	0	0	0	dst-reg			src-reg			data			AND d-reg, d-reg, imm3s
0	0	1	1	0	0	1	dst-reg			src-reg			data			OR d-reg, d-reg, imm3s
0	0	1	1	0	1	0	dst-reg			src-reg			data			XOR d-reg, d-reg, imm3s
0	0	1	1	0	1	1	register			data						LOAD d-reg, imm6s

TABLE 49: ARITHMETIC INSTRUCTIONS (CONTINUED)

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	Instruction
0	0	1	1	1	0	0	dst-reg			index						BITTST d-reg, bit-index
0	0	1	1	1	0	1	dst-reg			index						BITSET d-reg, bit-index
0	0	1	1	1	1	0	dst-reg			index						BITCLR d-reg, bit-index
0	0	1	1	1	1	1	dst-reg			index						BITTGL d-reg, bit-index
0	1	0	0	0	0	0	dst-reg			src-reg-1			src-reg-2			ADD d-reg, d-reg, d-reg
0	1	0	0	0	0	1	dst-reg			src-reg-1			src-reg-2			ADD.NX d-reg, d-reg, d-reg
0	1	0	0	0	1	0	0	0	0	src-reg-1			src-reg-2			CMP d-reg, d-reg
0	1	0	0	0	1	1	0	0	0	src-reg-1			src-reg-2			CMPU.NX d-reg, d-reg
0	1	0	0	1	0	0	dst-reg			src-reg-1			src-reg-2			SHLL d-reg, d-reg, d-reg
0	1	0	0	1	0	1	dst-reg			src-reg-1			src-reg-2			SHLL.NX d-reg, d-reg, d-reg
0	1	0	0	1	1	0	dst-reg			src-reg-1			src-reg-2			SHRA d-reg, d-reg, d-reg
0	1	0	0	1	1	1	dst-reg			src-reg-1			src-reg-2			SHRL.NX d-reg, d-reg, d-reg
0	1	0	1	0	0	0	dst-reg			src-reg-1			src-reg-2			AND d-reg, d-reg, d-reg
0	1	0	1	0	0	1	dst-reg			src-reg-1			src-reg-2			OR d-reg, d-reg, d-reg
0	1	0	1	0	1	0	dst-reg			src-reg-1			src-reg-2			XOR d-reg, d-reg, d-reg
0	1	0	1	0	1	1	reg-1			cond			reg-2			SWAP d-reg, d-reg, cond3
0	1	0	1	1	0	0	dst-reg			cond			data			ADD d-reg, imm3s, cond3
0	1	0	1	1	0	1	dst-reg			cond			data			ADD.DW d-reg, imm3s, cond3
0	1	0	1	1	1	0	dst-reg			src-reg-1			src-reg-2			ADDSH d-reg, d-reg, d-reg
0	1	0	1	1	1	1	dst-reg			src-reg-1			src-reg-2			SUBSH d-reg, d-reg, d-reg
0	1	1	0	0	0	0	dst-reg			src-reg-1			src-reg-2			SUB d-reg, d-reg, d-reg
0	1	1	0	0	0	1	dst-reg			src-reg-1			src-reg-2			SUB.NX d-reg, d-reg, d-reg
0	1	1	0	0	1	0	dst-reg			src-reg-1			src-reg-2			MAX d-reg, d-reg, d-reg
0	1	1	0	0	1	1	dst-reg			src-reg-1			src-reg-2			MIN d-reg, d-reg, d-reg
0	1	1	0	1	0	0	dst-reg			src-reg-1			src-reg-2			DIVST d-reg, d-reg, d-reg
0	1	1	0	1	0	1	0	0	0	0	0	0	0	0	0	NOP
0	1	1	0	1	1	0	dst-reg			src-reg			data			ADD.FR d-reg, d-reg, imm3s
0	1	1	0	1	1	1	dst-reg			0	0	0	src-reg			LOG2ABS d-reg, d-reg
0	1	1	1	0	0	0	dst-reg			0	0	0	src-reg			NEG d-reg, d-reg
0	1	1	1	0	0	1	dst-reg			0	0	0	src-reg			NLOG2ABS d-reg, d-reg
0	1	1	1	0	1	0	dst-reg			0	0	0	src-reg			ABS d-reg, d-reg
0	1	1	1	0	1	1	dst-reg			0	0	0	src-reg			MOVE.NX d-reg, d-reg
0	1	1	1	1	0	0	dst-reg			cond			src-reg			SUB d-reg, d-reg, cond3
0	1	1	1	1	0	1	dst-reg			cond			src-reg			MOVE d-reg, d-reg, cond3
0	1	1	1	1	1	0	dst-reg			cond			src-reg			ADD d-reg, d-reg, cond3

TABLE 49: ARITHMETIC INSTRUCTIONS (CONTINUED)

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	Instruction
0	1	1	1	1	1	1	dst-reg			cond			src-reg			NEG d-reg, d-reg, cond3
1	0	0	0	-	-	-	-	-	-	0	0	dst	0	s1	s2	ADD b-reg, b-reg, b-reg
1	0	0	0	-	-	-	-	-	-	0	0	dst	1	s1	s2	ADD b-reg, b-reg, a-reg
1	0	0	0	-	-	-	-	-	-	0	1	dst	0	s1	s2	SUB b-reg, b-reg, b-reg
1	0	0	0	-	-	-	-	-	-	0	1	dst	1	s1	s2	SUB b-reg, b-reg, a-reg
1	0	0	0	-	-	-	-	-	-	1	0	dst	0	s1	s2	ADDSH b-reg, b-reg, b-reg
1	0	0	0	-	-	-	-	-	-	1	0	dst	1	s1	s2	ADDSH b-reg, b-reg, a-reg
1	0	0	0	-	-	-	-	-	-	1	1	dst	0	s1	s2	SUBSH b-reg, b-reg, b-reg
1	0	0	0	-	-	-	-	-	-	1	1	dst	1	s1	s2	SUBSH b-reg, b-reg, a-reg
1	0	0	0	0	s1	s2	0	0	dst	-	-	-	-	-	-	MULADD a-reg, x-reg, x-reg
1	0	0	0	0	s1	s2	0	1	dst	-	-	-	-	-	-	MULSUB a-reg, x-reg, x-reg
1	0	0	0	0	s1	s2	1	0	dst	-	-	-	-	-	-	MUL a-reg, x-reg, x-reg
1	0	0	0	0	s1	s2	1	1	dst	-	-	-	-	-	-	MULNEG a-reg, x-reg, x-reg
1	0	0	0	1	s1	s2	0	0	dst	-	-	-	-	-	-	MULADD a-reg, x-reg, y-reg
1	0	0	0	1	s1	s2	0	1	dst	-	-	-	-	-	-	MULSUB a-reg, x-reg, y-reg
1	0	0	0	1	s1	s2	1	0	dst	-	-	-	-	-	-	MUL a-reg, x-reg, y-reg
1	0	0	0	1	s1	s2	1	1	dst	-	-	-	-	-	-	MULNEG a-reg, x-reg, y-reg
1	0	0	1	-	-	-	-	-	-	0	s1	s2	0	0	dst	MULADD b-reg, y-reg, y-reg
1	0	0	1	-	-	-	-	-	-	0	s1	s2	0	1	dst	MULSUB b-reg, y-reg, y-reg
1	0	0	1	-	-	-	-	-	-	0	s1	s2	1	0	dst	MUL b-reg, y-reg, y-reg
1	0	0	1	-	-	-	-	-	-	0	s1	s2	1	1	dst	MULNEG b-reg, y-reg, y-reg
1	0	0	1	-	-	-	-	-	-	1	s1	s2	0	0	dst	MULADD b-reg, y-reg, x-reg
1	0	0	1	-	-	-	-	-	-	1	s1	s2	0	1	dst	MULSUB b-reg, y-reg, x-reg
1	0	0	1	-	-	-	-	-	-	1	s1	s2	1	0	dst	MUL b-reg, y-reg, x-reg
1	0	0	1	-	-	-	-	-	-	1	s1	s2	1	1	dst	MULNEG b-reg, y-reg, x-reg
1	0	0	1	0	0	dst	0	s1	s2	-	-	-	-	-	-	ADD a-reg, a-reg, a-reg
1	0	0	1	0	0	dst	1	s1	s2	-	-	-	-	-	-	ADD a-reg, a-reg, b-reg
1	0	0	1	0	1	dst	0	s1	s2	-	-	-	-	-	-	SUB a-reg, a-reg, a-reg
1	0	0	1	0	1	dst	1	s1	s2	-	-	-	-	-	-	SUB a-reg, a-reg, b-reg
1	0	0	1	1	0	dst	0	s1	s2	-	-	-	-	-	-	ADDSH a-reg, a-reg, a-reg
1	0	0	1	1	0	dst	1	s1	s2	-	-	-	-	-	-	ADDSH a-reg, a-reg, b-reg
1	0	0	1	1	1	dst	0	s1	s2	-	-	-	-	-	-	SUBSH a-reg, a-reg, a-reg
1	0	0	1	1	1	dst	1	s1	s2	-	-	-	-	-	-	SUBSH a-reg, a-reg, b-reg
1	0	1	0	0	0	0	0	src1		src2		0	0	dst		MULADD ab-reg, xy-reg, xy-reg
1	0	1	0	0	0	0	0	src1		src2		0	1	dst		MULSUB ab-reg, xy-reg, xy-reg

TABLE 49: ARITHMETIC INSTRUCTIONS (CONTINUED)

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	Instruction
1	0	1	0	0	0	0	0	src1		src2		1	0	dst		MUL ab-reg, xy-reg, xy-reg
1	0	1	0	0	0	0	0	src1		src2		1	1	dst		MULNEG ab-reg, xy-reg, xy-reg
1	0	1	0	0	0	0	1	src1		src2		0	0	dst		MULADD.UU ab-reg, xy-reg, xy-reg
1	0	1	0	0	0	0	1	src1		src2		0	1	dst		MULSUB.UU ab-reg, xy-reg, xy-reg
1	0	1	0	0	0	0	1	src1		src2		1	0	dst		MUL.UU ab-reg, xy-reg, xy-reg
1	0	1	0	0	0	0	1	src1		src2		1	1	dst		MULNEG.UU ab-reg, xy-reg, xy-reg
1	0	1	0	0	0	1	0	src1		src2		0	0	dst		MULADD.SU ab-reg, xy-reg, xy-reg
1	0	1	0	0	0	1	0	src1		src2		0	1	dst		MULSUB.SU ab-reg, xy-reg, xy-reg
1	0	1	0	0	0	1	0	src1		src2		1	0	dst		MUL.SU ab-reg, xy-reg, xy-reg
1	0	1	0	0	0	1	0	src1		src2		1	1	dst		MULNEG.SU ab-reg, xy-reg, xy-reg
1	0	1	0	0	1	s1	s2	s3	0	0	0	0	0	dst		ADDMULADD ab-reg, x-reg, x-reg, y-reg
1	0	1	0	0	1	s1	s2	s3	0	0	0	0	1	dst		ADDMULSUB ab-reg, x-reg, x-reg, y-reg
1	0	1	0	0	1	s1	s2	s3	0	0	0	1	0	dst		ADDMUL ab-reg, x-reg, x-reg, y-reg
1	0	1	0	0	1	s1	s2	s3	0	0	0	1	1	dst		ADDMULNEG ab-reg, x-reg, x-reg, y-reg
1	0	1	0	0	1	s1	s2	s3	0	0	1	0	0	dst		SUBMULADD ab-reg, x-reg, x-reg, y-reg
1	0	1	0	0	1	s1	s2	s3	0	0	1	0	1	dst		SUBMULSUB ab-reg, x-reg, x-reg, y-reg
1	0	1	0	0	1	s1	s2	s3	0	0	1	1	0	dst		SUBMUL ab-reg, x-reg, x-reg, y-reg
1	0	1	0	0	1	s1	s2	s3	0	0	1	1	1	dst		SUBMULNEG ab-reg, x-reg, x-reg, y-reg
1	0	1	0	0	1	s1	s2	s3	0	1	0	0	0	dst		ADDMULADD ab-reg, x-reg, y-reg, y-reg
1	0	1	0	0	1	s1	s2	s3	0	1	0	0	1	dst		ADDMULSUB ab-reg, x-reg, y-reg, y-reg
1	0	1	0	0	1	s1	s2	s3	0	1	0	1	0	dst		ADDMUL ab-reg, x-reg, y-reg, y-reg
1	0	1	0	0	1	s1	s2	s3	0	1	0	1	1	dst		ADDMULNEG ab-reg, x-reg, y-reg, y-reg
1	0	1	0	0	1	s1	s2	s3	0	1	1	0	0	dst		SUBMULADD ab-reg, x-reg, y-reg, y-reg
1	0	1	0	0	1	s1	s2	s3	0	1	1	0	1	dst		SUBMULSUB ab-reg, x-reg, y-reg, y-reg
1	0	1	0	0	1	s1	s2	s3	0	1	1	1	0	dst		SUBMUL ab-reg, x-reg, y-reg, y-reg
1	0	1	0	0	1	s1	s2	s3	0	1	1	1	1	dst		SUBMULNEG ab-reg, x-reg, y-reg, y-reg
1	0	1	0	0	1	s1	s2	s3	1	0	0	0	0	dst		ADDMULADD ab-reg, y-reg, x-reg, y-reg
1	0	1	0	0	1	s1	s2	s3	1	0	0	0	1	dst		ADDMULSUB ab-reg, y-reg, x-reg, y-reg
1	0	1	0	0	1	s1	s2	s3	1	0	0	1	0	dst		ADDMUL ab-reg, y-reg, x-reg, y-reg
1	0	1	0	0	1	s1	s2	s3	1	0	0	1	1	dst		ADDMULNEG ab-reg, y-reg, x-reg, y-reg
1	0	1	0	0	1	s1	s2	s3	1	0	1	0	0	dst		SUBMULADD ab-reg, y-reg, x-reg, y-reg
1	0	1	0	0	1	s1	s2	s3	1	0	1	0	1	dst		SUBMULSUB ab-reg, y-reg, x-reg, y-reg
1	0	1	0	0	1	s1	s2	s3	1	0	1	1	0	dst		SUBMUL ab-reg, y-reg, x-reg, y-reg
1	0	1	0	0	1	s1	s2	s3	1	0	1	1	1	dst		SUBMULNEG ab-reg, y-reg, x-reg, y-reg
1	0	1	0	0	1	s1	s2	s3	1	1	0	0	0	dst		ADDMULADD ab-reg, y-reg, x-reg, x-reg

TABLE 49: ARITHMETIC INSTRUCTIONS (CONTINUED)

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	Instruction
1	0	1	0	0	1	s1	s2	s3	1	1	0	0	1	dst		ADDMULSUB ab-reg, y-reg, x-reg, x-reg
1	0	1	0	0	1	s1	s2	s3	1	1	0	1	0	dst		ADDMUL ab-reg, y-reg, x-reg, x-reg
1	0	1	0	0	1	s1	s2	s3	1	1	0	1	1	dst		ADDMULNEG ab-reg, y-reg, x-reg, x-reg
1	0	1	0	0	1	s1	s2	s3	1	1	1	0	0	dst		SUBMULADD ab-reg, y-reg, x-reg, x-reg
1	0	1	0	0	1	s1	s2	s3	1	1	1	0	1	dst		SUBMULSUB ab-reg, y-reg, x-reg, x-reg
1	0	1	0	0	1	s1	s2	s3	1	1	1	1	0	dst		SUBMUL ab-reg, y-reg, x-reg, x-reg
1	0	1	0	0	1	s1	s2	s3	1	1	1	1	1	dst		SUBMULNEG ab-reg, y-reg, x-reg, x-reg
1	0	1	1	-	-	-	0	-	-	0	0	dst	-	s1	s2	MULADD b-reg, y-reg, y-reg
1	0	1	1	-	-	-	0	-	-	0	1	dst	-	s1	s2	MULSUB b-reg, y-reg, y-reg
1	0	1	1	-	-	-	0	-	-	1	0	dst	-	s1	s2	MUL b-reg, y-reg, y-reg
1	0	1	1	-	-	-	0	-	-	1	1	dst	-	s1	s2	MULNEG b-reg, y-reg, y-reg
1	0	1	1	-	-	-	1	-	-	0	0	dst	-	s1	s2	MULADD b-reg, y-reg, x-reg
1	0	1	1	-	-	-	1	-	-	0	1	dst	-	s1	s2	MULSUB b-reg, y-reg, x-reg
1	0	1	1	-	-	-	1	-	-	1	0	dst	-	s1	s2	MUL b-reg, y-reg, x-reg
1	0	1	1	-	-	-	1	-	-	1	1	dst	-	s1	s2	MULNEG b-reg, y-reg, x-reg
1	0	1	1	0	0	dst	0	s1	s2	-	-	-	0	-	-	MULADD a-reg, x-reg, x-reg
1	0	1	1	0	0	0	-	0	0	-	-	-	1	-	-	NOP
1	0	1	1	0	0	dst	1	s1	s2	-	-	-	0	-	-	MULADD a-reg, x-reg, y-reg
1	0	1	1	0	1	dst	0	s1	s2	-	-	-	0	-	-	MULSUB a-reg, x-reg, x-reg
1	0	1	1	0	1	dst	1	s1	s2	-	-	-	0	-	-	MULSUB a-reg, x-reg, y-reg
1	0	1	1	1	0	dst	0	s1	s2	-	-	-	0	-	-	MUL a-reg, x-reg, x-reg
1	0	1	1	1	0	dst	1	s1	s2	-	-	-	0	-	-	MUL a-reg, x-reg, y-reg
1	0	1	1	1	1	dst	0	s1	s2	-	-	-	0	-	-	MULNEG a-reg, x-reg, x-reg
1	0	1	1	1	1	dst	1	s1	s2	-	-	-	0	-	-	MULNEG a-reg, x-reg, y-reg
1	1	0	-	-	-	-	0	-	-	0	0	dst	-	s1	s2	ADD b-reg, b-reg, b-reg
1	1	0	-	-	-	-	0	-	-	0	1	dst	-	s1	s2	SUB b-reg, b-reg, b-reg
1	1	0	-	-	-	-	0	-	-	1	0	dst	-	s1	s2	ADDSH b-reg, b-reg, b-reg
1	1	0	-	-	-	-	0	-	-	1	1	dst	-	s1	s2	SUBSH b-reg, b-reg, b-reg
1	1	0	-	-	-	-	1	-	-	0	0	dst	-	s1	s2	ADD b-reg, b-reg, a-reg
1	1	0	-	-	-	-	1	-	-	0	1	dst	-	s1	s2	SUB b-reg, b-reg, a-reg
1	1	0	-	-	-	-	1	-	-	1	0	dst	-	s1	s2	ADDSH b-reg, b-reg, a-reg
1	1	0	-	-	-	-	1	-	-	1	1	dst	-	s1	s2	SUBSH b-reg, b-reg, a-reg
1	1	0	0	0	0	dst	0	s1	s2	-	-	-	0	-	-	ADD a-reg, a-reg, a-reg
1	1	0	0	0	0	dst	0	s1	s2	-	-	-	1	-	-	ADD.NX a-reg, a-reg, a-reg
1	1	0	0	0	0	dst	1	s1	s2	-	-	-	0	-	-	ADD a-reg, a-reg, b-reg

TABLE 49: ARITHMETIC INSTRUCTIONS (CONTINUED)

30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	Instruction
1	1	0	0	0	0	dst	1	s1	s2	-	-	-	1	-	-	ADD.NX a-reg, a-reg, b-reg
1	1	0	0	0	1	0	0	s1	s2	-	-	-	0	-	-	CMP a-reg, a-reg
1	1	0	0	0	1	0	0	s1	s2	-	-	-	1	-	-	CMPU.NX a-reg, a-reg
1	1	0	0	0	1	0	1	s1	s2	-	-	-	0	-	-	CMP a-reg, b-reg
1	1	0	0	0	1	0	1	s1	s2	-	-	-	1	-	-	CMPU.NX a-reg, b-reg
1	1	0	0	1	0	dst	0	s1	s2	-	-	-	0	-	-	SUB a-reg, a-reg, a-reg
1	1	0	0	1	0	dst	0	s1	s2	-	-	-	1	-	-	SUB.NX a-reg, a-reg, a-reg
1	1	0	0	1	0	dst	1	s1	s2	-	-	-	0	-	-	SUB a-reg, a-reg, b-reg
1	1	0	0	1	0	dst	1	s1	s2	-	-	-	1	-	-	SUB.NX a-reg, a-reg, b-reg
1	1	0	0	1	1	dst	0	s1	s2	-	-	-	0	-	-	MAX a-reg, a-reg, a-reg
1	1	0	0	1	1	dst	0	s1	s2	-	-	-	1	-	-	MIN a-reg, a-reg, a-reg
1	1	0	0	1	1	dst	1	s1	s2	-	-	-	0	-	-	MAX a-reg, a-reg, b-reg
1	1	0	0	1	1	dst	1	s1	s2	-	-	-	1	-	-	MIN a-reg, a-reg, b-reg
1	1	0	1	0	0	dst	0	s1	s2	-	-	-	0	-	-	AND a-reg, a-reg, a-reg
1	1	0	1	0	0	dst	0	s1	s2	-	-	-	1	-	-	OR a-reg, a-reg, a-reg
1	1	0	1	0	0	dst	1	s1	s2	-	-	-	0	-	-	AND a-reg, a-reg, b-reg
1	1	0	1	0	0	dst	1	s1	s2	-	-	-	1	-	-	OR a-reg, a-reg, b-reg
1	1	0	1	0	1	dst	0	s1	s2	-	-	-	0	-	-	ADDSH a-reg, a-reg, a-reg
1	1	0	1	0	1	dst	0	s1	s2	-	-	-	1	-	-	SUBSH a-reg, a-reg, a-reg
1	1	0	1	0	1	dst	1	s1	s2	-	-	-	0	-	-	ADDSH a-reg, a-reg, b-reg
1	1	0	1	0	1	dst	1	s1	s2	-	-	-	1	-	-	SUBSH a-reg, a-reg, b-reg
1	1	0	1	1	0	dst	0	0	src	-	-	-	0	-	-	NEG a-reg, a-reg
1	1	0	1	1	0	dst	0	0	src	-	-	-	1	-	-	NLOG2ABS a-reg, a-reg
1	1	0	1	1	0	dst	1	0	src	-	-	-	0	-	-	NEG a-reg, b-reg
1	1	0	1	1	0	dst	1	0	src	-	-	-	1	-	-	NLOG2ABS a-reg, b-reg
1	1	0	1	1	1	dst	0	0	src	-	-	-	0	-	-	ABS a-reg, a-reg
1	1	0	1	1	1	dst	0	0	src	-	-	-	1	-	-	MOVE.NX a-reg, a-reg
1	1	0	1	1	1	dst	1	0	src	-	-	-	0	-	-	ABS a-reg, b-reg
1	1	0	1	1	1	dst	1	0	src	-	-	-	1	-	-	MOVE.NX a-reg, b-reg

This page left blank for printing purposes.

C Glossary

The following abbreviations and terms are used in this manual:

AGU

Address Generation Unit. The two AGUs provide addresses for indirect memory accesses, and handle auto-increment, auto-decrement and circular buffers. They also execute the address arithmetic instructions in the *Long* and *Move* classes.

Arithmetic-Move instruction

An instruction word containing an arithmetic part that is executed by the DCU and a data movement part that is executed by the DMU and AGUs. The arithmetic and data movement parts are executed in parallel.

DCU

Data Computation Unit. Performs arithmetic, logical, multiplication and multiply-accumulate (MAC) operations. Also does shifting, leading bit detection, and some data movement operations.

delay slot

A clock cycle available in the execution pipeline after a control-flow instruction is fetched.

DMU

Data Movement Unit. Handles loading data from memory, storing data to memory, and moving data between registers. Also handles conversion of data moved from a source to a target of a different width.

Double-Width data transfer

The *Long* and *Move* instruction mnemonics use the **`.DW`** suffix to indicate the transfer width, which is 48 bits.

Dual Arithmetic instruction

Contains two operations that are executed in parallel using both the X and Y sides of the DCU.

Dual Move instruction

Contains two independent data movement instructions that are executed in parallel by the DMU.

Extended Single-Width data transfer

Also called *extended* data transfers. The instruction mnemonics use the **`.EX`** suffix. 32-bit operations.

HEAR Configurable Accelerator

The ON Semiconductor device core that is a microcode configurable signal processing engine used to execute a variety of signal processing functions;

designed to work with the CFX to reduce the signal processing load on the CFX.

ILPC
interrupt-link program counter

ILSR
interrupt-link status register

LC
loop count register

LEA
loop end address register

Long instruction
The entire instruction word contains just a single instruction.

LR
link register

LSA
loop start address register

LSB
least significant bit

MAC
See “multiply-accumulate”.

Move-Move instruction
An instruction word that contains two data movement instructions that are executed sequentially by the DMU and AGUs.

MSB
most significant bit

multiplication instructions
All instructions that multiply, which means both *multiply* instructions and *multiply-accumulate* instructions, including those with pre-add and pre-subtract.

multiply-accumulate instructions
Multiplication instructions that add or subtract their product to an accumulator and have mnemonics *MULADD or *MULSUB.

multiply instructions
Also called *multiply-only* instructions. They do not accumulate (add to or subtract from) their product, so their mnemonics are *MUL or *MULNEG.

PC
program counter

PCU

Program Control Unit. Fetches and decodes instructions, and controls their execution.

rounding

Eliminates the low part of the accumulator when a full 56-bit accumulator is the source of a single-width (24-bit) or extended (32-bit) transfer.

register file

A set of equivalent registers. The instruction set is specified in terms of register files, so wherever one of the registers in a set can be used, another is also valid. For example, if `X0` is valid, `X1` is equally valid.

saturation

Eliminates the extension part of the accumulator when a full 56-bit accumulator is the source of a single-width (24-bit) or double-width (48-bit) transfer.

Single Arithmetic instruction

Carries out a single operation using the X side of the DCU.

Single Move instruction

Moves just one datum or performs addressing register arithmetic.

Single-Width data transfer

All `LOAD`, `STORE`, and `MOVE` instructions (in the *Long* and *Move* classes) that do *not* have a suffix (**`.EX`** or **`.DW`**). 24-bit operations.

SR

status register

stack

The CFX stack grows upwards (increasing addresses) in X memory and the stack pointer points to the next unused word at the top of the stack.

X ALU

The arithmetic logic unit of the DCU that is used for *Single Arithmetic* instructions and the *X Arithmetic* half of *Dual Arithmetic* instructions.

X Arithmetic instruction

The part of a *Dual Arithmetic* instruction that operates on the X side of the DCU.

X Move instruction

One part of a *Dual Move* instruction. Accesses X memory or performs a register operation.

X MUL

The multiplier in the DCU that is used for *Single Arithmetic* instructions and the *X Arithmetic* half of *Dual Arithmetic* instructions.

XY memory

Also referred to as *XYMEM*. A logical 48-bit memory generated by combining X memory and Y memory. The low-order word is stored in XMEM and the high-order word in YMEM.

Y ALU

The arithmetic logic unit of the DCU that is used for the *Y Arithmetic* half of *Dual Arithmetic* instructions.

Y Arithmetic instruction

The part of a *Dual Arithmetic* instruction that operates on the Y side of the DCU.

Y Move instruction

One part of a *Dual Move* instruction. Accesses Y memory or performs a register operation.

Y MUL

The multiplier in the DCU that is used only for the *Y Arithmetic* half of *Dual Arithmetic* instructions.

This page left blank for printing purposes.

Global Headquarters
ON Semiconductor
5005 East McDowell Road
Phoenix, Arizona 85008 USA
Tel: +1 602 244 6600

Website
www.onsemi.com

Copyright © 2011 Semiconductor Components Industries, LLC d/b/a ON Semiconductor. ON Semiconductor and the ON logo are registered trademarks of Semiconductor Components Industries, LLC (ON Semiconductor). All other brands, product names and company names mentioned herein may be trademarks or registered trademarks of their respective holders.